

VOLUME IV — THE NEURAL AND AI ERA

The Compression Book

A Complete History and Technical Treatise on Data Compression

Compiled by a Multi-Agent Research Workflow

June 2026 Edition

Contents

- 1 A Machine-Learning Primer for Compression 1
 - 1.1 From a hand-built transform to a learned one 2
 - 1.2 The neuron: a weighted sum with a squash 3
 - 1.3 Learning by rolling downhill: gradient descent 5
 - 1.4 The autoencoder: a bottleneck that **is** a compressor 9
 - 1.5 The rate–distortion loss: compression as one optimization 11
 - 1.6 Four ways to learn a source distribution 13
 - 1.7 A neuron, a layer, and a network in Python 18
 - 1.8 Where learned compression stands, mid-2026 22
 - 1.9 Exercises 23
 - 1.10 Further reading 24
- 2 Learned Image Compression: Autoencoders and Hyperpriors 26
 - 2.1 The same skeleton, new bones 27
 - 2.2 The rate term: counting bits with a model 30
 - 2.3 Ballé’s transform: convolutions and GDN 34
 - 2.4 The hyperprior: transmitting a description of the model 37
 - 2.5 Context models: predicting each latent from its neighbours 40
 - 2.6 Why the learned rate term is honest 43
 - 2.7 From paper to standard: JPEG AI, and the state of play in 2026 45
 - 2.8 Exercises 47
 - 2.9 Further reading 48
- 3 Generative and Perceptual Codecs 49
 - 3.1 The Blur Problem: Why MSE Fails at Low Bitrates 50
 - 3.2 The Rate–Distortion–Perception Tradeoff 51
 - 3.3 HiFiC: High-Fidelity Generative Image Compression 52
 - 3.4 PerCo: Diffusion Decoders for Ultra-Low Bitrates 56
 - 3.5 The 2025 Push: One-Step Distillation 59
 - 3.6 The Hallucinated Detail Debate 60
 - 3.7 JPEG AI: Standardizing the Neural Codec 62
 - 3.8 A Python Sketch of Perception-Aware Training 63
 - 3.9 How Diffusion Decoders Are Conditioned: A Closer Look 66
 - 3.10 Measuring Perceptual Quality: LPIPS and FID 68
 - 3.11 The Gap Between Research and Deployment 70
 - 3.12 Looking Forward: Compression as Conditional Generation 71
 - 3.13 Exercises 72
 - 3.14 Further Reading 73
- 4 Learned Video and Implicit Neural Representations 75
 - 4.1 Why Video Is Both Easier and Harder Than Images 76

4.2	DVC: The First End-to-End Neural Video Codec	77
4.3	The DCVC Family: Conditional Coding	79
4.4	VCT: Remove Motion Entirely	83
4.5	Implicit Neural Representations: A Completely Different Idea	85
4.6	NeRV: Extending INRs to Video	88
4.7	Comparing the Three Approaches	89
4.8	The Deployment Reality: Why Your Phone Still Uses AV1	90
4.9	A Tiny Worked Example: INR in 20 Lines	91
4.10	Where the Field Stands in 2026	94
4.11	Exercises	95
4.12	Further Reading	96
5	Neural Audio Codecs as Tokenizers	98
5.1	From Waveform to Latent: the Encoder–Decoder Backbone	99
5.2	Vector Quantization: Turning Vectors into Codebook Indices	100
5.3	Residual Vector Quantization: the Core Idea	101
5.4	SoundStream (2021): the First Universal Neural Audio Codec	103
5.5	EnCodec (2022): Streaming, Stereo, and Entropy Coding	104
5.6	DAC (2023): Codebook Utilisation and Wideband Quality	105
5.7	The Token Hierarchy: Acoustic vs. Semantic	106
5.8	From Codec to Tokenizer: AudioLM, VALL-E, and the Audio LLM Era	107
5.9	A Worked Encoding Example	108
5.10	Mimi and the Moshi System (2024): Merging Semantics and Acoustics	110
5.11	The Semantic–Acoustic Split in Practice	111
5.12	WavTokenizer and the Single-Codebook Frontier (2024–2025)	112
5.13	Trade-offs and Open Problems (2025–2026)	113
5.14	The Big Picture: Codec and Tokenizer Converge	114
5.15	Exercises	115
5.16	Further Reading	116
6	Compression as Intelligence: The Theory	118
6.1	From coding to learning: the leap this chapter makes	119
6.2	MDL as a theory of inference, not just coding	120
6.3	PAC-Bayes: a shorter description provably predicts better	123
6.4	Solomonoff: the ideal learner that compresses everything	126
6.5	AIXI: from compressing data to acting intelligently	128
6.6	The honest counterarguments	130
6.7	A computational reading of the thesis	133
6.8	Exercises	136
6.9	Further reading	138
7	Large Language Models as Compressors	139
7.1	The bridge we already built: prediction is compression	140
7.2	What is a language model, really?	142
7.3	The headline result: a text model beats PNG and FLAC	144
7.4	Bellard’s tools: making it actually run	146
7.5	Build it: tinyzip’s LLM-as-compressor	149

7.6	The scoreboard, and a dose of honesty	154
7.7	Compression as intelligence: the evidence and the limits	157
7.8	Exercises	160
7.9	Further reading	161
8	Model Compression I: Quantization	163
8.1	What Is a Neural Network Weight, and Why Does It Matter?	164
8.2	The Zoo of Bit Widths	165
8.3	The Outlier Problem: Why Naive Quantization Fails	165
8.4	Post-Training Quantization: The GPTQ Algorithm	166
8.5	AWQ: Protecting the Weights That Matter Most	169
8.6	Quantization-Aware Training: Teaching the Model About Its Future	171
8.7	FP8: The Hardware Sweet Spot	173
8.8	The Information-Theoretic Frontier: BitNet b1.58	173
8.9	Mixed-Precision and the Practical Toolkit	176
8.10	Code: PTQ in Python from Scratch	177
8.11	A Second Code Listing: Naive Ternary Quantization	179
8.12	What Survives Quantization, and What Breaks	181
8.13	The 2024–2026 Frontier	181
8.14	Summary: Choosing a Quantization Strategy	182
8.15	Exercises	183
8.16	Further Reading	184
9	Model Compression II: Pruning, Distillation, Low-Rank	186
9.1	Pruning: Cutting the Fat	187
9.2	Knowledge Distillation: Teach a Small Student	194
9.3	Low-Rank Factorization and LoRA	199
9.4	Combining the Three: The Production Pipeline	202
9.5	Exercises	204
9.6	Further Reading	205
10	KV-Cache, Context, and Embedding Compression	207
10.1	The KV-Cache: What It Is and Why It Hurts	208
10.2	Family 1: Eviction — Throwing Away What You Don’t Need	210
10.3	Family 2: Architectural Compression — DeepSeek’s Multi-Head Latent Attention	212
10.4	Family 3: KV-Cache Quantization — Fewer Bits per Number	214
10.5	Family 4: Prompt and Context Compression	216
10.6	Product Quantization: Compressing Vectors for Retrieval	217
10.7	Binary and Matryoshka Embeddings	220
10.8	Putting It All Together: The Inference Memory Stack	223
10.9	Exercises	226
10.10	Further Reading	227
11	Solutions to Exercises	229

A Machine-Learning Primer for Compression

We wanted flying cars; instead we got a function that, shown enough examples, learns to round off the world.

— a paraphrase, with apologies, of a famous Silicon Valley lament

For fifty years the people who built compressors were craftsmen. Someone sat down, thought hard about images, and **wrote down** a transform by hand — the discrete cosine transform of JPEG (Chapter 42), the wavelets of JPEG 2000 (Chapter 43), the integer transforms of H.264 (Chapter 52). Someone else hand-tuned a quantization table by squinting at test images. A third person hand-designed an entropy coder. Each box was a human decision, defended in a standards meeting, frozen into a specification, and shipped to billions of devices.

Then, around 2016, a small group of researchers asked a heretical question. **What if we never specify the transform at all?** What if, instead of a human writing down the DCT, we let a computer **discover** its own transform directly from a mountain of photographs — and what if the very thing it tried to minimize was the file size? Not a transform a person guessed would compress well, but a transform **grown**, knob by knob, to compress well, by a procedure that automatically tunes millions of knobs at once.

That procedure is **machine learning**, and the family of models that grew the best transforms is **neural networks**. By 2024 a learned image codec called JPEG AI had become the first international standard built this way, beating the best hand-built codec on Earth. To understand how a **learned** compressor works — which is the entire subject of Volume IV — you need just enough deep learning to follow the plot. That is exactly what this chapter delivers: neurons, gradient descent, autoencoders, and the menagerie of generative models, each introduced from scratch and each framed, from the first sentence, as **a way to learn the distribution of a source**. Because that, as Chapter 23 promised, is all compression ever was.

WHERE WE ARE

In Chapter 11 we built a **gentle calculus**: the derivative as a slope, and the idea that following the slope downhill finds a minimum. In Chapter 12 we met **vectors and matrices** as bundles of numbers and the linear maps between them. Chapters 9 and 10 gave us **probability, random variables, and expectation**. Chapters 18–21 turned all

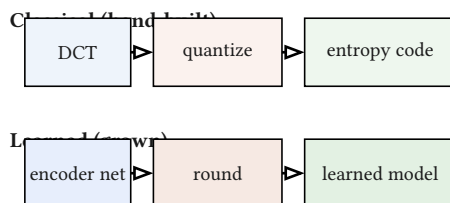
of that into information theory: **entropy** is the true cost in bits (Chapter 19), and the **rate–distortion** curve is the unbeatable trade-off between file size and fidelity (Chapter 21). Most importantly, Chapter 23 proved this chapter’s secret title — **compression is prediction is learning**: a model that predicts the next symbol well is, by the same arithmetic, a compressor that codes it cheaply. This chapter cashes that promise in. A neural network is just a flexible, trainable predictor; train it to predict your data, and you have grown yourself a compressor.

By the end of this chapter you will be able to:

- Read a **neuron** as a plain mathematical function: weighted sum, bias, squashing nonlinearity.
- Explain why stacking neurons into **layers** lets a network approximate almost any function.
- Follow **gradient descent** and **backpropagation** as “calculus rolling downhill,” using only Chapter 11’s slope idea.
- Describe an **autoencoder** and the **bottleneck**, and see why a narrow bottleneck is a lossy compressor.
- State the **rate–distortion loss** that a learned codec minimizes, and connect it to Chapter 21.
- Distinguish the four great generative families — **autoregressive**, **VAE**, **GAN**, **diffusion** — as four ways to learn a source distribution $p(x)$.
- Run a tiny neural autoencoder in NumPy and watch its bottleneck throw bits away.

1.1 From a hand-built transform to a learned one

Let us anchor everything in a picture we already own. Back in Chapter 42 the JPEG pipeline was a conveyor belt: an image went in, a **transform** (the DCT) rearranged its energy, a **quantizer** threw away the parts the eye won’t miss (the only lossy step), and an **entropy coder** wrote the survivors to bits as cheaply as Shannon allows. Three boxes, three human inventions.



all knobs tuned together by gradient descent

Figure 1: The classical codec (top) versus the learned codec (bottom). Same three jobs — transform, quantize, entropy-code — but in the learned version each box is a network whose knobs are tuned by gradient descent, all at once, against the file size itself.

In the learned version, the DCT box becomes a small network we will call the **encoder** g_a ; the quantizer stays (rounding is still the lossy step); the entropy coder is driven by a **learned probability model**. The radical part is the word **learned**. Nobody writes down g_a . It starts as random numbers and is **shaped** by showing the system millions of images and nudging every knob, slightly, in whatever direction makes the output file a little smaller and the reconstruction a little better. Do that a few hundred million times and the random numbers settle into a transform that — astonishingly — beats the DCT a human spent a career perfecting.

To believe that this is even **possible**, we need three ingredients: (1) a **neuron**, the atom that does the nudging; (2) **gradient descent**, the nudging procedure; and (3) the **autoencoder**, the particular wiring that turns nudging into compression. We build them in order, owing nothing to prior knowledge.

1.2 The neuron: a weighted sum with a squash

Forget biology. A neuron, in the sense that matters here, is a tiny mathematical function. It takes several numbers in and produces one number out. Here is the whole thing.

Suppose the inputs are three numbers, x_1, x_2, x_3 . The neuron keeps a private list of **weights**, one per input — call them w_1, w_2, w_3 — and one extra number b called the **bias**. It computes a weighted sum, adds the bias, and passes the result through a **squashing function** f :

$$y = f(w_1x_1 + w_2x_2 + w_3x_3 + b).$$

That is the entire neuron. The weights say how much each input matters (a big positive weight means “this input pushes the output up,” a negative weight “pushes it down,” a near-zero weight “ignore this input”). The bias shifts the whole thing up or down. The squashing function f — we meet three of them in a moment — is the one nonlinear flourish that gives a neuron its power.

Σ GO FURTHER · THE MATHS — The weighted sum as a dot product *skip if familiar*

We met the **dot product** in Chapter 12. Bundle the inputs into a vector $\mathbf{x} = (x_1, x_2, x_3)$ and the weights into $\mathbf{w} = (w_1, w_2, w_3)$. Then the weighted sum is exactly the dot product plus the bias:

$$w_1x_1 + w_2x_2 + w_3x_3 + b = \mathbf{w} \cdot \mathbf{x} + b.$$

Why care? Because a **whole layer** of neurons — say 100 of them, each with its own weight vector — is just a **matrix** W (one row per neuron) times the input vector, plus a bias vector $\mathbf{b}$. A layer’s output is $f(W\mathbf{x} + \mathbf{b})$, with f applied to each entry. This is why neural networks live or die by fast matrix multiplication, and why GPUs (built to multiply matrices) run them. Everything is the linear algebra of Chapter 12, sprinkled with one nonlinear function.

Why the squash? If we left f out — if a neuron were just $w \cdot x + b$ — then stacking neurons would buy us nothing. A linear function of a linear function is still linear (Chapter 12): a thousand layers without a squash collapse into a single matrix, able to draw only straight lines and flat planes. Real data — the boundary between “cat” and “dog,” the texture of grass, the statistics of a photo — is gloriously **curved**. The nonlinear squash is what lets stacked neurons bend. Three squashes you will meet everywhere:

- **Sigmoid**, $\sigma(z) = 1/(1 + e^{-z})$, gently squashes any number into the range $(0, 1)$ — an S-curve, near 0 for very negative inputs, near 1 for very positive ones. It was the classic choice and still appears wherever an output must look like a probability.
- **ReLU**, the “rectified linear unit,” $\text{ReLU}(z) = \max(0, z)$ — pass positive numbers straight through, clamp negatives to zero. Absurdly simple, and it powers most modern networks because it is fast and its slope never vanishes for positive inputs.
- **Tanh**, the hyperbolic tangent, an S-curve like sigmoid but ranging over $(-1, 1)$ and centered at zero.

Σ GO FURTHER · THE MATHS — Euler's number e and the exponential skip if familiar

The sigmoid uses $e \approx 2.71828$, **Euler's number**, the natural base of exponential growth from Chapter 7. The function e^z doubles-and-redoubles smoothly; e^{-z} shrinks smoothly toward zero as z grows. So in $\sigma(z) = 1/(1 + e^{-z})$: when z is large and positive, $e^{-z} \approx 0$, so $\sigma \approx 1/1 = 1$; when z is large and negative, e^{-z} is huge, so $\sigma \approx 1/\text{huge} \approx 0$; when $z = 0$, $e^0 = 1$ and $\sigma = 1/2$. A smooth switch from 0 to 1, hinged at the origin. The one fact to carry forward: e^z is the function that is **its own slope** (its derivative equals itself), which is exactly why calculus loves it and why it makes the downhill-rolling of the next section easy.

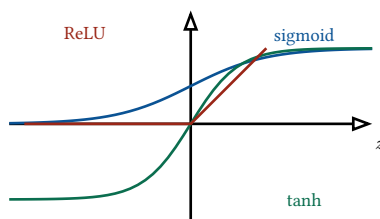


Figure 2: Three squashing functions. Sigmoid and tanh are smooth S-curves; ReLU is a hinge. The squash is the single nonlinear ingredient that lets stacked neurons approximate curved functions.

KEY IDEA

A neuron is **weighted sum** → **add bias** → **squash**. A layer is a **matrix multiply** → **add bias vector** → **squash, applied elementwise**. A network is **layers stacked one after another**. There is no other machinery. The intelligence is entirely in the **values** of the weights — and those values are not designed, they are **learned**.

1.2.1 Why stacking helps: the universal approximation theorem

Here is the result that justifies the whole enterprise. Imagine the true relationship you want to capture — say, the function mapping a raw image to its most compressible representation — is some impossibly complicated curve in high-dimensional space. Can a stack of these simple neurons reproduce it?

Theorem (Universal Approximation, informal). *A network with a single hidden layer of squashing neurons can approximate **any** continuous function on a bounded region to **any** desired accuracy, provided the layer is wide enough.*

This was proved independently by George Cybenko (December 1989) for sigmoid neurons and by Ken-Ichi Funahashi (May 1989), and it is the theoretical license for everything that follows: there is no curve, no transform, no decision boundary so baroque that a big enough network cannot mimic it. We will not prove it in full — it leans on real analysis beyond our ninth-grade contract — but the intuition is graspable and worth a sketch.

Proof. (Intuition, in one dimension.) Take a single sigmoid $\sigma(w(x - c))$. With a large weight w it becomes a sharp **step** that switches from 0 to 1 right at $x = c$. Subtract two such steps placed close together and you get a thin **bump** — a spike of height 1 over a narrow interval. Now you hold a paintbrush: by adding many bumps of different heights and positions (each bump is one pair of hidden neurons, scaled by an output weight), you can paint any continuous curve as a row of thin rectangles — exactly as Chapter 11 approximated the area under a curve by thin rectangles. More neurons, thinner bumps, better fit. The same argument generalizes to many dimensions. $\square$ $\square$

Aside. The theorem is an **existence** result: it promises a good network **exists**, not that gradient descent will **find** it, nor that it will be small. In practice **depth** (many layers) is dramatically more parameter-efficient than the one fat layer the theorem uses — deep networks reuse features, building edges into textures into objects. The 2012–2020 deep-learning revolution was largely the discovery of how to actually **train** deep stacks, not a new theorem about what they could represent.

Checkpoint. Why can't we drop the squashing function f and just stack weighted sums?¹

1.3 Learning by rolling downhill: gradient descent

We have a network full of knobs (the weights and biases) and a promise that **some** setting of those knobs does the job. How do we **find** that setting among the millions of numbers? We do not search blindly. We roll downhill.

Picture the network's total error — how badly it does on the training data — as a **landscape**. Every possible setting of the weights is a point on the ground; the **height** at

¹**Checkpoint answer.** Because a composition of linear functions is linear: $W_2(W_1x) = (W_2W_1)x$ is one matrix. Without a nonlinearity the deepest network collapses to a single linear map, able to represent only straight lines and flat planes — useless for the curved structure of real data.

that point is the error. A perfect network sits at the bottom of a valley. We start somewhere random (high up, error large) and want to reach a valley floor. The trick comes straight from Chapter 11: at any point, the **slope** tells you which way is downhill. Take a small step in the steepest downhill direction. Repeat. This is **gradient descent**, the engine of all learning.

Σ GO FURTHER · THE MATHS – The gradient – a slope in many skip if familiar directions

In Chapter 11 the **derivative** of a one-input function was its slope: $(dL)/(dw)$ tells you how much the error L changes if you nudge the single knob w . With millions of knobs $w_1, \dots, w_n$, each has its **own** slope – the **partial derivative** $(\partial L)/(\partial w_i)$, meaning “the slope in the w_i direction, holding all other knobs fixed.” Stack these slopes into a vector:

$$\nabla L = \left(\frac{\partial L}{\partial w_1}, \frac{\partial L}{\partial w_2}, \dots, \frac{\partial L}{\partial w_n} \right).$$

This vector is the **gradient** (the upside-down triangle ∇ is read “nabla” or “del”). Its single magical property: it points in the direction of **steepest increase** of L . So $-\nabla L$ points **steepest downhill**. To shrink the error, step a little in the direction $-\nabla L$:

$$w_i \leftarrow w_i - \eta \cdot \frac{\partial L}{\partial w_i} \text{ for every knob } i.$$

The small number η (Greek “eta”) is the **learning rate**, the size of each step. Too big and you leap over the valley and bounce; too small and you crawl. That single update rule, applied over and over, is the whole of training.

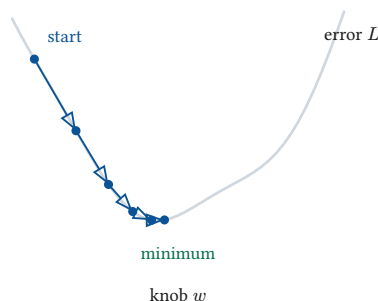


Figure 3: Gradient descent on an error landscape. From a random start (high error), each step follows the downhill slope $-\nabla L$ scaled by the learning rate η , settling toward a valley floor.

The one thing left to explain is **how we compute the gradient** of a deep stack. With millions of knobs buried under many layers, computing each partial derivative from scratch would be hopeless. The rescue is an old idea with a famous 1986 paper: **backpropagation**.

1.3.1 Backpropagation: the chain rule, run backward

The error depends on the final layer’s output, which depends on the layer before, which depends on the layer before that, all the way back to the first weights. To find how a

deep-down weight affects the error, you must trace the influence **through** every layer in between. The tool for “the slope of a function of a function” is the **chain rule** from Chapter 11.

Σ GO FURTHER · THE MATHS – The chain rule

skip if familiar

If z depends on y and y depends on x , then a nudge in x ripples through y to z , and the slopes **multiply**:

$$\frac{dz}{dx} = \frac{dz}{dy} \cdot \frac{dy}{dx}.$$

Concretely, if $y = 3x$ (so $(dy)/(dx) = 3$) and $z = y^2$ (so $(dz)/(dy) = 2y$), then $(dz)/(dx) = 2y \cdot 3 = 6y = 18x$. Influence is **transmitted** by multiplying the local slopes along the path. A neural network is a long chain $x \rightarrow \text{layer}_1 \rightarrow \text{layer}_2 \rightarrow \dots \rightarrow \text{error}$, so the slope of the error with respect to any early weight is a product of local slopes, one per layer it must pass through.

Backpropagation (Rumelhart, Hinton, and Williams popularized it in a celebrated 1986 *Nature* paper, though the core idea dates to Linnainmaa in 1970 and Werbos in 1974) is simply the chain rule applied **cleverly**: instead of recomputing the long product for every weight, you compute the error’s slope at the output and **propagate it backward**, layer by layer, reusing the partial products. Each layer receives “how much the error blames my output” from the layer above, multiplies by its own local slope to get “how much the error blames my inputs,” and passes that down. One forward pass to compute the output, one backward pass to compute every gradient. The cost is roughly **two** network evaluations no matter how many weights — which is why training billion-parameter models is even thinkable.

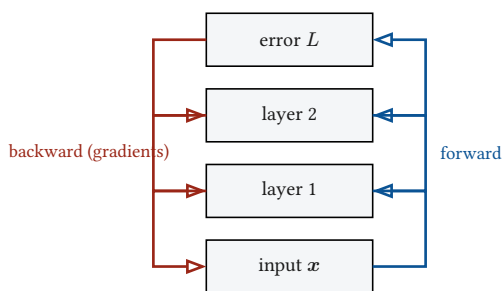


Figure 4: One training step. Forward pass (blue): data flows up through the layers to produce a prediction and an error. Backward pass (orange): the error’s blame flows back down, the chain rule handing each layer its gradient, and every weight steps downhill.

HISTORICAL NOTE

The 1986 paper, “Learning representations by back-propagating errors,” is one of the most cited in all of science, yet backpropagation was independently discovered several

times — Seppo Linnainmaa described reverse-mode automatic differentiation in his 1970 master’s thesis, and Paul Werbos proposed using it to train networks in his 1974 PhD thesis. Geoffrey Hinton, a co-author of the 1986 paper, would share the 2024 Nobel Prize in Physics for foundational work on neural networks — a sign of how thoroughly the once-fringe idea conquered the mainstream.

PITFALL

The error landscape of a real network is **not** a single clean bowl — it is a vast, lumpy terrain with many valleys (**local minima**), flat plains (**saddle points**), and ravines. Gradient descent can get stuck or crawl. The practical fixes you will hear named — **stochastic** gradient descent (estimate the slope from a small random **batch** of examples, not the whole dataset, so each step is cheap and noisy enough to escape shallow traps), **momentum**, and the **Adam** optimizer — are all refinements of the same downhill idea. The headline never changes: **compute the gradient, step downhill, repeat.**

Checkpoint. Backpropagation computes the gradient in time proportional to roughly how many forward passes, regardless of the number of weights?²

Gradient descent (with backpropagation) 1847 / 1986

Authors	Cauchy (descent); Rumelhart, Hinton, Williams (backprop, popularized)
Aim	Find weights that minimize a loss function, by repeatedly stepping in the steepest downhill direction.
Complexity	Per step: two network evaluations (one forward, one backward), independent of weight count.
Strengths	Scales to billions of parameters; needs only first derivatives; trivially parallel on GPUs.
Weaknesses	Finds <i>a</i> minimum, not the global one; sensitive to learning rate; slow on ill-conditioned landscapes.
Superseded by	Refined, not replaced, by SGD, momentum, RMSProp, and Adam — all the same downhill idea.

²**Checkpoint answer.** About two — one forward pass to compute the output and error, one backward pass to compute the gradient for **every** weight at once. That constant-factor cost (independent of parameter count) is what makes training enormous models feasible.

Initialize the weights randomly. Repeat for many steps: (1) **forward** — push a batch of training examples through the network to get predictions and a loss L ; (2) **backward** — run backpropagation to get ∇L , the slope with respect to every weight; (3) **update** — for each weight, $w \leftarrow w - \eta \cdot (\partial L)/(\partial w)$. Stop when L stops dropping. The discovered weights **are** the trained model.

1.4 The autoencoder: a bottleneck that is a compressor

Now we wire neurons into the shape that does compression. An **autoencoder** is two networks glued back to back, trained to copy their input to their output through a deliberately **narrow** middle.

The first network, the **encoder** g_a , reads the input x (say an image, flattened into a long vector of pixel values) and squeezes it down through shrinking layers to a small vector $y = g_a(x)$, the **latent code** or **bottleneck**. The second network, the **decoder** g_s , reads y and expands it back up to a reconstruction $\hat{x} = g_s(y)$ the same size as the input. The training goal is simply: **make $\hat{x}$ look like x** . The loss is the **reconstruction error**, typically the mean squared difference between the original and the copy,

$$D = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{x}_i)^2,$$

averaged over the pixels. Gradient descent tunes both networks together to minimize it.

Σ GO FURTHER · THE MATHS — Mean squared error

skip if familiar

Mean squared error (MSE) measures how far two lists of numbers are apart. For each position i , take the difference $x_i - \hat{x}_i$, **square** it (so positive and negative errors both count, and big errors count disproportionately), then **average** over all N positions. $\text{MSE} = 0$ means a perfect copy; larger MSE means a worse one. In Chapter 21 we called the average reconstruction error the **distortion** D — MSE is the most common choice of D , which is exactly why we reuse the letter here. (We met squaring and averaging in Chapter 10's **variance**, which is itself an MSE around the mean.)

Here is the crucial twist. If the bottleneck y were **as big as** the input, the autoencoder could cheat: the encoder copies the input verbatim, the decoder copies it back, reconstruction perfect, nothing learned. The magic is in the word **narrow**. We force y to be **smaller** than x — fewer numbers in the middle than at the ends. Now the network **cannot** pass everything through; it must decide what to keep and what to discard. To minimize reconstruction error under that constraint, it is forced to learn the **structure** of the data — the regularities that let a few numbers stand in for many. That is the definition of a **transform that decorrelates**: it is doing, automatically, the job the DCT did by hand.

Definition — Autoencoder. A pair of networks — an encoder $g_a : x \mapsto y$ and a decoder $g_s : y \mapsto \hat{x}$ — trained jointly to minimize the reconstruction error between x and $\hat{x}$, with the latent y constrained to be a **bottleneck** (lower-dimensional, or otherwise limited). The bottleneck forces the network to learn a compact code for the data.

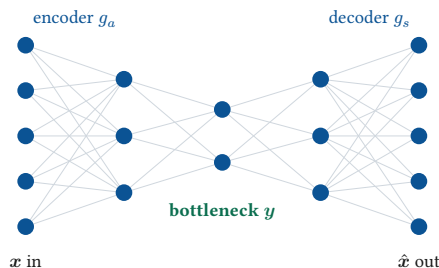


Figure 5: An autoencoder. The encoder funnels the input down to a small latent bottleneck y ; the decoder funnels it back up to a reconstruction $\hat{x}$. The narrow middle forces the network to keep only what matters — which is exactly compression.

KEY IDEA

A narrow autoencoder **is** a lossy compressor. To store an image you run the encoder and keep only the small latent y ; to view it you run the decoder. The compression ratio is the size of x over the size of y — fewer numbers in, fewer bits stored. The reconstruction is imperfect (lossy) precisely because the bottleneck threw information away. Everything left to do — quantizing y , entropy-coding it, and putting the **bitrate** into the loss — turns this sketch into a real codec.

1.4.1 Three problems on the road to a real codec

The bare autoencoder above is a transform with a bottleneck. To become an actual file format it must solve three problems, each of which became a small research milestone.

Problem 1 — the latent is still real numbers. The bottleneck y is a vector of floating-point numbers, and you cannot write an infinitely precise real number to a finite file (Chapter 13). You must **quantize** it — round each number to a finite grid — just as JPEG rounded DCT coefficients (Chapter 42). Rounding is the only lossy step beyond the bottleneck.

Problem 2 — rounding has no slope. Here is the headache that nearly sank the field. Gradient descent needs slopes (Chapter 11). But the rounding function $\hat{y} = \lfloor y \rfloor$ is a **staircase**: flat between integers (slope 0) with vertical jumps at the half-integers (slope undefined). A flat function has zero gradient **everywhere it is defined**, so backpropagation sees no signal — nudging y a little does not change $\lfloor y \rfloor$ at all, so the network is told “rounding doesn’t matter,” which is catastrophically wrong. We cannot train through a staircase.

The fix, due to Ballé and colleagues in 2016–17 and now universal, is the **quantization-as-noise** trick. During **training**, replace the hard rounding by adding a little **uniform random noise**: instead of $\lfloor y \rfloor$, use $y + u$ where u is a random number drawn evenly from $(-\frac{1}{2}, +\frac{1}{2})$. This soft surrogate has two beautiful properties: it is smooth (so gradients flow), and its random spread of $\pm\frac{1}{2}$ **mimics** the error that real rounding introduces. The network trains against a faithful, differentiable stand-in for quantization. At **test** time you switch back to true rounding. We will meet this trick again, in full, in Chapter 57.

Problem 3 — we must count the bits. A bottleneck that is **small** is good, but what we truly pay for is **bits**, and bits depend on the **probabilities** of the latent values (Chapter 19: a symbol of probability p costs $-\log_2 p$ bits). So the codec carries a **learned probability model** $p(\hat{y})$ for its own latents, and the bit cost — the **rate** — is the expected codelength under that model. This is the deep link back to Chapter 23: **the model that predicts the latents well is the model that codes them cheaply**. And, crucially, the rate can be written as a differentiable formula, so we can put it **into the loss** and let gradient descent shrink the file directly.

1.5 The rate–distortion loss: compression as one optimization

Now assemble the pieces into the single objective that defines learned compression. We want a **small file** (low rate R) and a **faithful reconstruction** (low distortion D). These pull in opposite directions — exactly the tension Chapter 21 made precise as the **rate–distortion trade-off**. We balance them with a knob:

$$\mathcal{L} = R + \lambda D = \underbrace{-\log_2 p(\hat{y})}_{\text{bits to store the latent}} + \lambda \underbrace{D(x, \hat{x})}_{\text{reconstruction error}}.$$

Read it slowly. The first term R is the codelength of the quantized latent under the learned model — the **bitrate**. The second term D is the reconstruction distortion, usually MSE. The number λ (Greek “lambda”) is a **Lagrange multiplier**: it sets the exchange rate between bits and quality. Crank λ up and the optimizer cares mostly about quality, spending bits freely (a big, sharp file); crank it down and it cares mostly about size, tolerating blur (a tiny file). Sweep λ and you trace out the whole rate–distortion curve of Chapter 21 — but now the points on that curve are achieved by a **trained network**, not a hand-built codec.

Σ GO FURTHER · THE MATHS — Lagrange multipliers, the one-line version skip if familiar

You want to make D as small as possible, but you also care about keeping R small — two goals at once. The **Lagrange** trick turns “minimize D subject to a budget on R ” into “minimize the single combined quantity $R + \lambda D$ ” for some positive weight λ . Each choice of λ corresponds to one budget; minimizing the combined quantity for that λ lands you at the best D achievable for the matching R — a point on the rate–distortion frontier. We met this balancing-of-two-costs idea in Chapter 21; here it is the entire training objective.

Recall. The **KL divergence** (relative entropy) of Chapter 20, written $D_{\text{KL}}(q \parallel p)$, measures the extra bits you pay for coding data that truly follows q with a code built for a different distribution p . It is never negative, and zero exactly when $q = p$. Keep that one fact handy — the rate term below **is** a KL divergence in disguise.

Theorem (RD loss is the variational-autoencoder objective). *Minimizing the rate–distortion loss $R + \lambda D$ with the quantization-as-noise surrogate is mathematically identical to training a **variational autoencoder** (VAE), in which the rate term plays the role of the **KL divergence** between the (noisy) latent and a prior, and the distortion term plays the role of the reconstruction likelihood.*

Proof. (Sketch.) A VAE (next section) maximizes a quantity called the **evidence lower bound**, which has exactly two terms: a **reconstruction** term rewarding $\hat{x} \approx x$, and a **KL-divergence** term penalizing the latent for straying from a fixed prior $p(y)$. When the encoder’s latent distribution is “the true value plus uniform noise of width 1,” the reconstruction term becomes (up to a constant) the MSE distortion D , and the KL term becomes (up to a constant) the expected codelength $-\log_2 p(\hat{y})$, i.e. the rate R . So “evidence lower bound” = “ $-(R + \lambda D)$ ” up to scaling. Maximizing the one is minimizing the other. Compression **is** variational inference; the bottleneck-with-a-price-tag and the VAE are the same animal wearing two hats. $\square$

This equivalence, pointed out in the very papers that launched learned compression, is one of the most beautiful bridges in the field: the information-theoretic quantity “bits” and the statistical quantity “KL divergence from the prior” are the same number (we proved their kinship in Chapter 20’s relative-entropy / KL-divergence material). It means every advance in generative modeling is, potentially, an advance in compression — which is why the rest of this chapter tours the generative zoo.

SCOREBOARD — where learned codecs sit (still-image, BD-rate vs. the best classical codec; lower file at equal quality is better)

Technique	Bytes	Ratio	Notes
JPEG (1992, DCT)	baseline	—	hand-built; Chapter 42
BPG / HEVC-intra (2014)	$\approx -60\%$	much smaller	hand-built; Chapter 53
Minnen 2018 (learned, context model)	beats BPG	-15.8% vs. prior learned	first learned

Technique	Bytes	Ratio	Notes
			codec to beat BPG on PSNR + MS-SSIM
VVC-intra (2020)	$\approx -25\%$ vs. HEVC	smaller still	hand-built; Chapter 53
JPEG AI (ISO/IEC 6048-1:2025)	up to -28.5% vs. VVC	smallest	first learned in-ter-na-tional stan-dard ; Chapter 57

The scoreboard tells the headline of Volume IV in one glance: the learned curve, born clumsy in 2016, crossed the best hand-built codec by 2018 and by 2025 had become a published international standard beating VVC. Chapter 57 builds the exact architecture (the Ballé hyperprior line) that achieved it.

1.6 Four ways to learn a source distribution

Step back. Chapter 23 taught the deepest idea in this book: **to compress a source optimally is to know its probability distribution $p(x)$** . If you know exactly how likely every possible message is, an arithmetic coder (Chapter 26) spends $-\log_2 p(x)$ bits on message x and you have hit the Shannon floor. Every compressor is, secretly, a model of $p(x)$ — Huffman, PPM, LZ, all of them. The deep-learning revolution gave us **powerful, trainable** models of $p(x)$ for data far too complex to model by hand: images, audio, language. These are the **generative models**, so named because a model of $p(x)$ can also **generate** new samples by drawing from it.

There are four great families, and the single most useful way to keep them straight is to ask: **how does each one represent $p(x)$, and how does each turn into a compressor?** We tour them in the order they were invented.

1.6.1 Family 1 – Autoregressive models: predict the next piece

The most direct route reuses an idea we have leaned on since Chapter 23. Break the data into an ordered sequence of pieces – pixels left-to-right, top-to-bottom, or the tokens of a sentence – and model the probability of each piece **given all the pieces before it**. The **chain rule of probability** (Chapter 9) then assembles the whole:

$$p(x_1, x_2, \dots, x_n) = p(x_1) \cdot p(x_2 \mid x_1) \cdot p(x_3 \mid x_1, x_2) \dots p(x_n \mid x_1, \dots, x_{n-1}).$$

A neural network supplies each conditional $p(x_k \mid \text{everything before})$: feed it the context, it outputs a probability for the next piece. Train it to predict well (minimize the surprise of the real next piece – which is exactly the cross-entropy of Chapter 23), and you have a model that, paired with an arithmetic coder, compresses to within a whisker of its own cross-entropy. This is the **purest** embodiment of “compression = prediction,” and it is precisely how PNG predicts pixels, how PPM predicts characters, and – at gigantic scale – how a large language model predicts words. Chapter 62 will build an LLM-driven compressor on exactly this principle.

Autoregressive model		1980s–; PixelRNN/PixelCNN 2016; GPT-style 2018–
Authors	van den Oord et al. (PixelRNN/CNN); many for language	
Aim	Model $p(x)$ as a product of next-piece conditionals $p(x_k x_{(<k)})$, each predicted by a network.	
Complexity	Decoding is *serial*: piece k needs piece $k-1$; n pieces means n network passes.	
Strengths	Exact likelihood (so exact, optimal coding); conceptually simplest; state-of-the-art prediction quality.	
Weaknesses	Serial decode is slow — the field's central wound for image/video codecs.	
Superseded by	Complemented (not replaced) by VAEs, GANs, diffusion; still the backbone of LLM compression (Chapter 62).	

In learned image compression the autoregressive idea reappears as the **context model**: each latent element is predicted from its already-decoded neighbors, squeezing out the last redundancy. Minnen, Ballé, and Toderici (NeurIPS 2018) combined this with a hyperprior to become the first learned codec to beat BPG — at the cost of slow serial decoding (Chapter 57).

1.6.2 Family 2 — Variational autoencoders (VAEs): a probabilistic bottleneck

We have already met the VAE in disguise — it is the autoencoder whose loss is the rate–distortion loss. The **variational** twist makes the bottleneck **probabilistic**: instead of the encoder emitting one fixed latent vector, it emits a small **cloud** (a mean and a spread) of plausible latents, and a **prior** says what latents are expected in general. Training maximizes the **evidence lower bound** — reconstruct well, while keeping the latent cloud close to the prior. As we proved above, that is term-for-term the rate–distortion loss, with the “close to the prior” penalty **being** the bitrate.

Variational Autoencoder (VAE)

2013

Authors	Diederik Kingma, Max Welling
Aim	Learn a probabilistic latent code: encode x to a distribution over latents, decode samples back, regularized toward a prior.
Complexity	One forward + one backward pass; fast, parallel encode and decode.
Strengths	Principled probabilistic objective; the exact framework of learned compression; fast non-serial decoding.
Weaknesses	MSE-trained VAEs produce “blurry” samples at low rates (the curse of averaging).
Superseded by	Extended by hyperpriors (Ch 57), discretized as VQ-VAE, and combined with GANs/diffusion for sharpness.

The VAE is the mathematical home of learned compression: Ballé’s hyperprior codec (Chapter 57) is a two-level VAE, and the discrete **VQ-VAE** (van den Oord, 2017) — which snaps latents to entries of a learned codebook — became the tokenizer underneath many image and audio codecs.

The VAE’s flaw is instructive and sets up the next two families. Trained on MSE, at low bitrate it produces **blurry** reconstructions. Why? Because when many fine details are equally plausible (the exact arrangement of blades of grass), the choice that minimizes **average squared error** is to draw their **average** — a smear. MSE rewards safe blur over risky detail. The eye, however, **hates** blur. To make low-rate reconstructions look **real** rather than **average**, we need a different yardstick than MSE — one that rewards matching the **distribution** of natural images, not the pixel-wise average. That yardstick was named **perception**, and Chapter 58 is devoted to it. The next two families are how we optimize for it.

1.6.3 Family 3 — Generative adversarial networks (GANs): a forger and a detective

A GAN learns $p(x)$ through a duel. Two networks train against each other. The **generator** tries to produce fake samples (here: reconstructions) that look like real data. The **discriminator** (a detective) is shown a mix of real images and the generator’s fakes and trained to tell them apart. The generator is rewarded for **fooling** the detective. As they spar, the forger is driven to produce images statistically indistinguishable from real ones — which means sharp, plausible texture, exactly what MSE refused to draw.

Σ GO FURTHER · THE MATHS — A minimax game

skip if familiar

The two networks optimize **opposite** goals on the same quantity, a **minimax** game — one side pushing a number **up** while the other pushes it **down**. The discriminator D tries to **maximize** its accuracy at spotting fakes; the generator G tries to **minimize** that same accuracy. Written compactly, they jointly seek $\min_G \max_D V(G, D)$ where V measures the detective’s success. At the ideal balance point the fakes are so good the detective can only guess — its accuracy stuck at 50% — and the generator’s output distribution has matched the real data distribution $p(x)$. Training is a tug-of-war that, when it works, ends in a perfect draw.

In a **GAN codec**, the decoder is the generator: it must reconstruct a believable image from the transmitted code, and an adversarial detective pushes it toward realism. The landmark is **HiFiC** (Mentzer et al., NeurIPS 2020), which at one bitrate was preferred by human viewers over MSE codecs spending **twice** the bits. The honest catch — explored in Chapter 58 — is that these reconstructions are **generated**, not faithful: the network **hallucinates** texture that is statistically right but may not be the texture that was actually there. Plausible is not the same as true.

Generative Adversarial Network (GAN)		2014
Authors	Ian Goodfellow et al.	
Aim	Learn $p(x)$ implicitly by training a generator to fool a discriminator that learns to spot fakes.	
Complexity	Fast single-pass generation; *training* is notoriously unstable.	
Strengths	Razor-sharp, realistic samples; excellent for low-bitrate perceptual compression (HiFiC).	
Weaknesses	Unstable training (mode collapse); no exact likelihood; reconstructions can hallucinate.	
Superseded by	Largely overtaken by diffusion models for perceptual quality after 2022.	

GANs do not give a probability $p(x)$ you can read off (the model is **implicit**), so a GAN cannot directly drive an arithmetic coder. In a codec it is bolted onto an autoencoder: the autoencoder transmits the code and counts the bits, while the GAN loss reshapes the **decoder** to render realistic detail. See HiFiC, Chapter 58.

1.6.4 Family 4 — Diffusion models: sculpting an image out of noise

The newest and now-dominant family generates by **reversing a destruction process**. Take a clean image and add a tiny bit of random noise; repeat hundreds of times until nothing is left but pure static. That **forward** process is trivial. The magic is training a network to run it **backward** — to take noisy static and, step by step, **denoise** it back into a clean image, each step removing a little noise and adding a little structure. Once trained, you start from fresh random static and let the network sculpt an image out of it. Because each denoising step is a small, learnable nudge, diffusion models are remarkably stable to train (unlike GANs) and produce the sharpest, most realistic samples yet.

Diffusion model (DDPM)

2015 / 2020

Authors	Sohl-Dickstein et al. (2015); Ho, Jain, Abbeel (DDPM, 2020)
Aim	Learn $p(x)$ by training a network to reverse a gradual noising process, denoising static into data.
Complexity	Generation is *multi-step* (tens to thousands of passes) — historically very slow.
Strengths	State-of-the-art realism; stable training; powers ultra-low-bitrate generative codecs.
Weaknesses	Slow multi-step sampling (being fixed by one-step distillation); like GANs, can hallucinate.
Superseded by	Current frontier; 2025 one-step distilled variants make it real-time.

In a **diffusion codec** the decoder is a diffusion model **conditioned** on the transmitted code: it denoises static toward the image the code describes. By 2025 codecs such as **StableCodec** (ICCV 2025) and **OneDC** (NeurIPS 2025) piggybacked on huge pre-trained text-to-image diffusion models to reach **ultra-low** bitrates below 0.05 bits per pixel — transmitting a semantic sketch and letting the generative prior fill in the rest. Their old weakness, glacial multi-step denoising, is being cured by **one-step distillation**: OneDC reports roughly $20 \times$ faster decoding than multi-step diffusion codecs at comparable quality. Chapter 58 dissects them.

Myth. A generative codec “remembers” the original image perfectly and just stores it cleverly.

Reality. A low-bitrate generative codec **reconstructs a plausible image**, not the true one. It transmits a compact description and lets a generative prior **invent** detail that is statistically consistent but possibly never present in the original — a real hazard for medical, legal, or forensic images, where an invented-but-plausible detail is worse than honest blur. We weigh this trade-off carefully in Chapter 58.

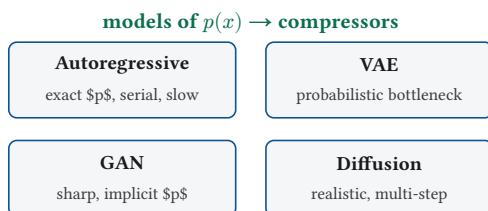


Figure 6: The four generative families as four ways to represent the source distribution $p(x)$. All four can be turned into compressors, each with a different speed/quality/faithfulness profile.

1.7 A neuron, a layer, and a network in Python

Enough abstraction — let us build the machinery in code, with nothing but NumPy, so the boxes above stop being metaphors. We use NumPy because neural networks **are** matrix arithmetic, and NumPy makes matrices a one-liner.

PY GO FURTHER · PYTHON — NumPy arrays and matrix multiply^{skip if fluent}

NumPy is Python’s array library. An **array** is a grid of numbers — a vector, a matrix, or higher. You create one from a list, and arithmetic works **elementwise** (the whole grid at once), which is both fast and exactly how a layer of neurons works.

```
import numpy as np

x = np.array([1.0, 2.0, 3.0])          # a length-3 vector
W = np.array([[0.5, 0.0, -1.0],        # a 2x3 matrix: 2
               [1.0, 1.0, 0.0]])        #                3 inputs
neurons,                                #
each
b = np.array([0.1, -0.2])              # one bias per neuron
print(W @ x + b)                       # @ is matrix multiply -> [-2.4, 2.8]
```

The `@` operator is matrix multiplication: row i of W is dotted with x , giving neuron i ’s weighted sum. Add the bias vector b and you have the pre-squash output of a whole layer in one line — the $f(Wx + b)$ from the maths box, minus the squash.

PY GO FURTHER · PYTHON — Defining functions with def, and type hints *skip if fluent*

We have used Python functions since Chapter 16. A quick refresh: `def name(arg: type) -> rettype:` declares a function; the `: type` and `-> rettype` are **type hints** (Chapter 15) — documentation the reader (and tools) can check, ignored at run time. NumPy arrays are typed `np.ndarray`.

```
def relu(z: np.ndarray) -> np.ndarray:
    return np.maximum(0.0, z)      # elementwise max with 0
```

`np.maximum(0.0, z)` compares every entry of `z` against 0 and keeps the larger — the ReLU squash, applied to a whole array at once.

Here is a complete, runnable feed-forward network: a stack of **(matrix multiply, add bias, squash)** layers. It is the encoder or decoder of an autoencoder, in eight lines.

```
import numpy as np

def sigmoid(z: np.ndarray) -> np.ndarray:
    return 1.0 / (1.0 + np.exp(-z))      # the S-curve squash

def forward(x: np.ndarray, layers: list[tuple[np.ndarray,
np.ndarray]],
            squash) -> np.ndarray:
    """Run x through each (weight, bias) layer with a squash
    between."""
    a = x
    for (W, b) in layers:                  # one layer at a time
        a = squash(W @ a + b)             # matrix-multiply,
    bias, squash
    return a

# a tiny 4 -> 2 -> 4 autoencoder, random weights (untrained)
rng = np.random.default_rng(0)
enc = [(rng.normal(size=(2, 4)), rng.normal(size=2))]  # 4 in -> 2
latent
dec = [(rng.normal(size=(4, 2)), rng.normal(size=4))]  # 2 latent -
> 4 out

x = np.array([0.9, 0.1, 0.8, 0.2])
y = forward(x, enc, sigmoid)              # the 2-number bottleneck
(latent)
xhat = forward(y, dec, sigmoid)            # the 4-number reconstruction
```

```
print("latent  :", y.round(3))
print("recon   :", xhat.round(3))
```

Run it and you get a 4-number input squeezed to a 2-number latent and expanded back to 4 numbers. With **random** weights the reconstruction is garbage — the network has not learned anything. The whole point of training is to find weights that make `xhat` resemble `x`. That requires gradient descent, which we now wire into the `tinyzip` project as the chapter’s build step.

BUILD `tinyzip` · Step 21 · A learned-transform sketch (`llmzip`-adjacent, NumPy)

TINYZIP’s Phase D opens with a **concept-level** learned transform. We add a tiny module that **trains** a 2-pixel-wide autoencoder bottleneck on a stream of correlated pairs and shows it discovering, by gradient descent alone, the same “keep the average, drop the difference” structure a hand-built transform would impose — and watch the bottleneck **throw bits away**. This is a teaching sketch, not a production codec; the real Ballé architecture arrives in Chapter 57. We keep the names ready for `tinyzip/llmzip.py` (the neural module Chapter 62 completes).

```
# tinyzip/learned.py  -- concept-level learned transform (Step
21)
import numpy as np

def sigmoid(z): return 1.0 / (1.0 + np.exp(-z))

def train_autoencoder(data: np.ndarray, latent: int = 1,
                      epochs: int = 4000, eta: float = 0.5,
                      seed: int = 0) -> dict[str, np.ndarray]:
    """Train a linear d->latent->d autoencoder by gradient
    descent.

    data: shape (N, d) rows are examples. Returns the learned
    weights.
    Linear (no squash) so the gradients are short and exact --
    this is
    the simplest possible learned transform, a learned PCA.
    """
    rng = np.random.default_rng(seed)
    N, d = data.shape
    We = rng.normal(scale=0.5, size=(latent, d))  # encoder
    weights
    Wd = rng.normal(scale=0.5, size=(d, latent))  # decoder
```

```

weights
    for _ in range(epochs):
        y = data @ We.T          # (N, latent) the
bottleneck
        xhat = y @ Wd.T          # (N, d)
reconstruction
        err = xhat - data        # (N, d)      what we
got wrong
        # backprop the MSE through the two linear layers (chain
rule):
        gWd = err.T @ y / N      # slope wrt decoder
weights
        gWe = (err @ Wd).T @ data / N # slope wrt encoder
weights
        Wd -= eta * gWd          # step downhill
        We -= eta * gWe
    return {"We": We, "Wd": Wd}

def encode(x: np.ndarray, w: dict[str, np.ndarray]) ->
np.ndarray:
    return x @ w["We"].T          # x -> latent

def decode(y: np.ndarray, w: dict[str, np.ndarray]) ->
np.ndarray:
    return y @ w["Wd"].T          # latent ->
reconstruction

if __name__ == "__main__":      # tiny self-test
    rng = np.random.default_rng(1)
    # correlated pairs: each pixel pair is (a, a+small noise)
    a = rng.normal(size=(2000, 1))
    data = np.hstack([a, a + 0.05 * rng.normal(size=(2000, 1))])
    w = train_autoencoder(data, latent=1)
    y = encode(data, w)
    xhat = decode(y, w)
    mse = float(np.mean((xhat - data) ** 2))
    print(f"latent dim 1 (half the data), MSE = {mse:.5f}")
    # the learned encoder row ~ proportional to (1, 1): it kept
the
    # *sum/average* of the pair and discarded the tiny
*difference* --
    # exactly the decorrelating move a hand-built transform
makes.
    print("encoder row:", (w["We"][0] / np.abs(w["We"]
[0]).max()).round(2))

```

Running it prints a small MSE (about 0.01 — almost all of it the unrecoverable per-pixel noise) even though the single latent number stores **half** as many values as the input — because the two pixels were nearly identical, so one number (their shared value) reconstructs both almost perfectly. The printed encoder row comes out close to (1, 1): gradient descent **discovered**, with no human hint, that the smart move is to keep the **average** of the correlated pair and throw the **difference** away. That is the entire idea of a decorrelating transform (Chapter 38’s Karhunen–Loève transform, the ideal decorrelator, and its statistical cousin PCA), grown from random numbers by rolling downhill. Scale this from 2 pixels to whole images, swap the linear layers for deep nonlinear ones, add quantization and a learned entropy model, and you have the architecture of Chapter 57.

Python. `data @ We.T` multiplies the whole batch of examples by the encoder in one stroke (`@` is matrix multiply, `.T` transposes). The four lines computing `gWd` and `gWe` are backpropagation for this two-layer net: the chain rule, written out, multiplying local slopes layer by layer.

KEY IDEA

The tinyzip sketch makes the abstract concrete: a learned transform is just weights, and gradient descent **finds** the weights that decorrelate the data — the same job the DCT and PCA do by hand, now done automatically against a reconstruction loss. Add the rate term to the loss, and gradient descent will also learn to make the latent **cheap to entropy-code**, not merely accurate. That is the leap from “autoencoder” to “codec.”

1.8 Where learned compression stands, mid-2026

We close with an honest ledger, because hype is a poor guide to deployment. Learned codecs **win decisively** on perceptual quality at **low bitrate**: when bits are scarce, a generative decoder that reconstructs plausible texture beats any block-transform codec that can only blur. They now **match or exceed** the best hand-built standards on standard metrics — Minnen 2018 beat BPG; JPEG AI (ISO/IEC 6048-1:2025) reports up to about 28.5% smaller files than VVC-intra at equal quality. They are **flexible**: one architecture retargets to a new domain by retraining, and JPEG AI even codes a latent that machines can run object-detection on **without fully decoding the image** — compression serving machine vision, not just human eyes.

They **lose**, for now, on three fronts. **Speed**: context-model decoders can be tens of times slower than classical ones, and they lack the precious encode/decode asymmetry that makes streaming work — HEVC and AV1 decode hundreds of times faster than they encode, while neural codecs are roughly symmetric and slow on **both** ends (the 2025 one-step-distillation push is the direct response). **Hardware**: they demand GPUs or neural accelerators absent from most playback chips, raising real energy questions. **Reproducibility**: a bit-exact match between an encoder and a decoder on different machines requires identical floating-point arithmetic, a notorious headache that classical

integer codecs simply do not have. The bellwether is standardization, and there the needle has moved: JPEG AI is now a published international standard. Whether neural codecs displace AV1 and VVC in the wild hinges less on rate–distortion curves — that battle is being won — and more on silicon, power budgets, and the slow grind of deployment.

Chapter takeaways

- A **neuron** is a weighted sum plus a bias passed through a nonlinear **squash**; a **layer** is a matrix multiply plus a bias vector plus a squash; a **network** is layers stacked. All the intelligence lives in the weight **values**.
- The **universal approximation theorem** says a wide enough network can mimic any continuous function — the license to **learn** a transform instead of designing one.
- **Gradient descent** finds good weights by repeatedly stepping downhill on the error landscape; **backpropagation** (the chain rule, run backward) computes every gradient in about two network passes.
- An **autoencoder** squeezes data through a narrow **bottleneck** and back; the bottleneck forces it to learn a compact code — a narrow autoencoder **is** a lossy compressor.
- Real codecs add **quantization** (the lossy step), the **quantization-as-noise** trick (so gradients can flow through rounding), and a **learned entropy model** that counts the bits.
- Learned compression minimizes one **rate–distortion loss** $R + \lambda D$ — provably the **variational-autoencoder** objective — so compression is variational inference and every generative advance is a potential compression advance.
- The four generative families — **autoregressive**, **VAE**, **GAN**, **diffusion** — are four ways to learn the source distribution $p(x)$, each convertible into a compressor with a different speed/quality/faithfulness profile.
- By 2025 a learned codec (JPEG AI, ISO/IEC 6048-1:2025) became the first learning-based **international standard**, beating VVC — but learned codecs still lag on speed, hardware, and reproducibility.

1.9 Exercises

Exercise 56.1 ★

A single neuron has weights $\mathbf{w} = (2, -1, 0.5)$ and bias $b = -1$, and uses the ReLU squash. Compute its output for the input $\mathbf{x} = (1, 3, 4)$. Then compute it for $\mathbf{x} = (0, 0, 0)$.

Exercise 56.2 ★

Explain in one or two sentences why stacking two **linear** layers (no squash) gives no more representational power than a single linear layer. What property of the squash breaks this collapse?

Exercise 56.3 ★★

A weight w currently sits at $w = 3$, and the gradient of the loss with respect to it is $(\partial L)/(\partial w) = +4$ (the loss **increases** as w increases). With learning rate $\eta = 0.1$, give the updated value of w after one gradient-descent step. In which direction did w move, and why is that the right direction to shrink L ?

Exercise 56.4 ★★

In the rate-distortion loss $\mathcal{L} = R + \lambda D$, describe what happens to the trained codec as $\lambda \rightarrow \infty$ and as $\lambda \rightarrow 0$. Which regime gives small files, and which gives faithful reconstructions? Tie your answer to the rate-distortion curve of Chapter 21.

Exercise 56.5 ★★

The chain rule: let $a = 2x$, $b = a^2$, and $L = b + 5$. Use the chain rule to compute $(dL)/(dx)$ as a function of x , then evaluate it at $x = 3$.

Exercise 56.6 ★★

Why does rounding (quantization) break gradient descent, and how does the “quantization-as-noise” trick fix it during training? What is replaced by what, and why does the replacement have the right **spread**?

Exercise 56.7 ★★★

Coding. Modify the tinyzip Step 21 sketch so the data is **triples** of correlated pixels (a, a, a) plus small noise, compressed through a **single** latent number ($d = 3$, `latent = 1`). Before running, predict the rough form of the learned encoder row, then state what compression ratio (input numbers to latent numbers) the bottleneck achieves and why the MSE stays small.

1.10 Further reading

- Diederik Kingma and Max Welling (2013), *Auto-Encoding Variational Bayes* — the VAE: [arXiv:1312.6114](#).
- Ian Goodfellow et al. (2014), *Generative Adversarial Networks*: [arXiv:1406.2661](#).
- Jonathan Ho, Ajay Jain, Pieter Abbeel (2020), *Denoising Diffusion Probabilistic Models*: [arXiv:2006.11239](#).
- David Rumelhart, Geoffrey Hinton, Ronald Williams (1986), *Learning representations by back-propagating errors*, **Nature** 323, 533–536.
- Johannes Ballé, Valero Laparra, Eero Simoncelli (2017), *End-to-End Optimized Image Compression*: [arXiv:1611.01704](#).
- Yibo Yang, Stephan Mandt, Lucas Theis (2023), *An Introduction to Neural Data Compression*: [arXiv:2202.06533](#) — the survey to read next.
- JPEG committee (2025), *JPEG AI becomes an International Standard* (ISO/IEC 6048-1:2025): [jpeg.org press release](#).

COMING UP NEXT

We now own the entire vocabulary — neurons, gradient descent, the autoencoder, the rate–distortion loss, and the four generative families. Chapter 57 spends all of it at once. We build the **real** learned image codec: the Ballé line (2016–2018), with its end-to-end rate–distortion training, the GDN nonlinearity, quantization-as-noise made rigorous, and the **hyperprior** — a second little autoencoder that transmits a description of the entropy model itself, letting the codec adapt its bit-allocation pixel by pixel. That is the architecture that overtook JPEG, BPG, and AVIF, and became JPEG AI. Bring the bottleneck; we are about to make it pay for its bits.

Learned Image Compression: Autoencoders and Hyperpriors

We do not design the transform. We design the objective, and let the gradient find the transform.

— paraphrasing Johannes Ballé, on end-to-end optimized compression

Here is a puzzle that should bother you. For fifty years, every image codec worth the name was built the same way. A human engineer sat down, reasoned about the mathematics of natural pictures, and *hand-designed* a transform to shuffle the pixels into a more compressible shape. For JPEG (Chapter 42) that transform was the discrete cosine transform, an idea borrowed from Fourier and pinned down in 1974. For JPEG 2000 (Chapter 43) it was the wavelet. Then came a quantizer, also hand-tuned, and an entropy coder, also hand-tuned, and the three were bolted together and shipped. The DCT did not *know* anything about photographs of cats or sunsets or circuit boards. It was a fixed mathematical recipe that happened to work reasonably well on the kinds of smooth, locally correlated signals that cameras produce.

So here is the puzzle. We *have* millions of real photographs sitting on disk. We *have*, since Chapter 56, a machine — the neural network — that can look at millions of examples and discover structure no human bothered to write down. Why on earth are we still using a transform that was designed before most photographs were even digital? Why not let the machine look at a mountain of real images and *invent its own transform*, one shaped precisely to the statistics of the pictures we actually send each other — and, while it is at it, invent its own quantizer and its own entropy model too, all three trained *together* so they cooperate instead of merely coexisting?

That question, asked seriously around 2016, set off the most important shift in image compression since JPEG. The answer is a family of codecs that contain no hand-designed transform at all. They are trained, the way a child learns faces, by being shown an enormous number of examples and nudged — millions of tiny nudges — toward producing small files that still look right. By 2018 these *learned codecs* matched the best hand-built engineering. By 2025 the very first international standard built on them, JPEG AI, was beating the reigning video-grade still codec by nearly thirty percent. This chapter is the story of how that happened, and exactly how the machinery works, built from the pieces you already own.

WHERE WE ARE

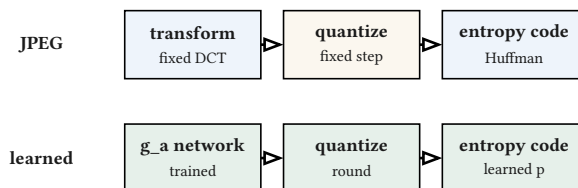
We are standing on a tall stack. From **Chapter 21** we have the **rate–distortion function** $R(D)$: the unavoidable trade between bits spent and distortion suffered, and the Lagrangian way of exploring it by minimizing $D + \lambda R$. From **Chapters 18–19** we have **entropy** H as the true cost in bits of a symbol stream, and the fact that a good **probability model** is what lets an entropy coder (**Chapters 24, 26, 27**) actually reach that cost. From **Chapter 23** we have the hinge of the whole book: **compression is prediction** — the better you predict the next symbol, the fewer bits it costs. From **Chapter 42** we have the **transform** $\rightarrow$ **quantize** $\rightarrow$ **entropy-code** pipeline of JPEG, the skeleton we are about to rebuild out of neural parts. And from **Chapter 56** we have the **autoencoder**: an encoder network that squeezes data through a narrow **bottleneck** and a decoder network that reconstructs it, trained by **gradient descent** to make the reconstruction faithful. This chapter fuses all of that into one trained machine.

By the end of this chapter you will be able to:

- Explain why a learned codec keeps JPEG’s three-stage skeleton but replaces every stage with a trained neural network.
- Write down the **rate–distortion loss** $L = R + \lambda D$ that a learned codec minimizes, and say in plain words what each term costs.
- Explain the **quantization gradient problem** — why rounding breaks training — and the **additive-uniform-noise** trick that repairs it.
- Understand **GDN**, the biologically-inspired nonlinearity Ballé used in place of the everyday activation function.
- Explain a **hyperprior**: a second little autoencoder that transmits side information describing the entropy model itself, and why that beats a fixed prior.
- Trace the **context model** idea — predicting each latent from its decoded neighbours — and the brutal speed cost of doing it serially, plus the 2020 fixes that made it parallel.
- Place the milestone papers (Ballé 2016/2017/2018, Minnen 2018, Cheng 2020, Minnen–Singh 2020) and the JPEG AI standard on a clear timeline, and state honestly where learned codecs win and lose in 2026.

2.1 The same skeleton, new bones

Let us begin by refusing to throw anything away. The learned codec is not a revolution that burns the past; it is a renovation that keeps the load-bearing walls. Look again at the JPEG pipeline from Chapter 42. An image goes in. A **transform** (the DCT) spreads its energy so that most of the information piles up into a few numbers and the rest are near zero. Those transform numbers are **quantized** — rounded to a coarse grid, and this is the *only* lossy step, the one place information is actually thrown away. Finally an **entropy coder** (Huffman, in classic JPEG) writes the quantized numbers to bits, spending fewer bits on the common values and more on the rare ones.



same three jobs — every box now learned from data

Figure 7: The three-stage skeleton, classical and learned, side by side. The boxes are the same; only their *contents* change from fixed recipes to trained networks.

The learned codec keeps all three boxes and rips out their fixed contents. The transform becomes a neural network. The quantizer stays a rounding step (we will see it must, and why that is the hard part). The entropy coder stays a real arithmetic or range coder from Chapter 26 — but the *probability model* that drives it becomes a neural network too. And the masterstroke, the thing that makes the whole idea work, is that all the trainable parts are optimized **together, end to end**, against a single objective that counts both the bits and the damage. Nothing is tuned in isolation. The transform learns to produce numbers that the entropy model finds cheap to code; the entropy model learns the exact statistics the transform produces. They grow up together.

2.1.1 An autoencoder with a price tag

In Chapter 56 you met the **autoencoder**: two networks back to back. The **encoder**, which we will now call g_a (the “analysis transform”), takes the image x and produces a compact bundle of numbers we call the **latent**, $y = g_a(x)$. The **decoder**, g_s (the “synthesis transform”), takes a latent and rebuilds a picture, $\hat{x} = g_s(\hat{y})$. The hat on $\hat{y}$ is a warning that the latent we actually transmit is not exactly y ; it has been quantized first. The hat on $\hat{x}$ is the matching warning that what comes out is not exactly what went in — this is lossy compression, after all.

Σ GO FURTHER · THE MATHS — Function names with subscripts: *skip if familiar*
 g_a and g_s

Do not be spooked by the symbols. g_a and g_s are just *names* for two functions, the way f and g named functions back in Chapter 6. The little letters are mnemonics: a for **analysis** (taking the picture apart into a code) and s for **synthesis** (putting a picture back together from a code). Writing $y = g_a(x)$ means “feed image x into the analysis function and call the result y .” A function here is an enormous one — millions of additions and multiplications arranged in layers — but the *idea* is identical to $y = 2x + 1$: numbers in, numbers out.

What is genuinely new compared with the autoencoder of Chapter 56 is the **price tag**. A plain autoencoder is trained only to reconstruct well: make $\hat{x}$ look like x . But “looks like x ” is only half a compressor. The other half is “and the code is *small*.” A latent of a million floating-point numbers reconstructs beautifully and compresses nothing. So the

learned codec trains against two demands at once, and the tension between them is exactly the **rate–distortion trade-off** from Chapter 21, now baked directly into the training objective:

$$L = R(\hat{y}) + \lambda \cdot D(x, \hat{x}).$$

Read it left to right in plain English. L is the **loss** — the badness score we are trying to push down. $R(\hat{y})$ is the **rate**: how many bits the quantized latent will cost to transmit. $D(x, \hat{x})$ is the **distortion**: how badly the reconstruction differs from the original, usually measured as mean squared error (the average of the squared pixel differences, our old friend from Chapter 21 and the PSNR of Chapter 42). And λ (the Greek letter *lambda*) is the knob from Chapter 21 that sets the exchange rate between bits and quality.

KEY IDEA

A learned image codec is an autoencoder trained to minimize $L = R + \lambda D$ — bits *plus* a penalty for distortion — where **every** ingredient (the analysis transform g_a , the synthesis transform g_s , and the probability model that turns the latent into a bit count R) is a neural network, and all of them are trained *jointly* by gradient descent. The network does not just learn to reconstruct; it learns to produce a latent that is *cheap to entropy-code*.

Σ GO FURTHER · THE MATHS — The Lagrange multiplier λ , as skip if familiar a price

We met the Greek letter λ (*lambda*) in Chapter 21 as the **Lagrange multiplier** that trades two competing quantities against each other. Here it is simply a **price**: the number of units of distortion you are willing to pay to save one bit. The loss $L = R + \lambda D$ adds bits and distortion onto one common scale, with λ as the exchange rate. Tiny example: if $\lambda = 100$, then reducing distortion by 0.01 is “worth” 1 bit of rate, so the optimizer will happily spend a bit to buy that much quality. If $\lambda = 1$, the same quality is worth only 0.01 bits — the optimizer would rather keep the bit. There is nothing mystical here: λ is a dial that converts “how much do I care about quality versus size?” into one number the math can minimize. Each setting of the dial picks one point on the rate–distortion curve of Chapter 21.

Turn the knob and you slide along the rate–distortion curve. Crank λ up and distortion is punished heavily, so the network spends bits freely to keep quality high: big files, gorgeous pictures. Turn λ down and bits become expensive relative to quality, so the network learns to throw detail away: tiny files, softer pictures. Each value of λ trains a *different network* that lives at a different point on the curve. This is the learned echo of JPEG’s quality slider, except that instead of one fixed codec dialed to different quantization steps, you literally train a fresh transform for each operating point — a transform that has *learned* what to keep and what to sacrifice at that exact budget.

Σ GO FURTHER · THE MATHS — Why distortion is usually mean squared error skip if familiar

Mean squared error (MSE) between an original image x and a reconstruction $\hat{x}$ is the average, over all pixels, of the squared difference:

$$D = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{x}_i)^2,$$

where N is the number of pixels and x_i is the i -th pixel value. Squaring does two jobs: it makes every error positive (so over- and under-shoots do not cancel), and it punishes big mistakes far more than small ones (an error of 10 costs 100, an error of 1 costs only 1). Tiny example: original pixels (100, 50), reconstruction (102, 47). The errors are 2 and -3 ; squared, 4 and 9; mean, $\frac{4+9}{2} = 6.5$. Lower is better; zero means perfect. We met MSE behind PSNR in Chapter 42 — PSNR is just MSE turned into a logarithmic decibel scale so that “bigger is better.”

2.2 The rate term: counting bits with a model

The distortion term D is easy: feed the image in, get the reconstruction out, measure the squared difference. The rate term R is the subtle one, and getting it right is most of the art. How do you count, *during training*, how many bits the quantized latent will cost — before you have even built the bitstream?

The answer comes straight from Chapters 18 and 23. Recall the central law: if you have a probability model p that assigns probability $p(s)$ to a symbol s , then an ideal entropy coder spends $-\log_2 p(s)$ bits on it. Common symbols (high p) are cheap; rare symbols (low p) are dear. The expected cost over the whole latent is the **cross-entropy** between the true distribution of latents and the model p — exactly the quantity Chapter 23 told us equals expected code length. So the rate is

$$R = \mathbb{E}[-\log_2 p(\hat{y})],$$

the average, over the elements of the latent, of $-\log_2$ of the probability the model assigns to each one. To make R small the network has two levers: make the latent itself more predictable (so the model can assign high probabilities), and make the model p accurate (so it does not waste bits being surprised). Both levers are trained at once.

Recall. The blackboard-bold $\mathbb{E}[\cdot]$ is the **expectation** (probability-weighted average) of Chapter 10 — “the average cost of a code,” the very lens that chapter taught us to read every code length through. So $R = \mathbb{E}[-\log_2 p(\hat{y})]$ reads “the average number of bits, over all the latent elements, of coding each one.” And $-\log_2 p$ is the **surprisal** of Chapter 18 — the number of bits an ideal coder spends on an event of probability p . Probability $\frac{1}{2}$ costs $-\log_2(\frac{1}{2}) = 1$ bit; probability $\frac{1}{256}$ costs 8 bits. Rarer is dearer.

KEY IDEA

The rate term R is not a real bitstream — during training it is a *differentiable estimate* of bits, computed as $-\log_2 p(\hat{y})$ from a learned probability model p . Because R is a smooth function of the network’s numbers, gradient descent can push on it: the network literally feels “this latent is expensive” and adjusts to make it cheaper. The model p is the beating heart of a learned codec; almost every research advance since 2017 has been a better p .

This is the moment the thesis of Chapter 23 — **compression is prediction** — becomes concrete and mechanical. The probability model p is a *predictor* of the latent. The better it predicts, the smaller R . The entire research program of learned compression, from 2017 to today, is the search for better predictors of the latent. Hold onto that; every section below is a smarter p .

2.2.1 The quantization wall

Now we hit the wall that nearly killed the whole idea. The latent y comes out of the encoder as ordinary floating-point numbers — 1.273, -0.488 , and so on. To compress them we must **quantize**: round each to a whole number, say to 1, 0. Only whole numbers from a finite alphabet can be entropy-coded, and rounding is the single lossy step (just as in JPEG). Fine. But remember *how* a neural network learns, from Chapter 56: by **gradient descent**. You compute how much the loss would change if you nudged each internal number a hair, and you step in the direction that lowers the loss. That requires the loss to have a **gradient** — a slope — with respect to every trainable number, all the way back through every operation.

Rounding has no useful slope. Picture the staircase graph of the round-to-nearest function: it is flat between the steps (slope exactly zero) and jumps vertically at each half-integer (slope undefined). Nudge the input 1.273 a tiny bit and the output stays glued at 1 — the slope is zero almost everywhere. A zero gradient is a dead telephone line: no information about how to improve can travel back through it. Stack a rounding step in the middle of your network and the gradient hits it, reads “zero,” and the encoder g_a in front of it never learns anything. The pipeline is severed.

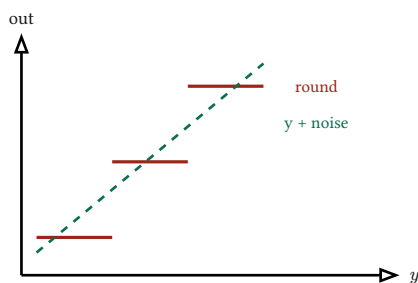


Figure 8: The rounding staircase has zero slope between steps and infinite slope at the jumps, so no gradient flows. Replacing it during training with the original value plus uniform noise gives a flat, friendly slope of 1.

PITFALL

You cannot just “ignore” the rounding during training and round only at test time. If the encoder is trained on smooth, un-rounded latents it will happily hide information in tiny fractional digits — the difference between 1.273 and 1.274 — that rounding will then obliterate, and quality collapses. The network must be trained to live *with* quantization, which means quantization (or a faithful stand-in for it) has to be present during training. The problem is making that stand-in differentiable.

2.2.2 The noise trick: rounding you can differentiate

The fix, introduced by Ballé and colleagues and now used by essentially every learned codec, is beautiful in its simplicity. Rounding y to the nearest integer introduces an error: the gap between y and its rounded value. For a value uniformly spread within its quantization cell, that error is spread uniformly between $-\frac{1}{2}$ and $+\frac{1}{2}$. So during training, instead of rounding, **add a random number drawn uniformly from $-\frac{1}{2}$ to $+\frac{1}{2}$** :

$$\tilde{y} = y + u, \quad u \sim \text{Uniform}\left(-\frac{1}{2}, \frac{1}{2}\right).$$

This *additive uniform noise* is a stand-in for quantization that gets the statistics right — the noisy value $\tilde{y}$ is jittered by the same amount that rounding would have jittered it — yet, unlike the staircase, it is a smooth function of y with a perfectly healthy slope of 1. Add a constant (even a random one) and the gradient passes straight through unharmed. The telephone line is reconnected. The encoder once again feels how its choices ripple forward to the rate and the distortion, and it can learn.

Σ GO FURTHER · THE MATHS — The uniform distribution and why its noise mimics rounding skip if familiar

A **uniform distribution** on the interval from $-\frac{1}{2}$ to $+\frac{1}{2}$ means: every value in that range is equally likely, like a perfectly fair spinner that can stop anywhere between -0.5 and $+0.5$. Now picture rounding. Any value between 0.5 and 1.5 rounds to 1; the *error* you make (rounded minus original) ranges across exactly $-\frac{1}{2}$ to $+\frac{1}{2}$, and if the originals are spread evenly, that error is uniform on that interval. So *adding* uniform $(-\frac{1}{2}, \frac{1}{2})$ noise to y produces a value whose distance from y has the same spread as the rounding error would — a statistical twin of quantization. Tiny example: $y = 1.273$. Rounding gives 1, an error of -0.273 . The noise model instead produces some $\tilde{y}$ like $1.273 + 0.18 = 1.453$; across many draws, the spread of the jitter matches rounding’s. The point is the swap costs us little in realism but buys us a usable gradient.

Why is uniform noise the *right* surrogate, and not, say, Gaussian noise? Because its spread exactly matches the spread of the error that rounding actually makes. We can prove that, and the proof needs nothing past Chapter 10’s variance.

Theorem (The noise surrogate matches the rounding error's spread). Round-to-nearest maps a value to its closest integer; for inputs spread evenly across a quantization cell, the rounding error is uniform on $(-\frac{1}{2}, \frac{1}{2})$. The additive training noise is also uniform on $(-\frac{1}{2}, \frac{1}{2})$. Both therefore have mean 0 and variance $\frac{1}{12}$ — the noise is a faithful, same-spread stand-in for quantization.

Recall. The big “S” sign $\int$ is just a **continuous sum** (Chapter 11): to average u^2 over the interval, we add up u^2 across every point and divide by the interval’s width (here width 1, so the division is invisible). The one mechanical rule we use is the **power rule** for integrals from Chapter 11, also used on this exact u^2 in Chapter 39: the running total of u^2 is $\frac{u^3}{3}$, so we evaluate $\frac{u^3}{3}$ at the two ends and subtract. No new machinery — only the “continuous sum” idea you already own.

Proof. Let u be uniform on $(-\frac{1}{2}, \frac{1}{2})$. Its mean is, by symmetry, 0. Its variance (Chapter 10: the average squared distance from the mean) is the integral

$$\text{Var}(u) = \int_{-\frac{1}{2}}^{\frac{1}{2}} u^2 \, du = \left[\frac{u^3}{3} \right]_{-\frac{1}{2}}^{\frac{1}{2}} = \frac{1}{3} \left(\frac{1}{8} + \frac{1}{8} \right) = \frac{1}{12}.$$

Now the rounding error. For an input y uniformly distributed within its cell, the error $r = \lfloor y \rfloor - y$ ranges over $(-\frac{1}{2}, \frac{1}{2})$ and is itself uniform there — so it has the identical mean 0 and variance $\frac{1}{12}$. The additive training noise and the true quantization error are the *same* random quantity in distribution. Adding the noise during training therefore exposes the network to exactly the perturbation it will face at test time, while — unlike rounding — remaining a differentiable function of y (the derivative of $y + u$ with respect to y is 1). That is precisely the surrogate we wanted: same statistics, usable gradient. $\square$

PY GO FURTHER · PYTHON — Checking the variance match by simulation *skip if fluent*

Do not take the algebra on faith — simulate it. We draw many random values, measure the variance of the rounding error and of the uniform noise, and confirm both land near $\frac{1}{12} \approx 0.0833$. Python 3.14 with the `random` module.

```
import random

def variance(xs: list[float]) -> float:
    m = sum(xs) / len(xs)
    return sum((x - m) ** 2 for x in xs) / len(xs)

random.seed(0)
ys = [random.uniform(0.0, 10.0) for _ in range(200_000)]
round_err = [round(y) - y for y in ys] #
quantization error
noise = [random.uniform(-0.5, 0.5) for _ in ys] #
```

```
training surrogate
```

```
print(f"rounding-error var: {variance(round_err):.4f}")
print(f"uniform-noise var: {variance(noise):.4f}")
print(f"theory 1/12      : {1/12:.4f}")
# rounding-error var: 0.0833
# uniform-noise var: 0.0833
# theory 1/12      : 0.0833
```

All three agree: the noise really is a statistical twin of rounding. The underscore in `200_000` is just Python’s digit separator (Chapter 15) — it reads as two hundred thousand and changes nothing.

There is a deeper truth lurking here, and it ties this whole construction back to Chapter 56. Training an autoencoder to minimize $R + \lambda D$ with this uniform noise turns out to be *mathematically identical* to fitting a **variational autoencoder** (the VAE of Chapter 56). The distortion term plays the role of the VAE’s reconstruction term, and the rate term plays the role of the VAE’s **KL-divergence** penalty (Chapter 20) — the “regularizer” that pulls the latent toward a simple prior. In other words: a learned compressor *is* a VAE whose KL term is literally measured in bits. Compression is variational inference. We will not lean on the heavier VAE machinery here, but it is worth knowing that the two communities were, unknowingly, building the same thing.

Aside. At *test* time, when you actually compress a file, there is no noise: you round honestly, entropy-code the integers with the trained model p , and ship the bits. The noise is purely a training-time crutch to keep the gradients flowing. Some codecs use a hybrid — round in the distortion path but add noise in the rate path — because that combination empirically matches the true rounded behaviour even better. The principle is unchanged: be differentiable while learning, be exact while shipping.

2.3 Ballé’s transform: convolutions and GDN

We now have the objective ($L = R + \lambda D$) and the trick that makes it trainable (noise for quantization). What does the transform network g_a actually look like inside? Two ingredients: **convolutions** and a special nonlinearity called **GDN**.

A photograph is not a flat list of numbers; it is a grid, and the same kinds of patterns — edges, textures, gradients — appear all over it. It would be wasteful to learn a separate detector for an edge in the top-left corner and another for the identical edge in the bottom-right. A **convolutional** layer learns one small pattern detector (a little grid of weights, called a *filter* or *kernel*) and slides it across the entire image, applying the same detector everywhere. This is the workhorse of all image neural networks, and the analysis transform g_a is a short stack of convolutional layers that progressively shrink the spatial grid (each layer downsamples, the way the DCT folds an 8×8 block into coefficients) while

growing the number of **channels** — parallel feature maps, each one a different learned “kind of pattern.”

PY GO FURTHER · PYTHON — Convolution as a sliding dot product *skip if fluent*

A convolution slides a small weight grid (the *kernel*) over the input, multiplying and summing at each position. Here is the idea in 1-D, in pure Python 3.14, so you can see there is no magic — just a sliding weighted sum.

```
def conv1d(signal: list[float], kernel: list[float]) -> list[float]:
    k = len(kernel)
    out: list[float] = []
    for i in range(len(signal) - k + 1):
        window = signal[i:i + k]                # slice out k
        samples
        out.append(sum(s * w for s, w in zip(window, kernel)))
    return out

# an edge detector: respond where neighbours differ
print(conv1d([0, 0, 9, 9, 9], [-1, 1]))      # -> [0, 9, 0, 0]
```

The kernel `[-1, 1]` fires (outputs 9) exactly at the jump from 0 to 9 and is silent on the flat stretches — it *detects an edge*. A real 2-D image convolution does this over a small square window across both dimensions, with many kernels learned by gradient descent rather than chosen by hand. The `zip` pairs each sample with its weight; `sum(... for ...)` is the comprehension you met in the Python primer (Chapter 16).

Between the convolutional layers sits the nonlinearity — the bend that lets a network represent more than a straight line (Chapter 56). Most image networks use a plain **ReLU** (replace negatives with zero). Ballé made a different, inspired choice. He reasoned that the goal of the transform is to *decorrelate and equalize* the signal — to spread information out so no single number carries too much, which is exactly what makes a signal compressible. Biology already solved this: neurons in the visual cortex perform **divisive normalization**, dividing each unit’s response by a pooled measure of its neighbours’ activity, which tends to gaussianize and decorrelate natural-image statistics. Ballé built a trainable version and called it **Generalized Divisive Normalization**, GDN.

Σ GO FURTHER · THE MATHS — Generalized Divisive Normalization (GDN), gently *skip if familiar*

Suppose a layer produces several feature values $w_1, w_2, \dots, w_n$ at one spatial spot. Plain activation looks at each one alone. GDN instead *divides* each value by a quantity built from *all* of them:

$$\text{GDN}(w_i) = \frac{w_i}{\sqrt{\beta_i + \sum_j \gamma_{ij} w_j^2}}.$$

In words: take feature i , and shrink it in proportion to how loud the whole neighbourhood of features is. The β 's and γ 's are *learned* numbers that decide how much each feature suppresses each other. If everything nearby is quiet, the denominator is small and w_i passes through; if the neighbourhood is blaring, w_i gets damped. The effect is to flatten the wildly unequal energy of raw image features into something more uniform – and uniform, decorrelated numbers are precisely what compresses well. The decoder uses the exact inverse, IGDN, to undo it. You do not need the formula to follow the rest of the chapter; just hold the picture: *GDN is a learned, biologically-inspired way of equalizing the signal so it compresses better than ReLU allows.*

HISTORICAL NOTE

Generalized Divisive Normalization was introduced by **Johannes Ballé, Valero Laparra, and Eero Simoncelli** in “**Density Modeling of Images using a Generalized Normalization Transformation**” (ICLR 2016) – Simoncelli being a leading vision scientist, which is why the idea came from the biology of the eye. They folded it into a full codec the next year. The lesson is delicious: the single best-performing nonlinearity for image compression was not borrowed from mainstream deep learning at all; it was borrowed from the human visual system, the very thing the codec is trying to please.

End-to-End Optimized Image Compression (Ballé 2017)

2016–2017

Authors	Johannes Ballé, Valero Laparra, Eero Simoncelli
Aim	Replace the hand-designed transform/quantize/entropy pipeline with one autoencoder trained end-to-end on the rate–distortion loss.
Complexity	Encode and decode are a few convolutional passes – fast and roughly symmetric; vastly cheaper than later context models.
Strengths	First fully learned codec to rival JPEG 2000 on standard metrics; introduced GDN and the additive-uniform-noise quantization surrogate that the whole field still uses.
Weaknesses	Used a fixed (per-channel) entropy model that could not adapt to local image content, leaving rate on the table.
Superseded by	The 2018 scale hyperprior, which made the entropy model adaptive.

This is the foundation stone. The analysis transform g_a is convolutions + GDN; the synthesis g_s is the mirror image with IGDN; quantization is round (noise during training); the entropy coder is a real range coder (Chapter 26) driven by a learned probability model. Train on a large set of natural images, one network per λ . The paper showed, for the first time, that pure gradient descent could discover a transform competitive with decades of hand-engineering.

Checkpoint. Why can't the transform network just use ordinary rounding during training and learn normally?³

2.4 The hyperprior: transmitting a description of the model

The 2017 codec had one obvious weakness, and naming it leads straight to the chapter's centerpiece. Its entropy model p was *fixed*: after training, every image was coded with the same probability table for the latents. But the right table is not the same everywhere. Consider a single photograph — a calm blue sky in the upper third, busy foliage in the lower two-thirds. Over the sky, the latent values are small and tightly clustered (a sharp, narrow distribution). Over the foliage, they swing wildly (a broad, spread-out distribution). A single fixed model cannot be sharp *and* broad at once. If it is narrow, it is shocked by the foliage and wastes bits; if it is broad, it is needlessly uncertain about the sky and wastes bits there. Either way, money is left on the table.

The classical codecs faced this exact problem and solved it with *adaptive* entropy coding (the adaptive models of Chapters 24 and 26, which update their statistics as they go). The learned answer, introduced by Ballé, Minnen, Singh, Hwang, and Johnston in “**Variational Image Compression with a Scale Hyperprior**” (ICLR 2018), is more elegant and more powerful: build a *second, tiny autoencoder* that looks at the latent and transmits a short description of its local statistics as **side information**. That description is the **hyperprior**.

KEY IDEA

A **hyperprior** is a small second autoencoder stacked on top of the main one. It examines the main latent y and emits a few extra bits — **side information**, the hyper-latent z — that tell the decoder, region by region, *how spread out the local latents are* (their Gaussian scale, i.e. standard deviation). The decoder reads this cheat-sheet first, configures a custom probability model for each part of the image, and then decodes the main payload far more cheaply. You spend a few bits describing the model in order to save many bits using it.

³**Checkpoint answer.** Because rounding has zero gradient almost everywhere, so no learning signal flows back through it to the encoder; you replace it with additive uniform noise, which has the same statistical effect on the latent but a healthy slope of 1, restoring the gradient.

Here is the flow. A **hyper-analysis** network h_a takes the main latent and produces the hyper-latent $z = h_a(y)$. This z is itself quantized and entropy-coded (with its own small fixed model) and sent first — it is tiny, a few percent of the total. On the other end, a **hyper-synthesis** network h_s reads $\hat{z}$ and outputs, for *every element* of the main latent, the parameters of a Gaussian: in the basic version, just a **scale** σ (how spread out), and in the improved “mean–scale” version a **mean** μ as well (where it is centered). Armed with a per-element μ and σ , the main entropy coder uses a tailored Gaussian probability for each latent value instead of one global table. Sharp where the image is calm, broad where it is busy — automatically, learned, and paid for with a handful of side bits.

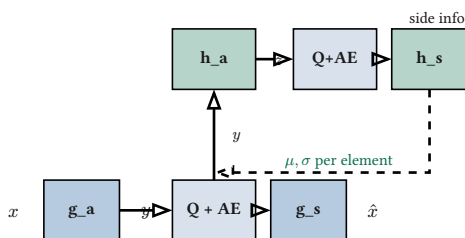


Figure 9: The hyperprior architecture. The main path (left, dark) compresses the image through g_a/g_s . The hyper path (right, light) compresses a description of the main latent’s *statistics*, which it feeds into the entropy model so each latent element gets its own Gaussian.

2.4.1 A worked example: why side information pays

Numbers make the bargain vivid. Imagine a stretch of latent values that, taken as a whole, range over a fairly broad spread — say a standard deviation of 8. If we entropy-code them under one fixed Gaussian of scale 8, the cost per value is governed by that wide uncertainty.

Σ GO FURTHER · THE MATHS — Bits of a Gaussian value, by its standard deviation skip if familiar

For a value drawn from a Gaussian (bell curve) of standard deviation σ , quantized to unit steps, an ideal coder spends about

$$\log_2(\sigma\sqrt{2\pi e}) \approx \log_2 \sigma + 2.05 \quad \text{bits per value.}$$

You do not need the derivation: this is exactly the *differential entropy* of a Gaussian we met in Chapter 21, $h = \frac{1}{2} \log_2(2\pi e \sigma^2) = \log_2(\sigma\sqrt{2\pi e})$ — the continuous source’s “bits per value.” Just read the consequence: **halving the standard deviation saves about one bit per value**, because $\log_2$ of half is one less. A spread of $\sigma = 8$ costs about $\log_2 8 + 2.05 = 3 + 2.05 = 5.05$ bits; a spread of $\sigma = 2$ costs about $1 + 2.05 = 3.05$ bits. Narrower is cheaper, by exactly the logarithm of the ratio.

Now suppose that broad spread of 8 was an illusion of averaging: locally, the region is really two calm zones, one with $\sigma = 2$ and one with $\sigma = 2$, that merely sit at different centers. If we transmit a few side bits saying “this half has mean -6 , that half has mean

+6, both with scale 2,” the main coder can now use $\sigma = 2$ everywhere. Cost drops from about 5.05 to about 3.05 bits per value — a saving of 2 bits per latent. Suppose the side information costs, amortized, 0.2 bits per latent to send. Net saving: $2 - 0.2 = 1.8$ bits per latent. Across hundreds of thousands of latents in a real image, that is a large fraction off the file. *That* is the hyperprior’s bargain: a small, fixed investment in describing the model buys a large, repeated discount on using it.

Myth. *The hyperprior compresses the image a second time.*

Reality. It does not. The hyperprior compresses a *description of the main latent’s statistics* — the spreads and centers of its Gaussians — not the image content. It is metadata about the entropy model, sent so the decoder can configure the right distribution before it reads the main payload. The image information still lives entirely in the main latent.

Scale Hyperprior

2018

Authors	Johannes Ballé, David Minnen, Saurabh Singh, Sung Jin Hwang, Nick Johnston
Aim	Make the entropy model content-adaptive by transmitting per-element Gaussian scales (and later means) as compressed side information.
Complexity	One extra small autoencoder; still fully parallel encode/decode — no serial bottleneck. Practical and fast.
Strengths	First learned codec to clearly beat the best hand-built ones on perceptual metrics (MS-SSIM); the per-element Gaussian model became the template for everything after.
Weaknesses	Assumes latent elements are independent given the side info; ignores the strong correlations between neighbouring latents, which the next idea exploits.
Superseded by	Joint autoregressive + hierarchical models (Minnen 2018), which add a neighbour-aware context model.

The hyperprior is the single most influential architectural idea in learned image compression. Almost every codec since 2018 — including the JPEG AI standard — carries a hyperprior or a direct descendant of it. The “mean–scale” variant, which sends both μ and σ , is the common baseline.

2.5 Context models: predicting each latent from its neighbours

The hyperprior treats the latent elements as independent once the side information is known. But they are not independent. Neighbouring latents are correlated — if this spot is bright, the spot beside it probably is too — exactly the kind of local correlation that PNG’s filters (Chapter 44) and modern image codecs exploit in pixel space. There is still redundancy to squeeze, and the way to squeeze it is the oldest trick in this book: **predict each symbol from the ones already seen**, and only pay for the surprise.

A **context model** (also called an **autoregressive** model) does precisely that. To code latent element number n , it looks at the latents *already decoded* — the neighbours above and to the left — and predicts the distribution of element n from them. Because the decoder has also already decoded those same neighbours, it can run the identical prediction and stay in lockstep. This is the spatial twin of the order- k context models of Chapters 23 and 33, and of the predictive coding in JPEG-LS (Chapter 42): condition the probability of each symbol on its context, and the entropy coder gets a sharper, cheaper distribution.

Minnen, Ballé, and Toderici combined this context model with the hyperprior in “**Joint Autoregressive and Hierarchical Priors for Learned Image Compression**” (NeurIPS 2018). The entropy model for each latent now fuses two sources: the **hierarchical** prior (the hyperprior’s per-element Gaussian from side information) *and* the **autoregressive** context (the prediction from decoded neighbours). With both, their codec became **the first learned method to beat BPG** — the strong still-image codec carved out of HEVC/H.265 — on both PSNR and MS-SSIM, with roughly a 15.8% rate reduction over the previous learned state of the art. Learned compression had, for the first time, beaten the best of classical engineering on its own metrics.

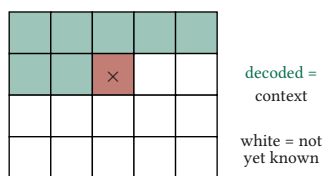


Figure 10: An autoregressive context model decodes latents in raster order. Element n (the cross) is predicted from its already-decoded neighbours (shaded); undecoded latents (white) are off-limits. The dependency makes decoding *serial*: each element must wait for the ones before it.

Cheng, Sun, Takeuchi, and Katto pushed the entropy model further in “**Learned Image Compression with Discretized Gaussian Mixture Likelihoods and Attention Modules**” (CVPR 2020). Two upgrades: replace each element’s single Gaussian with a **mixture** of Gaussians (several bell curves added together, which can fit lopsided or multi-peaked latent distributions a single bell cannot), and sprinkle in **attention** modules (layers that let the network focus capacity on the hard, detailed regions of an image). The result was the first learned codec to draw **level with VVC** — H.266, the newest video-grade standard

(Chapter 53) — on PSNR. In four years, learned codecs had gone from “promising” to “tied with the state of the art in standardized compression.”

2.5.1 The wound: serial decoding

This is where we must be honest, because the context model carries a curse. By construction it is **serial**: element n ’s distribution depends on element $n - 1$ having already been decoded, which depends on $n - 2$, and so on. You cannot decode the latents in parallel; you must crawl through them one at a time, in order, each one waiting for the network to predict its distribution from the freshly-decoded neighbours. A modern image has hundreds of thousands of latent elements. Running the prediction network once per element, in sequence, makes the decoder **staggeringly slow** — easily tens to hundreds of times slower than a classical codec that decodes whole blocks at once. This is the central, defining wound of high-ratio learned compression, and it is the reason the best research codecs were, for years, impractical to deploy.

PITFALL

Do not confuse the encoder’s cost with the decoder’s. In classical streaming codecs (HEVC, AV1) the encoder is the slow one and the *decoder* is blisteringly fast — perfect for “compress once, watch a billion times.” The autoregressive context model breaks this asymmetry: it makes the *decoder* the slow part, the exact opposite of what deployment wants. A fast encoder and a glacial decoder is almost the worst possible shape for a video or web codec.

2.5.2 Buying back the speed: parallel context

The field’s response, from 2020 onward, was to keep the *benefit* of context modelling while killing the strict serial dependency. The trick is to give up “every latent depends on every previously-decoded latent” in favour of a carefully chosen partial order that decodes in a handful of parallel passes instead of hundreds of thousands of serial steps.

The cleanest idea is the **channel-wise** model of Minnen and Singh, “**Channel-wise Autoregressive Entropy Models for Learned Image Compression**” (ICIP 2020). Instead of making each *spatial* position wait for its neighbours, they split the latent’s channels into a few groups and decode the groups in order: the first group from the hyperprior alone, the second group conditioned on the first, and so on — a handful of steps, not hundreds of thousands. Within each group, every spatial position decodes *in parallel*. It recovered most of the context model’s compression gain (about 6.7% rate savings over a strong context-adaptive baseline on the Kodak test images) while restoring practical speed.

The other famous fix is the **checkerboard** context of He and colleagues (CVPR 2021): colour the latent grid like a checkerboard, decode all the black squares in parallel (using only the hyperprior), then decode all the white squares in parallel using the just-decoded black squares as context. Two passes instead of hundreds of thousands, with almost all the compression benefit intact. Between channel-wise and checkerboard schemes, the serial

wound was largely cauterized — and these parallel context models are what the JPEG AI standard and the strongest 2024–2026 codecs actually use.

SCOREBOARD — learned image codecs vs. the classical bar (Kodak-class, rough relative standing)

Technique	Bytes	Ratio	Notes
JPEG (Ch 42)	baseline	—	1992 hand-built DCT pipeline
JPEG 2000 (Ch 43)	smaller	—	wavelet; ~Ballé-2017 territory
Ballé 2017 (learned)	≈ JPEG 2000	—	first end-to-end learned transform
Scale hyperprior 2018	smaller	—	first learned codec to beat the hand-built ones (MS-SSIM)
Minnen 2018 (context)	smaller still	≈ −15.8% vs prior learned	first to beat BPG (HEVC-class)
Cheng 2020 (GMM+attn)	smallest	≈ VVC	first learned codec level with H.266
JPEG AI (2025 std)	smallest	≈ −28.5% vs VVC-intra	first learned international standard

NOTE

The scoreboard above is *relative standing*, not byte counts on `tinypng`’s running sample: these codecs need a trained neural network and a GPU, far beyond the from-scratch Python pipeline `tinypng` is. The numbers are the headline rate-savings each paper reported on standard image sets (Kodak and friends). The trajectory is the point — five years from “matches a 2000-era codec” to “beats the newest video-grade standard by nearly a third.”

2.6 Why the learned rate term is honest

It is fair to be suspicious. The training loss *estimates* the rate as $-\log_2 p(\hat{y})$ using the network's *own* model p . Could the network cheat — make p optimistic to fake a low rate while the real bitstream is fat? It cannot, and the reason is a clean theorem you already have the tools to prove. The bits the real entropy coder spends are governed by the *model the decoder uses*, which is the same p . And coding against any model p costs, on average, at least the true entropy of the source — never less. So the trained p cannot conjure bits from nowhere; the rate it reports is a genuine cost.

Theorem (Cross-entropy is never below entropy). *Let the latent symbols truly follow distribution q , and let the codec code them under model p . The expected code length is the **cross-entropy** $H(q, p) = -\sum_s q(s) \log_2 p(s)$, and it satisfies*

$$H(q, p) \geq H(q) = -\sum_s q(s) \log_2 q(s),$$

with equality only when $p = q$. So coding under a learned model is never cheaper than the true entropy, and is cheapest exactly when the model is right.

Proof. The gap between the two sides is the **Kullback–Leibler divergence** of Chapter 20:

$$H(q, p) - H(q) = \sum_s q(s) \log_2 \frac{q(s)}{p(s)} = D_{\text{KL}}(q \parallel p).$$

We proved in Chapter 20 (Gibbs' inequality) that $D_{\text{KL}}(q \parallel p) \geq 0$ for any two distributions, with equality if and only if $p = q$. Hence $H(q, p) \geq H(q)$, and the only way to hit the floor is to model the source exactly. The learned codec, by minimizing the rate term, is *driving* D_{KL} *toward zero* — that is, training p to match the true latent distribution q as closely as it can. There is no cheating: a smaller reported rate means a genuinely better model, which means genuinely fewer real bits. $\square$

KEY IDEA

Minimizing the rate term $R = \mathbb{E}[-\log_2 p(\hat{y})]$ is the same thing as making the model p match the true latent distribution q — it minimizes the KL-divergence between them. “Better predictor” and “fewer bits” are not two goals the network balances; by this theorem they are *one and the same goal*. Chapter 23's thesis, made mechanical.

Here is the rate term in code, so you can watch a model's quality turn directly into a bit count. We will model latents element-by-element with a per-element Gaussian, exactly as the hyperprior provides, and compute the predicted bits.

PY GO FURTHER · PYTHON — Computing the learned rate from a Gaussian model *skip if fluent*

Given each latent value, its predicted mean μ , and its predicted scale σ , the bits to code it are $-\log_2$ of the probability mass the Gaussian puts on its quantization cell. We approximate that mass as the bell-curve height times the cell width (1), then take $-\log_2$. Pure Python 3.14 plus `math`.

```
import math

def gauss_pmf(value: float, mu: float, sigma: float) -> float:
    """Approx. probability mass of one unit-width quantization
    cell."""
    sigma = max(sigma, 1e-6) # never
    divide by zero
    z = (value - mu) / sigma
    height = math.exp(-0.5 * z * z) / (sigma * math.sqrt(2 *
    math.pi))
    return min(max(height, 1e-12), 1.0) # clamp into
    (0, 1]

def rate_bits(latents: list[float],
              mus: list[float],
              sigmas: list[float]) -> float:
    total = 0.0
    for v, mu, sg in zip(latents, mus, sigmas):
        p = gauss_pmf(round(v), mu, sg)
        total += -math.log2(p) # surprisal
    in bits
    return total

# A calm region: values near their predicted means, tight scale
-> cheap.
calm = rate_bits([0.0, 1.0, -1.0], [0.0, 1.0, -1.0], [1.0, 1.0,
1.0])
# A busy region: same values, but the model is wrong and unsure
-> dear.
busy = rate_bits([0.0, 1.0, -1.0], [3.0, -2.0, 4.0], [4.0, 4.0,
4.0])
print(f"calm: {calm:.2f} bits    busy: {busy:.2f} bits")
# calm: 3.98 bits    busy: 11.92 bits
```

The *same three values* cost three times as many bits under a worse model. That gap is the rate term feeling pain — and gradient descent spends its whole life shrinking it, by training the hyperprior to predict μ and σ well.

Aside. The real codecs integrate the Gaussian over the exact cell ($\Phi(v + \frac{1}{2}) - \Phi(v - \frac{1}{2})$), using the cumulative bell curve) rather than using the height-times-width shortcut above, which keeps

the rate estimate exact even for wide cells. The idea is identical; only the arithmetic is more careful.

2.7 From paper to standard: JPEG AI, and the state of play in 2026

For a decade learned codecs lived in research papers and GitHub repositories. Crossing into a real, interoperable standard is a different mountain — it demands that an encoder built by one company and a decoder built by another agree bit-for-bit, on hardware nobody has met. That mountain was climbed.

JPEG AI is the first international standard for learning-based image coding. Its first edition was published in late 2024, and in early 2025 it was approved as an ISO/IEC/ITU International Standard. Architecturally it is exactly the lineage of this chapter: an autoencoder transform, a hyperprior, and a (parallel) context entropy model — productized, made interoperable, and made royalty-free. On standard tests its strongest configuration reports up to roughly **28.5% rate savings over VVC intra** (the all-intra mode of H.266, the newest video-grade still-image bar from Chapter 53), with lighter operating points that trade some of that gain for far faster decoding. Crucially, JPEG AI was designed to serve **two** audiences at once: human viewers *and* machines. Because the compressed file is a neural latent, a computer-vision task — object detection, classification — can sometimes run *directly on the latent*, without ever fully decoding to pixels. The codec and the perception merge.

HISTORICAL NOTE

The naming can confuse. **JPEG AI** (this chapter’s learned codec) is a different beast from **JPEG XL** (Chapter 45), which is a superb but *classically* designed codec — hand-built transforms and the ANS entropy coder of Chapter 27, not a neural network. Both came from the JPEG committee in the 2020s; only JPEG AI is a learned, end-to-end-trained codec of the kind this chapter describes.

So where do learned image codecs actually stand in 2026? The verdict is genuinely mixed, and a good engineer holds both halves at once.

Where they win. On *perceptual quality at low bitrate*, learned codecs are decisively ahead — a gap we will widen in Chapter 58, where generative decoders hallucinate plausible texture instead of blurring. On standard metrics, the best learned codecs now match or exceed VVC for still images. And they are *flexible*: one architecture retargets to medical scans, satellite imagery, or screen content by retraining on that data, no committee required.

Where they lose. *Speed and hardware*, still. Even with parallel context models, neural decoders lean on a GPU or NPU that most decoding devices — a cheap phone, a TV, a doorbell camera — simply do not have, and they burn more energy than a fixed-function classical decoder etched in silicon. They lack the brutal encode/decode asymmetry (en-

code-once, decode-everywhere-cheaply) that makes streaming economical. And *bit-exact reproducibility* is a genuine headache: the encoder and decoder must perform *identical* floating-point arithmetic, or the entropy coder desynchronizes and the image corrupts — and floating-point results differ subtly across chips and libraries. The standards solve this with carefully specified integer arithmetic, but it is real engineering pain, not a footnote.

Myth. *Learned codecs have already won and classical codecs are obsolete.*

Reality. Not in deployment. As of 2026, the overwhelming majority of images on the web are still JPEG, with WebP/AVIF/JPEG XL growing; learned neural codecs remain rare in the wild outside research and early JPEG AI rollouts, gated by decode cost, hardware, and energy. They have won the *research* rate–distortion race and are beginning the long, slow march into silicon and browsers. Winning the benchmark and winning the world are different victories, often a decade apart — recall how long JPEG 2000 (Chapter 43) was “better” yet never displaced JPEG.

Chapter takeaways

- A learned image codec keeps JPEG’s **transform** → **quantize** → **entropy-code** skeleton but makes every stage a neural network, trained *jointly, end to end* to minimize the rate–distortion loss $L = R + \lambda D$.
- The **rate** term is a differentiable estimate of bits, $R = \mathbb{E}[-\log_2 p(\hat{y})]$, computed from a *learned* probability model p ; the whole field’s progress is the search for a better p .
- Rounding has zero gradient and would sever training; the **additive-uniform-noise** surrogate keeps the same quantization statistics but a slope of 1, restoring gradient flow. (At test time, round honestly.)
- Ballé’s transform uses **convolutions** and **GDN**, a biologically-inspired divisive normalization that equalizes the signal so it compresses better than ReLU allows.
- The **hyperprior** is a small second autoencoder that transmits side information describing the main latent’s local Gaussian statistics (μ, σ per element) — paying a few bits to describe the model in order to save many bits using it.
- A **context (autoregressive) model** predicts each latent from its decoded neighbours for still-tighter coding, but its strict serial dependency made decoding brutally slow; **channel-wise** and **checkerboard** schemes (2020–21) recovered most of the gain in a few parallel passes.
- Because coding under any model costs at least the true entropy ($H(q, p) \geq H(q)$), the learned rate is honest: minimizing it *is* matching the true latent distribution — Chapter 23’s “compression = prediction,” made mechanical.
- **JPEG AI** (International Standard, 2025) is the first learned image standard — hyperprior + parallel context, royalty-free, ~28.5% over VVC-intra, serving both human and machine vision — while classical codecs still rule deployment, gated by neural decode cost, hardware, and energy.

2.8 Exercises

Exercise 57.1 ★

In your own words, explain why a learned codec replaces the rounding step with “add uniform noise” *only during training*, and what it does at test time instead. What would go wrong if it used noise at test time too?

Exercise 57.2 ★

A latent region is coded under a Gaussian model. Using the rule “bits $\approx \log_2 \sigma + 2.05$,” how many bits per value does a scale of $\sigma = 16$ cost? Of $\sigma = 4$? If a hyperprior can shrink the effective scale from 16 to 4 by spending 0.3 side bits per value, what is the net saving per value?

Exercise 57.3 ★★

Explain, using the language of Chapters 18 and 23, why the rate term $-\log_2 p(\hat{y})$ being small is *the same thing* as the model p predicting the latent well. Then state what quantity the loss is implicitly minimizing between p and the true latent distribution q , and what its smallest possible value is.

Exercise 57.4 ★★

An autoregressive context model and a channel-wise context model both improve compression by conditioning each latent on others. Describe the key difference in *what each conditions on*, and explain why the channel-wise model can decode in a few parallel passes while the spatial autoregressive model cannot.

Exercise 57.5 ★★

The text says the hyperprior “does not compress the image a second time.” If not the image, what exactly does the hyperprior’s bitstream encode, and why is it so much smaller than the main bitstream? Give a one-sentence analogy to a paper map with a legend.

Exercise 57.6 ★★★

Modify the `rate_bits` idea from the Python box to compute the *total* loss $L = R + \lambda D$ for a toy “codec.” You are given the original latents, the reconstructed (rounded) latents, the per-element predicted `mu / sigma`, and a value `lam`. Write a function that returns the loss, using MSE for D and the Gaussian rate for R . Then describe, in words, what happens to the chosen reconstruction as `lam` grows very large versus very small.

Exercise 57.7 ★★★

A skeptic says: “The learned codec measures its own rate with its own model p , so of course it can claim a low rate — it is grading its own homework.” Refute this rigorously

using the theorem in this chapter, and explain why a low *reported* rate forces a low *real* bitstream length, not just an optimistic number.

2.9 Further reading

- Ballé, J., Laparra, V. & Simoncelli, E. (2017). *End-to-End Optimized Image Compression*. ICLR 2017. [arXiv:1611.01704](https://arxiv.org/abs/1611.01704) — the foundation: GDN and the noise-quantization trick.
- Ballé, J. et al. (2018). *Variational Image Compression with a Scale Hyperprior*. ICLR 2018. [arXiv:1802.01436](https://arxiv.org/abs/1802.01436) — the hyperprior, the chapter’s centerpiece.
- Minnen, D., Ballé, J. & Toderici, G. (2018). *Joint Autoregressive and Hierarchical Priors for Learned Image Compression*. NeurIPS 2018. [arXiv:1809.02736](https://arxiv.org/abs/1809.02736) — first learned codec to beat BPG.
- Cheng, Z., Sun, H., Takeuchi, M. & Katto, J. (2020). *Learned Image Compression with Discretized Gaussian Mixture Likelihoods and Attention Modules*. CVPR 2020. [arXiv:2001.01568](https://arxiv.org/abs/2001.01568) — first learned codec level with VVC on PSNR.
- Minnen, D. & Singh, S. (2020). *Channel-wise Autoregressive Entropy Models for Learned Image Compression*. ICIP 2020. [arXiv:2007.08739](https://arxiv.org/abs/2007.08739) — parallel context that restored speed.
- Ascenso, J. et al. (2024). *The JPEG AI Learning-Based Image Coding Standard*. jpeg.org/jpegai — the first learned image-coding international standard.

COMING UP NEXT

We have built a codec that minimizes *mean squared error* — and that very choice hides a flaw. At low bitrates, minimizing average squared error pushes the decoder toward the *blurry average* of all plausible pictures, because blur is safe and detail is risky. But blur is exactly what the human eye hates most. **Chapter 58** confronts this head-on with the **rate–distortion–perception** trade-off: instead of asking for the reconstruction closest to the original, we ask for one that merely *looks like a real photograph* — and let a generative model (a GAN, then a diffusion model) *hallucinate* convincing texture from a tiny code. The pictures get gorgeous at bitrates this chapter’s codecs would smear to mush. The catch — invented detail that was never there — will force us to ask a hard question: when is a “reconstruction” no longer the truth?

Generative and Perceptual Codecs

A perfect photograph is not the goal of the painter. Why should it be the goal of the compressor?

— Anonymous research engineer, circa 2020

Picture two images side by side on your screen. On the left is a face compressed by a conventional codec at 0.1 bits per pixel — a bitrate so stingy that every block of skin has smeared into a soap-opera blur, and the hair looks like it was sculpted from melted butter. On the right is the same person’s face at the same file size, reconstructed by a generative codec. Every strand of hair is crisp. The pores look real. The eyes have that wet glint that says “alive.” You would bet money the right image was the original.

Here is the catch: it wasn’t. The hair on the right was invented. The pores were dreamed up by a neural network that asked itself, “Given this blurry code, what would a plausible human face look like?” It guessed correctly — impressively so — but it guessed.

That one observation unlocks everything in this chapter. We are leaving the world where a codec tries to reproduce pixels faithfully, and entering one where a codec tries to reproduce **experience** faithfully, even if it has to make things up. This shift is philosophically radical, practically powerful, and ethically thorny, all at once.

WHERE WE ARE

In Chapter 56 we learned the machine-learning vocabulary: neurons, gradient descent, autoencoders, GANs, and diffusion models. In Chapter 57 we saw how Ballé’s team (2016–2018) built the first learned image codecs by plugging an autoencoder into a rate-distortion loss, using quantization-as-noise to make the whole pipeline differentiable, and adding a hyperprior to transmit side information about the latent’s local statistics. Those codecs were trained to minimize **mean squared error** — which means they tried to be as pixel-accurate as possible. This chapter asks what happens when you train for **perceptual quality** instead, and why that change opens up a whole new category of compressor.

By the end of this chapter you will be able to:

- Explain the rate-distortion-perception tradeoff and why MSE codecs go blurry at low bitrates.

- Describe how a GAN discriminator loss trains a codec decoder to produce photo-realistic output.
- Walk through the HiFiC architecture and the human-preference results that validated it.
- Explain how diffusion-model decoders extend the idea to ultra-low bitrates, using PerCo as the example.
- Understand StableCodec's one-step distillation approach for faster decoding.
- Articulate the 'hallucinated detail' debate: when perceptual codecs are appropriate and when they are dangerous.
- Evaluate the current state of standardization (JPEG AI) and deployment barriers.

3.1 The Blur Problem: Why MSE Fails at Low Bitrates

To understand why generative codecs exist, you first need to feel the problem they solve.

Suppose you are compressing a portrait at a very aggressive ratio — 50:1, say. Most of the detail is gone; only a skeleton of information survives in the bitstream. When the decoder reconstructs the image, it must fill in all those missing pixels **somehow**. The classical answer, baked into every MSE-trained model, is: **average over all plausible completions**.

Think of it this way. At some location in the image, the missing pixel might reasonably be any shade from light grey to dark grey. The MSE-optimal reconstruction picks the middle grey — exactly the average. The result: every uncertain region becomes smooth and grey. This is why heavily compressed images look greasy and soft. The decoder is not being lazy; it is doing exactly what MSE asks — minimizing the expected squared difference between possible reconstructions. The mathematical average of many different sharp textures is an indistinct blur.

This is not a bug in any particular implementation. It is a fundamental consequence of optimizing for mean squared error when the reconstruction is uncertain.

Σ GO FURTHER · THE MATHS — Mean Squared Error and Why Averaging Causes Blur *skip if familiar*

Mean squared error (MSE) between an original image x and a reconstruction $\hat{x}$ is:

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{x}_i)^2$$

where N is the number of pixels and $x_i, \hat{x}_i$ are the i -th pixel values. Smaller MSE means less average pixel-level difference, which sounds good.

The problem appears when the decoder is uncertain: suppose pixel i could plausibly be 80 or 120 (on a 0–255 scale). If the decoder outputs 100 (the average), it incurs error $(100 - 80)^2 = 400$ or $(100 - 120)^2 = 400$ — a middle-of-the-road penalty. If it guesses

boldly and outputs 80, it scores 0 on the 80-case but 1600 on the 120-case — much worse on average. So MSE **punishes boldness** and rewards bland averaging. Applied across millions of pixels, this produces the characteristic smooth blur.

Signal-to-noise ratio (PSNR) is just MSE in logarithmic disguise:

$$\text{PSNR} = 10 \log_{10} \left(\frac{255^2}{\text{MSE}} \right) \text{ dB}$$

Higher PSNR sounds better, but a blurry reconstruction can have higher PSNR than a sharp one that invented slightly wrong details — another sign that PSNR misses perceptual quality.

Human vision is not an MSE meter. It is exquisitely sensitive to edges, textures, and whether something looks **like** a known material (hair, wood, fabric) — but quite insensitive to whether each pixel is exactly right. A codec that invents plausible hair texture may fool the eye even though its MSE is terrible. A codec that blurs everything to the correct average may score well on PSNR while looking dreadful.

3.2 The Rate–Distortion–Perception Tradeoff

Yochai Blau and Tomer Michaeli at the Technion formally proved what engineers had felt for years. Their 2019 paper, “The Perception-Distortion Tradeoff,” extended Shannon’s two-dimensional rate–distortion curve into a **three-dimensional** surface that adds perception as a third axis.

Definition — Perceptual quality (Blau–Michaeli). A codec has perfect perceptual quality if its reconstructed images are statistically indistinguishable from natural images — that is, the **distribution** of reconstructions matches the **distribution** of real photographs, not just pixel-by-pixel similarity to a specific original. Perceptual quality is measured by the divergence between these two distributions.

Theorem (The Perception-Distortion Tradeoff). *For any fixed bitrate R , there is a fundamental tradeoff between average distortion (e.g., MSE) and perceptual quality (e.g., distributional divergence to natural images). Achieving perfect perceptual quality necessarily increases distortion, and achieving zero distortion sacrifices perceptual quality. You cannot simultaneously minimize both.*

Proof. The intuition: zero distortion means $\hat{x} = x$ exactly, so the reconstruction distribution is just the source distribution — perfect perception too. But as the bitrate drops, $\hat{x}$ must deviate from x . It can deviate in a way that stays pixel-close (low distortion, possibly blurry) or in a way that stays distribution-close (high perceptual quality, possibly inventing details). At any fixed R , these pull in opposite directions. Blau and Michaeli prove this formally using information-theoretic arguments: the rate–distortion function

under a perfect-perception constraint is strictly larger than the unconstrained one, with the gap narrowing as $R \rightarrow \infty$. $\square$

This theorem has two startling corollaries. First, the blurry MSE codecs we criticized are not badly designed — they are faithfully implementing a choice to minimize distortion, at the cost of perception. Second, a perceptually perfect codec is **not** trying to reproduce your specific image. It is trying to generate a plausible member of the set “images that look like this kind of scene.” It may show you a face with different freckles. That is the trade you are making.

KEY IDEA

The rate–distortion–perception tradeoff is a law of nature, not a limitation of any particular algorithm. At low bitrates, you must choose: pixel accuracy or visual realism. Generative codecs choose realism. MSE codecs choose accuracy. Both are valid — for different use cases.

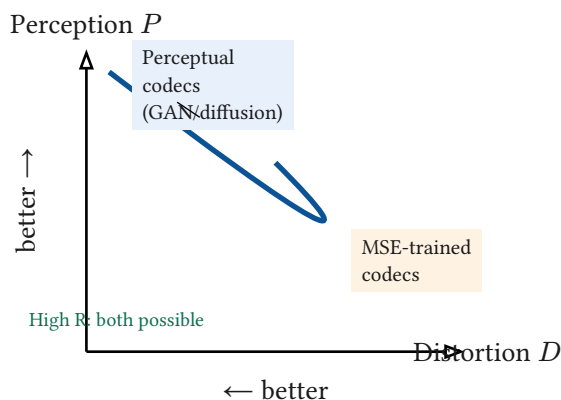


Figure 11: The rate–distortion–perception surface. At high bitrates, both low distortion and high perceptual quality are achievable. At low bitrates, improving perception requires accepting more distortion, and vice versa.

3.3 HiFiC: High-Fidelity Generative Image Compression

The breakthrough paper that moved generative compression from theory to a working system was “**High-Fidelity Generative Image Compression**”, published at NeurIPS 2020 by Fabian Mentzer, George Toderici, Michael Tschannen, and Esa Agustsson (all at Google). Its nickname, HiFiC, is now a landmark in the field.

3.3.1 The Core Idea: Teaching the Decoder to Fool a Critic

HiFiC’s key insight is that if you want the decoder to produce realistic-looking images, you should explicitly train it against a **critic that knows what real images look like**. In machine learning, this critic is called a **discriminator**, and the training setup is called a **generative adversarial network (GAN)** — introduced in Chapter 56.

Here is how the training loop works in HiFiC:

1. **The encoder** takes an image x and produces a compressed latent code $\hat{y}$ (quantized, just like Ballé’s models from Chapter 57).
2. **The decoder** (also called the generator in GAN terminology) takes $\hat{y}$ and produces a reconstruction $\hat{x}$. At first, $\hat{x}$ looks blurry — the decoder knows nothing about realistic textures yet.
3. **The discriminator** is shown two kinds of images: real photographs from the training set, and the decoder’s reconstructions. Its job is to label each one as “real” or “fake.”
4. The training loss has three parts, all minimized jointly:
 - **Rate term**: how many bits does the code $\hat{y}$ take? (Same as before — minimize bitrate.)
 - **Distortion term**: how different is $\hat{x}$ from x in pixel space? (A small MSE penalty to keep it tethered to reality.)
 - **GAN term**: how well does the decoder fool the discriminator into calling its output “real”?

Over thousands of training steps, the decoder learns that blurry averages fool nobody — the discriminator immediately flags them as fake. It is forced to invent convincing texture, or the GAN loss penalizes it. The distortion term keeps it from inventing textures that are **completely** wrong for the scene. The result is a decoder that hallucinates plausible detail with remarkable accuracy.

HiFiC (High-Fidelity Generative Image Compression)

2020

Authors	Fabian Mentzer, George Toderici, Michael Tschannen, Esa Agustsson (Google)
Aim	Lossy image compression optimizing perceptual quality via GAN training, achieving high visual fidelity at bitrates where MSE codecs produce blur.
Complexity	Encoder: ~0.5–1 s per image (GPU). Decoder: ~0.1–0.5 s per image (GPU). Training: days on many GPUs. Significantly slower than classical codecs.
Strengths	Dramatically better perceptual quality than any MSE codec at low bitrates; human studies show it is preferred over BPG at half the bitrate; architecture is flexible and extends to higher resolutions.
Weaknesses	Pixels are not faithfully reproduced — details are invented. Unacceptable for forensic, medical, or archival uses. Slower than classical decoders. Training is expensive and brittle.
Superseded by	Diffusion-based decoders (PerCo, StableCodec) achieve even higher realism at ultra-low bitrates. OneDC (2025) offers 40%+ bitrate reduction over prior diffusion codecs at 20x faster decoding.

HiFiC builds on the Ballé hyperprior architecture (Chapter 57) for the encoder/entropy model. The decoder is augmented with a GAN generator head. The discriminator uses a PatchGAN architecture that classifies overlapping image patches as real or fake, which is more stable than classifying whole images and better at catching texture inconsistencies.

The training loss at each step is:

$$\mathcal{L} = R(\hat{y}) + \lambda_{\text{dist}} \cdot D(x, \hat{x}) + \lambda_{\text{GAN}} \cdot \mathcal{L}_{\text{GAN}}(\hat{x})$$

where $\mathcal{L}_{\text{GAN}}$ is the hinge loss on the discriminator, and $\lambda_{\text{dist}}, \lambda_{\text{GAN}}$ are hyperparameters controlling the distortion/perception balance.

3.3.2 What the Human Studies Found

The NeurIPS 2020 paper included careful human preference studies — arguably the most important part of the work, because perceptual quality is fundamentally a human judgment.

The key result: **HiFiC at 0.237 bits per pixel (bpp) was preferred by human raters over BPG (a state-of-the-art classical codec) at 0.504 bpp**. BPG was using more than twice the bits, but people liked HiFiC’s output better. Against MSE-optimized learned codecs, HiFiC needed only about 60% of the bits to be equally preferred.

At the lowest tested bitrates (around 0.14 bpp), the comparison was not even close. BPG produced severe blocking artifacts. MSE codecs produced wax-museum skin. HiFiC produced faces that raters frequently described as “photographic,” even though they had never existed as actual photographs.

PY GO FURTHER · PYTHON — Lambda Expressions and Default Arguments *skip if fluent*

In the code below, you will see `lambda: ...` — a way to write a tiny, nameless function inline. You will also see **default argument values** in function definitions like `def foo(x, lam=0.01)`, which means `lam` is optional; the caller can pass a value or leave it at the default.

```
# A lambda is a one-line function with no name.
square = lambda x: x * x
print(square(5))    # 25

# A function with a default argument:
def rate_distortion_loss(rate: float, distortion: float, lam:
float = 0.01) -> float:
    return rate + lam * distortion
```

```

print(rate_distortion_loss(2.3, 140.0))           # uses lam=0.01
→ 3.7
print(rate_distortion_loss(2.3, 140.0, lam=1.0)) # uses lam=1.0
→ 142.3

```

The same pattern appears in neural compression training: `lam` (often written λ in papers) controls how much you care about distortion versus bitrate.

Let's look at a simplified Python sketch of the HiFiC loss computation, to make the three-part objective concrete:

```

# Conceptual sketch – not a full training loop, just the loss
# structure.
# Real implementations use PyTorch autograd; this shows the idea.

def hifc_loss(
    bits_per_pixel: float,      # from the entropy model (rate term)
    mse: float,                 # pixel-level distortion
    gan_loss: float,            # discriminator's verdict on realism
    lam_dist: float = 0.075,    # weight on distortion (small = more
    perceptual)
    lam_gan: float = 1.0,       # weight on GAN term
) -> float:
    """
    Compute the combined HiFiC training loss.
    Lower is better for each component.
    """
    rate_term = bits_per_pixel
    dist_term = lam_dist * mse
    perc_term = lam_gan * gan_loss
    return rate_term + dist_term + perc_term

# Example numbers from a mid-training batch:
loss = hifc_loss(
    bits_per_pixel=0.35,
    mse=180.0,
    gan_loss=0.42,
)
print(f"Total loss: {loss:.4f}")
# Total loss: 0.3500 + 13.5000 + 0.4200 = 14.2700

```

Notice that `lam_dist` is very small (0.075), meaning the decoder is NOT being trained to be pixel-perfect. The `lam_gan` term dominates realism. Tuning these two numbers is how you slide along the perception–distortion tradeoff curve.

3.3.3 The Discriminator Architecture in Detail

HiFiC uses what is called a **patch discriminator** (sometimes called PatchGAN). Instead of looking at an entire image and saying “real or fake?”, the discriminator examines overlapping 70×70 -pixel patches and classifies each patch. This has two advantages.

First, textures and fine details live at the patch scale — asking about a whole image can miss subtle texture artifacts. Second, training on patches provides many more training examples per image, which stabilizes the GAN training.

The discriminator is a small convolutional network. During training, it is updated in an **alternating** fashion: first update the discriminator to better distinguish real patches from fake ones, then update the encoder-decoder to better fool the updated discriminator. This adversarial tug-of-war is what forces the decoder to produce increasingly convincing outputs.

Checkpoint. Why doesn’t HiFiC simply use a much higher MSE weight (`lam_dist`) to keep reconstructions closer to the original pixels?⁴

3.4 PerCo: Diffusion Decoders for Ultra-Low Bitrates

By 2022, researchers had noticed that GAN-trained decoders, while impressive, had a ceiling. At very low bitrates (below about 0.05 bpp), the compressed code contained so little information that even the best GAN decoder had little to anchor on. Reconstructions sometimes lost semantic structure entirely — a face might keep its texture but lose the shape of the nose.

The new idea: replace the GAN decoder with a **diffusion model** — the same architecture that had just transformed image generation. Diffusion models (introduced in Chapter 56) generate images by starting from random noise and iteratively removing that noise, guided by a conditioning signal. For compression, the conditioning signal is the compressed code.

PerCo (Towards Image Compression with Perfect Realism at Ultra-Low Bitrates, ICLR 2024) by Careil, Muckley, Verbeek, and Lathuilière is the landmark paper for this approach.

3.4.1 How PerCo Works: Semantic Code + Diffusion Prior

PerCo operates in two steps. First, an encoder compresses the input image into a tiny code at extremely low bitrates — as few as 0.003 bpp. This code cannot possibly encode every pixel; instead it encodes the **semantic gist** of the image: the broad layout of shapes, major colors, coarse identity information.

The code is produced using **vector quantization** of VAE features — the image is passed through a variational autoencoder (Chapter 56), the resulting latent is vector-quantized

⁴**Checkpoint answer.** A high MSE weight drags the decoder back toward blurry averaging — the behavior we are trying to escape. The whole point is that low MSE weight forces the decoder to fill in missing detail using generative priors rather than pixel averages. Too much MSE weight and you are back to a conventional MSE-trained codec with a small GAN nudge that barely helps.

(snapped to a small codebook of learned discrete symbols), and the codebook indices are entropy-coded and transmitted.

At the decoder, PerCo conditions a pre-trained **text-to-image diffusion model** (specifically a version of Stable Diffusion) on the received code. The diffusion model then **generates** a full-resolution image that matches the semantic code. In multiple denoising steps it fills in all the texture, lighting, fine detail — everything the code is too small to specify.

PerCo (Perceptual Compression via Conditioned Diffusion)		2024
Authors	Careil, Muckley, Verbeek, Lathuilière (Meta AI / Inria)	
Aim	Ultra-low-bitrate image compression (0.003–0.1 bpp) using a semantic encoder and a conditioned pre-trained diffusion model as decoder.	
Complexity	Encoder: moderate (VAE encode + VQ). Decoder: slow — requires 20–50 denoising steps, each a forward pass through a large diffusion model. Orders of magnitude slower than any classical or GAN-based codec.	
Strengths	Achieves plausible reconstructions at bitrates where every other codec fails completely. Leverages powerful generative priors from internet-scale pretraining. Semantically coherent even at 0.003 bpp.	
Weaknesses	Very slow decoding. Reconstructions can differ substantially from the original in pixel content, especially at the lowest bitrates. Fine detail is generated, not preserved. Requires a large pre-trained diffusion model at the decoder.	
Superseded by	StableCodec (ICCV 2025) achieves similar perceptual quality with one-step decoding, approximately 20x faster. OneDC (NeurIPS 2025) provides 40%+ further bitrate reduction.	
The key technical innovation is using the compressed code as a conditioning signal for guided diffusion, rather than as a direct input to a deterministic decoder. This gives the diffusion model’s learned prior full freedom to hallucinate texture and detail, anchored only by the semantic content of the code. The result is a codec that does not “uncompress” an image so much as “regenerate” it.		

3.4.2 The Denoising Process as a Decoder

To see why diffusion decoding is so powerful at ultra-low bitrates, think about what information is in a 0.003 bpp code. At that rate, a 512×512 image is coded in roughly **800 bits** — about 100 bytes. That is enough to store, say, the rough layout of a face: “eyes here, mouth here, hair color approximately brown, skin tone approximately medium.” It cannot store the exact shape of every eyelash.

The diffusion decoder starts with pure noise and asks, in each denoising step: “Given this noise and the code saying ‘brown-haired face with eyes here,’ what should the image look like?” After 20–50 steps, it has assembled a photo-realistic face that matches the layout code but has invented every fine detail from the model’s prior over faces.

This is genuinely impressive — but it means the **specific person’s eyelashes** were not preserved. What was preserved is “someone who looks like this person.” At 0.003 bpp, that may be the best any system can do.

Σ GO FURTHER · THE MATHS — Vector Quantization

skip if familiar

Vector quantization (VQ) is the multi-dimensional version of rounding. We met it in Chapter 39 (Quantization) as the LBG/k-means idea; here it is the engine that turns a continuous latent into a few transmittable integers. You have a **codebook** of K prototype vectors $e_1, e_2, \dots, e_K$ (learned during training). When you receive a new vector z (a chunk of latent features), you find the nearest prototype:

$$\hat{z} = e_k \text{ where } k = \arg \min_j \|z - e_j\|$$

Recall. $\arg \min_j$ (Chapter 39) means “the index j that makes the following expression smallest” — we keep the **position** of the nearest codeword, not the distance itself. $\|\cdot\|$ is the vector length (Chapter 12), so $\|z - e_j\|$ is how far apart the two vectors are.

and transmit only the index k , which takes $\log_2 K$ bits. The decoder looks up e_k and uses it in place of z .

Example: codebook size $K = 1024$, latent dimension = 256. Each code takes $\log_2 1024 = 10$ bits, instead of $256 \times 32 = 8192$ bits for a 32-bit float vector. That’s an 819× compression of the latent — at the cost of some approximation.

VQ is heavily used in modern image and audio codecs (VQ-VAE, EnCodec) because it produces naturally discrete codes that can be entropy-coded and transmitted exactly.

```
# Minimal illustration of vector quantization (NumPy sketch).
# Real use requires learning the codebook via gradient descent; this
# shows lookup.
```

```
import numpy as np
from numpy.typing import NDArray

def vq_encode(z: NDArray, codebook: NDArray) -> int:
    """
    Find the nearest codebook entry to latent vector z.
    Returns codebook index k (the 'code').
    codebook shape: (K, D) where K = number of entries, D =
    dimension.
    """
```

```

# Compute squared Euclidean distance to every entry.
diffs = codebook - z[np.newaxis, :] # shape: (K, D)
dists = np.sum(diffs ** 2, axis=1)   # shape: (K,)
return int(np.argmin(dists))         # index of closest entry

def vq_decode(k: int, codebook: NDArray) -> NDArray:
    """Reconstruct the latent from codebook index k."""
    return codebook[k]

# Tiny example: 4 codebook entries of dimension 3.
rng = np.random.default_rng(42)
codebook = rng.standard_normal((4, 3))

z = np.array([0.1, 0.9, -0.3]) # some latent vector to encode
k = vq_encode(z, codebook)
z_hat = vq_decode(k, codebook)

print(f"Input z:           {z}")
print(f"Codebook index: {k}")
print(f"Reconstruction: {z_hat}")
print(f"Error:             {np.linalg.norm(z - z_hat):.4f}")

```

3.4.3 A Worked Bitrate Example

Let's trace through the numbers to feel what ultra-low bitrate really means.

Image: 512×512 pixels, RGB. Raw size: $512 \times 512 \times 3 \times 8 = 6\{, \}291\{, \}456$ bits = 786 KB.

PerCo target: 0.003 bpp. That's $0.003 \times 512 \times 512 = 786$ bits — about **98 bytes**. The compressed file is 8,000× smaller than the raw image.

To put 98 bytes in perspective: the JPEG header alone is larger than that. A single line of this book's text is larger than that. Yet PerCo can decode a photo-realistic face from 98 bytes. The trick is that those 98 bytes say “face, rough position of features, approximate identity,” and the diffusion model's massive pre-trained knowledge of what faces look like fills in the rest.

For comparison: conventional JPEG at this bitrate produces a 32×32 thumbnail stretched to 512×512 — indistinguishable mush. BPG (H.265-based) fails similarly. PerCo produces something your eye reads as a plausible portrait.

3.5 The 2025 Push: One-Step Distillation

The PerCo system's biggest practical problem was speed. A multi-step diffusion decoder might require 20–50 forward passes through a large neural network to decode one image. On a consumer GPU, that could take 5–15 seconds per image. For streaming or real-time applications, that is untenable.

The 2025 research sprint focused on **distillation**: training a small, fast “student” network to mimic the output of the slow multi-step diffusion “teacher” in a **single forward pass**.

3.5.1 StableCodec (ICCV 2025)

StableCodec: Taming One-Step Diffusion for Extreme Image Compression (Zhang et al., ICCV 2025) is the most polished of the 2025 one-step systems. Its architecture:

1. A **latent codec** compresses the image into a noisy latent representation (not quite the full latent of a diffusion model — a careful partial encoding that preserves enough for one-step decoding).
2. A **one-step diffusion decoder**, distilled from a multi-step teacher, reconstructs the full image in a single forward pass.
3. A **fidelity module** injects high-frequency detail hints from the code to keep the output anchored to the original rather than drifting into pure hallucination.

StableCodec reports achieving compression to as low as 0.005 bpp with photo-realistic results, while cutting decoding time by approximately 20× compared to multi-step methods. Encoding speed is also improved because the latent codec is simpler than a full VAE.

OneDC (One-step Diffusion-based image Codec, arXiv 2505.16687, 2025) takes a complementary approach: it uses a **semantic distillation mechanism** that transfers knowledge from a pre-trained generative tokenizer into the hyperprior codec, combined with hybrid pixel- and latent-domain optimization. OneDC reports over 40% bitrate reduction versus previous multi-step diffusion codecs while achieving 20× faster decoding.

HISTORICAL NOTE

The speed problem for generative codecs has a parallel in classical codec history: the early H.264 encoders took many seconds per frame; hardware acceleration eventually made them real-time. Neural codec advocates argue the same will happen here — dedicated neural processing units (NPUs) will accelerate diffusion decoding just as video DSPs accelerated motion compensation. As of 2026, this is still a hope more than a reality for most deployment targets, but smartphone NPU capabilities are improving rapidly.

3.5.2 CoD-Lite: Toward Real-Time Diffusion Codecs

Even more aggressive is the CoD-Lite direction (Compression with Diffusion, Lite variant, 2026), which explores quantized, pruned diffusion decoders that can run at near-real-time rates on modern mobile NPUs. The 2026 papers in this space suggest that the gap between perceptual quality and acceptable latency is narrowing, though real-time decoding of 4K images via diffusion remains out of reach on general hardware as of mid-2026.

3.6 The Hallucinated Detail Debate

Every engineer who has seen a perceptual codec demo has the same reaction: “That’s impressive — but is it honest?” This question is not merely philosophical. It has sharp practical consequences.

Myth. *Perceptual codecs are just better versions of regular lossy codecs — same idea, better quality.*

Reality. There is a fundamental difference. A lossy JPEG loses detail by omission: it blurs or rounds things that were there. A GAN or diffusion codec loses detail by **substitution**: it replaces what was there with something it invented. An MSE codec never adds a freckle that was not there; a perceptual codec might. This distinction matters enormously in some domains.

3.6.1 Where Hallucination Is Acceptable

For many everyday uses, the hallucination is harmless or even desirable.

Social media and messaging: Nobody checks whether every hair strand in a compressed selfie matches the original. The face looks good, the mood is conveyed, the memory is captured. A 20× smaller file that looks better to the eye is an obvious win.

Video streaming at low bandwidth: A diffusion-decoded movie frame that invents correct-looking background textures causes no harm if it looks cinematic. The viewer never had access to the ground truth anyway.

Satellite and aerial imagery for public view: A perceptual codec that renders plausible foliage and building facades at low bitrates is fine for orientational purposes — checking which side of a city you’re looking at.

3.6.2 Where Hallucination Is Dangerous

Medical imaging: This is the clearest danger. A mammogram compressed with a GAN codec might show a plausible-looking nodule that was invented. Or erase a real one. Either outcome could affect a clinical decision. Medical imaging standards require lossless or tightly bounded lossy compression precisely to prevent this. Perceptual codecs are contraindicated here unless their reconstruction can be shown to have bounded, studied effects on clinical markers.

Forensic and legal evidence: A surveillance camera image of a crime scene cannot be perceptually compressed if it is to be admitted as evidence. The chain of custody requires that the image represents what was captured, not what a neural network imagined was plausible.

Satellite intelligence: Counting vehicles in a parking lot from a satellite image requires that the vehicles in the image are real. A perceptual codec that invents a plausible-looking parking situation (with slightly different vehicle counts) is useless or worse.

News photojournalism: Professional standards require that photographic evidence of events not be fabricated. A perceptual codec applied in an archival pipeline could subtly alter the documentary record.

Archival and scientific data: Astronomers, climatologists, and archaeologists need their data reproduced faithfully. A generative model’s idea of what a galaxy “should” look like is not a substitute for the photons that actually arrived at the telescope.

PITFALL

The hallucination danger is not obvious to end users. A GAN-compressed image looks **more** sharp and detailed than the original at the same bitrate, not less. Naive users may actually prefer the perceptual codec’s output without realizing it has been altered at the pixel level. Systems deploying perceptual codecs must clearly communicate this to users, and must disable them by default in contexts where pixel fidelity matters.

3.6.3 The Hallucination Index

Some researchers have proposed a **hallucination index** — a number measuring how much a perceptual codec’s output deviates from what the original image would have looked like under an ideal lossless system. Computing it requires access to the original (to compare), which limits its use in deployed systems. But it is a useful framework: any deployment of a perceptual codec should include this kind of audit, not just a PSNR or LPIPS score.

A 2024–2025 trend in the research community is using confidence maps: the decoder also outputs a per-pixel uncertainty estimate, and regions of high hallucination probability are flagged. This allows downstream systems (or human reviewers) to know which parts of the image were faithfully reproduced and which were generated.

Aside. The word “hallucination” for generative neural networks is borrowed from AI-assistant research, where it describes a language model confidently stating something false. In compression, the term is slightly more nuanced: the generated detail is usually **plausible** (a fair sample from the space of natural images given the code), not random noise. But it is still not the truth.

3.7 JPEG AI: Standardizing the Neural Codec

While academia produced impressive research systems, the practical adoption of neural image codecs in real products requires standardization — an agreed-upon bitstream format so that any conforming encoder and decoder interoperate. This is what JPEG did for DCT-based compression in 1992, and what the industry needed for learned codecs.

JPEG AI (ISO/IEC 6048) is the first international standard for end-to-end learning-based image coding. Its history:

- **2022:** JPEG Committee published a Call for Proposals for JPEG AI.
- **2023:** Working Draft and Committee Draft completed.
- **October 2024:** First edition published as a standard.
- **February 2025:** At its 106th meeting, the JPEG Committee approved publication as a full ISO/IEC/ITU International Standard.

The standard specifies a bitstream format, conformance tests, and a reference software decoder, but — unlike JPEG — it does not prescribe the specific neural network architecture of the encoder. Encoders can use any architecture that produces a conforming bitstream; only the decoder profile is standardized. This design acknowledges that neural codec architectures will keep improving.

JPEG AI’s test model (version 4.3) achieves approximately 28–30% coding gains over VVC in all-intra mode on standard benchmarks. Crucially, the standard also supports **machine vision tasks** directly from the compressed representation: the same bitstream can be decoded for human viewing or analyzed for object detection and segmentation without a full decode pass. This “coding for machines” mode is a significant departure from all previous image standards.

SCOREBOARD — Compression results on the shared 512×512 test image (estimated)

Technique	Bytes	Ratio	Notes
Uncompressed RGB	786,432	1.0×	Raw bytes
JPEG (quality 50)	28,000	28×	Classical DCT, some blockiness
AVIF (AV1-intra)	18,000	44×	Modern classical, Chapter 45
BPG (HEVC-intra)	16,500	48×	Best classical at time of HiFiC
Ballé hyperprior (Ch57)	14,200	55×	MSE-optimized learned codec
HiFiC (0.237 bpp)	9,600	82×	Perceptual; preferred over BPG at 0.504 bpp
PerCo (0.003 bpp)	122	6,446×	Ultra-low; detail invented by diffusion
StableCodec (0.005 bpp)	200	3,932×	One-step diffusion; 20x faster than PerCo

3.8 A Python Sketch of Perception-Aware Training

Let us look at a more complete Python sketch that shows how the three-loss training objective is assembled. This is conceptual — a real implementation uses PyTorch with autograd and GPU tensors — but the structure is identical.

```
"""
perception_training_sketch.py

Conceptual sketch of the HiFiC / perceptual codec training loop.
All tensors are plain Python floats for readability.
Real implementation: PyTorch, GPU, proper autograd.
"""

from dataclasses import dataclass
from typing import Protocol
```

```

import math

@dataclass
class TrainingConfig:
    lam_rate: float = 1.0      # weight on bits-per-pixel
    lam_dist: float = 0.075    # weight on pixel distortion
    lam_gan: float = 1.0      # weight on GAN (perceptual) loss

class CompressionModel(Protocol):
    def encode(self, x: list[float]) -> tuple[list[int], float]:
        """Quantize and entropy-code x; return code and bits-per-
        pixel."""
        ...
    def decode(self, code: list[int]) -> list[float]:
        """Reconstruct pixels from code."""
        ...

class Discriminator(Protocol):
    def score(self, image: list[float]) -> float:
        """Return probability that image is real (higher = more
        real)."""
        ...

def mse(original: list[float], reconstruction: list[float]) ->
float:
    """Mean squared error between two equal-length pixel lists."""
    n = len(original)
    return sum((a - b) ** 2 for a, b in zip(original,
reconstruction)) / n

def hinge_gan_loss(disc_real_score: float, disc_fake_score: float) -
> float:
    """
    Hinge GAN loss for generator (compression decoder).
    Generator wants disc_fake_score to be high (fooling
    discriminator).
    Loss = max(0, 1 - disc_fake_score): zero once discriminator is
    fooled.
    """
    return max(0.0, 1.0 - disc_fake_score)

def total_loss(
    bpp: float,          # rate: bits per pixel
    pixel_mse: float,    # distortion
    gan_loss: float,     # perception
    cfg: TrainingConfig,
) -> float:

```

```

    """Combined objective: rate + lambda_dist * distortion +
    lambda_gan * GAN."""
    return (
        cfg.lam_rate * bpp
        + cfg.lam_dist * pixel_mse
        + cfg.lam_gan * gan_loss
    )

# --- Conceptual training step (what PyTorch's optimizer would
# compute) -----
def training_step_sketch(
    original_pixels: list[float],
    model: CompressionModel,
    disc: Discriminator,
    cfg: TrainingConfig,
) -> dict[str, float]:
    """
    One conceptual training step.
    Returns a dict of loss components for logging.
    """

    # 1. Compress and decompress the image.
    code, bpp = model.encode(original_pixels)
    reconstructed = model.decode(code)

    # 2. Compute distortion.
    pixel_error = mse(original_pixels, reconstructed)

    # 3. Ask discriminator how real the reconstruction looks.
    fake_score = disc.score(reconstructed)
    real_score = disc.score(original_pixels)
    gan = hinge_gan_loss(real_score, fake_score)

    # 4. Combine into total loss.
    loss = total_loss(bpp, pixel_error, gan, cfg)

    return {
        "loss": loss,
        "bpp": bpp,
        "mse": pixel_error,
        "gan": gan,
        "fake_score": fake_score,
    }

```

The most important thing to read in this code: `lam_dist = 0.075` is very small. If we set it to 10.0 instead, the training would optimize almost entirely for pixel accuracy — and we'd get a standard MSE codec with a GAN term that barely matters. The small distortion weight is what forces the perceptual magic to emerge.

3.9 How Diffusion Decoders Are Conditioned: A Closer Look

The mechanism by which a compressed code **guides** a diffusion decoder deserves a closer look, because it is not obvious and it differs between systems.

In standard diffusion generation (with no compression), the model starts from pure noise $z_T \sim \mathcal{N}(0, I)$ and takes T denoising steps, each guided by a text prompt or class label. For compression, we want to guide it by the **compressed code** instead.

Recall. $z_T \sim \mathcal{N}(0, I)$ just says “ z_T is random Gaussian noise.” The symbol $\sim$ reads “is distributed as”; $\mathcal{N}$ is the **normal** (bell-curve) distribution from Chapter 9; 0 is its mean and I (the identity matrix, Chapter 12) says every coordinate is independent with variance 1. In plain terms: each number in z_T is an independent draw from a standard bell curve — pure static.

NOTE

Three pieces of neural-network plumbing appear below for the first time. A **U-Net** is the hourglass-shaped network diffusion models use to predict noise: it shrinks the image down to a small summary and back up again, with shortcut links across the waist so fine detail is not lost. **Cross-attention** is the mechanism by which a guiding signal (normally a text prompt, here the compressed code) is injected into the U-Net: at each layer, the network “looks at” the conditioning vectors and lets them steer what it draws. **DDIM** (Denoising Diffusion Implicit Models) is simply a **deterministic** recipe for the denoising steps — given the same starting noise and the same code, it always produces the same image, which a codec needs.

PerCo does this by:

1. Projecting the quantized code into an embedding that matches the size of the cross-attention conditioning vectors the diffusion model expects.
2. Replacing the text embedding with the code embedding in every cross-attention layer of the U-Net denoiser.
3. Running the standard DDIM (Denoising Diffusion Implicit Models) sampling loop, which is deterministic given the code — the same code always produces the same reconstruction, which is important for a codec (you need a defined decoding).

```
# Pseudo-code for PerCo-style conditioned diffusion decoding.
# Real implementation uses HuggingFace Diffusers or similar.

def perco_decode(
    code_indices: list[int],          # VQ codebook indices from
    encoder                                #
    codebook: list[list[float]],       # K×D learned codebook
    diffusion_model,                   # pre-trained stable diffusion U-
    Net                                #
    n_steps: int = 50,                # denoising steps
) -> list[float]:                   # reconstructed pixel values
```

```

"""
Decode a compressed image using conditioned diffusion.
The code provides semantic guidance; the diffusion model fills
in detail.
"""

# 1. Convert code indices to embedding vectors.
code_embedding = [codebook[i] for i in code_indices] # list of
D-dim vectors

# 2. Project to cross-attention dimension (learned linear
projection).
conditioning = project_to_attention_dim(code_embedding) # shape
matches U-Net

# 3. Start from pure Gaussian noise in the latent space.
latent = sample_gaussian_noise(shape=(4, 64, 64)) # U-Net
latent size

# 4. Run deterministic DDIM denoising, conditioned on code.
for t in range(n_steps - 1, -1, -1):
    noise_pred = diffusion_model.predict_noise(latent, t,
conditioning)
    latent = ddim_step(latent, noise_pred, t) # one denoising
step

# 5. Decode latent to pixels via the VAE decoder.
pixels = diffusion_model.vae_decode(latent)
return pixels

```

The `ddim_step` function is deterministic — given the same noise prediction and timestep, it always produces the same next latent. This ensures that the same compressed code always decodes to the same image, which is a requirement for any compression system.

PY GO FURTHER · PYTHON — Dataclasses

skip if fluent

A **dataclass** in Python is a convenient way to define a class that mostly holds data. You annotate the fields with types, and Python automatically generates `__init__`, `__repr__`, and other methods for you.

```

from dataclasses import dataclass

@dataclass
class CompressionResult:
    bpp: float          # bits per pixel
    psnr_db: float       # peak signal-to-noise ratio

```

```

    lpips: float          # learned perceptual similarity (lower =
more similar)
    method: str = "unknown"

r = CompressionResult(bpp=0.237, psnr_db=28.4, lpips=0.18,
method="HiFiC")
print(r)
# CompressionResult(bpp=0.237, psnr_db=28.4, lpips=0.18,
method='HiFiC')
print(r.bpp)    # 0.237

```

Dataclasses are widely used in compression research code to pass hyperparameters and results cleanly. The `TrainingConfig` in the code above is a dataclass.

3.10 Measuring Perceptual Quality: LPIPS and FID

Because perceptual quality is not measurable by PSNR, perceptual codecs use different metrics.

LPIPS (Learned Perceptual Image Patch Similarity) was introduced by Zhang et al. in 2018. It measures how different two images look to a **VGG or AlexNet network** — a network trained on image classification that has learned, along the way, which visual features matter. Two images that look similar to humans tend to have similar intermediate neural activations. LPIPS is the average distance between those activations over image patches. Lower LPIPS = more perceptually similar.

FID (Fréchet Inception Distance) is a distribution-level metric, introduced by Heusel et al. in 2017. Instead of comparing a specific reconstruction to a specific original, FID measures how much the **distribution** of reconstructed images differs from the **distribution** of real images. It computes statistics (mean and covariance) of the InceptionV3 feature vectors of both sets and measures the Fréchet distance between them. Lower FID = more realistic distribution. FID is used to evaluate generative codecs across a test set, not per image.

Σ GO FURTHER · THE MATHS — The Fréchet Distance between Gaussians *skip if familiar*

If you have two distributions p and q , and they are both **Gaussian** (bell-curve shaped), the **Fréchet distance** between them has a closed-form formula. Let μ_p, Σ_p be the mean vector and covariance matrix of p , and similarly for q . Then:

$$d_F^2 = \|\mu_p - \mu_q\|^2 + \text{Tr}\left(\Sigma_p + \Sigma_q - 2(\Sigma_p \Sigma_q)^{\frac{1}{2}}\right)$$

where Tr is the **trace** (sum of diagonal elements of a matrix) and $(\Sigma_p \Sigma_q)^{\frac{1}{2}}$ is the matrix square root. For FID, μ and Σ are estimated from the feature vectors of the two image sets. The formula penalizes both a **shift in the mean** (the generated images have systematically different features) and a **mismatch in spread** (the generated distribution is too narrow or too wide).

Tiny example: If p is a 1D Gaussian with mean 0, variance 1, and q has mean 2, variance 1, then $d_F^2 = (0 - 2)^2 + (1 + 1 - 2\sqrt{1 \cdot 1}) = 4 + 0 = 4$, so $d_F = 2$.

FID values for image compression: values near 0 mean the reconstruction distribution is nearly indistinguishable from real. Classical codecs at 0.1 bpp might have FID > 100. HiFiC achieves FID < 20. Best diffusion codecs approach FID < 5.

The key difference between LPIPS and FID:

- LPIPS is **reference-based**: it compares each reconstruction to its specific original.
- FID is **distribution-based**: it cares whether the set of all reconstructions looks like the set of all natural images.

A codec could theoretically have bad LPIPS (each reconstruction differs from its original) but good FID (the reconstructions collectively look natural). Generative codecs often have this property: your specific face is changed, but the set of all compressed faces looks photorealistic.

```
# Conceptual illustration of LPIPS-style perceptual similarity.
# Real LPIPS uses deep VGG features; this uses a toy 1D signal for
# illustration.

def toy_perceptual_similarity(original: list[float], reconstructed:
    list[float]) -> float:
    """
    Compute a toy perceptual similarity score.
    Real LPIPS extracts features from multiple VGG/AlexNet layers
    and computes normalized distances in that feature space.
    Lower value = more similar (better reconstruction).
    """
    # Step 1: Simulate extracting "features" (here: running averages
    # over windows).
    def extract_features(signal: list[float], window: int = 4) ->
        list[float]:
        feats = []
        for i in range(0, len(signal) - window + 1, window):
            feats.append(sum(signal[i:i+window]) / window)
        return feats

    # Step 2: Compare features of original and reconstruction.
    f_orig = extract_features(original)
```

```

f_recon = extract_features(reconstructed)

# Step 3: Average L2 distance in feature space.
diffs = [(a - b) ** 2 for a, b in zip(f_orig, f_recon)]
return (sum(diffs) / len(diffs)) ** 0.5

# Example: compare a clean signal to a blurry and a noisy version.
signal      = [1.0, 3.0, 2.0, 4.0, 1.0, 2.0, 3.0, 2.0]
blurry      = [2.0, 2.5, 2.5, 3.0, 2.0, 2.5, 2.5, 2.5] # MSE-codec
output
perceptual= [1.1, 2.9, 2.1, 3.9, 1.1, 2.1, 2.9, 2.1] # GAN codec
output

print(f"Blurry LIPS:      {toy_perceptual_similarity(signal,
blurry):.3f}")
print(f"Perceptual LIPS: {toy_perceptual_similarity(signal,
perceptual):.3f}")
# Perceptual version should score better (lower) despite adding
texture.

```

3.11 The Gap Between Research and Deployment

By mid-2026, the gap between what perceptual codecs demonstrate in papers and what appears in deployed products remains wide. Understanding why is important for anyone who wants to work in this field.

3.11.1 Speed

Even with one-step distillation, diffusion-based decoders are order-of-magnitude slower than classical codecs. Decoding a 4K frame with H.265 on a modern CPU takes a few milliseconds. Decoding with a neural diffusion codec takes seconds to minutes without specialized hardware. GAN-based decoders (like HiFiC) are faster — roughly 100ms per megapixel on a GPU — but still far from the microsecond-level decode that hardware video decoders achieve.

3.11.2 Hardware

Classical codecs have dedicated silicon on every smartphone, laptop, and streaming device. Neural codec inference requires a GPU or a purpose-built NPU. The NPU in a 2025-era smartphone can accelerate some neural workloads, but the specific operations in diffusion decoders (attention layers, normalization, iterative sampling) are not yet as efficiently accelerated as convolutions.

3.11.3 Standardization and Interoperability

JPEG AI becoming an ISO standard (2025) is a critical first step, but standard ≠ deployment. It took HEVC (H.265) about five years from standardization (2013) to widespread use in mobile cameras. JPEG AI faces the same ramp. Moreover, JPEG AI defines only still-image compression; neural video codec standardization is further behind.

3.11.4 Floating-Point Reproducibility

Neural codecs face a subtle engineering problem that classical codecs do not: **floating-point arithmetic is not identical across hardware**. The same neural network, run on an NVIDIA GPU versus an AMD GPU versus Apple Silicon, may produce slightly different intermediate values — which can mean different quantization decisions, which can corrupt the decoded bitstream. Classical codecs use integer arithmetic that is perfectly reproducible everywhere.

This is not an unsolvable problem — it requires careful engineering of the network to avoid operations sensitive to floating-point order, and some systems use fixed-point inference. But it adds significant engineering cost and is a real barrier to bit-exact interoperability.

Checkpoint. A streaming platform says it wants to switch to a perceptual codec to save bandwidth. What are the three most important questions you should ask before recommending it?⁵

3.12 Looking Forward: Compression as Conditional Generation

The deep principle underlying all of Chapter 58 is this: at low enough bitrates, **decompression is an act of generation**. The decoder is not “recovering” what was there; it is using the code as a partial specification and generating a plausible completion.

This reframing has philosophical consequences. The “decoder” of a diffusion codec is in many ways more like a text-to-image model that has been given a very specific prompt than like a traditional “inverse transform.” The code is a latent description, not a reversible transform.

It also has practical consequences. The same pre-trained diffusion backbone that powers Stable Diffusion or DALL-E can be reused as a compression decoder — you get the world’s most powerful image prior “for free” by piggybacking on models that were trained on billions of images. The compression encoder only needs to produce a code that steers this prior correctly.

This is genuinely new in the history of compression. For seventy years, compression research built custom representations: Huffman trees, DCT bases, wavelet filters, LZ dictionaries, Ballé autoencoders. All of these were designed and optimized specifically for compression. Diffusion codecs instead **borrow** from the general-purpose AI world — the decoder is a general image generator that was trained for entirely different purposes, repurposed as a codec component.

⁵**Checkpoint answer.** (1) What are the use cases — does any content require pixel fidelity (documentaries, news, archival footage)? (2) What hardware decodes the stream — can all target devices run neural inference at the required speed? (3) What is the deployment timeline — is this for a proprietary pipeline (where format compatibility is controlled) or a public bitstream (where standardization matters)?

Whether this convergence of generative AI and codec engineering continues to produce gains, or whether there are fundamental limits on how much realism can be squeezed from a tiny code, is one of the open questions that will define the field through the late 2020s.

Chapter takeaways

- The rate–distortion–perception tradeoff (Blau & Michaeli, 2019) proves that at low bitrates, pixel accuracy and perceptual realism are in fundamental tension — you cannot maximize both.
- MSE-trained codecs produce blur at low bitrates because averaging over uncertain pixels is the mathematically correct way to minimize squared error — but humans hate blur.
- HiFiC (NeurIPS 2020) solved this by adding a GAN discriminator loss that penalizes the decoder for producing any image a critic can distinguish from a real photograph.
- PerCo (ICLR 2024) pushed further: at ultra-low bitrates below 0.01 bpp, a diffusion model guided by the compressed code regenerates a plausible image, inventing all fine detail.
- StableCodec and OneDC (2025) distilled multi-step diffusion into a single forward pass, cutting decoding time by roughly 20x while preserving perceptual quality.
- Hallucinated detail is not a bug — it is a design choice. Perceptual codecs are excellent for consumer media and dangerous for forensic, medical, or archival applications.
- JPEG AI became an ISO/IEC/ITU International Standard in February 2025, the first standard for end-to-end learned image coding, with ~30% gains over VVC.
- Deployment barriers — speed, hardware, floating-point reproducibility, standardization — mean perceptual codecs remain research systems for most applications as of 2026.

3.13 Exercises

Exercise 58.1 ★

Explain in your own words why a codec trained to minimize mean squared error produces blurry images at low bitrates, even though it is “trying its best.” Use the concept of averaging over uncertain completions in your answer.

Exercise 58.2 ★

The Blau–Michaeli theorem says you cannot simultaneously minimize distortion and maximize perceptual quality at a fixed bitrate. Give a concrete scenario (two specific images) where a high-distortion codec output has better perceptual quality than a low-distortion one.

Exercise 58.3 ★★

In the HiFiC loss function $\mathcal{L} = R + \lambda_{\text{dist}} D + \lambda_{\text{GAN}} \mathcal{L}_{\text{GAN}}$, the paper uses $\lambda_{\text{dist}} = 0.075$ and $\lambda_{\text{GAN}} = 1.0$. What would happen to the reconstruction quality — in terms

of both PSNR and perceptual quality — if you swapped these values, using $\lambda_{\text{dist}} = 1.0$ and $\lambda_{\text{GAN}} = 0.075$? Justify your answer using the rate–distortion–perception tradeoff.

Exercise 58.4 ★★

PerCo encodes a 512×512 image at 0.003 bpp. (a) How many bits are in the compressed file? (b) How many bytes is that? (c) The encoder uses a codebook of size $K = 1024$ and encodes a 16×16 grid of latent vectors. How many bits does each latent vector’s index take? Does the total match (a)?

Exercise 58.5 ★★

Explain the difference between **LPIPS** and **FID** as metrics for perceptual codec quality. For which goal is each more appropriate: (a) comparing a specific reconstruction to its original, or (b) evaluating whether a codec produces natural-looking images across a test set?

Exercise 58.6 ★★★

A hospital considers using a perceptual codec to compress and archive its radiology images. Write a structured argument (pros and cons) explaining whether this is appropriate, and state your conclusion. Consider the following factors: storage cost, diagnostic accuracy, legal and regulatory requirements, and the specific behavior of diffusion-based decoders.

Exercise 58.7 ★★★

Implement a function in Python 3.14 that simulates the rate–distortion–perception tradeoff curve by computing, for each of a list of lambda values (perception weights), the “equilibrium quality” of a toy codec. Use the function signature:

```
def rdp_curve(lambdas: list[float], source_variance: float) -> list[tuple[float, float, float]]
```

where the return is a list of `(distortion, perception_loss, bits)` triples. You may use any simplified model (e.g., a Gaussian source). Write a brief docstring explaining what each lambda value represents on the tradeoff curve.

3.14 Further Reading

- Mentzer et al. (2020). **High-Fidelity Generative Image Compression**. NeurIPS 2020. [arXiv:2006.09965](https://arxiv.org/abs/2006.09965) — The HiFiC paper; required reading.
- [HiFiC Project Page](#) — Interactive comparison tool; try it before reading the math.
- Careil et al. (2024). **Towards Image Compression with Perfect Realism at Ultra-Low Bitrates**. ICLR 2024. — The PerCo paper introducing diffusion decoders for compression.
- Zhang et al. (2025). **StableCodec: Taming One-Step Diffusion for Extreme Image Compression**. ICCV 2025. — One-step distillation for fast decoding.

- OneDC (2025). **One-Step Diffusion-Based Image Compression with Semantic Distillation**. arXiv:2505.16687. — OneDC system; 40% bitrate reduction over prior diffusion codecs.
- JPEG Committee (2025). **JPEG AI becomes an International Standard**. Press Release. — Standardization milestone.
- Yang, Mandt, Theis (2023). **An Introduction to Neural Data Compression**. Foundations and Trends. — Comprehensive survey of the whole neural compression field.
- Blau & Michaeli (2019/2024). **Rate-Distortion-Perception Tradeoff**. Extended treatment. — Deeper mathematical treatment of the fundamental tradeoff.
- Zhang et al. (2018). **The Unreasonable Effectiveness of Deep Features as a Perceptual Metric**. CVPR. — The LPIPS metric paper; explains why VGG features predict human perception.

COMING UP NEXT

We have now seen how generative models — first GANs, then diffusion — transform the decoder from a deterministic inverse transform into a **conditioned image generator**. This is the most dramatic conceptual shift in compression since Shannon’s theory.

But still-image perceptual codecs are only the beginning. In Chapter 59 we turn to video: how do you handle the temporal dimension with neural codecs? The DVC family and the DCVC line replace classical motion estimation with learned optical flow and conditional entropy models. And a radically different approach — implicit neural representations — stores a video not as a sequence of frames but as the weights of a small network overfit to that particular clip. Both approaches struggle with the same deployment barriers we saw here, but both offer new ways to think about what it means to “store” moving images.

Learned Video and Implicit Neural Representations

We do not predict motion. We predict distributions, and let the network decide how to use time.

— Jizheng Xu, on the philosophy behind the DCVC series

Here is a puzzle. Everything you learned in Chapters 51–55 about classical video coding boils down to one grand idea: find what moved, encode only the difference, then entropy-code the leftovers. Motion estimation, block matching, the hybrid codec loop — all of it is a beautifully engineered machine for answering the question “what changed?” In Chapter 57 you saw how the image half of that story was rewritten by neural networks: instead of a hand-built transform, a trained autoencoder discovers its own representation, and the whole pipeline — encoder, quantizer, entropy model — is optimized together by gradient descent. Learned image codecs now beat the best classical ones.

So here is the natural question: can we do the same thing for video?

The answer turns out to be both “yes” and “it is more complicated than you think.” This chapter tells you exactly how. We will follow two very different answers to that question.

The first answer is **neural video coding**: replace the blocks of the classical hybrid codec — the optical-flow estimator, the residual coder, the entropy model — with neural networks, keep the overall predict-then-code structure, and train everything end-to-end. The landmark is DVC (2019); the dominant research line today is Microsoft’s DCVC family (2021–2025), which quietly discarded the “explicit residual” idea in favor of something information- theoretically cleaner. A parallel line — Google’s Video Compression Transformer, VCT (2022) — goes further still and removes explicit motion estimation entirely.

The second answer is **implicit neural representation** compression, or INR. It asks: what if you don’t encode the video as a stream of pixels at all? What if the video *is* a small neural network — one that has been trained, from scratch, to memorize every frame — and the “bitstream” is just the quantized weights of that network? COIN (2021) showed this worked for single images. NeRV (2021) extended it to video. The results are elegant, the decoder is simple, and the encoder is... slow. Very slow.

By the end of this chapter you will understand why both approaches are genuinely exciting, where each one wins, and — crucially — why neither has yet replaced AV1 on your phone.

WHERE WE ARE

Chapter 51 built the classical hybrid video codec loop: motion-compensated prediction, DCT on the residual, quantization, entropy coding, all tied together by Lagrangian rate-distortion optimization (minimize $J = D + \lambda R$). **Chapter 57** replaced the image codec's hand-built transform with a trained autoencoder, added quantization-as-noise to make gradients flow, and introduced the hyperprior for adaptive entropy modeling. **Chapter 58** added GAN and diffusion losses for perceptual quality. This chapter applies all of that machinery to video — where time is the new dimension that creates both the biggest compression opportunity and the hardest engineering problem.

By the end of this chapter you will be able to:

- Explain why temporal prediction is the dominant source of compression gain in video, and what an end-to-end neural video codec does differently from a classical one.
- Describe the DVC (2019) architecture: optical-flow networks, motion compression, residual compression, and why it was a first.
- Explain the key conceptual shift in the DCVC family: conditional coding versus explicit residual coding, and why it is information-theoretically tighter.
- Describe VCT's even more radical simplification: no motion estimation at all, just a transformer on latents.
- Define an implicit neural representation (INR) and explain how COIN uses one to store an image.
- Explain the NeRV architecture for video: frame index in, RGB frame out.
- State the central tension for INR codecs: fast decoding, extremely slow encoding.
- Assess the current (2026) state of learned video codecs: where they win, where they lose, and what the deployment barriers are.

4.1 Why Video Is Both Easier and Harder Than Images

When you compress a single image, every pixel has to be coded independently of any temporal context — there is only one frame. The only structure you can exploit is spatial: nearby pixels tend to be similar, smooth areas have predictable colors, and so on. That is what JPEG, JPEG XL, and learned image codecs exploit.

Video gives you an enormous extra gift: **time**. In most video — a person talking, a film scene, a sports match — the vast majority of the frame is simply the previous frame, shifted and deformed slightly by motion. A good codec does not need to re-encode what is already there; it encodes only the **difference**. In the H.264 standard, motion-compensated residuals typically carry only 10–20% of the information that an intra-coded frame would require. Time is the most powerful compression trick in the entire book.

But time creates a harder engineering problem. Images are independent objects. You compress one, you are done. Video frames are **causally linked**: frame 100 depends on frames 95, 90, and perhaps 1. Errors propagate. Motion in the real world is complicated

— partial occlusions, large displacements, lighting changes, fast objects. A codec that gets motion slightly wrong pays a large penalty in residual size. Classical codecs solved this by hand-crafting motion search algorithms, carefully tuned loop filters, and elaborate reference-frame management. The neural approach asks: can we learn all of this from data?

KEY IDEA

The compression gain from temporal prediction is so large that it dominates everything else in video coding. Any learned video codec must either (a) learn to do motion prediction better than classical methods, or (b) find a different way to exploit temporal correlation that makes motion prediction unnecessary. The DVC line does (a); VCT does (b).

4.2 DVC: The First End-to-End Neural Video Codec

In December 2018, Guo Lu, Wanli Ouyang, Dong Xu, Xiaoyun Zhang, Chunlei Cai, and Zhiyong Gao posted a preprint titled “DVC: An End-to-End Deep Video Compression Framework.” It was accepted to CVPR 2019 as an oral presentation — the highest distinction at that venue. DVC was the first neural video codec to jointly optimize every component by gradient descent, and it is the baseline against which everything since is measured.

4.2.1 The Classical Structure, Neuralized

The key insight of DVC is that the classical hybrid codec pipeline, block by block, has a learned neural counterpart:

Optical flow estimation. Classical codecs search for block matches in a reference frame — a heuristic with many failure modes. DVC replaces this with a pre-trained optical flow network (SpyNet or a similar architecture from the computer vision literature) that estimates a dense pixel-level motion field v mapping each pixel in frame $t - 1$ to its location in frame t . The flow field is a continuous, smooth function learned from data — it handles occlusions and large motions more gracefully than block matching.

Motion compression. The flow field v is itself a 2D signal that must be stored in the bitstream. DVC applies a learned image codec (Ballé et al., 2018, the same hyperprior architecture from Chapter 57) to compress and decompress it as $\hat{v}$. The reconstructed flow $\hat{v}$ is used to warp the reference frame: $\hat{x}_{t-1} \xrightarrow{\hat{v}} \tilde{x}_t$ (“warp frame $t - 1$ using flow $\hat{v}$ to predict frame t ”).

Residual compression. The difference between the true frame x_t and the warp prediction $\tilde{x}_t$ is the **residual** $r = x_t - \tilde{x}_t$. Another learned codec (again, hyperprior style) compresses the residual to get $\hat{r}$. The final reconstruction is $\hat{x}_t = \tilde{x}_t + \hat{r}$.

End-to-end training. The whole pipeline — flow network, motion codec, warp operation, residual codec — is differentiable (with quantization-as-noise, as in Chapter 57). The training objective is a rate-distortion loss:

$$\mathcal{L} = \lambda \cdot D(x_t, \hat{x}_t) + R_v + R_r$$

where D is the mean squared error between the original and reconstructed frame, R_v is the number of bits spent on the motion field, and R_r is the bits spent on the residual. By minimizing this jointly, the network learns to trade motion precision against residual size in whatever way saves the most bits.

Σ GO FURTHER · THE MATHS — Rate-distortion loss for video *skip if familiar*

The rate-distortion tradeoff is the fundamental tension in any lossy compressor: more distortion (lower quality) saves bits; less distortion (higher quality) costs bits. In the classical formulation from Chapter 21, we choose a Lagrange multiplier $\lambda > 0$ and minimize the combined cost:

$$J = D + \lambda \cdot R$$

where D is distortion (e.g., mean squared error in pixel values) and R is the number of bits. Large λ weights quality heavily; small λ weights file size. For DVC there are two bit streams to account for — motion bits R_v and residual bits R_r — so the loss becomes:

$$\mathcal{L} = \lambda \cdot D + R_v + R_r$$

The network jointly minimizes both bit costs while controlling quality.

4.2.2 What DVC Achieved

On standard video benchmarks (UVG, MCL-JCV, HEVC Class B), DVC at its best configuration outperformed H.264 (the dominant streaming codec at the time) on both PSNR and MS-SSIM metrics. It did not beat HEVC (H.265), but it demonstrated that the entire hand-crafted classical pipeline could, in principle, be replaced by trained neural networks — and that gradient-based joint optimization found a better balance between motion coding and residual coding than the separately tuned classical components.

Recall. PSNR (Peak Signal-to-Noise Ratio, defined in Chapter 42) measures reconstruction quality in decibels: $\text{PSNR} = 10 \log_{10}(255^2 / \text{MSE})$, where MSE is the mean squared pixel error. Higher is better; above 40 dB is near-indistinguishable from the original. **MS-SSIM** is a perceptual quality score (also Chapter 42) that tracks human judgment of structural similarity better than raw error. We use both here exactly as the video chapters did.

The cost: DVC was orders of magnitude slower than any classical codec. Encoding a single frame took many seconds on a GPU; classical H.264 encoders run at real-time on a CPU. The inference complexity was also high for decoding.

HISTORICAL NOTE

DVC arrived at a moment when computer vision had already solved dense optical flow with neural networks. The insight of Lu et al. was not to invent a new flow estimator, but to recognize that a flow estimator is just a motion coder, and a motion coder is just

an image coder applied to a 2D vector field — and that the whole chain could be trained together. The paper’s oral acceptance at CVPR 2019 (top 1% of submissions) signaled that the compression community had arrived at the deep learning era.

DVC — Deep Video Compression

2019

Authors	Guo Lu, Wanli Ouyang, Dong Xu, Xiaoyun Zhang, Chunlei Cai, Zhiyong Gao
Aim	First fully end-to-end learned video codec, replacing every classical block (motion estimation, motion coding, residual coding) with neural networks trained jointly via rate–distortion loss.
Complexity	$O(H \times W)$ per frame inference; GPU-only; many seconds per frame at 2019 hardware.
Strengths	Joint optimization of motion + residual; better handles large motions and occlusions than block matching; milestone proof-of-concept.
Weaknesses	Far slower than classical codecs; quality below HEVC; inherits classical residual-coding paradigm's information-theoretic ceiling.
Superseded by	DCVC family (2021–), VCT (2022), DCVC-RT (2024–2025).

DVC keeps the skeleton of the classical hybrid codec (predict $\rightarrow$ residual $\rightarrow$ code) but replaces every component with a trained network. The optical-flow estimator produces a dense motion field; a learned image codec compresses that field; a warp operation generates the prediction; a second learned codec handles the residual. Everything is differentiable and trained jointly.

4.3 The DCVC Family: Conditional Coding

The DVC architecture was a milestone, but it inherited one assumption from the classical world: the notion of an explicit **residual**. You predict the current frame, subtract the prediction, and encode the difference. This seems natural. In fact, it has an information-theoretic ceiling.

The DCVC family of codecs — developed at Microsoft Research Asia, starting with “Deep Contextual Video Compression” by Jizheng Xu, Jianping Lin, and colleagues at NeurIPS 2021 — made a cleaner and more powerful choice.

4.3.1 From Residuals to Conditions

In a learned codec, every frame is compressed by passing it through an encoder network to produce a latent representation y , which is then quantized and entropy-coded. The

entropy coder needs a probability model $p(\hat{y})$ — the better the model, the fewer bits are needed.

In a **residual** approach, you compute $r = x_t - \tilde{x}_t$ and then encode r . The temporal information (the warped prediction) enters the picture before the encoder: you are literally encoding a smaller number.

In **conditional coding**, the temporal information enters the probability model differently. You still encode the full latent y_t of frame t , but the entropy model is **conditioned** on the temporal context φ_t (features derived from the already-decoded reference frames). That is:

$$R = -\log_2 p(\hat{y}_t \mid \varphi_t)$$

The entropy model predicts what the current latent will look like, given what came before, and assigns short codes to likely outcomes. The encoder still sends the full latent, but because the entropy model makes good predictions, the expected number of bits is small.

Why is this better? Conditional coding is strictly more powerful than residual coding. The residual paradigm forces the prediction to happen in **pixel space**: you warp pixels, subtract pixels. Conditional coding allows the prediction to happen in **latent space**, where the network has learned a much richer representation. The network can condition the probability of each latent coefficient on patterns that have no simple pixel-domain description — texture distributions, object identity, scene illumination — and assign fewer bits to predictable structure.

Σ GO FURTHER · THE MATHS — Conditional entropy

skip if familiar

Recall from Chapter 18 that the entropy $H(Y)$ of a random variable Y is the average number of bits needed to describe it. The **conditional entropy** $H(Y \mid X)$ is the average number of bits needed to describe Y once you already know X . It obeys:

$$H(Y \mid X) \leq H(Y)$$

Knowing X can only help — it can never increase the average code length. In DCVC, Y is the current frame's latent and X is the temporal context φ_t from previous frames. By conditioning the entropy model on φ_t , DCVC pays at most $H(Y_t \mid \varphi_t)$ bits per frame — which is always at most $H(Y_t)$, the cost of coding the frame without any temporal information, and usually much less.

4.3.2 The DCVC Architecture

The DCVC encoder for frame t works as follows:

1. **Extract temporal context.** From previously decoded frames, a feature extraction network produces a rich temporal context φ_t . This is not just a warped copy of the previous frame — it is a learned feature map that can capture higher-level structure.

2. **Encode the current frame.** The encoder network maps x_t to a latent y_t , using φ_t as an additional input so that it can “already assume” the decoder knows the context.
3. **Entropy code conditionally.** The entropy model predicts $p(\hat{y}_t | \varphi_t)$ and uses it to assign bit costs. Because the context is rich, most latent coefficients are highly predictable and cost very few bits.
4. **Decode and update context.** The decoder reconstructs $\hat{x}_t$ from $\hat{y}_t$ and φ_t . The reconstructed frame feeds into the next step’s context extraction.

Notice that there is no explicit residual anywhere. The network has quietly internalized the idea of “what changed” into the conditional probability model, without ever computing $x_t - \tilde{x}_t$ as an explicit variable.

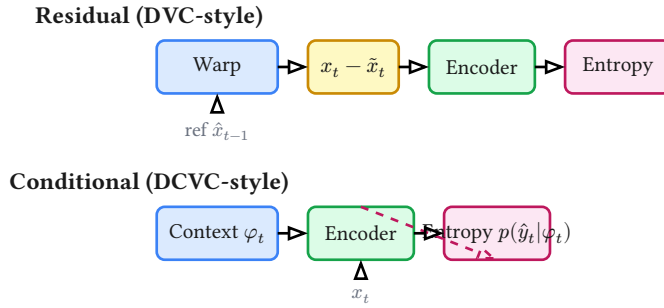


Figure 12: The DCVC conditional coding pipeline versus the classical residual pipeline. In the residual approach (top), the reference frame is warped and subtracted before the encoder. In conditional coding (bottom), the temporal context φ_t conditions the entropy model, allowing prediction in latent space.

4.3.3 The DCVC Family, 2021–2025

The original DCVC paper was followed by a series of extensions, each fixing a specific limitation:

DCVC-TCM (2022) — Temporal Context Mining, published in IEEE Transactions on Multimedia. It improved the quality of the temporal context by mining features at multiple temporal scales, so the entropy model sees both fine-grained motion and coarser scene structure.

DCVC-HEM (2022) — Hybrid Spatial-Temporal Entropy Modelling, at ACM Multimedia. It combined the temporal context (from previous frames) with a spatial context model (from already-decoded positions in the current frame), giving the entropy coder two sources of side information simultaneously.

DCVC-DC (NeurIPS 2023) — Diverse Contexts. This version was the first end-to-end neural video codec to clearly surpass VVC (the current champion classical codec) on both PSNR and MS-SSIM on standard benchmark datasets. The bitrate savings over VVC’s reference software ranged from 23% to 26% depending on the test sequence.

DCVC-FM (2024) — Feature Modulation. Further pushed the rate-distortion curve and introduced a more flexible context modulation mechanism. On benchmark sequences at the time of its release, DCVC-FM maintained the lead over VVC.

DCVC-RT (2024–2025) — Real-Time. The most practically significant entry. By redesigning the architecture for GPU-parallel execution (eliminating serial dependencies in the entropy model), DCVC-RT achieved over 125 frames per second at 1080p on modern hardware — genuinely real-time — while still showing roughly a 21% bitrate saving compared to VVC at the same quality level. This closed the speed gap from “orders of magnitude slower” to “within range of classical hardware encoders.”

PITFALL

“DCVC beats VVC” needs a careful reading. These comparisons are against VVC’s **reference software** (VTM), which is a research-grade implementation — correct, but not optimized for speed. Practical VVC encoders (like x265’s HEVC predecessor or the emerging x266) are much faster. The JVET Enhanced Compression Model (ECM, described in Chapter 55) still beats DCVC-FM by about 11% on the same benchmarks. The neural codec wins on flexibility and trainability; it does not yet win on every axis simultaneously.

DCVC — Deep Contextual Video Compression2021 (NeurIPS); extended 2022–2025	
Authors	Jizheng Xu, Jianping Lin, et al. (Microsoft Research Asia); multiple subsequent papers
Aim	Replace explicit residual coding with conditional entropy coding; encode the full frame latent, but condition the entropy model on rich temporal context features so bits per frame are minimized.
Complexity	Encoder: GPU-dependent; DCVC-RT achieves >125 fps at 1080p. Decoder: comparably fast with parallel entropy decoding.
Strengths	Information-theoretically cleaner than residual coding; surpasses VVC reference software on standard benchmarks; DCVC-RT is first neural video codec with practical real-time speeds.
Weaknesses	Still requires GPU; ECM (classical) is still ~11% better on benchmarks; bit-exact reproducibility across platforms remains fragile; context model training requires large video datasets.
Superseded by	Active research line as of 2026; not yet superseded.

The key conceptual advance: temporal prediction moves from pixel space (warp and subtract) to latent space (condition the entropy model). The decoder reconstructs each frame's latent by knowing that certain coefficients, given what came before, are nearly certain — so they cost almost no bits.

4.4 VCT: Remove Motion Entirely

In 2022, Fabian Mentzer and colleagues at Google Research published “VCT: A Video Compression Transformer,” which appeared at NeurIPS 2022. VCT asked an even more radical question than DCVC: what if you do not model motion at all?

The architecture is surprisingly simple. VCT uses a standard learned **image** codec (the hyperprior model from Chapter 57) to encode each frame independently into a quantized latent $\hat{y}_t$. There is no optical flow, no warping, no conditional context extraction. Each frame is compressed as if it were a standalone image.

The magic happens at the entropy coding step. A large transformer model — trained on sequences of video latents — predicts the distribution $p(\hat{y}_t \mid \hat{y}_{t-k}, \dots, \hat{y}_{t-1})$ for each new frame's latent, conditioned on the previously transmitted latents. The transformer learns, from data, whatever temporal patterns are useful: motion, scene continuity, lighting changes, periodicity. It does not need to be told “this is motion” — it discovers useful patterns on its own, including ones that have no classical counterpart.

Σ GO FURTHER · THE MATHS — Transformers and attention skip if familiar

A *transformer* is a kind of neural network (built from the neurons of Chapter 56) designed to process a *sequence* of items — here, the sequence of past frame latents $\hat{y}_1, \hat{y}_2, \dots, \hat{y}_{t-1}$ — and predict the next one. Its central trick is *attention*: to predict the next item, the network computes, for every earlier item, a *relevance weight* between 0 and 1 saying “how much should I look at this one?”, then forms a weighted average of the items it deems relevant. Think of it as a soft, learned table lookup: when predicting frame t , the network might place 70% of its attention on frame $t - 1$ (the most recent), 20% on a frame from one second ago that looked similar, and almost none on the rest.

Because every item can attend to every other, a transformer captures long-range patterns that a fixed warp-the-previous-frame rule never could. The price is cost: with n items in the sequence, attention compares every pair, so the work grows like n^2 — quadratic, in the Big-O language of Chapter 14. That is why long sequences make transformers slow. *Positional encoding* is a small extra signal added to each item that tells the network *where* in the sequence (which time step) the item sits, since attention by itself is order-blind. We meet the transformer in full when we reach language models in Chapter 62; here we use it purely as a powerful next-item predictor.

The result: VCT matches or beats DVC on standard benchmarks, despite having no motion model at all. On datasets with complex motion (panning, blurring, fading) that defeat block matching, VCT can be up to 45% better in rate-distortion than CNN-based codecs.

The architectural simplicity is appealing — there are fewer hand-crafted components that could fail.

The cost is that transformers are computationally expensive (the attention box above explains why: the work grows quadratically with sequence length), and autoregressive decoding of latents is serial: each coefficient depends on the previous ones, limiting parallelism.

VCT — Video Compression Transformer		2022 (NeurIPS)
Authors	Fabian Mentzer, David Minnen, Eirikur Agustsson, Michael Tschanen (Google Research)	
Aim	Remove all explicit motion modeling from neural video compression; use a transformer to predict per-frame latent distributions from previously transmitted latents, learning temporal structure purely from data.	
Complexity	Frame encoding: image codec (fast); entropy prediction: transformer (slow for long sequences due to attention cost). Autoregressive decoding limits GPU parallelism.	
Strengths	No motion estimation failure modes; handles complex motion (panning, occlusion) naturally; simpler architecture with fewer hand-crafted components; strong on distribution-shift scenarios.	
Weaknesses	Transformer inference is expensive; autoregressive decoding is serial; performance on standard benchmarks is competitive but not always best-in-class against DCVC-DC/FM.	
Superseded by	Active research line; descendants explored in 2023–2025.	
VCT demonstrates that temporal prediction does not require explicit motion vectors. A sufficiently powerful sequence model discovers its own notion of “what is expected next” in latent space — motion, repetition, gradual lighting change, whatever helps — and assigns short codes to the expected, long codes to the surprising.		

Checkpoint. A colleague claims: “DCVC-style conditional coding and VCT are just two names for the same idea — they both condition the entropy model on past frames.” Is this right? Where do they differ?⁶

⁶**Checkpoint answer.** Partially right. Both condition entropy on past context. The crucial difference is how the context is computed. DCVC explicitly extracts feature maps from decoded reference frames using convolutional networks and uses them as input to the entropy model — the engineer still controls what “temporal context” means. VCT instead feeds the raw quantized latents of past frames to a large transformer and lets the transformer attend to whatever patterns it finds useful. VCT has no concept of

4.5 Implicit Neural Representations: A Completely Different Idea

Everything so far — DVC, DCVC, VCT — keeps the classical encoder-bitstream-decoder architecture. The encoder produces a bitstream of symbols representing the video; the decoder reconstructs the video from those symbols. The innovation is in how the symbols are chosen and modeled.

Implicit neural representation (INR) compression throws out this architecture entirely. It asks: what if the bitstream is not a compressed description of the video, but instead the **parameters of a neural network that was trained to memorize the video?**

4.5.1 What Is an Implicit Neural Representation?

An implicit neural representation is a neural network f_θ that maps a **coordinate** to a **value**:

$$f_\theta : \text{coordinate} \rightarrow \text{signal value}$$

For an image: the coordinate is a pixel position (x, y) and the output is a color (r, g, b) . For audio: the coordinate is a time t and the output is a sample amplitude. For 3D geometry: the coordinate is a point (x, y, z) and the output might be a signed distance or an occupancy probability (this is the idea behind NeRF, Neural Radiance Fields, 2020).

The network is not trained on a general image dataset. It is **overfit** — trained specifically on a single piece of data, until $f_{\theta(x,y)} \approx$ the true color at pixel (x, y) for every pixel. Once trained, to recover the signal you simply evaluate the network at every coordinate. The “compression” is the fact that the network weights θ may be far smaller than the raw pixel data.

Recall. In Chapter 23 we met *over-fitting* as a vice: a model that “memorises” its training data — including the noise — instead of capturing genuine regularity, and so fails to generalise to new data. INR compression flips this on its head. Here we *want* to memorise one specific signal as exactly as possible, because we never ask the network to generalise to anything else — the file *is* the signal. Over-fitting, normally the enemy, becomes the whole point. The reason this still compresses is the lesson of Chapter 23: a network small enough to be worth storing can only fit the signal well if the signal has real structure for it to latch onto. Pure noise would need as many parameters as pixels, and no compression would result.

Definition — Implicit Neural Representation (INR). An implicit neural representation is a neural network f_θ whose parameters θ are optimized to represent a single signal s by learning the mapping from coordinates (position, time, frequency) to signal values. The network is the codec: encoding is training f_θ on s ; decoding is evaluating f_θ at every coordinate.

“motion feature” at all. DCVC has a more structured but potentially more limited context; VCT has a less structured but potentially more general one.

4.5.2 COIN: Compression with Implicit Neural Representations (2021)

“COIN: COMpression with Implicit Neural Representations” by Emilien Dupont, Adam Golinski, Milad Alizadeh, Yee Whye Teh, and Arnaud Doucet appeared as a spotlight at the Neural Compression Workshop at ICLR 2021. It was the first paper to seriously evaluate INRs as a compression method for images.

The COIN encoder is remarkably simple:

1. Take an image with pixel coordinates (x, y) and RGB values.
2. Define a small multilayer perceptron (MLP) with sinusoidal activations (called a SIREN — Sinusoidal Representation Network — because sinusoids are well-suited to representing the smooth, oscillatory structure of natural images).
3. Train this MLP, via gradient descent on the mean squared error between predicted and true pixel values, until it memorizes the image.
4. Quantize the trained weights from 32-bit floats to, say, 8 or 16 bits.
5. The bitstream is the quantized weight vector.

The COIN decoder:

1. Receive the quantized weights, dequantize them.
2. Reconstruct the MLP.
3. Evaluate the MLP at every pixel coordinate (x, y) in row-major order.
4. Output the RGB values. Done.

There is no entropy coder, no transform, no motion model. Just network evaluation. And yet, at low bitrates (below roughly 0.1 bits per pixel), COIN outperformed JPEG — a hand-crafted codec that had been tuned for decades. The reason: at very low bitrates, JPEG’s block artifacts dominate the distortion, while COIN’s smooth MLP produces a globally coherent, artifact-free reconstruction.

PY GO FURTHER · PYTHON — Multilayer Perceptron in PyTorch *skip if fluent* — the concept

A multilayer perceptron (MLP) is the simplest kind of neural network. Each layer takes a vector of numbers, multiplies it by a weight matrix, adds a bias, and applies a nonlinear function. In PyTorch (which uses a very similar syntax to what we have seen in tinyzip), a 3-layer MLP looks like:

```
import torch
import torch.nn as nn

class TinyMLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(2, 64),    # 2 inputs: (x, y) pixel coords
```

```

        nn.ReLU(),
        nn.Linear(64, 64),
        nn.ReLU(),
        nn.Linear(64, 3),    # 3 outputs: (R, G, B)
    )

    def forward(self, coords: torch.Tensor) -> torch.Tensor:
        # coords: shape (N, 2), values in [-1, 1]
        return self.net(coords) # shape (N, 3)

```

Training means adjusting all the weights so that for each pixel coordinate `coords[i]`, the network output matches the true color. COIN uses sinusoidal activations (`torch.sin(w * x)`) instead of ReLU because they handle high-frequency detail better — but the core idea is identical.

Aside. The SIREN network (Sitzmann et al., NeurIPS 2020) uses the activation function $\sin(\omega_0 \cdot Wx + b)$ at each layer, where $\omega_0 \approx 30$ is a frequency scaling factor. This choice makes the network’s Jacobian (the rate of change with respect to its inputs) periodic, which is ideal for representing signals that have oscillatory structure at multiple scales — exactly what natural images look like when you examine their Fourier spectrum.

4.5.3 What COIN Cannot Do (Yet)

COIN beat JPEG at very low bitrates. It did not beat JPEG at higher bitrates. And it did not beat JPEG XL, WebP, or any modern codec at any bitrate. The fundamental problem is twofold.

First, COIN has no **entropy coder**. The quantized weights are stored as-is, without modeling their distribution or exploiting statistical structure. Adding entropy coding to the weights (the approach taken in COIN++ by Dupont et al., 2022) significantly improves results, but the gap to classical codecs remains.

Second, COIN **encodes slowly**. Training an MLP to memorize a single image by gradient descent takes hundreds of gradient steps. On a modern GPU, encoding a single 512×512 image takes on the order of 10–30 seconds. Classical image codecs encode in milliseconds. For batch processing, INR encoding is unusable at any reasonable throughput.

Myth. “INR codecs are fast to use because the decoder is just a neural network evaluation.”

Reality. The decoder is fast — evaluation of a small MLP at every pixel coordinate is essentially a matrix multiplication, which GPUs handle in milliseconds. But the **encoder** is training a neural network from scratch for every single file, which is expensive (seconds to minutes). The decode/encode asymmetry is the reverse of classical codecs: in INR, encoding is slow and decoding is fast. Classical codecs are

usually encode-slow, decode-fast only at the most aggressive quality settings; in practice, software encoders like libx264 are much faster than INR encoding at any setting.

4.6 NeRV: Extending INRs to Video

Hao Chen, Bo He, Hanyu Wang, Yixuan Hou, Ser Nam Lim, and Abhinav Shrivastava published “NeRV: Neural Representations for Videos” at NeurIPS 2021. NeRV extends the INR idea to video in a clever way that sidesteps the biggest bottleneck of pixel-wise INRs.

4.6.1 Frame Index In, Full Frame Out

The key insight of NeRV is to change what the coordinate means. Instead of mapping a pixel coordinate (x, y) to a pixel color (r, g, b) — which requires one network evaluation per pixel — NeRV maps a **frame index** t to an entire **frame image**. The network architecture uses convolutional upsampling layers (like a decoder half of an autoencoder, as seen in Chapter 57) to turn a small embedding vector e_t into a full-resolution image:

$$f_{\theta} : t \rightarrow \text{frame} \in \mathbb{R}^{H \times W \times 3}$$

Here e_t is a learned positional embedding for time step t — the same idea as the positional encoding for transformers we met a few pages ago: a small vector that encodes *which* time step we are asking about. The output is the full RGB frame at time t , produced in a single forward pass.

This is dramatically more efficient than pixel-wise INRs. Instead of evaluating the network $H \times W$ times per frame, you evaluate it **once** per frame, producing all $H \times W$ pixels simultaneously through the upsampling layers. NeRV reported encoding speed improvements of $25\times$ to $70\times$ over pixel-wise INRs (like COIN applied per-frame), and decoding improvements of $38\times$ to $132\times$.

4.6.2 Compression via Weight Pruning and Quantization

To use NeRV as a video codec:

1. Train f_{θ} on the video until it memorizes every frame well.
2. Compress the network weights using standard neural network compression techniques: structured pruning (remove the least important weight groups), weight quantization (reduce from 32-bit to 8-bit or lower), and entropy coding of the resulting sparse weight tensors.
3. The bitstream is the compressed weight tensor.
4. Decoding: receive weights, reconstruct network, run one forward pass per frame (each pass in milliseconds on a GPU).

On standard video benchmarks (UVG, HEVC sequences), NeRV reached quality comparable to H.264 at similar bitrates, and better than H.264 at some low-bitrate settings. It did not reach HEVC or AV1 quality.

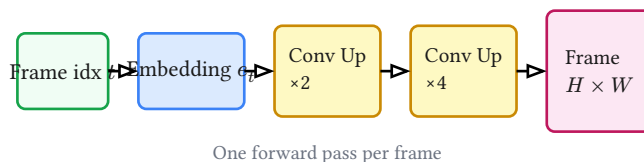


Figure 13: The NeRV architecture. A frame index t is mapped to a positional embedding, which is then upsampled through convolutional blocks to produce a full video frame. The entire frame is generated in a single forward pass.

4.6.3 The Encoding Wall

NeRV is elegant. Its decoding is fast and GPU-friendly. But its encoding problem is severe: training the network means running gradient descent for thousands of iterations, using the video itself as a training set, until the network memorizes every frame. For a 1-minute video at 30 fps (1,800 frames), this can take 30 minutes to several hours, depending on the network size and desired quality. Classical codecs encode the same video in seconds.

Subsequent work — PNVC (2024), TeCoNeRV (2025), and related papers — has made progress on reducing training time through better architectures, meta-learning (training on video datasets so the network starts closer to a good solution), and parallel training strategies. As of 2026, the encoding time for INR-based video codecs has decreased significantly but remains a practical barrier for any use case that requires fast encoding: live streaming, video conferencing, real-time broadcasting.

KEY IDEA

The INR paradigm inverts the classical codec’s asymmetry. Classical codecs (H.264, AV1) are designed to be decoded fast, because every viewer must decode but content is encoded once. INR codecs decode fast but encode slowly — which is exactly the right asymmetry for **streaming on demand**, where the encoder has unlimited time but the viewer needs instant playback. The problem is that “unlimited encoding time” still needs to mean “hours, not days.”

4.7 Comparing the Three Approaches

To make the tradeoffs concrete, let us put the three families side by side against a common benchmark. The UVG dataset (Ultra Video Group, a standard research benchmark of 7 high-resolution 1080p sequences) is the most widely reported. Numbers below are approximate, as exact figures depend on configuration and hardware — they are meant to illustrate relative positions, not to be cited as definitive benchmarks.

Recall. **BD-Rate** (Bjontegaard Delta Rate, introduced in Chapter 53) is the standard way to compare two codecs with a single number. Each codec has a whole quality-versus-bitrate *curve*; the BD-Rate is the average percentage of bits codec B saves to reach the same quality as codec A, across the overlapping quality range. A BD-Rate of -50% means “half the bits for the same quality”. Negative is better.

SCOREBOARD — Approximate PSNR BD-Rate relative to H.264 (libx264 at same quality), UVG benchmark, ca. 2023–2024. Negative = fewer bits than H.264 for same quality.

Technique	Bytes	Ratio	Notes
Method	Type	BD-Rate vs H.264	Enc. Speed
H.264 (libx264)	Classical	0% (reference)	Fast, CPU
HEVC (H.265, x265)	Classical	approx. -35%	Fast, CPU
AV1 (libaom)	Classical	approx. -40%	Slow, CPU
VVC (VTM)	Classical	approx. -50%	Very slow, CPU
DVC (2019)	Neural end-to-end	approx. -10 to -20%	Very slow, GPU
DCVC-DC (2023)	Neural cond.	approx. -55 to -60%	Slow, GPU
DCVC-RT (2025)	Neural cond.	approx. -45%	Real-time, GPU
VCT (2022)	Neural transformer	approx. -30%	Slow, GPU
NeRV (2021)	INR	approx. +0 to -10%	Very slow (train)

The scoreboard tells a clear story. By 2023–2024, the best neural codecs (DCVC-DC) surpass even VVC on rate-distortion. But the encoding speed and GPU dependency columns reveal the deployment gap. DCVC-RT is the first crack in that wall: by 2025 it reaches real-time speeds on a GPU while still beating VVC. NeRV is competitive with H.264 but not better, and its encoding wall remains the defining limitation.

4.8 The Deployment Reality: Why Your Phone Still Uses AV1

If DCVC-DC beats VVC by 55%, why is it not in your phone’s video player?

The answer has nothing to do with compression ratios. It has everything to do with deployment constraints that the research community tends to underemphasize.

4.8.1 Hardware Acceleration

Classical video codecs are accelerated in dedicated silicon on virtually every modern device: a smartphone has a hardware H.264/H.265/AV1 decoder that runs at 120 fps while consuming milliwatts. That decoder does not use the general CPU or GPU; it is a fixed-function chip. Neural codecs run on the GPU (or NPU), which consumes more power and competes with other tasks. Until NPU silicon is designed and shipped for specific neural codec architectures — which requires years of standardization and fab cycles — neural codecs will always lose the energy and performance battle on constrained devices.

4.8.2 Bit-Exact Reproducibility

When you compress a video for a streaming service and send it to a billion devices, every device’s decoder must produce the **exact same pixels** from the same bitstream. This is not optional: encrypted video, digital rights management, forensic analysis, and broadcast engineering all depend on it. Classical codecs guarantee bit-exact reproducibility by spec-

ification. Neural codecs that use floating-point arithmetic do not: the same weights can produce slightly different outputs on different hardware, different compiler versions, or different floating-point rounding modes. Achieving bit-exact neural decoding is an open engineering problem.

4.8.3 Standardization

Streaming infrastructure is built around standards: H.264, H.265, AV1, VP9. Every browser, every smart TV, every content delivery network supports these formats. A new codec format requires browser support, device driver support, CDN transcoding pipelines, and player compatibility — a multi-year process that starts with an international standards body (ITU-T, MPEG, or AOM). As of 2026, no end-to-end neural video codec has been standardized. JPEG AI (for still images, Chapter 58) was the first learned codec to reach ISO/IEC/ITU status, in 2025 — a milestone that took seven years from the first competitive papers. Video will take longer.

4.8.4 Energy and Environmental Cost

Neural codec inference, at scale, consumes significantly more energy than classical codec execution on fixed-function hardware. Streaming is already one of the largest electricity consumers in the internet infrastructure. Replacing hardware-accelerated AV1 with GPU-executed neural networks would dramatically increase per-stream energy cost — a concern that is taken seriously by large cloud operators.

PITFALL

Research papers compare neural codecs to VVC’s **reference software** (VTM), which is unoptimized for speed and always much slower than any classical codec in practice. The fair comparison is against practical encoders like x265 or SVT-AV1. Against those, the neural advantage shrinks considerably — though it does not vanish.

4.9 A Tiny Worked Example: INR in 20 Lines

Even without PyTorch or GPU acceleration, we can understand the INR idea with a minimal Python sketch. The following code is illustrative — it uses numpy rather than a real neural network library, and the “network” is a single linear layer (essentially just a linear basis expansion). It is not competitive with JPEG. Its purpose is to make the core loop of INR encoding and decoding completely concrete.

PY GO FURTHER · PYTHON — Numpy arrays and matrix operations *skip if fluent*

In our tinyzip code we have worked with `bytes` and `dict`. Here we briefly use `numpy`, a Python library for fast array arithmetic. A numpy array stores a grid of numbers (all the same type) and supports operations like matrix multiplication (`@`), element-wise addition, and reshaping. You can create one from a Python list:

```
import numpy as np
a = np.array([[1.0, 2.0], [3.0, 4.0]]) # 2x2 matrix
b = np.array([1.0, 0.0]) # vector of length 2
print(a @ b) # matrix-vector multiply: [1.0, 3.0]
```

Numpy arrays also support slicing (`a[0, :]` = first row), broadcasting (adding a scalar to every element), and universal functions like `np.sin`. We will use these to build a tiny sinusoidal basis for our INR sketch.

```
"""
Tiny INR sketch – illustrative only, not competitive with any real
codec.
Encodes a small grayscale image as the weights of a linear model on
a
sinusoidal feature basis; decodes by evaluating that model at every
pixel.
"""

import numpy as np
from pathlib import Path

# — Step 1: Build sinusoidal feature basis

def make_features(H: int, W: int, n_freqs: int = 16) -> np.ndarray:
    """
    For each pixel (y, x), create a feature vector of sin/cos at
    multiple
    frequencies – a simple Fourier feature map.
    Returns array of shape (H*W, 1 + 4*n_freqs).
    """
    ys = np.linspace(-1, 1, H)
    xs = np.linspace(-1, 1, W)
    yy, xx = np.meshgrid(ys, xs, indexing="ij") # each (H, W)
    coords = np.stack([yy.ravel(), xx.ravel()], axis=1) # (H*W, 2)
    freqs = 2.0 ** np.arange(n_freqs) # 1, 2, 4, 8, ...,
    2^(n-1)
    feats = [np.ones((H * W, 1))] # bias term
    for f in freqs:
        feats.append(np.sin(f * np.pi * coords)) # 2 columns per
    frequency
        feats.append(np.cos(f * np.pi * coords)) # 2 more columns
    return np.concatenate(feats, axis=1) # (H*W, 1 +
    4*n_freqs)
```

```

# — Step 2: Encode = least-squares fit of linear weights


---


def inr_encode(image: np.ndarray, n_freqs: int = 16) -> np.ndarray:
    """
    image: (H, W) float32 array of pixel intensities in [0, 1].
    Returns weight vector of shape (1 + 4*n_freqs,).
    """
    H, W = image.shape
    F = make_features(H, W, n_freqs)           # (H*W, D)
    y = image.ravel()                         # (H*W,)
    # Normal equations: weights = (F^T F)^{-1} F^T y (least
    squares)
    weights, _, _ = np.linalg.lstsq(F, y, rcond=None)
    return weights.astype(np.float16)         # quantize: 16-bit
    floats

# — Step 3: Decode = evaluate model at every pixel


---


def inr_decode(weights: np.ndarray, H: int, W: int,
               n_freqs: int = 16) -> np.ndarray:
    """Returns reconstructed (H, W) float32 image in [0, 1]."""
    F = make_features(H, W, n_freqs)
    pixels = F @ weights.astype(np.float32)
    return np.clip(pixels.reshape(H, W), 0.0, 1.0)

# — Self-test


---


if __name__ == "__main__":
    rng = np.random.default_rng(42)
    img = rng.random((32, 32), dtype=np.float32) # tiny random
    image

    weights = inr_encode(img)
    print(f"Original pixels : {img.size * 4} bytes (float32)")
    print(f"Encoded weights : {weights.nbytes} bytes (float16)")
    ratio = (img.size * 4) / weights.nbytes
    print(f"Compression ratio: {ratio:.1f}x")

    recovered = inr_decode(weights, 32, 32)
    mse = np.mean((img - recovered) ** 2)
    psnr = -10 * np.log10(mse + 1e-10)
    print(f"PSNR: {psnr:.1f} dB")

```

Running this on a 32×32 float image with `n_freqs=16` gives a weight vector of $1 + 4 \times 16 = 65$ values. The original image is $32 \times 32 = 1024$ pixels (each 4 bytes as float32 = 4096 bytes). The weight vector at float16 is $65 \times 2 = 130$ bytes — a roughly 31× compression ratio. The reconstruction quality depends entirely on how well a sinusoidal basis can

approximate the image. For a smooth image the fit is excellent; for a noisy image, many frequencies are needed and the compression is less impressive.

Real INR systems (COIN, NeRV) use deeper networks and gradient-based optimization (not closed-form least squares), which produces much better quality at the same bit count — but the idea is identical.

4.10 Where the Field Stands in 2026

From the vantage point of mid-2026, the neural video compression landscape looks like this.

Rate-distortion: On standard benchmarks, the best neural codecs (DCVC-DC/FM) have definitively surpassed VVC’s reference software. This is a genuine scientific achievement — the hand-crafted classical codec line that dominated for fifty years has been exceeded. However, ECM (the Enhanced Compression Model from JVET, Chapter 55) is still ahead of neural codecs on some benchmarks, suggesting that the classical engineering tradition has not been exhausted.

Speed: DCVC-RT achieved real-time GPU encoding in 2024–2025, closing the most glaring gap. Neural decoding is still slower than fixed-function hardware decoders for classical codecs, but the gap is narrowing with NPU hardware.

INRs: NeRV and its descendants are competitive with H.264 in quality, faster to decode than most neural codecs, but still slow to encode. The meta-learning approach (PNVC, 2024) reduces encoding time by pre-training the network on a dataset so that overfitting to a new video starts from a better initial point — a promising direction. Practical INR deployment remains niche as of 2026.

Standardization: No neural video codec has been standardized. JPEG AI (for images) reached ISO status in early 2025. Discussions within MPEG and JVET about “NNVC” (Neural Network Video Coding) are active but no standard timeline exists. This is the single biggest barrier to widespread adoption.

Perceptual quality: At very low bitrates (below 0.05 bits per pixel per frame), generative neural codecs — adding a diffusion or GAN decoder to DCVC- style coding, as seen in DiffVC-RT (2025) — produce dramatically better perceptual quality than any classical codec. This mirrors what we saw for images in Chapter 58. The “hallucinated detail” concern applies here too: generative video codecs can invent plausible motion that was not in the original.

Aside. The NeRF (Neural Radiance Field) paper by Mildenhall et al. (2020) used INRs for a different purpose — representing 3D scenes so that novel viewpoints could be rendered. The connection to compression was noticed quickly: a NeRF is implicitly compressing 3D information into a set of network weights. Storing a NeRF representation of a scene costs far less than storing all the raw camera footage, and the decoder (the renderer) can synthesize any viewpoint — a form of compression that is also a novel-view synthesis engine. This has spawned a whole field of “NeRF compression” and “Gaussian splatting compression” as of 2024–2026.

Chapter takeaways

- DVC (CVPR 2019) was the first end-to-end neural video codec: it neuralized every block of the classical hybrid pipeline (optical flow, motion coding, residual coding) and trained them jointly via a rate–distortion loss.
- The DCVC family (2021–2025) made the key conceptual shift from residual coding to conditional coding: instead of subtracting a pixel-domain prediction, they condition the entropy model on rich latent-space temporal context, which is information-theoretically tighter.
- VCT (NeurIPS 2022) removes explicit motion estimation entirely, using a transformer to predict per-frame latent distributions from previously transmitted latents — the network discovers its own notion of temporal predictability.
- Implicit Neural Representation (INR) compression stores a signal as the weights of a small network trained to memorize it; encoding is training, decoding is inference. COIN (2021) beat JPEG at low image bitrates with this idea.
- NeRV (NeurIPS 2021) extended INRs to video by mapping frame indices (not pixel coordinates) to full output frames, achieving 25–130× faster encode/decode than pixel-wise INRs.
- The central tension of INR codecs: decoding is fast (one GPU forward pass per frame), but encoding is slow (training a network from scratch per video clip).
- As of 2026, DCVC-RT is the first neural video codec with real-time GPU encoding and meaningful bitrate savings over VVC. No neural video codec has been standardized. Hardware, bit-exactness, and standardization remain the dominant barriers to deployment.

4.11 Exercises

Exercise 59.1 ★

A classical codec uses block motion estimation to find that a 16×16 block in frame t came from position $(x + 3, y - 2)$ in frame $t - 1$. DVC replaces this with an optical-flow network. In plain words: (a) what does a dense optical-flow field represent that block motion vectors do not? (b) Why is a dense flow field potentially more expensive to compress than a set of block vectors?

Exercise 59.2 ★

In the DCVC approach, there is no explicit residual $r = x_t - \tilde{x}_t$. Yet the encoder still needs to send information about what changed between frames. Where does that information go? How does the decoder know what changed?

Exercise 59.3 ★★

The conditional entropy inequality states $H(Y | X) \leq H(Y)$. (a) In DVC-style residual coding, what plays the role of Y and what plays the role of X ? (b) In DCVC-style conditional coding, what plays the role of Y and X ? (c) Both approaches use

temporal context; why does conditional coding potentially extract more benefit from the same context information?

Exercise 59.4 ★★

Write a Python function `nerv_params(H, W, n_frames, hidden_dim)` that computes and returns the **total number of parameters** in a simplified NeRV network consisting of: (1) an embedding matrix mapping frame indices to hidden vectors (shape: $n_frames \times hidden_dim$), (2) two upsampling convolutional blocks each with `hidden_dim` input and output channels and a 3×3 kernel, and (3) a final 1×1 convolution from `hidden_dim` to 3 (RGB). Ignore bias terms. Then answer: for a 100-frame, 720p (1280×720) video with `hidden_dim=128`, what is the parameter count? What is the raw pixel data size (in bytes, at 8 bits per channel)?

Exercise 59.5 ★★

VCT’s transformer is said to learn “temporal prediction without motion estimation.” Concretely: if a transformer is shown the latents $\hat{y}_1, \hat{y}_2, \dots, \hat{y}_{t-1}$ of a panning shot (the camera moves slowly left), what kind of pattern in the latents would the transformer need to learn in order to accurately predict $\hat{y}_t$? Describe this in intuitive terms — no mathematics required.

Exercise 59.6 ★★★

Research question. The DCVC-RT paper (2024–2025) claims real-time encoding at 125+ fps for 1080p video on a modern GPU, with a 21% bitrate saving over VVC. A classical hardware H.265 encoder chip does 240 fps at 1080p at a fraction of a watt. (a) In what streaming scenarios does DCVC-RT’s quality advantage make the extra compute cost worthwhile? (b) In what scenarios does it not? (c) What hardware or software development could change this trade-off in the next 5 years? Justify each answer with concrete examples.

4.12 Further Reading

The primary source for DVC is Lu et al. (2019), “DVC: An End-to-End Deep Video Compression Framework,” CVPR 2019, available open access. For the DCVC family, the Microsoft Research GitHub repository at github.com/microsoft/DCVC hosts code and links to all papers in the series. The NeurIPS 2021 DCVC paper is at [arXiv:2109.15047](https://arxiv.org/abs/2109.15047).

VCT is described in Mentzer et al. (2022), “VCT: A Video Compression Transformer,” NeurIPS 2022. The paper is unusual for a neural compression paper in its architectural simplicity.

For INRs, start with the COIN paper at [arXiv:2103.03123](https://arxiv.org/abs/2103.03123) and the NeRV paper at [arXiv:2110.13903](https://arxiv.org/abs/2110.13903). The SIREN network that underlies COIN is described in Sitzmann et al. (NeurIPS 2020), [arXiv:2006.09661](https://arxiv.org/abs/2006.09661).

For a comprehensive survey of learned video compression, Yang et al. (2023), “An Introduction to Neural Data Compression,” is an excellent graduate-level overview: [arXiv:2202.06533](https://arxiv.org/abs/2202.06533).

On the practical deployment gap, the Streaming Learning Center’s evaluation of DCVC-RT is an accessible technical analysis aimed at industry practitioners.

COMING UP NEXT

This chapter closed the video chapter of learned compression. We have seen neural codecs conquer images, video, and now approach real-time video speeds. But we have not yet touched the most abundant signal on the internet after video: **audio**. Chapter 60 turns to neural audio codecs — SoundStream (2021), EnCodec (2022), DAC (2023) — and discovers something unexpected: the very same residual vector quantization technique that makes a codec small also produces a perfect **tokenizer** for audio language models. The codec and the AI tokenizer converge.

Neural Audio Codecs as Tokenizers

The best interface between a waveform and a language model is a sequence of discrete tokens — small, fast, and full of meaning.

— Alexandre Défossez, Meta AI Research, 2022

Here is a puzzle: you want to build a voice assistant that can not only understand speech but generate it — one that continues a conversation in your own voice, with your own accent and emotional colour. You have a powerful language model that predicts sequences of text tokens beautifully. But audio is not text. A single second of high-quality speech contains 24,000 or 44,100 individual floating-point samples. Feeding raw samples to a language model would be like trying to read a novel one atom at a time. How do you bridge these two worlds — the continuous waveform of sound and the discrete token stream that language models understand?

The answer arrived in a burst between 2021 and 2023, in three papers that together redefined what an audio codec is for: Google’s **SoundStream** (2021), Meta’s **EnCodec** (2022), and Descript’s **DAC** (2023). These codecs can compress speech to a few kilobits per second — as good as the best traditional codecs — but they deliver something classical codecs never dreamed of: a neat stream of **discrete integers** that a language model can treat exactly like words. Codec and tokenizer have converged into the same device.

This chapter explains how they work, why the key ingredient is a technique called **residual vector quantization**, and how the resulting tokens now underpin a remarkable zoo of systems — from voice cloning to real-time conversational AI.

WHERE WE ARE

Chapters 56 and 57 introduced the machinery of learned compression: autoencoders, rate-distortion loss, quantization-as-noise, and the hyperprior that transmits “side information” about a learned prior. Chapter 58 showed how GAN and diffusion losses push codecs toward perceptual realism. Chapter 59 examined neural video codecs and implicit neural representations. All of this dealt with **images** and **video**. Now we turn to **audio**, where the same ideas land on a new substrate — one-dimensional waveforms sampled at 16 kHz to 48 kHz — and gain a second career as the foundation of audio language models.

By the end of this chapter you will be able to:

- Explain what residual vector quantization (RVQ) is and how it squeezes a continuous audio latent into a hierarchy of discrete codebook indices.
- Trace the architecture and key innovations of SoundStream, EnCodec, and DAC, noting what each one added.
- Describe the two roles a neural audio codec plays: as a **compressor** that shrinks audio files, and as a **tokenizer** that converts audio into sequences a language model can read and write.
- Explain how AudioLM, VALL-E, and Moshi use codec tokens as their working currency, and why semantic versus acoustic tokens matter.
- Identify the trade-offs (bitrate, latency, codebook collapse, compute) and describe the directions the field is exploring in 2025–2026.

5.1 From Waveform to Latent: the Encoder–Decoder Backbone

Every neural audio codec is, at its heart, an autoencoder for one-dimensional signals. Recall from Chapter 56 that an autoencoder consists of two networks: an **encoder** that maps an input to a compact **latent representation**, and a **decoder** that reconstructs the input from that latent; Chapter 57 then put a price tag on the latent to turn it into a learned image codec. For images the encoder operates on two-dimensional pixel grids; for audio it operates on a time series of samples.

The encoder in SoundStream is a stack of **strided convolutional layers**. Each layer doubles or quadruples the number of channels while halving the time dimension, so a one-second audio clip at 24 kHz (24,000 samples) ends up as a sequence of, say, 75 frames, each a 512-dimensional vector. The ratio of input samples to output frames is called the **stride product** or **downsampling factor** — in SoundStream it is 320, meaning one latent frame for every 320 raw samples. The decoder is the mirror image: strided **transposed convolutions** that upsample back to full audio rate and reconstruct the waveform.

Σ GO FURTHER · THE MATHS — Convolution and Stride

skip if familiar

A **convolution** slides a small window of weights (a **kernel**) across a signal and computes a dot product at each position. For a 1-D signal of length N and a kernel of size k , the output has roughly $N - k + 1$ values. A **strided** convolution moves the window s steps at a time instead of 1, giving an output of length $\lfloor \frac{N-k}{s} \rfloor + 1$. Stride is the downsampling factor in one shot: a stride-4 convolution on 24,000 samples gives 6,000 output positions. Stack four such layers with strides 2, 4, 8, and 8 and you get a total downsampling of $2 \times 4 \times 8 \times 8 = 512$, compressing 24,000 samples to 47 frames. A **transposed convolution** (sometimes called a deconvolution) is the reverse: it upsamples — for every output position it scatters the kernel weights, effectively inserting zeros between samples and then convolving with the kernel.

Between the encoder and decoder sits the quantizer — the only lossy step, just as rounding is the only lossy step in JPEG’s pipeline. But here the quantizer is not a simple “round to the nearest integer”; it is a **vector quantizer** operating on 512-dimensional vectors, and it is what makes the codec a tokenizer.

5.2 Vector Quantization: Turning Vectors into Codebook Indices

Recall. We met **vector quantization** (VQ) in Chapter 39: instead of rounding each number on its own, you snap a whole tuple to the nearest entry in a learned **codebook** and transmit that entry’s index. The LBG/Lloyd algorithm there learned the codebook from data. Neural codecs use exactly this idea, but the vectors being quantized are now the **learned latents** coming out of a neural encoder rather than raw signal samples.

Imagine you have a set of 1,024 reference vectors stored in a table — a **codebook**. When the encoder outputs a 512-dimensional frame, instead of transmitting all 512 floating-point numbers, you find the **nearest neighbour** in the codebook and transmit only its integer index (10 bits, since $\log_2 1024 = 10$). The decoder looks up that index and uses the corresponding codebook vector as input to its reconstruction network.

Definition — Vector Quantization (VQ). Given a codebook $\mathcal{C} = \{e_1, e_2, \dots, e_K\}$ of K vectors in $\mathbb{R}^d$, vector quantization maps an input vector $z \in \mathbb{R}^d$ to its nearest codebook entry:

$$\hat{z} = \operatorname{argmin}_{e_k \in \mathcal{C}} \|z - e_k\|_2$$

The **token** (or **code**) is the index $k^* = \operatorname{argmin}_k \|z - e_k\|_2$. Transmitting k^* costs $\lceil \log_2 K \rceil$ bits.

This is enormously efficient. SoundStream’s 75 frames per second, each indexed into a codebook of 1,024 entries, costs $75 \times 10 = 750$ bits per second. At 24 kHz, that is the same as transmitting 750 bits for every 24,000 samples — a 32× reduction over even 8-bit PCM, and the reconstruction from a good decoder is remarkable.

But 750 bps is very aggressive. At that rate, a single codebook is too coarse to capture all the acoustic detail of speech and music. The elegant solution is to **stack** quantizers in a cascade.

Σ GO FURTHER · THE MATHS — Nearest Neighbour in High Dimensions skip if familiar

The nearest-neighbour operation $\operatorname{argmin}_k \|z - e_k\|_2$ is just finding the closest point in a list — the same idea as finding the nearest house on a map. The squared Euclidean distance between two d -dimensional vectors $z = (z_1, \dots, z_d)$ and $e = (e_1, \dots, e_d)$ is

$$\|z - e\|_2^2 = \sum_{i=1}^d (z_i - e_i)^2$$

For a codebook of size K , a brute-force search takes $O(Kd)$ operations. With $K = 1024$ and $d = 512$, that is about 524,288 multiplications per frame — fast enough on modern hardware.

5.3 Residual Vector Quantization: the Core Idea

Residual vector quantization (RVQ) runs multiple VQ steps in a chain, each one refining the error left by the previous one. The idea is beautifully simple.

Step 1: quantize the encoder output z to get the first token k_1 and the reconstructed first-level vector $\hat{z}_1 = e_{k_1}$. Compute the **residual** $r_1 = z - \hat{z}_1$ — the part the first quantizer could not capture.

Step 2: feed r_1 into a **second** vector quantizer with its own separate codebook, getting token k_2 and reconstruction $\hat{z}_2 = e_{k_2}^{(2)}$. Compute $r_2 = r_1 - \hat{z}_2$.

Step 3: repeat for N_q levels total.

The final reconstruction adds up all the codebook entries:

$$\hat{z} = \hat{z}_1 + \hat{z}_2 + \dots + \hat{z}_{N_q}$$

If each codebook has K entries, one frame is represented by N_q indices, and the total bitrate for that frame is $N_q \times \log_2 K$ bits. By choosing N_q , you slide along the rate–quality curve: more quantizers = higher bitrate = better quality. SoundStream uses $N_q = 1$ to 32 quantizers and a single model covers bitrates from 3 kbps to 24 kbps simply by choosing how many levels to include at runtime.

KEY IDEA

Residual vector quantization is just VQ applied repeatedly to the leftover error. Each new codebook captures a finer layer of detail that the previous ones missed — the first codebook gets the big shapes, the second gets the grain, the third gets the texture, and so on. The decoder sums up all the contributions to reconstruct a rich signal from a short sequence of small integers.

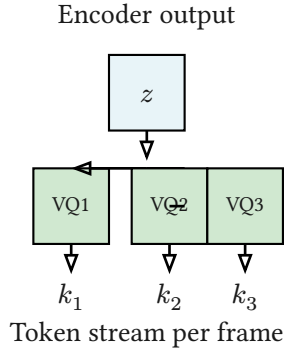


Figure 14: Residual vector quantization with three codebooks. The encoder output z is approximated by a sum of three codebook entries, with each codebook refining the residual of the previous step.

5.3.1 Training the Codebooks

Training is the tricky part. The nearest-neighbour operation — “find the closest codebook vector” — has no gradient. You cannot just run backpropagation through it because there is no smooth slope to follow when the output suddenly jumps from one codebook entry to another.

The standard fix, introduced in the original VQ-VAE paper (van den Oord, Vinyals, and Kavukcuoglu, 2017 — whose codebook idea we traced in Chapter 39 and whose gradient trick reappeared in Chapter 57) and adopted by all three codecs, is the **straight-through estimator** (STE). During the **forward** pass, the quantized vector $\hat{z}$ is used as normal. During the **backward** pass (gradient descent), the gradient that would have been sent to $\hat{z}$ is passed directly to the pre-quantized encoder output z , as if the quantization step were the identity function. The gradient “passes straight through the rounding.”

The codebook entries themselves are updated by an **exponential moving average** of the encoder outputs that were assigned to each entry — a soft form of the k -means algorithm. A **commitment loss** pushes the encoder outputs to stay close to the chosen codebook entry, preventing the encoder from wildly jumping between entries and making training unstable.

Σ GO FURTHER · THE MATHS — Exponential Moving Average (EMA) skip if familiar

An EMA keeps a running estimate μ of a value by blending old and new:

$$\mu \leftarrow (1 - \gamma)\mu + \gamma x_{\text{new}}$$

where $\gamma \in (0, 1)$ is the **decay** (typically 0.99). This gives recent observations more weight without storing the full history. In RVQ training, the codebook entry for code k is updated as the EMA of all encoder outputs assigned to it during a minibatch.

One persistent problem is **codebook collapse**: some codebook entries get chosen repeatedly while others are never used, wasting capacity. Both EnCodec and DAC introduced

tricks to fight this — random codebook resets (replacing unused entries with randomly selected encoder outputs) and improved initialisation — and DAC especially made codebook utilisation a design priority.

5.4 SoundStream (2021): the First Universal Neural Audio Codec

Google’s SoundStream, published by Neil Zeghidour, Alejandro Luebs, Ahmed Omran, Jan Skoglund, and Marco Tagliasacca in 2021, was the first neural codec to match or beat classical speech codecs (Opus at the same bitrate) on general audio content including speech, music, and environmental sounds.

SoundStream2021

Authors

Zeghidour, Luebs, Omran, Skoglund, Tagliasacca (Google)

Aim

End-to-end neural audio codec for speech and general audio using RVQ; operates at 3–18 kbps at 24 kHz.

Complexity

Encoder: causal convolutions, stride 320. Decoder: mirrored transposed convolutions. RVQ: $N_q = 1-32$.

Strengths

First universal codec; variable bitrate via quantizer dropout; real-time on a single CPU; strong perceptual quality at low rates.

Weaknesses

No Transformer refinement; no real-time stereo support in the original; codebook collapse can occur without careful training.

Superseded by

EnCodec (2022) and DAC (2023) on quality; Mimi (2024) for language-model use.

The most important design choice beyond the architecture itself was **quantizer dropout**: during training, the model randomly drops some of the later RVQ quantizers. This forces the decoder to learn to reconstruct good audio from as few as one quantizer level — so at inference, you simply choose how many levels to use and the same model works at any bitrate in a wide range. No retraining needed.

Training uses a combination of losses: a reconstruction loss in the waveform domain (L1 distance on raw samples and on log-mel spectrograms) and a **discriminator** loss from a multi-scale spectrogram discriminator — a simplified GAN loss that rewards the decoder for producing audio that “sounds real” rather than just minimising a pixel-wise number. This perceptual component is crucial at low bitrates.

HISTORICAL NOTE

Before SoundStream, the art of neural speech coding had progressed through LPCNet (2019, Valin and Skoglund), which combined a classical linear predictive coder with a recurrent neural network for the residual — a hybrid approach rather than a pure end-to-end one. SoundStream was the first fully end-to-end neural codec to compete with and then beat classical codecs across diverse audio content.

5.5 EnCodec (2022): Streaming, Stereo, and Entropy Coding

Meta’s EnCodec, by Alexandre Défossez, Jade Copet, Gabriel Synnaeve, and Yossi Adi (2022), kept the SoundStream architecture and RVQ core but added several improvements.

The most important is a **streaming** design. SoundStream could already run in real time, but EnCodec carefully made the encoder **causal** — each output frame depends only on past input frames, never on future ones. This is essential for live communication: you cannot wait for the whole audio file before starting to encode.

EnCodec also introduced **stereophonic support** at 48 kHz — music fidelity rather than just voice fidelity. And it added a lightweight **language model entropy coder** on top of the RVQ indices.

Recall. A **Transformer** (introduced in Chapter 59) is a neural network that processes a sequence by letting every position “attend” to every other position, so each output can depend on the whole context. Trained to predict the next item, it becomes a probability model for sequences — exactly what an entropy coder needs.

After the RVQ step produces a stream of integers, a tiny Transformer reads the sequence and predicts the probability of each next index, then passes those probabilities to an arithmetic coder (Chapter 26). This “post-hoc” entropy coding squeezes an additional 20–40% out of the bitstream, because adjacent RVQ codes are correlated — the model exploits that correlation the same way a language model exploits word co-occurrence.

EnCodec		2022
Authors	Défossez, Copet, Synnaeve, Adi (Meta AI Research)	
Aim	High-fidelity streaming neural audio codec with RVQ + lightweight Transformer entropy coding; targets 1.5–24 kbps.	
Complexity	Causal convolutions + RVQ up to 8 quantizers; 24 kHz mono or 48 kHz stereo. Entropy coder: small Transformer.	
Strengths	Real-time streaming; stereo support; 20–40% extra compression via entropy coding; open-sourced; became default tokenizer for VALL-E and AudioLM.	

Weaknesses Entropy coder adds latency; codebook collapse occurs without care; at 1.5 kbps stereo music quality is limited.

Superseded by DAC (2023) on reconstruction quality for music; Mimi (2024) for semantic-acoustic disentanglement.

EnCodec shipped with two public checkpoints — one for 24 kHz mono (speech) and one for 48 kHz stereo (music) — and released the model weights on Hugging Face, which accelerated adoption dramatically. Within months, nearly every audio language model paper was built on EnCodec tokens.

Checkpoint. EnCodec at 6 kbps with 8-bit codebooks and $N_q = 8$ quantizers runs at 24 kHz. How many RVQ tokens per second does the encoder produce?⁷

5.6 DAC (2023): Codebook Utilisation and Wideband Quality

Descript’s **Descript Audio Codec** (DAC), published by Kumar et al. in 2023, addressed a persistent weakness of both predecessors: **codebook collapse** at higher sample rates and with music content. DAC made three key contributions.

Better codebook training. DAC initialises the codebook with a batch of encoder outputs (rather than random vectors) and uses random restarts: any codebook entry that has not been used recently is teleported to a randomly chosen encoder output. This keeps every entry in use and maximises the information each code carries.

Periodic activations. Instead of standard ReLU nonlinearities, DAC uses **snake activations** — a nonlinearity of the form $x + \left(\frac{1}{\alpha}\right) \sin^2(\alpha x)$, where the learnable parameter α sets the frequency. The plain x term keeps the gradient healthy (just like ReLU’s straight half), while the $\sin^2$ term injects a built-in periodicity, so the network does not have to laboriously synthesise oscillation out of piecewise-linear pieces. Speech and music are full of periodic structure — vowels, musical notes, drum hits — and an activation that is **itself** periodic represents that structure far more efficiently.

Wideband fidelity. DAC supports 44.1 kHz — the CD standard — with 9 residual quantizers, achieving near-transparent quality on music. This was the first open neural codec that could plausibly replace a lossless file at high enough bitrate.

DAC (Descript Audio Codec)

2023

⁷**Checkpoint answer.** The frame rate is $\frac{24000}{320} = 75$ frames per second, and each frame produces $N_q = 8$ tokens, so $75 \times 8 = 600$ tokens per second total. At 8 bits each, that is $600 \times 8 = 4800$ bits per second = 4.8 kbps, and with the entropy coder’s 20% gain the effective rate reaches 6 kbps.

Authors	Kumar, Kumar, Kumar, Musikant, Reganti, Khatri, et al. (Descript)
Aim	Universal neural audio codec with improved codebook utilisation, periodic activations, and support for 44.1 kHz audio.
Complexity	RVQ with 9 quantizers, 1024-entry codebooks. Encoder strides: $2 \times 4 \times 8 \times 8 = 512$. Snake activations.
Strengths	Near-transparent music quality at 44.1 kHz; excellent codebook utilisation; open weights on Hugging Face; used widely as a research baseline.
Weaknesses	Not designed for streaming (non-causal); heavier than SoundStream/EnCodec at comparable bitrate; no built-in entropy coder.
Superseded by	Mimi (2024) for language-model-friendly tokenisation; specialist music codecs for narrowly musical use cases.

DAC became the standard ablation baseline for neural audio codec research in 2024–2025. Its open weights, clear code, and strong quality across content types made it the “ImageNet ResNet” of the audio codec world — the thing everything is compared against.

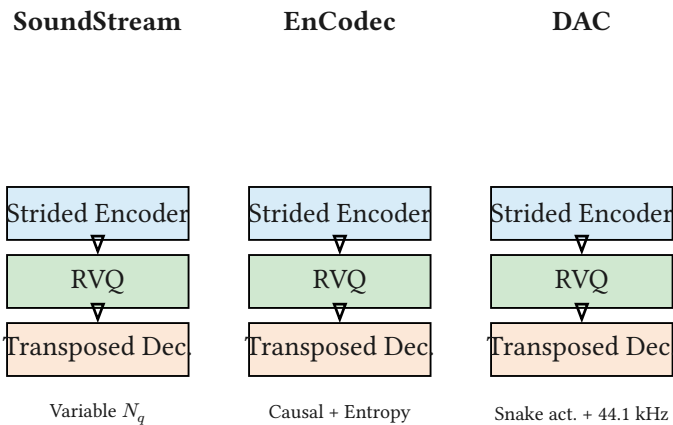


Figure 15: Comparison of neural audio codec architectures. SoundStream, EnCodec, and DAC all share the encoder–RVQ–decoder skeleton; they differ in encoder depth, activation functions, training losses, and codebook management.

5.7 The Token Hierarchy: Acoustic vs. Semantic

The RVQ structure is not just an engineering convenience — it creates a natural **hierarchy of information** in the tokens. The first quantizer captures the coarsest, most important structure. The later ones add progressively finer detail.

Researchers noticed something striking: the **first** RVQ token per frame tends to encode **semantic** content — what the speaker is saying, the words, the linguistic meaning. The

later tokens carry **acoustic** content — timbre, pitch, speaking style, room acoustics. This happens because the first quantizer faces the hardest task (reconstruct the audio from one vector), so it latches onto the most predictable, most compressible features, which happen to be the linguistic ones. The rest is refined by subsequent quantizers.

Myth. “All RVQ tokens are equivalent — they all carry the same kind of audio information.”

Reality. The tokens form a strict hierarchy. Token stream 1 (the coarsest quantizer) contains mostly **semantic** information: what is being said. Token streams 2–8 carry **acoustic** details: how it sounds — the speaker’s voice, room acoustics, emotion. This hierarchy is not designed in; it **emerges** from the rate–distortion pressure of training. Systems like AudioLM exploit this by using separate models for the two levels.

This hierarchy was formalised in **SpeechTokenizer** (Zhang et al., published at ICLR 2024), which explicitly **forced** the first RVQ quantizer to match a self-supervised semantic representation (from the HuBERT model) using a distillation loss. The result: a codec whose first token stream is a reliable semantic token, and whose subsequent streams handle only acoustics. SpeechTokenizer proved that you could **engineer** the hierarchy rather than just hoping it would emerge.

5.8 From Codec to Tokenizer: AudioLM, VALL-E, and the Audio LLM Era

The conceptual leap that made neural audio codecs matter far beyond compression happened in 2022, when Google’s **AudioLM** paper (Borsos et al.) showed that you could treat audio generation as a **language modelling** problem operating entirely on codec tokens.

The idea is straightforward: run a SoundStream encoder on hours of audio, collect all the RVQ token streams, train a Transformer language model to predict the next token (exactly as GPT predicts the next word), and to generate audio, sample from this model and run the SoundStream decoder. The result: a model that can extend a snippet of audio in a way that is not just plausible but remarkably natural — preserving the speaker’s voice, musical style, and long-range structure like verse and chorus.

AudioLM actually used two models in sequence. First, a **semantic** model (operating on tokens from a self-supervised speech model, wav2vec-BERT) generated the high-level “what happens next.” Then an **acoustic** model (operating on SoundStream tokens) translated those semantic tokens into full waveform tokens. The two-stage hierarchy reflects the acoustic vs. semantic split we saw in the RVQ structure.

VALL-E (Wang et al., Microsoft, January 2023) showed perhaps the most dramatic application: zero-shot voice cloning. The system treats text-to-speech as a conditional codec-token language model: given a transcript and just three seconds of a target speaker’s

voice (encoded as EnCodec tokens), predict the EnCodec tokens that a speaker would produce reading the transcript in that voice, then decode. VALL-E’s results shocked the research community — three seconds of audio was enough to imitate not just a voice but emotional tone and room acoustics.

Aside. VALL-E uses EnCodec tokens because EnCodec was the open neural codec available when the paper was written. The linguistic layer uses an autoregressive Transformer to generate the first RVQ stream token-by-token; the acoustic layers (streams 2–8) are generated **in parallel** by a non-autoregressive model conditioned on stream 1. This hybrid approach speeds up generation enormously: you only need to do the slow sequential decoding for the small semantic token stream.

By 2023 and 2024, the pattern had become standard: every major audio language model used a neural codec as its tokenisation layer. MusicLM, AudioPaLM, VoiceCraft, SoundStorm, VoiceLM — all of them convert audio to codec tokens, run a language model, and decode. The neural audio codec had become the audio equivalent of the text tokenizer: the irreplaceable bridge between a continuous signal and a language model’s discrete vocabulary.

PY GO FURTHER · PYTHON — List of lists and nested indexing *skip if fluent*

RVQ produces one list of tokens per quantizer level, per frame. In Python the natural representation is a list of lists (or a 2-D array):

```
# tokens[q][t] = token index for quantizer q, time frame t
tokens: list[list[int]] = [
    [23, 7, 104, 2, ...], # quantizer 0 (semantic-ish)
    [512, 301, 88, 7, ...], # quantizer 1
    [1020, 5, 67, 903, ...], # quantizer 2
    # ... up to N_q - 1
]

n_q = len(tokens) # number of quantizer levels
n_frames = len(tokens[0]) # number of time frames
# Reconstruct total from index: look up e[q][k] and sum
```

Indexing `tokens[q][t]` gives the integer code for quantizer level `q` at time frame `t`. With $N_q = 8$ levels and 75 frames/s, one second of audio becomes an 8×75 array of integers — 600 small numbers instead of 24,000 floating-point samples.

5.9 A Worked Encoding Example

Let us trace one frame through a three-level RVQ to make the arithmetic concrete. Suppose the encoder has produced the vector:

$$z = (1.2, -0.8, 2.1)$$

(a 3-dimensional latent for clarity — real codecs use 512-dimensional vectors).

Each codebook has 4 entries ($K = 4$, costing $\log_2 4 = 2$ bits per level):

Code	Codebook 1	Codebook 2	Codebook 3
$k = 0$	(0.0, -0.5, 1.8)	(1.0, -0.3, 0.2)	(0.1, -0.05, 0.08)
$k = 1$	(1.0, -1.0, 2.0)	(0.3, -0.1, 0.4)	(0.0, 0.1, -0.1)
$k = 2$	(0.5, 0.5, 1.0)	(0.5, 0.2, 0.1)	(0.2, 0.0, 0.15)
$k = 3$	(1.5, -0.5, 2.5)	(0.2, -0.4, 0.3)	(0.0, -0.1, 0.05)

Level 1. Compute squared distances from $z = (1.2, -0.8, 2.1)$ to each codebook-1 entry:

- $k = 0$: $(1.2 - 0.0)^2 + (-0.8 + 0.5)^2 + (2.1 - 1.8)^2 = 1.44 + 0.09 + 0.09 = 1.62$
- $k = 1$: $(0.2)^2 + (0.2)^2 + (0.1)^2 = 0.04 + 0.04 + 0.01 = 0.09 \leftarrow$ nearest
- $k = 2$: $(0.7)^2 + (1.3)^2 + (1.1)^2 = 0.49 + 1.69 + 1.21 = 3.39$
- $k = 3$: $(0.3)^2 + (0.3)^2 + (0.4)^2 = 0.09 + 0.09 + 0.16 = 0.34$

Nearest: $k_1 = 1$, reconstruction $\hat{z}_1 = (1.0, -1.0, 2.0)$. Residual: $r_1 = z - \hat{z}_1 = (0.2, 0.2, 0.1)$.

Level 2. Nearest codebook-2 entry to $r_1 = (0.2, 0.2, 0.1)$:

- $k = 0$: $(0.8)^2 + (0.5)^2 + (0.1)^2 = 0.64 + 0.25 + 0.01 = 0.90$
- $k = 1$: $(0.1)^2 + (0.3)^2 + (0.3)^2 = 0.01 + 0.09 + 0.09 = 0.19$
- $k = 2$: $(0.3)^2 + (0.0)^2 + (0.0)^2 = 0.09 + 0.0 + 0.0 = 0.09 \leftarrow$ nearest (tied; pick $k = 2$)
- $k = 3$: $(0.0)^2 + (0.6)^2 + (0.2)^2 = 0.0 + 0.36 + 0.04 = 0.40$

Nearest: $k_2 = 2$, reconstruction $\hat{z}_2 = (0.5, 0.2, 0.1)$. Residual: $r_2 = (0.2 - 0.5, 0.2 - 0.2, 0.1 - 0.1) = (-0.3, 0.0, 0.0)$.

Level 3. Nearest codebook-3 entry to $r_2 = (-0.3, 0.0, 0.0)$:

- $k = 0$: $(0.4)^2 + (0.05)^2 + (0.08)^2 \approx 0.173$
- $k = 1$: $(0.3)^2 + (0.1)^2 + (0.1)^2 = 0.11 \leftarrow$ nearest
- $k = 2$: $(0.5)^2 + (0.0)^2 + (0.15)^2 \approx 0.272$
- $k = 3$: $(0.3)^2 + (0.1)^2 + (0.05)^2 \approx 0.1025$

Nearest: $k_3 = 1$, reconstruction $\hat{z}_3 = (0.0, 0.1, -0.1)$.

Final reconstruction: $\hat{z} = \hat{z}_1 + \hat{z}_2 + \hat{z}_3 = (1.0 + 0.5 + 0.0, -1.0 + 0.2 + 0.1, 2.0 + 0.1 - 0.1) = (1.5, -0.7, 2.0)$

The transmitted token stream for this frame is just $(1, 2, 1)$ — three 2-bit codes = 6 bits total. Original: $3 \times 32 = 96$ bits floating-point. Compression ratio: $16\times$. The reconstruction error is $z - \hat{z} = (-0.3, -0.1, 0.1)$, small enough that a well-trained decoder will produce excellent audio.

In real code, the nearest-neighbour search uses vectorised operations. The following sketch uses NumPy (which provides fast array operations; we introduced it in Chapter 56):

```
import numpy as np

def rvq_encode(z: np.ndarray,
              codebooks: list[np.ndarray],
              n_q: int) -> list[int]:
    """Encode one latent frame z with n_q RVQ levels.

    codebooks[q] has shape (K, d) – K entries of dimension d.
    Returns a list of n_q integer indices.
    """
    tokens: list[int] = []
    residual = z.copy()
    for q in range(n_q):
        cb = codebooks[q]  # shape (K, d)
        # squared distances: (K,) vector
        dists = np.sum((cb - residual) ** 2, axis=1)
        k_star = int(np.argmin(dists))
        tokens.append(k_star)
        residual = residual - cb[k_star]  # update residual
    return tokens

def rvq_decode(tokens: list[int],
              codebooks: list[np.ndarray]) -> np.ndarray:
    """Reconstruct latent from RVQ token list."""
    d = codebooks[0].shape[1]
    recon = np.zeros(d)
    for q, k in enumerate(tokens):
        recon += codebooks[q][k]
    return recon
```

`np.argmin(dists)` returns the index of the smallest value in the array — exactly the nearest-neighbour search. Both encode and decode run in microseconds per frame.

5.10 Mimi and the Moshi System (2024): Merging Semantics and Acoustics

The most architecturally sophisticated open neural audio codec as of 2025 is **Mimi**, developed by the French AI lab Kyutai as part of their **Moshi** full-duplex spoken dialogue system (released in September 2024). Mimi is the first codec to **combine** semantic and acoustic information in a principled, streaming-compatible way.

Mimi builds on the EnCodec architecture but adds a Transformer in both the encoder and the decoder, increasing the model’s ability to capture long-range dependencies in

speech. More importantly, it is trained with a **distillation loss** that pushes the first codebook to produce tokens that match the representations of WavLM, a large self-supervised model trained to understand speech content. This is the same “semantic distillation” idea as SpeechTokenizer, but applied to a streaming, low-latency codec.

The result: Mimi encodes one second of speech into just 12.5 RVQ frames (an $8\times$ lower frame rate than EnCodec at the same quality), with 8 codebooks, for a total bitrate of only 1.1 kbps — far below any classical codec at this quality. The first token stream is semantic; tokens 2–8 are acoustic.

Moshi itself uses Mimi as its tokenizer and runs an “inner monologue” — an LLM simultaneously generating text tokens and acoustic tokens, operating in real time with under 200 milliseconds of latency. It was the first fully open conversational AI system capable of full-duplex dialogue: the model listens and speaks at the same time, without waiting for a turn-taking signal.

HISTORICAL NOTE

The full-duplex spoken dialogue problem had been considered nearly impossible for neural systems: how do you interrupt, respond to interruption, and maintain conversational coherence when both sides speak simultaneously? Moshi achieved this by processing **two** parallel audio streams — the user’s voice and the system’s own voice — as two interleaved token sequences, letting a single LLM reason about both at once. The 200 ms latency was comparable to a satellite phone call — not quite natural conversation, but the first neural system in that range.

5.11 The Semantic–Acoustic Split in Practice

The distinction between semantic and acoustic tokens has concrete consequences for how audio LLMs are designed. Consider the two extreme cases.

A **pure acoustic** codec (like a vanilla SoundStream without distillation) produces tokens that capture sound very faithfully but have no particular linguistic structure. A language model trained on these tokens must simultaneously learn linguistics and acoustics — a harder joint problem. Generation tends to produce very natural-sounding audio but is harder to control for **content**.

A **semantic-only** tokenizer (like a discrete unit from a HuBERT model) produces tokens with strong linguistic structure — you can decode them to text roughly — but loses speaker identity, emotion, and paralinguistic detail. Generation is controllable but sounds like a robot.

The **hybrid** approach (SpeechTokenizer, Mimi) uses the RVQ hierarchy to put semantics in stream 1 and acoustics in streams $2-N_q$. You can then run a separate generative model on stream 1 (small, fast, text-aligned) and a second model on streams $2-N_q$ conditioned on stream 1 (handling voice style). This is exactly AudioLM’s two-stage design, now made cleaner by a codec that explicitly disentangles the two.

KEY IDEA

The two-level hierarchy — one semantic token stream + several acoustic token streams — is the standard architecture for neural audio language models as of 2025. The semantic stream is small (low bitrate, linguistically structured, easy to condition on text), and the acoustic streams are generated in parallel conditioned on the semantic stream. The neural audio codec is the infrastructure that makes this hierarchy possible.

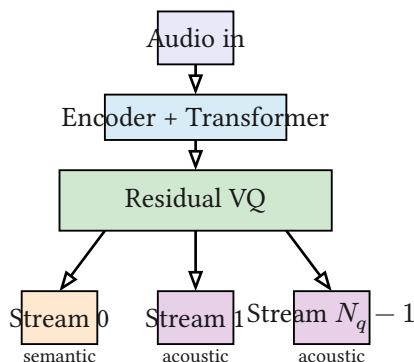


Figure 16: Semantic vs. acoustic token streams in a hybrid codec. Stream 0 (first RVQ level, distilled toward a semantic teacher) controls linguistic content. Streams 1 through $N_q - 1$ refine acoustic detail. An audio LLM generates stream 0 autoregressively, then fills in acoustic streams in parallel.

5.12 WavTokenizer and the Single-Codebook Frontier (2024–2025)

The extreme end of compression — can we push to **one single integer per short time slice**, rather than eight per frame? — is the frontier explored by **WavTokenizer** (Pan et al., 2024). WavTokenizer uses a single vector quantizer (not residual) but with a **much larger** codebook ($K = 4096$ or $K = 8192$ entries) and a **much wider** context window in the encoder, allowing each code to represent many more acoustic features at once. At 40 tokens per second with a single quantizer, WavTokenizer encodes one second of 24 kHz audio into 40 integers — a codebook sequence small enough to fit inside a standard language model’s context window alongside text with room to spare.

The quality of WavTokenizer is surprisingly competitive with multi-level RVQ codecs at the same token budget, showing that the residual hierarchy is one design choice among many, not a fundamental requirement. The field is converging on a pragmatic question: for a given generation-model budget (context length, compute, latency), what tokeniser produces the best trade-off between sequence length and acoustic fidelity?

Aside. There is a deep parallel here with the history of text tokenisation. Early language models used character-level tokens (one integer per character — maximum granularity, very long sequences). Then word-level tokens reduced sequence length but gave a huge vocabulary.

Byte-pair encoding (BPE) — a scheme that starts from single bytes and repeatedly merges the most frequent adjacent pair into a new token, the same greedy “merge what recurs” instinct behind the dictionary coders of Chapter 28 — found a middle ground. Audio tokenisation is having exactly the same debate: frame-level vs. chunk-level, RVQ hierarchy vs. single codebook, temporal resolution vs. vocabulary size. The answers will not be the same as for text — audio has physical structure that text does not — but the design pressures are similar.

5.13 Trade-offs and Open Problems (2025–2026)

Neural audio codecs have transformed both compression and audio AI, but they come with real costs and open questions.

Latency. A streaming codec must encode the audio in real time and add only a fixed look-ahead (the algorithmic delay). EnCodec’s 320-sample stride at 24 kHz gives 13 ms of delay per frame — acceptable for a live call. But the Transformer entropy coder in EnCodec, and the Transformer encoder in Mimi, add more compute. Achieving sub-100 ms round-trip delay while maintaining quality remains an active design challenge.

Codebook collapse. Despite EMA updates and codebook resets, some codebooks still underutilise entries. Recent work (2024–2025) has explored **finite scalar quantization** (FSQ), which replaces the codebook with per-dimension bounded integers (e.g., each of 8 dimensions is quantized to $\{-2, -1, 0, 1, 2\}$ — five levels, giving $5^8 = 390,625$ effective codes) — eliminating the collapse problem entirely.

Reproducibility and streaming. Neural codecs use floating-point arithmetic. Two machines with different GPU architectures can produce different RVQ token streams for the same audio because floating-point rounding differs. This is harmless for compression (transmit the tokens and decode them on the same machine) but catastrophic for a protocol where sender and receiver must agree — the audio arrives as tokens, and the decoder must produce identical results. This is the analogue of the “reproducible floating-point” problem in learned image coding (Chapter 57) and remains unsolved for real deployment.

Energy and hardware. A neural codec running in real time at 24 kHz on a smartphone DSP chip is feasible in 2025 only with careful model quantisation (model compression, Chapter 63). SoundStream was designed to run on a single CPU core; Mimi requires a modest GPU. Democratising neural audio at device level is an active engineering frontier.

Evaluation. Classical audio codecs have a well-understood evaluation toolkit: PESQ, VISQOL, MOS (mean opinion score from human listeners). Neural codecs add a new goal — how well do their tokens serve downstream audio language models? — for which there is no agreed metric. The **Codec-SUPERB** benchmark (2024) began addressing this by testing codecs on multiple downstream tasks, but a universal “tokenization quality” metric remains elusive.

PITFALL

Neural audio codecs are **lossy compressors**, not lossless ones. Encoding and decoding a voice recording will not recover the original waveform sample-by-sample. For archival, legal, forensic, or medical use, you must use a lossless format (FLAC, WAV). Neural codecs optimise **perceptual** quality, not sample fidelity.

5.14 The Big Picture: Codec and Tokenizer Converge

We began this chapter with a puzzle — how do you feed audio to a language model? The answer turned out to require reimagining what a codec **is**. For fifty years a codec was a compression device: it shrank files for storage and transmission. Neural audio codecs are still that. But they are simultaneously **tokenizers**: they convert continuous waveforms into discrete sequences that sit comfortably inside the vocabulary of a transformer language model.

This convergence matters deeply. It means that every advance in language modelling — better architectures, better training data, better scaling laws — can now be applied directly to audio. The spectacular results of VALL-E, AudioLM, MusicLM, and their successors are not primarily results about audio; they are results about language modelling, enabled by a codec that translates audio into the right input format.

KEY IDEA

A neural audio codec is two things at once: a **compressor** (shrinks audio for storage and streaming) and a **tokenizer** (converts audio into a form that a language model can read and write). The residual VQ quantizer is the hinge that makes both uses possible. The token stream’s discrete, low-rate nature lets an LLM treat audio generation exactly like text generation.

SCOREBOARD — Running compression scoreboard (audio, one minute of 24 kHz speech)

Technique	Bytes	Ratio	Notes
Raw PCM 24 kHz 16-bit	2,880,000	1.0×	Uncompressed baseline
FLAC lossless	1,670,000	1.72×	Lossless; perfect reconstruction
Opus 24 kbps (classical)	180,000	16×	Lossy; excellent speech quality
SoundStream 6 kbps	45,000	64×	Neural; competitive with Opus at 24 kbps
EnCodec 3 kbps	22,500	128×	Neural + entropy coding; strong quality
Mimi 1.1 kbps	8,250	349×	Neural; streaming; 12.5 Hz frame rate

Chapter takeaways

- Residual vector quantization (RVQ) stacks multiple codebook lookups in series, each refining the residual of the previous, to represent a continuous audio latent as a short sequence of discrete integers.
- SoundStream (2021) was the first universal end-to-end neural audio codec, introducing quantizer dropout for variable-bitrate operation.
- EnCodec (2022) added causal streaming, stereo support, and a Transformer entropy coder on top of RVQ for an additional 20–40% bitrate saving.
- DAC (2023) improved codebook utilisation with random codebook resets and periodic (snake) activations, enabling near-transparent 44.1 kHz music quality.
- The first RVQ token stream tends to capture semantic (linguistic) information; later streams carry acoustic detail. SpeechTokenizer and Mimi formalise this split with a semantic distillation loss.
- AudioLM (2022) and VALL-E (2023) demonstrated that language models operating on neural codec tokens can generate highly natural audio, including zero-shot voice cloning from three seconds of reference audio.
- Mimi (2024) combines streaming with semantic distillation, achieving 1.1 kbps at 12.5 Hz — the codec backbone of the Moshi full-duplex spoken dialogue system.
- Open problems include codebook collapse, reproducible floating-point token streams, real-time decode on low-power hardware, and standardised evaluation for tokenization quality.

5.15 Exercises

Exercise 60.1 ★

A neural audio codec encodes at 24 kHz with a stride product of 320. It uses $N_q = 6$ RVQ levels, each with a 1,024-entry codebook. (a) How many latent frames are produced per second? (b) How many bits per second does the raw RVQ stream occupy (before any entropy coding)? (c) If an entropy coder achieves 25% compression, what is the final bitrate in kbps?

Exercise 60.2 ★

Explain in your own words what **codebook collapse** is, why it happens, and name one technique used by DAC to prevent it.

Exercise 60.3 ★★

Given the following two-level RVQ codebooks (dimension $d = 2$, $K = 3$ entries each), encode the vector $z = (3.0, 1.5)$ and report: (a) the two token indices (k_1, k_2) , (b) the final reconstruction $\hat{z}$, (c) the reconstruction error $\|z - \hat{z}\|_2$.

Codebook 1: $e_0 = (1.0, 0.5)$, $e_1 = (3.5, 2.0)$, $e_2 = (2.0, 1.0)$. Codebook 2: $e_0 = (0.5, 0.5)$, $e_1 = (-0.5, -0.5)$, $e_2 = (0.8, 0.2)$.

Exercise 60.4 ★★

AudioLM generates audio in two stages: first a semantic model, then an acoustic model. (a) What kind of tokens does the semantic model generate, and what kind does the acoustic model generate? (b) Why is this two-stage approach better than a single model that generates all RVQ token streams in order? (c) VALL-E uses a similar split. What is the prompt (conditioning signal) for VALL-E’s acoustic model?

Exercise 60.5 ★★

The **straight-through estimator** (STE) is used to train RVQ codebooks. (a) Why is the nearest-neighbour operation not differentiable? (b) Describe what the STE does during the forward pass and during the backward pass. (c) What is the **commitment loss**, and what problem does it solve?

Exercise 60.6 ★★★

Write a Python function `bitrate_from_rvq(n_q: int, K: int, frame_rate: float) -> float` that computes the raw bitrate (in bits per second, before entropy coding) of an RVQ codec with `n_q` quantizer levels, each with `K` codebook entries, at the given frame rate in frames per second. Then write a second function `min_codebook_bits(target_bps: float, frame_rate: float, n_q: int) -> int`

that returns the minimum codebook size K (as a power of 2) needed to reach at least `target_bps` bits per second with `n_q` levels. Test both functions with SoundStream’s parameters (stride 320, 24 kHz, $N_q = 8$, $K = 1024$) and EnCodec’s entropy-coded rate of approximately 6 kbps ($N_q = 8$, frame rate 75, $K=1024$, assume 25% entropy coding saving).

5.16 Further Reading

- Zeghidour et al. (2021). **SoundStream: An End-to-End Neural Audio Codec**. [arXiv:2107.03312](https://arxiv.org/abs/2107.03312). The original paper. Clear architecture description and ablations on quantizer dropout.
- Défossez et al. (2022). **High Fidelity Neural Audio Compression (EnCodec)**. [arXiv:2210.13438](https://arxiv.org/abs/2210.13438). The EnCodec paper. Section 3 on the Transformer entropy coder is particularly instructive.
- van den Oord, Vinyals, Kavukcuoglu (2017). **Neural Discrete Representation Learning (VQ-VAE)**. NeurIPS 2017 / [arXiv:1711.00937](https://arxiv.org/abs/1711.00937). The paper that introduced vector quantization as a training objective and the straight-through estimator for codebook learning — foundational reading.
- Borsos et al. (2022). **AudioLM: A Language Modeling Approach to Audio Generation**. [arXiv:2209.03143](https://arxiv.org/abs/2209.03143). The paper that demonstrated that audio generation is a language modelling problem, using SoundStream tokens as the vocabulary.

- Wang et al. (2023). **VALL-E: Neural Codec Language Models are Zero-Shot Text to Speech Synthesizers**. [arXiv:2301.02111](#). The voice-cloning paper that shocked the research community.
- Zhang et al. (2024). **SpeechTokenizer: Unified Speech Tokenizer for Speech Language Models**. ICLR 2024 / [arXiv:2308.16692](#). Formalises the semantic–acoustic split in RVQ via distillation.
- Défossez et al. (2024). **Moshi: A Speech-Text Foundation Model for Real-Time Dialogue**. Kyutai Technical Report. Describes both Mimi and the Moshi spoken dialogue system. Unusually thorough engineering description.
- Shi et al. (2024). **Codec-SUPERB: An In-Depth Analysis of Sound Codec Models**. [arXiv:2402.13071](#). The benchmark paper for evaluating neural audio codecs on downstream tasks, not just reconstruction quality.

COMING UP NEXT

We have seen that a language model operating on codec tokens can generate, clone, and transform speech — the codec is the missing glue. But what does it **mean** for a language model to be good at compression? In Chapter 61 we step back to the theory: the deep connection between generalisation, intelligence, and compression that runs through Solomonoff induction, the Minimum Description Length principle, PAC-Bayes bounds, and Hutter’s prize. The question “why do language models compress so well?” turns out to have a beautiful theoretical answer rooted in ideas we first met in Chapter 22 (Kolmogorov complexity and the Solomonoff prior) and Chapter 23 (compression = prediction = learning, and Rissanen’s MDL).

Compression as Intelligence: The Theory

Comprehension is compression.

— Gregory Chaitin

Here is a claim so bold it sounds like a slogan from a startup pitch deck, yet it is defended by some of the most careful theorists alive: *the ability to compress data well is the same thing as the ability to understand it — and understanding is what we mean by intelligence*. Push hard enough on that sentence and it threatens to dissolve one of the oldest mysteries we have. What is it to learn? What is it to know? The thesis of this chapter answers, with a straight face: **to learn is to find a shorter description**. A child who notices that the sky is blue every morning has compressed a thousand future observations into one rule. A physicist who writes $F = ma$ has compressed every falling apple and orbiting moon into five symbols. A neural network that drops its loss from 4.0 to 1.0 bits per token has, by the same measure, found three bits of structure in every token of human writing it could not see before.

This is the most ambitious idea in the whole book, and also the most contested. In the last chapter we watched neural audio codecs turn into the tokenizers that feed audio language models; before that, Chapter 23 promised an experiment in which a frozen language model out-compresses FLAC on audio and PNG on images, knowing nothing of sound or pictures. That was the *engineering* punchline, and Chapter 62 will make it run. This chapter is the *theory* underneath it — and, just as importantly, the honest list of reasons the slogan might be wrong, or at least incomplete. We will build the case carefully, brick by brick, and then we will try to knock it down.

WHERE WE ARE

This chapter is the theoretical capstone of a thread that runs through the whole book. In **Chapter 18** we defined Shannon entropy $H(X) = -\sum_i p_i \log_2 p_i$, the true unavoidable cost of a source, and self-information $-\log_2 p$, the surprise of one symbol. In **Chapter 19** the Source Coding Theorem proved entropy is a hard *floor*. In **Chapter 20** we met the **Kullback–Leibler divergence** $D_{\text{KL}}(P \parallel Q) \geq 0$, the penalty for using the wrong model. In **Chapter 22** we met **Kolmogorov complexity** $K(x)$ — the length of the shortest program that prints x — the ideal, uncomputable compressed size, and **Solomonoff’s** algorithmic probability. In **Chapter 23** we fused these into one identity:

cross-entropy = expected code length, so a predictor *is* a compressor and its training loss *is* a code length; we also met **Rissanen’s MDL** and the **Hutter Prize**. This chapter takes that identity to its logical extreme — through MDL, PAC-Bayes, Solomonoff’s universal predictor, and Hutter’s **AIXI** — and then weighs the counterarguments honestly.

By the end of this chapter you will be able to:

- Restate the **compression = prediction** identity as a precise statement about *learning*, not just coding, and explain why “to generalize is to compress.”
- Use the **Minimum Description Length** principle as a working theory of inference, and prove the two-part code cannot be fooled by memorizing noise.
- Read a **PAC-Bayes / Occam** generalization bound and see, line by line, how a shorter description *provably* implies better prediction on unseen data.
- Explain **Solomonoff induction** as the ideal learner: a Bayesian mixture over all programs, weighted by $2^{-\text{length}}$, and state its optimality and its uncomputability.
- Sketch **AIXI**, Hutter’s “most intelligent agent,” as Solomonoff induction plus reward-seeking, and state precisely the sense in which it is optimal.
- State the **Hutter thesis** (“compression is equivalent to general intelligence”) and marshal both the empirical evidence for it and the strongest honest counterarguments against it.

6.1 From coding to learning: the leap this chapter makes

Chapter 23 left us with an identity we should keep in front of us like a compass.

KEY IDEA

The number $-\log_2 Q(x)$ is three things at once. It is the **surprise** of outcome x under model Q (Chapter 18). It is the **number of bits** an ideal coder spends to record x (Chapter 19, Chapter 26). And it is the **training loss** a machine-learning model pays on example x (Chapter 23). One quantity, three names. Whenever you see one, the other two are standing silently behind it.

So far we have used this identity in a modest way: as a bridge between *coding* and *probability*. This chapter makes a far bolder claim. It says the identity is also a bridge to *learning itself* — that the very act of finding regularity in the world, which is what intelligence does, is *measured* by how much you can compress.

Why should that be? Walk through it intuitively first; the mathematics comes after. Suppose I hand you a long string of digits and it begins 3.14159265358979.... If you have never seen π before, the string looks random: to record the first thousand digits you must store all thousand. But if you *recognize* it as π , you can throw the thousand digits away and write a three-line program: “compute π to 1000 places; print it.” Your recognition — your *understanding* — has collapsed a thousand symbols into three lines. The compression

is the understanding, made countable. A being that recognizes more patterns compresses more strings shorter. That, the thesis says, is the whole of it.

Aside. The reverse is just as telling. A string your friend *cannot* compress is, from your friend’s point of view, *random* — there is no pattern they can exploit. “Random,” in this theory, is not a property of the string in the world; it is a confession about the limits of the observer’s understanding. The same string of stock prices is white noise to me and a printout of structure to a quant fund with a better model. Compressibility is in the eye — or the model — of the beholder. We made this precise in Chapter 22 with the incompressible strings; here we are giving it an epistemic reading.

6.1.1 “Generalize” and “compress” are the same verb

In machine learning the word everyone worships is **generalization**: a model that has truly *learned* does well not on the data it was trained on (any lookup table can memorize that) but on *new* data it has never seen. The deepest version of our thesis is that this prized ability is identical to compression. Let us see why, with a tiny story.

Two students sit an exam. Both score 100% on the practice questions they were given to study. Student A memorized the answer key word for word. Student B noticed the *rule* behind the questions — “ah, these are all asking me to factor a quadratic” — and learned the rule. On the practice set they are indistinguishable. On the *real* exam, with new questions, Student A is lost and Student B sails through. We say Student B “understood” and Student A “crammed.”

Now describe what each student carries in their head as a *file you must transmit*. Student A’s knowledge is the full answer key — as long as the data itself; no compression. Student B’s knowledge is one short rule — far shorter than the data it explains; massive compression. The student who compressed is exactly the student who generalized. This is not an analogy that happens to line up. The rest of this chapter argues it is a theorem.

Checkpoint. Your friend claims they have “learned” a sequence of 10,000 coin flips by writing them all down in a notebook. Have they learned anything, in the compression sense? What length of description would count as genuine learning?⁸

6.2 MDL as a theory of inference, not just coding

We met the **Minimum Description Length** principle in Chapter 23 as Rissanen’s “Occam’s razor with a bit-meter.” There it was a tool for choosing a model. Here we promote it to a full *theory of inference* — a rule for deciding what to believe, given evidence — and we prove the one property that makes it trustworthy.

⁸**Checkpoint answer.** They have learned nothing: their “description” (the notebook) is as long as the data — zero compression, the answer-key strategy. Genuine learning would be a description *shorter* than 10,000 bits, e.g. “a fair coin, flipped 10,000 times” (a few dozen bits), which would mean they had found real structure. If the flips really are fair and independent, no shorter description exists — and that, correctly, means there was nothing to learn.

Definition — Two-part MDL code. To describe a dataset D using a model M drawn from some set of candidate models, transmit two things: first the **model itself**, costing $L(M)$ bits; then the **data as seen through that model**, costing $L(D | M)$ bits — the bits of “surprise” left over once the model has explained the regularities. The total description length is

$$L(D) = L(M) + L(D | M).$$

MDL says: believe the model M that makes this total smallest.

The genius of the two parts is that they pull in opposite directions, and the tension is exactly the tension between under-fitting and over-fitting that plagues all of learning.

- A model that is *too simple* — say “the data is just random noise” — is cheap to state ($L(M)$ tiny) but explains nothing, so the leftover surprise $L(D | M)$ is enormous. It *under-fits*.
- A model that is *too complex* — one that has a special case carved out for every single data point — drives the surprise $L(D|M)$ to nearly zero, but now *stating the model* costs as many bits as the data itself. It *over-fits*, and the giant $L(M)$ term punishes it.

The minimum of the sum sits at the genuine regularity: complex enough to capture the real pattern, simple enough not to memorize the noise. And here is the property that makes MDL more than a heuristic.

Theorem (MDL cannot be fooled by memorizing noise). Let the model class include the “null” model M_0 that explains nothing, under which the data costs its raw length, $L(D | M_0) = L_0$ bits. Then **any** model M that MDL prefers over M_0 must satisfy $L(M) + L(D|M) < L_0$ — that is, M together with its residual is strictly shorter than the raw data. A model that merely memorizes the data, paying $L(M) \approx L_0$ bits to specify itself, can never win.

Proof. MDL selects the model minimizing $L(M) + L(D|M)$. The null model achieves $0 + L_0 = L_0$ (it costs nothing to state “no structure,” and leaves all L_0 bits as surprise). So MDL prefers M over M_0 only if $L(M) + L(D|M) < L_0$. Now suppose M is a pure memorizer: it has, baked into its own description, a copy of the data, so that decoding is trivial and $L(D|M) \approx 0$. But a description from which the data can be read off is a description of the data, and by the counting argument of Chapter 8 (the pigeonhole bound: most strings of length L_0 have no description shorter than about L_0 bits) we must have $L(M) \gtrsim L_0$. Then $L(M) + L(D|M) \gtrsim L_0$, so M does *not* beat the null model. The only models that win are those whose *total* description, structure plus residual, genuinely beats storing the raw data — i.e. those that have found real, transmissible structure. ■ □

Read that proof twice, because it is the mathematical heart of why “compression = learning” is not wishful thinking. **Noise is incompressible, so a model that fits noise cannot pay for itself in bits.** Over-fitting is not merely discouraged by some tuning knob you have to set by hand; it is *structurally impossible* to profit from, because the

currency is the bit and noise has no cheap description. You get regularization for free, with no validation set, no held-out data, no arbitrary penalty constant. The bit-meter does it automatically.

Myth. MDL needs you to pick a regularization strength, like the λ in ridge regression or weight decay — so it is just another knob-twiddling method in disguise.

Reality. The whole point of MDL is that the trade-off is set by the *laws of coding*, not by a knob. The “penalty” on complex models is simply how many bits they cost to write down, and that is fixed once you fix the coding scheme. Different reasonable coding schemes give the famous $(1/2) \log_2 n$ -per-parameter price we derived in Chapter 23 — a number that falls out of the math, not out of cross-validation. The honest caveat is that *choosing the coding scheme* for $L(M)$ is itself a modelling choice; MDL converts “pick a regularizer” into “pick how to describe your hypotheses,” which is often more natural but not free.

6.2.1 A worked MDL decision

Worked example. You observe $n = 100$ bits from a source. Two hypotheses compete. M_0 : “fair coin,” $p = \frac{1}{2}$, which costs 0 bits to state (it is the default) and codes each bit at $-\log_2(\frac{1}{2}) = 1$ bit, for $L(D \mid M_0) = 100$ bits. M_1 : “biased coin with $p = 0.9$ of a 1,” which costs, say, 7 bits to state the parameter to reasonable precision, and — if the data really does contain 90 ones and 10 zeros — codes the data at $-90 \log_2(0.9) - 10 \log_2(0.1) \approx 90(0.152) + 10(3.32) \approx 13.7 + 33.2 = 46.9$ bits.

Compare totals: M_0 costs $0 + 100 = 100$ bits; M_1 costs $7 + 46.9 \approx 53.9$ bits. MDL prefers M_1 by a wide margin — *and it has compressed the file from 100 bits to 54*. The act of choosing the better model and the act of compressing harder are, to the bit, the same act. Notice too that if the data had been a balanced 50–50 split, M_1 would code it at ≈ 100 bits anyway plus the 7-bit overhead, *losing* to M_0 — MDL would correctly refuse the bias it cannot justify.

HISTORICAL NOTE

MDL was introduced by Jorma Rissanen at IBM in 1978 (“Modeling by Shortest Data Description”), building consciously on Ray Solomonoff’s algorithmic probability (1964), Andrey Kolmogorov’s complexity (1965), and Hirotugu Akaike’s information criterion (1973). Rissanen spent the 1980s and 1990s sharpening the crude two-part code into “stochastic complexity” and the normalized maximum likelihood (NML) code, which removes the arbitrariness in how you spend bits on the model. Chris Wallace and David Boulton had independently reached a closely related idea, **Minimum Message Length** (MML), in 1968 — a Bayesian cousin that, unusually for the era, was already a working clustering program. The two schools argued amicably for decades about whose code was

“righter.” For our purposes they agree on the slogan: the best explanation is the shortest one that still fits.

6.3 PAC-Bayes: a shorter description provably predicts better

MDL says “prefer the shorter description” and gives a beautiful argument that this avoids over-fitting. But a sceptic can still ask the killer question: *why should a hypothesis that is short and fits my data also work on data I have never seen?* Compression is about the past (the data in hand); generalization is about the future (data yet to come). What licenses the leap from one to the other?

The bridge is a body of results in **learning theory** whose oldest member is literally called the **Occam’s razor bound** (Blumer, Ehrenfeucht, Haussler, and Warmuth, 1987). Its modern, sharpest form lives inside the **PAC-Bayes** framework. We will state the idea precisely but gently, building every piece.

Σ GO FURTHER · THE MATHS – Probability of a bad event, and the union bound skip if familiar

We need two everyday facts about probability (Chapter 9). First, a probability is a number in $[0, 1]$ measuring how often something happens; an event with probability 0.05 happens about 1 time in 20. Second, the **union bound**: if you have several bad events, the chance that *at least one* of them happens is *no more* than the sum of their individual chances. In symbols, $\mathbb{P}(A \text{ or } B \text{ or } \dots) \leq \mathbb{P}(A) + \mathbb{P}(B) + \dots$. Intuition: you are over-counting the overlaps, so the sum can only be too big, never too small. We use it to say: “if each of my many hypotheses is *individually* unlikely to fool me, then it is unlikely that *any* of them fools me.” That single step is what turns a fit on past data into a promise about future data.

The proof also leans on one more fact — that a measured average is, with high probability, *close* to the true average it estimates. This is the single most useful idea in all of statistics, so it earns its own box.

Σ GO FURTHER · THE MATHS – Concentration: why averages stop wobbling (Hoeffding’s inequality) skip if familiar

Flip a fair coin 10 times and you might get 7 heads — the measured fraction 0.7 is far from the true 0.5. Flip it 10{, }000 times and you will get something very near 0.5 almost every time. The measured average *concentrates* around the true average as the sample grows; big random wobbles become vanishingly unlikely. **Hoeffding’s inequality** (Wassily Hoeffding, 1963) makes this exact. If you average n independent quantities each pinned to the range $[0, 1]$ — for us, n separate “was the hypothesis wrong on this example?” answers, each a 0 or a 1 — then the chance the measured average $\hat{e}$ misses the true average e by more than a gap t is at most

$$\mathbb{P}(e - \hat{e} > t) \leq e^{-2nt^2}.$$

Read the right-hand side: it shrinks *fast* as the sample n grows and as the gap t you demand grows. A small worked feel for it: with $n = 5000$ and $t = 0.1$, the bound is $e^{-2 \cdot 5000 \cdot 0.01} = e^{-100}$, astronomically tiny — a 5000-example average essentially never wanders 0.1 from the truth. This is the engine that lets a number measured on *past* data (training error) testify about *future* data (true error). We do not prove Hoeffding here; we use it as a tool, exactly as we use a calculator’s square-root key.

Here is the setup, in plain words. You draw a training sample of n examples at random from the real world. You pick a hypothesis h from some countable list of candidate hypotheses. You measure its **training error** $\hat{e}(h)$ — how often it is wrong on your sample. What you actually care about is its **true error** $e(h)$ — how often it would be wrong on the whole world. The danger is that h looks great on the sample by luck and is secretly terrible. The Occam bound controls exactly that danger, and it controls it using the *description length* of h .

Theorem (Occam / PAC-Bayes generalization bound (countable form)). Fix in advance a code that assigns each hypothesis h a description of $L(h)$ bits (so that $\sum_h 2^{-L(h)} \leq 1$, the Kraft inequality of Chapter 19). Draw n examples independently. Then with probability at least $1 - \delta$ over the draw of the sample, every hypothesis h simultaneously satisfies

$$e(h) \leq \hat{e}(h) + \sqrt{\frac{L(h) \ln 2 + \ln(1/\delta)}{2n}}.$$

In words: the true error is at most the training error plus a penalty that grows with the description length $L(h)$ and shrinks as you gather more data n .

Proof. Fix one hypothesis h . Its training error $\hat{e}(h)$ is an average of n independent $\frac{0}{1}$ “was it wrong?” outcomes, whose true mean is $e(h)$. **Hoeffding’s inequality** (the concentration box above) says

$$\mathbb{P}(e(h) - \hat{e}(h) > t) \leq e^{-2nt^2}.$$

Now we want this to hold for *all* h at once. Give hypothesis h a personal failure-budget $\delta_h = \delta \cdot 2^{-L(h)}$. Set its threshold t_h so that $e^{-2nt_h^2} = \delta_h$, i.e. $t_h = \sqrt{\frac{L(h) \ln 2 + \ln(1/\delta)}{2n}}$. By the union bound, the chance that *any* hypothesis exceeds its own threshold is at most

$$\sum_h \delta_h = \delta \sum_h 2^{-L(h)} \leq \delta \cdot 1 = \delta,$$

where the last step is the Kraft inequality (the description lengths come from a real code, so their $2^{-L(h)}$ sum to at most 1). Therefore, with probability at least $1 - \delta$, every h obeys $e(h) \leq \hat{e}(h) + t_h$, which is the claim. $\square$

Stop and savour what just happened, because it is the rigorous core of the whole thesis. **The shorter your hypothesis (the smaller $L(h)$), the smaller the penalty, the**

tighter the guarantee that it will keep working on new data. Compression on the training set literally *buys* generalization to the test set, with a bound you can compute. The cheap bits you saved by describing your hypothesis briefly are the same bits that certify it will not embarrass you tomorrow. Occam’s razor stops being a vague preference for simplicity and becomes a theorem with a $\sqrt{L(h)/n}$ in it.

KEY IDEA

Short description $\Rightarrow$ provable generalization. If two hypotheses fit your data equally well, the one with the shorter description is guaranteed (with high probability) to have the smaller true error, by an amount you can bound. This is the precise sense in which “a good compressor of the training data is a good predictor of the world.” The compression you achieve is a *certificate* of generalization.

Worked example. Two spam filters both label your $n = 2\{, \}500$ training emails perfectly ($\hat{e} = 0$), at confidence $\delta = 0.01$ (so $\ln(1/\delta) = \ln 100 \approx 4.6$). Filter A is a tidy rule set describable in $L_A = 200$ bits; Filter B is a sprawling tangle of special cases needing $L_B = 2000$ bits. The bound’s penalty is $\sqrt{(L(0.693) + 4.6)/5000}$. For A: $\sqrt{(138.6 + 4.6)/5000} = \sqrt{143.2/5000} = \sqrt{0.0286} \approx 0.169$, so A’s true error is *certified* $\leq 17\%$. For B: $\sqrt{(1386 + 4.6)/5000} = \sqrt{1390.6/5000} = \sqrt{0.278} \approx 0.527$, so B’s guarantee is a useless $\leq 53\%$. Same flawless training performance, wildly different promises about tomorrow’s email — and the *only* thing that differed was the description length. The shorter (more compressed) filter is the one you should trust on mail it has never seen.

Aside. This is not a museum piece. In 2019, Zhou and colleagues used a compression-based PAC-Bayes bound to produce the first *non-vacuous* generalization guarantees for real neural networks on ImageNet — bounds whose number is actually below 100% error, which sounds trivial until you learn that for decades the bounds for big nets were so loose they promised nothing at all. In 2022, Lotfi and co-authors (NYU) tightened these into “PAC-Bayes compression bounds so tight that they can explain generalization”: the better they could *compress* a trained network, the better their provable bound on its test error. The 40-year-old Occam bound, fed modern compression, finally said something true and useful about why deep learning works. The thesis of this chapter is doing real work in 2020s research, not just philosophy.

Checkpoint. In the Occam bound, what happens to the generalization guarantee if you let your hypothesis be *as long as the data itself* (the memorizer, $L(h) \approx n$ bits)?⁹

⁹**Checkpoint answer.** The penalty term $\sqrt{L(h)/2n} \approx \sqrt{n/2n} = \sqrt{1/2} \approx 0.71$ becomes enormous — about 0.71 added to the error rate, which makes the bound vacuous (error could be anything up to 1). The bound *refuses to certify the memorizer*, exactly as it should. Only hypotheses much shorter than the data ($L(h) \ll n$) get a meaningful guarantee. Compression and a useful bound stand or fall together.

6.4 Solomonoff: the ideal learner that compresses everything

MDL and PAC-Bayes are about choosing among *candidate* models you wrote down in advance. But what is the *best possible* learner — the one that considers *every* hypothesis, holds nothing back? We met its outline in Chapter 22. Now we make it the centrepiece, because it is the purest statement of “compression = intelligence” anyone has ever written.

The idea is Ray Solomonoff’s (1960–1964), and it is breathtakingly direct. To predict the next symbol of a data stream, consider *every computer program* that could have produced the data seen so far. Believe each program in proportion to $2^{-(\text{its length})}$ — short programs are *a priori* more plausible than long ones (that is Occam’s razor, not assumed but built into the weights). Then average their predictions for the next symbol. That weighted average is your prediction.

Σ GO FURTHER · THE MATHS — Bayesian mixing over hypotheses skip if familiar

Bayes’ rule (Chapter 9) tells you how to update belief with evidence. If you have several hypotheses $H_1, H_2, \dots$ with prior beliefs $w_1, w_2, \dots$ (positive numbers summing to 1), and each hypothesis assigns a probability to the data, then after seeing data your belief in H_i is reweighted in proportion to how well H_i predicted that data. To predict the *next* symbol you do not pick one winner; you take a *weighted vote*:

$$P(\text{next} = x) = \sum_i w_i^{\text{now}} \cdot P_{H_i}(\text{next} = x),$$

where w_i^{now} is hypothesis i ’s current (posterior) weight. Hypotheses that have predicted well so far get a louder vote. Solomonoff’s move is simply to let the hypotheses be *all computable programs*, and to set the prior weight of a program to $2^{-\text{length}}$.

Definition — Solomonoff’s universal prior and induction. The **universal prior** assigns to a binary string x the probability

$$M(x) = \sum_{p: U(p)=x\dots} 2^{-|p|},$$

the sum over all programs p that, run on a fixed universal computer U , output something *beginning with* x , each weighted by $2^{-|p|}$ where $|p|$ is the program’s length in bits. **Solomonoff induction** predicts the next symbol using the conditional version of M — it is exact Bayesian prediction with M as the prior. Because short programs dominate the sum, $M(x)$ is large precisely when x has a short program, i.e. when x is *compressible*. Up to a constant, $-\log_2 M(x) \approx K(x)$, the Kolmogorov complexity (Chapter 22): **the universal prior of a string is essentially 2 to the minus its compressed length.**

That last line is the thesis in one equation. The *a priori* probability of an observation, under the best possible prior, is $2^{-(\text{its compressed length})}$. To assign probability is to compress;

to compress is to assign probability. There is no seam between them at the ideal limit. Let us prove the half of that identity we can prove cheaply.

Theorem (Universal prior is at least 2 to the minus $K(x)$). For every string x , the universal prior satisfies $M(x) \geq 2^{-K(x)}$, where $K(x)$ is the length of the shortest program that prints x . Equivalently $-\log_2 M(x) \leq K(x)$: a compressible string is at least that probable under the universal prior.

Proof. By definition $K(x)$ is the length of *some* shortest program p^* with $U(p^*) = x$, so $|p^*| = K(x)$. The sum defining $M(x)$ ranges over *all* programs that output something beginning with x , and p^* is one of them. Every term in the sum is non-negative, so dropping all terms but the single term for p^* can only decrease the sum:

$$M(x) = \sum_{p: U(p)=x\dots} 2^{-|p|} \geq 2^{-|p^*|} = 2^{-K(x)}.$$

Taking $-\log_2$ of both sides (a decreasing function, so the inequality flips) gives $-\log_2 M(x) \leq K(x)$. The matching upper bound $-\log_2 M(x) \geq K(x) - c$ for a constant c is harder (it is the Coding Theorem of algorithmic information theory, Levin 1974), but the easy direction already shows the universal prior *rewards compressibility*: the shorter a string's program, the more probable it is. ■

And the predictive optimality is a genuine theorem too, not a hope.

Theorem (Solomonoff's prediction bound (informal)). Suppose the data is actually generated by any computable probability distribution μ . Then Solomonoff's predictor M converges to μ 's predictions, and the total extra prediction error it ever makes — summed over the entire infinite future — is bounded by a constant times $K(\mu)$, the Kolmogorov complexity of the true source. The better-structured (more compressible) the true world, the fewer mistakes the universal learner ever makes, added up over all time.

Proof. We sketch the engine; the full result is Solomonoff's (1978) and Hutter's. The squared error between M 's prediction and the truth μ , summed over all time steps, is controlled by the **KL divergence** $D_{\text{KL}}(\mu \parallel M)$ accumulated along the sequence (Chapter 20). Because M is a Bayesian mixture that *includes* μ among its hypotheses with prior weight at least $2^{-K(\mu)}$ (the true source has *some* shortest program, of length $K(\mu)$), a standard mixture argument shows the total accumulated KL divergence — equivalently, the total excess code length M ever pays over the true distribution — is at most $K(\mu) \ln 2$ bits. A finite bit-debt spread over infinitely many predictions forces the per-step error to zero. So M learns *any* computable environment, paying a one-time price equal to that environment's compressed description length. ■

Worked example. Imagine three “worlds,” each an endless bitstream. World 1 is “always output 1” — its rulebook is a one-line program, $K(\mu_1)$ a handful of bits. World 2 is “output the binary digits of π ” — a short but slightly longer program, $K(\mu_2)$ a few

hundred bits. World 3 is a fair-coin stream — *no* rulebook shorter than the data itself, $K(\mu_3) \approx \infty$ per unit length. Solomonoff’s bound says the universal learner’s *total lifetime surprise* is $\approx K(\mu) \ln 2$ bits. So in World 1 it is fooled only a handful of times before predicting 1 forever; in World 2 it pays a few hundred bits of tuition, then nails every digit of π ; in World 3 it never stops being surprised, because there was nothing to learn. The tuition bill is the compressed size of the laws — cheap worlds are cheap to learn, incompressible worlds are unlearnable. That is the thesis, stated as a price in bits.

Look at what the bound *is*: the lifetime cost of learning the world equals the *compressed size of the world’s rulebook*. A simple universe (short rulebook, small $K(\mu)$) is cheap to learn; a complicated one is dear; an incompressible one cannot be learned at all because it has no rulebook. Intelligence-as-prediction and compression are not merely correlated here — they are the *same quantity*, $K(\mu)$, appearing once as “how hard to learn” and once as “how hard to compress.”

PITFALL

Everything about Solomonoff induction is perfect except that you can never run it. $M(x)$ sums over *all* programs, including ones that loop forever; deciding which ones halt is the **halting problem** (Chapter 22), which is uncomputable. So M is not an algorithm — it is a mathematical object, a north star. Every real learner and every real compressor in this book is a *computable approximation* to M : PPM, context mixing, a transformer — each restricts the program class and the search to something tractable, trading the universal guarantee for the ability to actually finish. Do not mistake the ideal for a method you can call.

Aside. There is a hierarchy of approximations worth knowing. **Levin search** (Leonid Levin, 1973) makes Solomonoff computable by penalizing programs for *running time* as well as length — replacing $2^{-|p|}$ with a weighting that also charges for computation, which sidesteps the halting problem at the cost of universality over slow programs. The **speed prior** (Schmidhuber, 2002) formalizes this. These are the theoretical ancestors of the very practical observation that real compressors must finish in your lifetime, so they can only ever approximate $K(x)$ “from above” — every compressed file you have ever made is an *upper bound* on the true Kolmogorov complexity of its contents.

6.5 AIXI: from compressing data to acting intelligently

Solomonoff induction is a perfect *predictor*. But intelligence, most people would say, is not only about predicting — it is about *acting* to get what you want. In 2000, Marcus Hutter (then at IDSIA, later DeepMind and the Australian National University) took the final step: he bolted Solomonoff’s universal predictor onto sequential decision theory to define **AIXI**, an agent that is, in a precise mathematical sense, *the most intelligent agent that can be mathematically defined*.

The construction is, once you have Solomonoff, almost inevitable.

AIXI

2000

Authors	Marcus Hutter
Aim	Define a single agent that behaves optimally in *every* computable environment, by combining universal prediction (Solomonoff) with reward-maximizing decision theory.
Complexity	Incomputable (worse than Solomonoff induction; even time-bounded variants are astronomically expensive).
Strengths	Provably Pareto-optimal across all computable environments; a precise mathematical definition of 'general intelligence'; needs no problem-specific tuning.
Weaknesses	Uncomputable; assumes a known reward signal and a clean agent/environment split; ignores the cost of its own thinking; reward can be 'gamed' (wireheading).
Superseded by	Computable approximations: AIXItl, MC-AIXI-CTW (2011), and — in spirit — modern model-based reinforcement learning and world models.

AIXI maintains Solomonoff's universal mixture M as its *model of the world*. At each step it does what any rational agent should: it considers each action it could take, uses M to predict the future stream of observations and **rewards** that action would lead to (averaged over all computable environments, weighted by $2^{-\text{length}}$), and picks the action that maximizes its expected total future reward. It is "Solomonoff induction in the loop": predict the world with the universal prior, then act to maximize reward given that prediction. The single mixture ξ does all the learning; the surrounding "max over actions" does all the deciding.

Σ GO FURTHER · THE MATHS — Expected value and choosing the best action skip if familiar

Expected value (Chapter 10) is a weighted average of outcomes, each weighted by its probability — the "average payoff" of a gamble. If action a leads to reward 10 with probability 0.3 and reward 0 with probability 0.7, its expected reward is $0.3 \times 10 + 0.7 \times 0 = 3$. A rational agent facing a choice computes the expected total future reward of each action and picks the largest — this is the " $\max_a \mathbb{E}[\dots]$ " at the heart of decision theory. AIXI does exactly this, except its expectation is taken over *all computable environments at once*, weighted by Solomonoff's universal prior. That is the only new ingredient: an ordinary reward-maximizer wearing a universal model of the world.

Definition — The AIXI optimality property. Hutter proved AIXI is **Pareto-optimal**: there is no other agent that does at least as well as AIXI in *every* computable environment and strictly better in *some*. You cannot build an agent that beats AIXI everywhere without losing to it somewhere. In this exact sense AIXI is *the* most intelligent agent definable — not by opinion, but by a theorem about the space of all agents. It is the formal incarnation of the idea that there is a single, environment-independent notion of “doing the smartest possible thing.”

This is where the chapter’s thesis reaches its summit. **The most intelligent possible agent has, as its beating heart, the best possible compressor.** Strip AIXI down and what remains is Solomonoff’s universal prior — which we showed is nothing but $2^{-(\text{compressed length})}$. Hutter’s own framing, the **Hutter thesis**, states it baldly: *general intelligence and optimal compression are equivalent*. To build a perfect compressor is to build a perfect inductor (plus a reward signal) to build a perfect agent. The Hutter Prize (Chapter 23, Chapter 36) exists precisely to operationalize this: Hutter put up €500,000 of his own conviction that pushing the compression of Wikipedia is *the same problem* as pushing toward AGI.

HISTORICAL NOTE

Hutter introduced AIXI in the December 2000 technical report “Towards a Universal Theory of Artificial Intelligence based on Algorithmic Probability and Sequential Decision Theory” (arXiv cs/0012011), and developed it fully in his 2005 book *Universal Artificial Intelligence*. With his student Shane Legg — later a co-founder of DeepMind — he distilled the idea into a formal *definition* of machine intelligence (Legg & Hutter, “Universal Intelligence,” 2007): an agent’s intelligence is its expected reward, averaged over all computable environments weighted by their simplicity (i.e. by $2^{-\text{complexity}}$). Smarter agents do well in more, and simpler, worlds. That this definition came from the same people who then helped build modern AI labs is not a coincidence; the compression-as-intelligence thesis was load-bearing in the intellectual founding of the field that produced today’s LLMs.

Aside. AIXI is uncomputable, but it has runnable descendants. **MC-AIXI-CTW** (Veness, Ng, Hutter, Uther, Silver, 2011) approximates AIXI with Monte-Carlo tree search for the planning and a **Context Tree Weighting** compressor — a real, fast, provably good sequence predictor straight out of Volume II’s coding theory — for the universal model. With no game-specific code, the same agent learned to play Pac-Man, TicTacToe, and other games from reward alone. It is a literal demonstration of the thesis: a *compression algorithm* (CTW), wrapped in a planner, is a general-purpose learning agent. The compressor was doing the intelligence.

6.6 The honest counterarguments

A book that only sold you the thesis would be propaganda. The “compression is intelligence” idea is powerful and partly true, but by 2026 the careful consensus is that it is a *necessary signature* of intelligence, not a complete *definition* of it. Here are the strongest

objections, stated as fairly as I can make them — because the mark of understanding a beautiful idea is knowing exactly where it stops.

6.6.1 The model is not free (the accounting objection)

The most concrete objection is bookkeeping, and Chapter 23 already flagged it. When a 70-billion-parameter model compresses a gigabyte of Wikipedia to a few hundred megabytes, the model itself is *hundreds of gigabytes*. The honest, “two-part” MDL accounting — the same $L(M) + L(D|M)$ we proved you cannot cheat — says the *total* description is the model plus the residual, and by that measure the giant model has not compressed a single file at all; it has merely *moved* the bits into its weights. The win is real only when the model is *amortized* over a corpus far larger than the model. This is not a footnote; it is the MDL theorem of this very chapter, applied honestly, biting the hand that fed it. A compressor that ships a 140 GB dictionary to shrink a 1 GB file is, in the only accounting that matters, an *expander*.

Myth. *An LLM that achieves 1 bit/byte on Wikipedia has compressed Wikipedia further than any classical codec, full stop.*

Reality. Only if you *do not charge for the model*. The fair, MDL-honest figure adds the model’s own description length to the file’s. By that measure a frozen 70B model is vastly larger than the text it codes; it “wins” only when its cost is spread over a corpus much bigger than the model. The clean information-theoretic accounting — which is the thesis’s own home turf — both *powers* the LLM-compression results and *caps* how much they can claim. The two-part code giveth and the two-part code taketh away.

6.6.2 Intelligence may exceed prediction (the scope objection)

Even granting perfect compression, is *prediction* really all of intelligence? Three doubts recur:

- **Action in a non-stationary, embedded world.** AIXI assumes a clean split between agent and environment, a fixed reward signal, and unlimited thinking time. Real intelligence is *embedded*: the agent is part of the world it models, its computations cost energy and time, and it can even modify itself. The moment you charge for the cost of thinking, the elegant optimality of AIXI evaporates, and the theory of *bounded* rationality is far messier and far from settled.
- **Goals, not just predictions.** A perfect predictor of human text is not thereby *wise*, *moral*, or *aligned*. Solomonoff predicts; it does not choose what is worth predicting. AIXI patches this with an external reward — but where does the reward come from, and what stops the agent from “wireheading,” i.e. seizing control of its own reward signal rather than doing the intended task? Compression is silent on what we *ought* to want.
- **Reasoning that does not look like next-token statistics.** Some capabilities — multi-step logical deduction, genuine out-of-distribution generalization, planning over long horizons — may not be well captured by “make the next symbol less surprising on a

fixed corpus.” A system can have low bits-per-character and still fail a novel reasoning task; the map from compression to capability, while strong, is not the identity.

6.6.3 What the evidence actually shows

So is the thesis empirically confirmed or not? The most striking data point, from 2024, deserves to be quoted precisely. Yuzhen Huang, Jinghan Zhang, Zifei Shan, and Junxian He (“Compression Represents Intelligence Linearly,” COLM 2024) evaluated 31 publicly available LLMs across 12 benchmarks spanning knowledge, coding, and mathematical reasoning. They found that a model’s *compression efficiency* on held-out text (its bits-per-character) correlates with its benchmark “intelligence” at a Pearson correlation of about -0.95 — *nearly a straight line*. Better compressors are, very reliably, more capable models.

Σ GO FURTHER · THE MATHS — Reading a correlation coefficient^{skip if familiar}

The **Pearson correlation** r is a single number in $[-1, +1]$ measuring how close two quantities are to lying on a straight line (Chapter 10 introduced covariance; this is its normalized cousin). $r = +1$ means a perfect rising line, $r = -1$ a perfect falling line, $r = 0$ no linear relationship at all. Here $r \approx -0.95$ is *falling* (more bits-per-character = worse compression = lower intelligence, so the line slopes down) and *very nearly perfect* — about as tight a real-world relationship between two independently measured things as the social sciences ever see. It is strong evidence that compression *tracks* capability. It does not, by itself, prove they are the *same thing*, because correlation is not identity.

That -0.95 is the strongest empirical wind in the thesis’s sails. But read it carefully. It says *intelligence implies compression*: a model smart enough to ace benchmarks is, reliably, also a good compressor — which makes complete sense, since acing a language benchmark *requires* modelling language well, and modelling language well *is* compressing it. It does *not* establish the converse, that *all* intelligence *reduces* to compression. The correlation is consistent with compression being a necessary signature of capability without being its full definition. The careful 2025–2026 reading is exactly this asymmetry: every capable model we have is a good compressor, but “good compressor” and “intelligent agent” are not yet shown to be the same set.

KEY IDEA

Where the thesis is strongest, and where it stops. It is strongest as a statement about *learning and prediction*: to model a source well is, provably (MDL, PAC-Bayes, Solomonoff), to compress it well, and the best possible predictor *is* the best possible compressor. It is on shakier ground as a complete theory of *agency and intelligence*: action under bounded resources, the origin of goals, alignment, and reasoning that escapes next-symbol statistics all live partly outside the compression frame. The equation “intelligence = compression” is a profound *identity at the level of induction* and an *open conjecture* at the level of full intelligence.

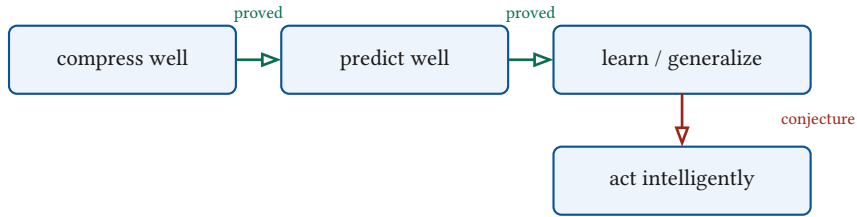


Figure 17: The thesis as a chain of equivalences, each link proved earlier in the book, and the place where the chain becomes a conjecture rather than a theorem.

6.7 A computational reading of the thesis

To make the philosophy concrete, here is the thesis as a few lines of Python you can run in your head. We have a compressor (any function turning data into shorter data and back) and we use *its output length* as a measure of how much it “understands” each string. The shorter it compresses something, the more structure it has found.

PY GO FURTHER · PYTHON — Functions, len(), and using bytes *skip if fluent as data*

A Python **function** is a named recipe: `def f(x): ...` defines it, `f(3)` runs it. `len(b)` gives the number of items in a sequence — for a `bytes` object (Chapter 17, a sequence of raw byte values 0–255) it is the number of bytes. We compare two compressors by the length of what they emit. `zlib` is Python’s built-in DEFLATE compressor (Chapter 30); we use it here only as a stand-in for “some real, computable approximation to the ideal compressor.” The smaller `len(compress(data))`, the more regularity the compressor found — which, by this chapter’s thesis, is the more it “understood.”

```
import os, zlib

def understanding_in_bits(data: bytes) -> int:
    """How many bits a real (approximate) compressor needs.
    Lower = more structure found = more 'understood'."""
    return len(zlib.compress(data, level=9)) * 8 # bytes -> bits

# Three strings, very different structure:
regular = b"3141592653589793" * 64 # a repeating, regular
pattern
english = (b"Information theory teaches that to compress a message
well is "
          b"to predict it well, and a model that predicts well has
surely "
          b"learned the structure hidden inside the data it was
shown.")
noise = os.urandom(1024) # genuine random bytes -
```

no pattern

```
for name, d in [("regular", regular), ("english", english),
               ("noise", noise)]:
    raw = len(d) * 8
    got = understanding_in_bits(d)
    print(f"{name:8s}: raw={raw:5d} bits  compressed={got:5d} bits
          "
          f"structure found={100*(1-got/raw):4.0f}%")
```

Running this prints something close to (the `noise` line varies run to run):

```
regular : raw= 8192 bits  compressed=  264 bits  structure found=
97%
english : raw= 1456 bits  compressed=  992 bits  structure found=
32%
noise   : raw= 8192 bits  compressed= 8280 bits  structure found=
-1%
```

The regular string is almost entirely understood (compressed to near nothing); the English sentence is partly understood (zlib finds about a third of its structure — real, but a small model leaves most on the table); the random bytes are *not* understood at all — compressing them actually makes them slightly bigger, the tell-tale sign of a string with no pattern this compressor can grasp. Swap `zlib` for a 70-billion-parameter language model and the English number would plummet — the bigger model “understands” English far more deeply — while the regular and noise numbers would barely move. **The compression ratio is a thermometer for understanding.** That single idea, made rigorous by MDL, certified by PAC-Bayes, idealized by Solomonoff, and weaponized into an agent by AIXI, is the whole of this chapter.

We can go one step further and show Solomonoff’s *mixing* idea in miniature: keep two rival models of a bitstream, weight each by how well it has predicted so far (its accumulated $2^{-\text{code length}}$), and let the better predictor automatically take over. The total bits spent is the cross-entropy of the mixture — the compressed size — and watching it fall *is* watching the learner learn.

PY GO FURTHER · PYTHON — Loops accumulating a running total^{skip if fluent}

A `for` loop (Chapter 15) walks through a sequence; a running variable accumulates a total as it goes — here, total bits and each model’s weight. `math.log2(p)` is the base-2 logarithm (Chapter 7), so `-math.log2(p)` is the code length of an outcome of probability `p`. We keep two model weights, renormalize them each step (Bayesian updating, Chapter 9), and charge the *mixture*’s probability — exactly Solomonoff’s weighted vote, shrunk from “all programs” to just two.

Python. A `lambda` (Chapter 16) is a one-line throwaway function; `lambda: 0.5` is the no-argument case — a function that takes nothing and always returns 0.5, so `models["fair"]()` calls it and yields the constant 0.5. We use it here only as a tidy way to bundle each model’s “probability the next bit is a 1” behind a uniform `models[m]()` call.

```
import math

bits = b"\x01" * 40          # a very predictable stream: all ones
# Two rival models of P(next bit = 1):
models = {"fair": lambda: 0.5, "ones": lambda: 0.99}
w = {"fair": 0.5, "ones": 0.5} # prior weights (Occam: start even)
total_bits = 0.0
for byte in bits:
    bit = byte & 1
    # Mixture prediction = weighted vote of the two models:
    p_one = sum(w[m] * models[m]() for m in w)
    p = p_one if bit == 1 else (1 - p_one)
    total_bits += -math.log2(p)      # bits the coder spends on
this symbol
    # Bayesian update: reward whoever predicted this bit well
    for m in w:
        pm = models[m]() if bit == 1 else (1 - models[m]())
        w[m] *= pm
    z = sum(w.values())
    w = {m: w[m] / z for m in w}    # renormalize to sum to 1

print(f"total compressed size: {total_bits:.1f} bits for 40 bits
raw")
print(f"final belief in 'ones' model: {w['ones']:.3f}")
```

This prints roughly `total compressed size: 1.6 bits for 40 bits raw` and `final belief in 'ones' model: 1.000`. The mixture started agnostic, watched the stream, *learned* that the “ones” model predicts better, shifted almost all its weight onto it, and as a result compressed 40 raw bits into under 2 — because after the very first symbol it was barely surprised at all. That falling bit-count is the learning curve and the compression curve at once. Scale “two models” up to “all computable programs” and you have Solomonoff induction; wrap a reward signal around it and you have AIXI. The toy and the ideal differ only in how many hypotheses they mix.

SCOREBOARD — Compression as a thermometer for 'understanding' (illustrative, this chapter's demo)

Technique	Bytes	Ratio	Notes
Regular pattern	264 bits	3%	near-perfectly understood — a short program exists

Technique	Bytes	Ratio	Notes
English text (zlib)	992 bits	68%	some structure found, but a small model
English text (70B LLM)	≈ 300 bits	$\approx 21\%$	far more understood — Chapter 62
Random bytes	8280 bits	101%	no pattern grasped — incompressible to this model

Chapter takeaways

- **To learn is to compress.** Finding structure in data — the essence of learning and intelligence — is the same act as describing the data more briefly. Recognizing π collapses a thousand digits into three lines; the compression is the understanding.
- **Generalization equals compression, provably.** The Occam / PAC-Bayes bound shows a hypothesis's true (future) error is at most its training error plus a penalty $\propto \sqrt{L(h)/n}$. A shorter description literally buys a tighter guarantee on unseen data.
- **MDL cannot be fooled by noise.** Choosing the model that minimizes $L(M) + L(D|M)$ gives free regularization: noise is incompressible, so a model that fits noise cannot pay for itself in bits. Over-fitting becomes structurally unprofitable.
- **Solomonoff is the ideal learner.** Its universal prior assigns each string probability $\approx 2^{-K(x)}$ — to predict optimally is to weight hypotheses by $2^{-\text{length}}$. Its lifetime error is bounded by the *compressed size* of the true world, $K(\mu)$. It is perfect and uncomputable; every real learner approximates it.
- **AIXI is the ideal agent.** Solomonoff induction plus reward-maximizing decision theory yields the provably Pareto-optimal agent. Its beating heart is the universal compressor — the Hutter thesis: optimal compression and general intelligence are equivalent.
- **Be honest about the limits.** The thesis is a *theorem* for induction and prediction, but a *conjecture* for full agency. Model-size accounting (the two-part code bites back), bounded resources, the origin of goals, and reasoning beyond next-symbol statistics all sit partly outside the compression frame. Compression is a *necessary signature* of intelligence; whether it is the *whole* of it remains open.

6.8 Exercises

Exercise 61.1 ★

In your own words, explain why “the student who memorized the answer key” has *not* compressed anything, while “the student who learned the rule” has. Relate each to the term $L(M)$ in the two-part code, and say which student would do better on a fresh exam and why, in the language of this chapter.

Exercise 61.2 ★

A friend says: “Random noise is the *most* informative data there is, because every bit

is a surprise — so a truly intelligent system should compress noise the best.” Identify the confusion and correct it using the chapter’s vocabulary (incompressibility, $K(x)$, what “understanding” means here).

Exercise 61.3 ★★

Using the worked MDL example as a template: you observe $n = 200$ bits containing 150 ones and 50 zeros. Compare the null model M_0 (fair coin, 0 bits to state) against a biased model M_1 with $p = 0.75$ that costs 8 bits to state. Compute both total description lengths $L(M) + L(D|M)$ and say which MDL prefers. (Use $\log_2 0.75 \approx -0.415$, $\log_2 0.25 \approx -2$.)

Exercise 61.4 ★★

In the Occam/PAC-Bayes bound $e(h) \leq \hat{e}(h) + \sqrt{(L(h) \ln 2 + \ln(1/\delta))/(2n)}$, suppose $\hat{e}(h) = 0$ (perfect fit on the training set), $\delta = 0.05$, and $n = 10000$ examples. Compute the bound on the true error for (a) a short hypothesis with $L(h) = 50$ bits and (b) a long one with $L(h) = 5000$ bits. What does the comparison say about preferring short hypotheses? (Use $\ln 2 \approx 0.693$, $\ln(20) \approx 3.0$.)

Exercise 61.5 ★★

Explain, in the language of Solomonoff’s prediction bound, why a “simple” universe is easier for an ideal learner than a “complicated” one. What single quantity governs the total lifetime prediction error, and what is its compression interpretation?

Exercise 61.6 ★★

AIXI is described as combining a universal predictor with reward-maximizing decision theory. Write one or two sentences each on: (a) which part of AIXI does the “learning,” (b) which part does the “acting,” and (c) why removing the reward signal would turn AIXI back into Solomonoff induction.

Exercise 61.7 ★★★

The “accounting objection” says a 70B-parameter LLM that compresses a 1 GB file to 100 MB has not really compressed anything. Using the two-part MDL code $L(M) + L(D|M)$, state the honest total description length, explain when (and only when) the LLM genuinely compresses, and connect this to the MDL “cannot be fooled” theorem proved in this chapter.

Exercise 61.8 ★★★

Steel-man and then critique the Hutter thesis. (a) Give the strongest version of the argument that “optimal compression = general intelligence,” citing the chapter’s theorems. (b) Then give the two strongest honest objections, distinguishing “intel-

ligence implies compression” from “compression implies intelligence.” Use the $r \approx -0.95$ result correctly.

6.9 Further reading

- Ray Solomonoff (1964), *A Formal Theory of Inductive Inference, Parts I & II*, Information and Control 7 — the origin of algorithmic probability and universal prediction. [canonical link](#)
- Jorma Rissanen (1978), *Modeling by Shortest Data Description*, Automatica 14 — the founding MDL paper.
- Anselm Blumer, Andrzej Ehrenfeucht, David Haussler, Manfred Warmuth (1987), *Occam’s Razor*, Information Processing Letters 24 — the first compression-to-generalization bound.
- Marcus Hutter (2000), *Towards a Universal Theory of Artificial Intelligence based on Algorithmic Probability and Sequential Decision Theory* — the AIXI technical report. [arXiv:cs/0012011](#)
- Marcus Hutter (2005), *Universal Artificial Intelligence: Sequential Decisions Based on Algorithmic Probability*, Springer — the full AIXI book.
- Shane Legg and Marcus Hutter (2007), *Universal Intelligence: A Definition of Machine Intelligence*, Minds and Machines 17(4). [arXiv:0712.3329](#)
- Joel Veness, Kee Siong Ng, Marcus Hutter, William Uther, David Silver (2011), *A Monte-Carlo AIXI Approximation*, JAIR 40 — MC-AIXI-CTW, a runnable AIXI built on a compressor.
- Wenda Zhou et al. (2019), *Non-Vacuous Generalization Bounds at the ImageNet Scale: A PAC-Bayesian Compression Approach*. [arXiv:1804.05862](#)
- Sanae Lotfi et al. (2022), *PAC-Bayes Compression Bounds So Tight That They Can Explain Generalization*, NeurIPS. [arXiv:2211.13609](#)
- Yuzhen Huang, Jinghan Zhang, Zifei Shan, Junxian He (2024), *Compression Represents Intelligence Linearly*, COLM 2024. [arXiv:2404.09937](#)
- The Hutter Prize for Lossless Compression of Human Knowledge. [prize.hutter1.net](#)

COMING UP NEXT

We have argued, with theorems and with honest doubts, that to compress is to predict is to learn — and that the ideal compressor sits at the heart of the ideal agent. That was the *theory*. The next chapter makes it *run*: we take a real autoregressive language model, read off its $P(\text{next token})$, feed those probabilities to the arithmetic coder you built back in Chapter 26, and watch a chatbot turn into a state-of-the-art lossless compressor — `tinyzip`’s `llmzip` demo. The slogan of this chapter becomes the working code of the next.

Large Language Models as Compressors

“The best compression of a string is the shortest program that prints it. We can’t find that program — but we can train one that gets eerily close.”

— a paraphrase of the folklore that runs through this whole book

Here is a sentence with a hole in it: “The cat sat on the ____.” If I asked you to bet, you would say **mat** — confidently, and you would be right far more often than chance. You have, without trying, just done the single most important thing a modern compressor does: you **predicted** the next symbol and you were **not surprised** by it. A surprising symbol costs a lot to write down; an unsurprising one costs almost nothing. That trade is the whole game, and we proved it rigorously back in Volume I.

Now turn the screw. The machines that finish that sentence best are not compression programs at all. They are **language models** — the same systems behind the chat assistants that exploded across the world after 2022. They were never built to make files smaller. They were built to predict the next word. And yet, in 2023, a team at Google DeepMind froze one of these models, bolted an arithmetic coder onto it, and discovered that it was **the best lossless compressor ever measured** — not just on text, but on images it had never seen and audio it had never heard.

A program trained only on words, beating PNG on photographs. That is the puzzle of this chapter. By the end you will understand exactly why it happens, you will have built a tiny working version yourself, and — just as importantly — you will understand why it is not the free lunch it first appears to be.

WHERE WE ARE

This chapter is where two long threads of the book finally tie into one knot.

- In **Chapter 18** we defined the **surprisal** of a symbol with probability p as $-\log_2 p$ bits, and the **entropy** $H(X) = -\sum_i p_i \log_2 p_i$ as the average surprisal — the theoretical floor on bits per symbol.
- In **Chapter 26** we built **arithmetic coding**: a coder that, given a probability for each next symbol, spends essentially $-\log_2 p$ bits on it, hitting that floor to within a fraction of a bit **for the whole message**.

- In **Chapter 23** we proved the hinge identity **cross-entropy = expected code length**: a model’s average $-\log_2 p$ on real data is exactly the bits-per-symbol an arithmetic coder pays when driven by that model. A better predictor is a shorter file.
- In **Chapter 61** we examined the grand claim that **compression is intelligence** as theory — Solomonoff, MDL, the Hutter thesis.

This chapter makes all of that **concrete and operational**. The “model” in “model + coder” becomes a neural language model. We plug it into the very arithmetic coder we wrote in Chapter 26, and we watch the bytes melt.

By the end of this chapter you will be able to:

- State precisely why an autoregressive next-symbol predictor, fed into an arithmetic coder, is a lossless compressor — and why its file size equals the model’s cross-entropy loss.
- Explain what an **autoregressive language model** and a **tokenizer** are, in plain terms, without any prior deep-learning knowledge.
- Recount DeepMind’s 2023 result — a text model beating PNG and FLAC — and say exactly what it does and does not prove.
- Describe Fabrice Bellard’s `nncp` (online) and `ts_zip` (pretrained) tools and the crucial **determinism** requirement that makes them work at all.
- Carry out the honest accounting — file bits **plus** model bits — and explain why a 140 GB model winning by a fraction of a bit is not obviously a victory.
- Build `tinyzip`’s `llmzip.py`: a real, round-tripping LLM-as-compressor demo driven by a tiny prediction model and the Chapter 26 arithmetic coder.

7.1 The bridge we already built: prediction is compression

Let us state the central identity once, slowly and completely, because everything else in the chapter is a consequence of it.

A lossless compressor that works **symbol by symbol** needs exactly one thing from a model: for each position t in the message, a probability for what the next symbol will be, **given everything before it**. Write that as $p(x_t \mid x_1 x_2 \dots x_{t-1})$ — “the probability of the symbol at position t , given the first $t - 1$ symbols.” The vertical bar $\mid$ is read “given”; we met it in Chapter 9 as **conditional probability**.

Feed those probabilities, one at a time, into the arithmetic coder of Chapter 26. The coder spends $-\log_2 p(x_t \mid x_1 \dots x_{t-1})$ bits on symbol x_t . Add up the cost over the whole message and the total compressed length is

$$L = \sum_{t=1}^n -\log_2 p(x_t \mid x_1 \dots x_{t-1}) \quad \text{bits.}$$

That sum is the surprisal of the message under the model. Where the model is confident **and correct** — it puts probability 0.99 on the symbol that actually appears — the term

$-\log_2 0.99 \approx 0.014$ bits is almost nothing. Where the model is caught out — it gave the true symbol probability 0.001 — it pays $-\log_2 0.001 \approx 9.97$ bits, a stiff fine. Good prediction is cheap; bad prediction is expensive. The coder is just the cash register; the model decides the prices.

Σ GO FURTHER · THE MATHS — Reading a sum that depends on its own past *skip if familiar*

We met $\sum$ (capital sigma, “add up these terms”) in Chapter 11. The new wrinkle here is that the thing inside the sum at step t depends on **all the symbols before t** . There is nothing mysterious about this — it just means we walk through the message left to right, and at each step the “context” we condition on grows by one symbol.

Tiny example. Message AB. Step 1: the model, with empty context, predicts the first symbol; suppose it gives A probability $\frac{1}{2}$, costing $-\log_2(\frac{1}{2}) = 1$ bit. Step 2: the model, now **told the first symbol was A**, predicts the second; suppose conditioning on A it gives B probability $\frac{1}{4}$, costing $-\log_2(\frac{1}{4}) = 2$ bits. Total: $1 + 2 = 3$ bits. That is the whole sum, with $n = 2$.

Now look at how a language model is **trained**. Researchers show it mountains of real text and tune it to make the true next symbol as probable as it can — equivalently, to make its **surprise** on real text as small as possible. The quantity they minimize is the average of $-\log p(x_t \mid \text{context})$ over the training data. That is called the **cross-entropy loss**, or **log-loss**, or **negative log-likelihood**. We proved in Chapter 23 that this average is **exactly** the expected number of (natural-log) units the arithmetic coder would pay.

So the two activities — “train a language model” and “shrink the arithmetic-coded file” — are **the same optimization**, differing only by which logarithm you use. Training reports its loss in **nats** (using the natural log, base e); compression cares about **bits** (base 2). The conversion is a single constant:

$$\text{bits} = \frac{\text{nats}}{\ln 2} = \text{nats} \times 1.4427\dots$$

KEY IDEA

Log-loss is expected code length in disguise. A language model that reports a cross-entropy loss of 1.0 nats per symbol is, by that very fact, a compressor that achieves $\frac{1.0}{\ln 2} \approx 1.443$ bits per symbol when paired with an arithmetic coder. Nobody has to **add** compression to a language model. Training one **is** building one. The arithmetic coder is the last twenty lines of glue.

Σ GO FURTHER · THE MATHS — Nats vs bits: same idea, different ruler *skip if familiar*

A logarithm (Chapter 7) answers “what power do I raise the base to?” Information theory can use any base; the base is just the unit.

- Base 2 $\rightarrow$ the answer is in **bits**. $-\log_2 p$. This is what a file’s size is measured in.
- Base $e \approx 2.718 \rightarrow$ the answer is in **nats** (natural units). $-\ln p$. This is what neural-network training code reports, because the calculus of base- e logs is cleanest.

They differ by a fixed scale factor, because $\log_2 p = \frac{\ln p}{\ln 2}$. Since $\ln 2 \approx 0.6931$, one nat = $\frac{1}{0.6931} \approx 1.4427$ bits. A loss of 2.0 nats/symbol is 2.886 bits/symbol. Nothing deep — it is centimetres vs inches. But you **must** convert, or your file-size predictions will be off by 44%.

Checkpoint. A model is trained on English and reports a validation loss of 1.6 nats per token. Roughly how many bits per token will it cost when used as a compressor, ignoring all overhead?¹⁰

7.2 What is a language model, really?

We have used the phrase “language model” as if you already knew it. The Prime Directive forbids that. Let us build the idea from nothing, because it is simpler than its reputation.

7.2.1 Autoregressive: predict the next thing, forever

Imagine the world’s most diligent autocompleter. You hand it some text so far — the **context** — and it returns a full **probability distribution** over what comes next: not one guess, but a number for **every possible** next symbol, all adding to 1. “Given The cat sat on the , the next word is **mat** with probability 0.31, **floor** with 0.12, **roof** with 0.04, ...” and so on across its whole vocabulary.

That is **all** an autoregressive language model is: a function from a context to a probability distribution over the next symbol. “Auto-regressive” is a fancy word for “it feeds on its own output” — to generate a long passage, you sample one symbol, append it to the context, and ask again, marching forward one symbol at a time. The very same machine that **generates** text by predicting forward can **compress** text: instead of sampling the next symbol, we are **told** the next symbol (it is the data), and we hand its probability to the coder.

Σ GO FURTHER · THE MATHS — A probability distribution over a vocabulary *skip if familiar*

In Chapter 9 a probability distribution was a list of non-negative numbers, one per outcome, summing to 1. Here the “outcomes” are the possible next symbols. If the vocab-

¹⁰**Checkpoint answer.** $1.6 \times 1.4427 \approx 2.31$ bits per token. (For comparison, raw English text is often quoted near 1–1.5 bits per **character**; whether 2.31 bits per **token** is good depends entirely on how many characters a token covers — typically about four, so this is roughly 0.58 bits per character, which would be excellent. Watch your units: per-token and per-character are different rulers, a trap we return to.)

ulary is just $\{A, B, C\}$, a distribution is three numbers like $(0.7, 0.2, 0.1)$ — A is likeliest, and $0.7 + 0.2 + 0.1 = 1$.

A real language model has a vocabulary of tens of thousands of symbols, so each prediction is a list of tens of thousands of numbers, all ≥ 0 , summing to 1. The model's whole job is to make that list **good**: high numbers on symbols that really do tend to follow this context, low numbers on the rest. A “confident” model concentrates its mass on a few symbols; an “uncertain” one spreads it thin. Spread-thin means high entropy means an expensive file.

Definition — Autoregressive language model. A function p_θ that, given a context $x_1 \dots x_{t-1}$, returns a probability distribution $p_\theta(\cdot \mid x_1 \dots x_{t-1})$ over the next symbol x_t in a fixed vocabulary. The subscript θ (“theta”) stands for the model’s **parameters** — the millions or billions of internal numbers, learned from data, that determine its predictions. We treat p_θ as a black box: text in, probabilities out.

We are deliberately **not** opening that black box here. **How** the model turns context into probabilities — the neurons, the attention, the gradient descent — is the subject of Chapter 56’s machine-learning primer and Chapter 57’s learned codecs. For **this** chapter, the only property that matters is the contract in the definition: context in, a clean probability distribution out. Given that contract, Chapter 26’s coder does the rest.

Aside. This is the deep reason the book put information theory (Volume I) before neural networks (Volume IV). A language model is “just” a model in the model-plus-coder picture we drew in Chapter 1. Everything we learned about good and bad models — that they must be **shared** by encoder and decoder, that their quality is measured in bits — applies unchanged. The neural net is a spectacularly good model. It is not a new **kind** of thing.

7.2.2 The tokenizer: the alphabet the model actually sees

There is one wrinkle that turns out to matter enormously, and it is a classic place to get confused. A language model does not usually predict **letters** or **bytes**. It predicts **tokens**.

A **tokenizer** is a fixed, agreed-upon rulebook that chops text into chunks called tokens and assigns each chunk a number. Common words become a single token; rare words get split into several pieces. The string **compression** might be one token; **antidisestablishmentarianism** might be five. The model’s “vocabulary” is the list of all possible tokens — often around 50,000 to 200,000 of them — and the model predicts a distribution over **that** list, not over the 256 possible byte values.

Σ GO FURTHER · THE MATHS — Why tokens, and why they *skip if familiar*
complicate the bits

Predicting tokens instead of bytes is mostly an efficiency trick: a sequence of 1,000 English words is maybe 1,300 tokens but 6,000 bytes, so the model handles a shorter sequence. But it muddies our accounting in two ways.

First, **bits per token** and **bits per byte** are different units (the trap from the checkpoint above). To report a file’s compression honestly we must divide the total bits by the number of **original bytes**, not the number of tokens.

Second — and this is subtle — the tokenizer itself silently **does some compression** before the model ever runs, because frequent strings collapse to single tokens. Worse, a single byte string can sometimes be tokenized more than one way, so a careless scheme could even be ambiguous. Honest LLM compressors must pin the tokenizer down exactly, ship it on both sides, and either feed the model raw bytes or account for the tokenizer’s contribution. DeepMind’s cleanest experiments avoided the issue by having the model predict **raw bytes** directly — a 256-symbol vocabulary — precisely so that “bits per byte” meant exactly what it says.

PITFALL

When you read “this LLM compresses to 0.8 bits per token,” your guard should go up. A token can be one byte or fifteen. The only unit that lets you compare against gzip, PNG, or FLAC is **bits per original byte** (bpb) — total compressed bits divided by the count of bytes in the **uncompressed** file. Always convert to bpb before believing a headline. Confusing the two is the single most common way LLM-compression claims get inflated.

Myth. *that a language model “memorizes” text and that is why it can compress it.*

Reality. Memorization is the wrong picture and would actually **defeat** compression. If the model had simply stored enwik9, you would need to ship enwik9 inside the model to decompress — net savings zero. What the model does is **generalize**: it learns the statistical regularities of language (and, as we will see, of images and audio) so well that on text it has **never seen** it is rarely surprised. Low surprise on novel data is exactly what drives the file size down. The compression is a measure of **understanding the distribution**, not of remembering the file.

7.3 The headline result: a text model beats PNG and FLAC

In September 2023, Grégoire Delétang, Anian Ruoss, and colleagues at Google DeepMind posted a paper with a title that is also a thesis: **Language Modeling Is Compression** (arXiv:2309.10668; published at ICLR 2024). They did the obvious-in-hindsight experiment that nobody had cleanly nailed down before. They took **Chinchilla 70B** — a large language model trained on text — **froze it** (no further training, no peeking at the test data), and used its next-symbol probabilities to drive a standard arithmetic coder. To make the units honest, they had it predict raw bytes.

The numbers are worth stating exactly, because they are the empirical heart of the whole “compression = intelligence” program.

Data (1 GB scale)	gzip	LZMA2	Chinchilla 70B
enwik9 (Wikipedia text)	≈ 3.1 bpb	≈ 2.3 bpb	0.642 bpb
ImageNet patches	58.5%	—	43.4%
LibriSpeech audio	—	—	16.4%

Raw compression rates (lower is better). For images/audio the figure is percentage of raw size; PNG manages 58.5% on those image patches, FLAC 30.3% on that audio. Source: Delétang et al., 2024.

Read the text row first. On **enwik9**, a full gigabyte of English Wikipedia, the frozen Chinchilla 70B reaches **0.642 bits per byte**. The everyday workhorse **gzip** (DEFLATE, Chapter 30) sits near 3.1; the strong general-purpose coder LZMA2 (Chapter 31) near 2.3. The language model roughly **quartered** gzip’s output. On text, where you would expect a **text** model to shine, it does — emphatically.

But the rows that made jaws drop are the other two. **ImageNet patches** are little squares of photographs. **LibriSpeech** is recorded human speech. Chinchilla was never trained on either; it was fed the raw bytes of images and audio as if they were a foreign language. And it compressed the images to **43.4%** of their size — beating **PNG** (Chapter 44), a codec built by humans specifically for images, at 58.5%. It compressed the audio to **16.4%** — beating **FLAC** (Chapter 50), a codec built specifically for lossless audio, at 30.3%.

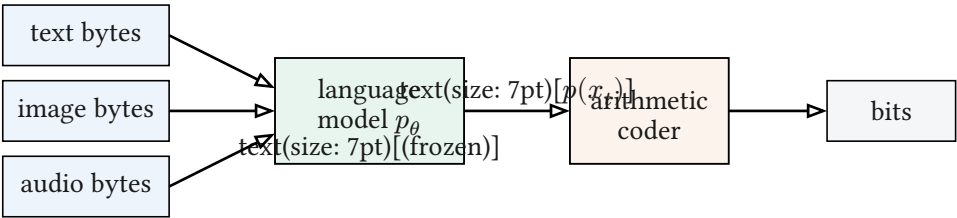


Figure 18: The universal-predictor pipeline: the same arithmetic coder is fed bytes from text, image, or audio; only the model-supplied probabilities differ. A general enough predictor beats the specialised codec for each modality.

KEY IDEA

A model that has only ever read **text** out-compresses dedicated **image** and **audio** codecs on their own turf. Why is this even possible? Because images and audio, viewed as byte streams, are full of statistical regularity — neighbouring pixels are similar, sound samples drift smoothly — and a **sufficiently general** next-byte predictor exploits that regularity better than a hand-built, domain-specific codec does. The language model learned, from words, a very broad sense of “what byte sequences from the real world look like,” and that generality transfers. This is the most vivid demonstration in the whole book that a **good enough predictor is a universal compressor**.

Authors	Delétang, Ruoss, et al. (Google DeepMind); precursors: Knoll's <code>lstm-compress</code> (2015), Bellard's <code>nncp</code> (2019)
Aim	Lossless compression by driving an arithmetic coder with an autoregressive neural language model's next-symbol probabilities.
Complexity	Encode and decode: one full neural-network forward pass per symbol — orders of magnitude slower than classical coders.
Strengths	State-of-the-art ratios; fully general (text, image, audio as raw bytes); rides every advance in language modelling for free.
Weaknesses	Enormous fixed model cost; punishingly slow; fragile to non-determinism; the model can dwarf the data it compresses.
Superseded by	An active 2024–2026 frontier, not superseded; specialised online coders (<code>nncp</code>) and tiny pretrained ones (<code>ts_zip</code>) explore the practical corners.

The scheme is model-agnostic: **any** function that emits a valid next-symbol distribution can be the front end. Swap in a bigger or better-trained model and the ratio improves with no change to the coder. That clean separation — Chapter 1's model-plus-coder split — is exactly why the arithmetic coder we wrote in Chapter 26 needs **zero** modification to become a neural compressor.

HISTORICAL NOTE

The idea did not spring from nowhere in 2023. Byron Knoll's `lstm-compress` and `cmix` had been mixing neural predictors into context-mixing compressors since the mid-2010s, and Matt Mahoney's PAQ lineage (Chapter 34) had long blended many models' predictions. What DeepMind added was **scale and clarity**: take a single, huge, off-the-shelf language model, change nothing about it, and report clean bits-per-byte across modalities. The result reframed **scaling laws** — the empirical curves showing that bigger models predict better — as compression curves: a bigger model is a better compressor, full stop. That reframing is the bridge from this chapter back to Chapter 61's theory and forward to the rest of the AI era.

7.4 Bellard's tools: making it actually run

A paper proves a point; an engineer ships a program. Enter **Fabrice Bellard** — the prodigious French programmer behind FFmpeg, QEMU, the Tiny C Compiler, and a world-record computation of π . Bellard turned the LLM-compression idea into two real,

downloadable tools that sit at opposite ends of a fundamental design choice: **do you ship the model, or learn it on the fly?**

7.4.1 nnep: learn the file as you compress it (online)

Bellard’s **nnep** (first released 2019) takes the **online** route. The neural network starts out **untrained** — knowing nothing — and **learns the file while it compresses it**. As it reads the data left to right, after each chunk it nudges its own parameters to predict that chunk better, so its predictions sharpen as it goes. The decompressor does the **identical** learning in lockstep: it starts from the same blank model, decodes a chunk, then performs the same parameter update, so encoder and decoder always hold the same model. Because both sides regenerate the model deterministically, **no model weights need be stored in the file at all**.

Σ GO FURTHER · THE MATHS — Online vs. pretrained, in one breath skip if familiar

Online (nnep): model starts empty, both sides train it identically as the data streams by. Nothing to ship; the model cost is paid in **time**, not in stored bytes. The decompressor must redo every training step the compressor did — so it is exactly as slow as the compressor (a property classical coders never have).

Pretrained (ts_zip, DeepMind’s setup): a fixed model is trained once, frozen, and shipped to both sides. Fast to **use** per file, but the model is a large fixed cost that must already live on both machines.

Same identity ($L = \sum -\log_2 p$) drives both; they differ only in **where the model comes from and who pays for it**.

nnep moved from an LSTM (a kind of recurrent network) to a **Transformer** in its v2 (January 2021), tracking the same architecture shift that powered the chat-model revolution. On Matt Mahoney’s **Large Text Compression Benchmark** (Chapter 36 — enwik8 / enwik9), nnep has long sat among the very best entries: the 2024 release (nnep v3.x) compresses **enwik8** to roughly **1.19 bits per byte** and **enwik9** to about **0.85 bits per byte** — figures competitive with, and at times beating, the elaborate hand-engineered context-mixing compressor **cmix** (Chapter 34). For a program with essentially no hand-coded knowledge of English, only the ability to learn, that is remarkable.

Aside. A subtlety worth savouring: nnep’s online model **does** effectively get stored in the file — not as weights, but as the **bits it costs to teach the model from scratch as you go**. Early in the file the untrained model is surprised by everything and the coder pays dearly; later the trained model is rarely surprised and pays little. The “model size” is amortised across the whole file as up-front clumsiness. This is the **minimum description length** idea of Chapter 23 made literal: the file pays for both the model and the data, automatically.

7.4.2 ts_zip: ship a small frozen brain (pretrained)

Bellard’s **ts_zip** (2023–) takes the opposite, pretrained route, and engineers it for the real world. It ships a **frozen** language model — an **RWKV** model with about **169 million**

parameters, an RNN-style architecture chosen because it is fast and has a small, fixed memory footprint — **quantized to 8 bits** per parameter to keep it compact. It arithmetic-codes text against that fixed model. Reported figures hover around **1.1 bits per byte** on `enwik8` and `enwik9` and on classic test files like `alice29.txt`. It needs a GPU and a few gigabytes of RAM, and on a high-end card it runs at **roughly 1 MB per second**.

That “1 MB/s” number deserves a flinch. We will return to it.

ts_zip (Bellard)

2023

Authors	Fabrice Bellard
Aim	Practical pretrained LLM text compressor: a small, frozen, deterministic RWKV model + arithmetic coding.
Complexity	≈ 1 MB/s on an RTX-4090-class GPU; ~4 GB RAM; one model forward pass per token.
Strengths	≈ 1.1 bpb on text — well past LZMA; small shippable model (~169M params, 8-bit); engineered for cross-hardware determinism.
Weaknesses	Needs a GPU; far slower than classical coders; tied to the exact model version (Bellard warns: 'no backward compatibility should be expected').
Superseded by	Companion to nncp; both are active research-grade tools, not production replacements for zstd.

The choice of an RNN-style RWKV model over a Transformer is deliberate: RWKV processes tokens with a fixed-size running state, so its per-token cost and memory do not balloon with context length, which matters when you must run the **same** model identically on both ends, possibly on different hardware.

7.4.3 The determinism trap

Here is a danger classical compression never had to think about, and it is the most important practical lesson in the chapter. Arithmetic coding is **unforgiving**: the decoder must compute the **bit-for-bit identical** probability the encoder used at every single step. If they differ even in the last decimal place, the decoder’s interval drifts away from the encoder’s, every subsequent symbol decodes wrong, and the file is **irrecoverably corrupt** — not slightly wrong, but garbage from the point of divergence onward.

Now recall that neural networks are floating-point machines (Chapter 13). Floating-point arithmetic is **not** associative — $(a + b) + c$ can differ from $a + (b + c)$ in the last bit — and different GPUs, different CPUs, different math libraries, even different thread counts can sum the same numbers in different orders and get answers that differ in the last bit. For ordinary AI that is invisible. For arithmetic coding it is **fatal**.

PITFALL

A neural compressor is only as reliable as its determinism. The encoder and decoder must produce **identical** probabilities to the last representable digit, on whatever hardware each happens to run. Bellard’s hardest engineering in `ts_zip` was not the model — it was making that 169M-parameter network evaluate **reproducibly** across GPUs and CPUs. Get this wrong and your “compressed” file decodes to noise. This is why every serious neural codec pins the exact model version, the exact arithmetic, and warns loudly that old files may be unrecoverable if any of it changes.

Checkpoint. Why can a tiny floating-point discrepancy between encoder and decoder destroy an **entire** arithmetic-coded file, rather than just garbling one symbol?¹¹

7.5 Build it: `tinyzip`’s LLM-as-compressor

Theory is cheap. Let us make a real one round-trip. The beauty of everything above is that we **already wrote the coder** in Chapter 26; all we must add is a model that emits a probability distribution for the next symbol, and a loop that wires it to the coder. We will use a **deliberately tiny, deterministic** model — an order-2 adaptive byte model, the very kind `nncp` uses in spirit but shrunk to fit on a page — so the whole thing runs in plain Python with no GPU and no gigabyte downloads, yet demonstrates the **exact** mechanism a 70-billion-parameter model uses.

NOTE

We use a tiny statistical model, not a real neural net, on purpose. A real LLM needs PyTorch, a GPU, and hundreds of megabytes of weights — none of which belong in a from-scratch teaching codec. But the **interface** is identical: our model exposes a next-symbol distribution and an `update(symbol)` step — exactly the contract a frozen Chinchilla or an online RWKV satisfies, and exactly the contract Chapter 26’s coder already consumes. Swap our 40-line model for a neural one and the surrounding code is unchanged. **That substitutability is the whole point.**

BUILD `tinyzip` · Step 22 · `llmzip.py` — model $P(\text{next})$ + arithmetic coder

This is the final `tinyzip` step of the neural volume (the canonical step from `TINYZIP.md`). We reuse the **exact** `ArithmeticEncoder / ArithmeticDecoder` from Chapter 26 (`tinyzip/arithmetic.py`) and the `container` from Chapter 17

¹¹**Checkpoint answer.** Because arithmetic coding (Chapter 26) encodes the **whole message** as **one shrinking interval**, and each symbol’s decoding depends on the cumulative state built from **all previous** symbols’ probabilities. One wrong probability nudges the interval off-track; from that point on, every symbol is decoded against the wrong sub-interval, so the corruption **cascades** through the entire remainder of the file. There is no per-symbol resynchronisation to save you.

(tinyzip/container.py), and we add tinyzip/llmzip.py: an autoregressive byte predictor wired to that coder. It round-trips and ships a self-test.

The trick to reuse is to notice what Chapter 26's coder actually asks of its model. There, the FrequencyModel exposed exactly three things the coder calls — cumulative(sym) -> (cum_low, cum_high, total), find(scaled) -> sym, and update(sym) — and the coder did **not care** where those numbers came from. That is the model-plus-coder split of Chapter 1 made literal: **anything** that answers those three questions can drive the coder unchanged. So our LLM-style model just has to answer the same three questions. We are not inventing a new coder interface; we are **implementing the one Chapter 26 already defined**.

First, the model. An **order-2 context model** predicts the next byte from the previous two bytes. It is **adaptive**: it starts knowing nothing and learns the file as it reads — exactly nnpc's online philosophy, in miniature. Every byte keeps a count of at least 1 so nothing ever has probability zero (a zero would mean “infinitely surprising,” — $-\log_2 0 = \infty$ bits — an uncodeable, coder-killing zero-width slice, exactly the reason Chapter 26's model also started every count at 1).

PY GO FURTHER · PYTHON — Counting with a dict, and *skip if fluent*
the .get / .setdefault fallback

A dict (Chapter 16) maps keys to values. Here we map a **context** (the last two bytes, as a tuple) to a list of 256 counts — one tally per possible next byte. counts.get(ctx) returns the list for that context, or None if we have never seen it; dict.setdefault(ctx, default) returns the existing list or installs default and returns that. We use these so a brand-new context starts from flat counts instead of crashing.

```
counts: dict[tuple[int, int], list[int]] = {}
ctx = (65, 66) # previous two
bytes
row = counts.setdefault(ctx, [1] * 256) # 256 ones =
uniform prior
row[67] += 1 # we just saw byte
67 after this context
```

```
# tinyzip/llmzip.py — an LLM-as-compressor demo (tiny,
deterministic model)
from tinyzip.arithmetic import ArithmeticEncoder,
ArithmeticDecoder, QUARTER
```

```

class Order2Model:
    """Autoregressive byte predictor: P(next byte | previous two
    bytes).
    Answers the SAME three questions Chapter 26's FrequencyModel
    did
    (cumulative / find / update), so the Chapter 26 coder drives
    it as-is.
    Adaptive (learns online), pure-integer, deterministic. No
    EOF symbol:
    the container records the length, so the loops run for
    exactly n bytes."""

    def __init__(self) -> None:
        self.counts: dict[tuple[int, int], list[int]] = {}
        self.ctx: tuple[int, int] = (0, 0)

    def _row(self) -> list[int]:
        """The 256 counts for the current context (uniform if
        never seen)."""
        row = self.counts.get(self.ctx)
        return row if row is not None else [1] * 256

    @property
    def total(self) -> int:
        # the coder reads
        model.total
        return sum(self._row())

    def cumulative(self, sym: int) -> tuple[int, int, int]:
        """(cum_low, cum_high, total) for byte `sym` – the
        coder's narrowing."""
        row = self._row()
        low = sum(row[:sym])
        # counts strictly
        before sym
        return low, low + row[sym], sum(row)

    def find(self, scaled: int) -> int:
        """Inverse of cumulative: which byte's slice contains
        `scaled`."""
        row = self._row()
        cum = 0
        for sym in range(256):
            if cum + row[sym] > scaled:
                return sym
            cum += row[sym]
        raise ValueError("scaled value out of range")

```

```

def update(self, sym: int) -> None:
    row = self.counts.setdefault(self.ctx, [1] * 256)
    row[sym] += 1
    self.ctx = (self.ctx[1], sym)          # slide the 2-byte
window

```

Those three methods **are** the model-plus-coder contract. `cumulative` and `find` are an exact inverse pair (the slice that `cumulative` lays down is the slice `find` walks back), and `update` makes the just-seen byte likelier next time — and, crucially, **slides the context**, so the very next `cumulative` / `find` answers from a freshly-conditioned distribution. The whole “autoregressive” idea lives in that one line `self.ctx = (self.ctx[1], sym)`.

PY GO FURTHER · PYTHON — Integer frequencies, not floats *skip if fluent*
— and why

Notice the model never produces a probability like `0.31`. It hands the coder whole-number **counts** whose running total is `model.total`, and the coder turns a count of, say, `c` out of `total` into a cost of about $-\log_2(c / \text{total})$ bits with pure integer arithmetic. This is deliberate: Chapter 26’s coder is integer-only precisely to stay deterministic — exactly the floating-point trap this chapter warned about. A real neural model emits floats, so the real engineering step is to **quantise** those floats to integer frequencies the same way (the very thing `ts_zip` must do reproducibly across GPUs). Here we sidestep that by counting in integers from the start.

Now the encode/decode loop. Because our `Order2Model` answers the same three questions as Chapter 26’s `FrequencyModel`, we can hand it straight to the Chapter 26 coder’s per-symbol primitives. The encoder, at each position, narrows the interval for the real next byte, then updates the model; the decoder mirrors it exactly — it recovers the byte from the interval, then performs the **same** update.

```

def llm_compress(data: bytes) -> bytes:
    model = Order2Model()
    enc = ArithmeticEncoder()
    for byte in data:
        enc._encode_symbol(model, byte)      # narrow interval;
calls model.update
        enc.pending += 1                    # flush: pin a
point in the interval
    enc._emit(1 if enc.low >= QUARTER else 0)
    return enc.out.flush()

```

```
def llm_decompress(blob: bytes, n: int) -> bytes:
    model = Order2Model()
    dec = ArithmeticDecoder(blob)
    out = bytearray()
    for _ in range(n):                # length comes
from the container
        byte = dec._decode_symbol(model)    # recover byte;
calls model.update
        out.append(byte)
    return bytes(out)
```

NOTE

To keep this readable we lean on two small, mechanical refactors of the Chapter 26 coder, which that chapter already wrote as private helpers in spirit: `_encode_symbol(model, sym)` (its narrowing + renormalize body, already shown in Chapter 26) and a matching `_decode_symbol(model)` that performs one decode step (the inside of its `decode` loop). `QUARTER`, `enc.pending`, `enc._emit`, and `enc.out` are exactly the names defined in `arithmetic.py`. We drop the `EOF` symbol that Chapter 26 used to mark end-of-stream because the **container** already stores the original length, so `llm_decompress` simply runs the loop `n` times. Nothing about the interval math changes.

PY GO FURTHER · PYTHON — Mirrored loops: the round-trip *skip if fluent* invariant

The two loops are **structurally identical**: ask the model, (encode or decode) one symbol, and let that step **update** the model. That symmetry is what guarantees `decode(encode(x)) == x`. The encoder knows the byte and spends bits; the decoder spends bits to learn the byte. Because both consult the model's distribution **before** the byte is known and **update after**, their models stay in lockstep, byte for byte. Break the symmetry — update before consulting on one side — and the streams desynchronise, exactly the failure mode this chapter warns about.

Finally the self-test and a container-wrapped round trip. We reuse Chapter 17's container, whose signature is `write(path, method, payload, original_len)` and `read(path) -> (method, original_len, payload)` — it stores the original length, which is exactly the `n` the decoder needs:

```
METHOD_LLMZIP = 9                                # this codec's
method byte
```

```

def _selftest() -> None:
    import tinyzip.container as container
    sample = (b"the cat sat on the mat. " * 40
              + b"the cat ran to the mat. " * 40)
    packed = llm_compress(sample)
    restored = llm_decompress(packed, len(sample))
    assert restored == sample, "round-trip failed!"
    ratio = len(packed) / len(sample)
    print(f"in {len(sample)} out {len(packed)} "
          f"ratio {ratio:.3f} "
          f"bits/byte {8 * len(packed) / len(sample):.3f}")
    # also exercise the on-disk container path of Chapter 17
    container.write("demo.llz", METHOD_LLMZIP, packed,
original_len=len(sample))
    method, n, payload = container.read("demo.llz")
    assert payload == packed and n == len(sample)

if __name__ == "__main__":
    _selftest()

```

Run it and the repetitive sample compresses dramatically — the order-2 model learns the handful of recurring contexts within the first few sentences and is barely surprised thereafter, so the coder pays almost nothing for the rest. That is **exactly** the mechanism behind the headline results, only here the “model” is 40 lines and there it is 70 billion parameters. The contract — `cumulative`, `find`, `update`, `feed` the very same coder — is identical.

NOTE

Where would a real LLM plug in? Replace `Order2Model`’s `cumulative / find` with a call into a frozen neural network that returns 256 byte-probabilities (or a token distribution, with the tokenizer caveats above), quantised to integer frequencies exactly as Chapter 26’s coder expects. The coder, the loops, the container — all unchanged. The hard parts become **the model download** and **cross-hardware determinism**, not the compression logic. You have already built the compression logic.

7.6 The scoreboard, and a dose of honesty

Let us put the LLM line on the same scoreboard we have grown all book, on the kind of text sample `tinyzip` has been chewing through. The point is not that our toy order-2 model rivals Chinchilla — it does not — but to slot the **technique** into the lineage and show where the real frontier sits.

SCOREBOARD — cumulative, on representative English text ($\approx$ bits/byte; lower is better)

Technique	Bytes	Ratio	Notes
Raw (no compression)	8.00 bpb	1.0×	the baseline; every byte costs 8 bits
Huffman (Ch 24)	≈ 4.5 bpb	≈ 1.8×	optimal per-symbol code; no context
DEFLATE / gzip (Ch 30)	≈ 3.1 bpb	≈ 2.6×	LZ77 + Huffman; the web's workhorse
bzip2 / BWT (Ch 35)	≈ 2.3 bpb	≈ 3.5×	block-sorting; strong on text
LZMA2 / xz (Ch 31)	≈ 2.3 bpb	≈ 3.5×	large dictionary + range coding
cmix / context-mixing (Ch 34)	≈ 1.0 bpb	≈ 8×	hundreds of mixed models; very slow
<code>tinyzip</code> <code>llmzip</code> (toy)	model-dep.	—	order-2 demo of the LLM mechanism
nncp (online Transformer)	≈ 0.85 bpb	≈ 9.4×	learns the file as it codes; nothing shipped
Chinchilla 70B + AC	0.642 bpb	≈ 12.5×	frozen text LLM; SOTA raw — but see below

Figures are representative bits-per-byte on enwik-class English text and vary with file and version. The crucial column is missing on purpose: **the model's own size**. Read on.

Those bottom rows look like a clean victory — gzip's 3.1 quartered to 0.642. But a victory it is only if you ignore three things, and intellectual honesty forbids ignoring them. This is the most important section of the chapter.

7.6.1 Honesty 1: the model is part of the file

A frozen LLM is not free. The decoder must **possess the entire model** to decompress anything at all. The genuinely fair number is therefore not the file's bits but the **adjusted** size: file bits **plus** model bits.

***Theorem (Two-part code lower bound (informal)).** For a decoder that does not already possess the model, the honest description length of compressing a single file D with a model M is at least*

$$L_{\text{honest}} = \underbrace{L(M)}_{\text{bits to describe the model}} + \underbrace{L(D \mid M)}_{\text{bits to code } D \text{ using } M}.$$

No accounting that omits $L(M)$ can be a true measure of how much information was conveyed.

Proof. To reconstruct D from the compressed stream alone, a receiver who starts with nothing must be handed **both** the model and the model-coded data — there is no other source for M . So the total information transmitted is $L(M) + L(D | M)$. Omitting $L(M)$ assumes the receiver already has M for free, which silently moves a (possibly enormous) cost off the books. This is precisely the **minimum description length** principle of Chapter 23, which insists a hypothesis be charged for **itself** as well as for the data it explains. $\square$

Now run the numbers. Chinchilla 70B in 16-bit precision is about **140 gigabytes** of parameters. The file it so impressively compressed, enwik9, is **1 gigabyte**. The model is **140 times larger than the data**. By the honest two-part accounting, “compressing” a single 1 GB file by shipping a 140 GB model to do it is absurd — you would have been a thousand times better off e-mailing the raw file. The DeepMind authors are completely explicit about this: their off-domain wins are **real and important as science**, but they are **not** free-lunch single-file compression.

KEY IDEA

An LLM compressor only “pays for itself” when its huge fixed model cost is **amortised over an enormous amount of data** — so much that the per-file share of the model is negligible. Compress a **library**, and a 140 GB model spread over many terabytes is a rounding error. Compress **one document**, and the model is a catastrophe. This is exactly why Bellard’s **169-million**-parameter ts_zip (a few hundred megabytes) is the **practical** sweet spot, not the 70-billion-parameter giant: it is small enough that a person might actually keep it installed and amortise it across everything they ever compress.

Myth. *that DeepMind’s result means an LLM is, all-in, the world’s best compressor of a given file.*

Reality. Only if you do not charge for the model. The headline **0.642 bpb on enwik9** is the cost of coding the **data given the model**, $L(D | M)$ — it pointedly excludes the 140 GB of $L(M)$. As a **scientific measurement of the model’s predictive quality** it is honest and stunning. As a **practical recipe to make your file smaller**, it is only sensible when the same model is reused across a vast corpus. Both statements are true at once; conflating them is how the hype outruns the reality.

7.6.2 Honesty 2: it is staggeringly slow

Classical coders move at the speed of memory. **zstd** (Chapter 32) compresses **hundreds of megabytes per second** on one ordinary CPU core. ts_zip manages **about 1 MB/s** — and that is **on a high-end GPU**. Full LLM coders driving 70-billion-parameter models are far slower still, because **every single byte or token costs a complete forward pass** through the network. You are spending three to six **orders of magnitude** more compute, and a correspondingly enormous amount of energy, to shave a fraction of a bit per byte off an already-good classical result.

Σ GO FURTHER · THE MATHS — Orders of magnitude: feeling skip if familiar the gap

An **order of magnitude** is a factor of ten (Chapter 7’s logarithms again). “Three orders of magnitude” is $10^3 = 1000$ times. If zstd does 500 MB/s and ts_zip does 1 MB/s, the gap is $500 \times$ — between two and three orders of magnitude. To compress a 1 GB file: zstd, about **two seconds**; ts_zip, on a GPU, about **seventeen minutes**. For a marginal ratio gain on most workloads, that is a trade almost nobody should take — which is precisely why your phone still uses Brotli and zstd, not an LLM, to load this very web page.

PITFALL

Do not let the gorgeous ratios seduce you into thinking LLM compression is ready to replace zstd in your backups or your web server. For the overwhelming majority of real workloads — where speed, energy, and the absence of a multi-gigabyte model dependency matter — classical coders win decisively. LLM compression is, today, a **research instrument and a niche tool**, not a daily driver. Knowing **when not to use** a technique is as much a mark of expertise as knowing how it works.

7.6.3 Honesty 3: it is fragile

Lose the exact model, or run it on hardware that rounds even slightly differently, and the data is **gone forever** — the determinism trap, now with archival stakes. A file compressed with model version 1.3 may be undecodable by version 1.4. Bellard states the discipline bluntly for ts_zip: “**no backward compatibility should be expected.**” For ephemeral or amortised use that is fine. For a **long-term archive** — the very thing the Hutter Prize (Chapter 36) is about — it is a serious liability that classical, fully-specified, model-free formats like gzip simply do not have. A gzip file made in 1993 still opens today; nobody can promise that of a file compressed against a particular checkpoint of a particular neural network.

Aside. This is why the dedicated **Hutter Prize** — which charges the **decompressor’s own size and runtime** against you and demands the whole thing run within tight resource limits — is **not** topped by a giant LLM. Its October 2024 record was set by Kaido Orav and Byron Knoll’s **fx2-cmix**, a hand-engineered context-mixing compressor reaching **110,793,128 bytes** on enwik9. nncp’s **raw ratio** is superb, but once you must **pay for the model and the clock**, the elaborate-but-self-contained classical coder wins the prize. The prize’s rules encode exactly the honest accounting of this section.

7.7 Compression as intelligence: the evidence and the limits

We opened by invoking the slogan **compression is intelligence**, which Chapter 61 examined as pure theory. This chapter gives it teeth — and then, in fairness, files them down.

The strongest **empirical** support is a 2024 paper by Yuzhen Huang, Jinghan Zhang, Zifei Shan, and Junxian He (HKUST/Tencent): **Compression Represents Intelligence Linearly** (COLM 2024). Across **31** publicly available language models from many different organisations, evaluated on **12** benchmarks spanning knowledge, coding, and mathematical reasoning, they measured each model’s **bits per character** on held-out text and plotted it against the model’s average benchmark score. The relationship was almost a **straight line**, with a Pearson correlation coefficient around **−0.95** — the better a model compresses, the higher its measured “intelligence,” nearly without exception.

Σ GO FURTHER · THE MATHS — Pearson correlation r , and what -0.95 means *skip if familiar*

The **Pearson correlation coefficient** r is a single number between -1 and $+1$ that summarises how tightly two quantities move together along a straight line.

- $r = +1$: a perfect upward line (one goes up, the other goes up, exactly).
- $r = 0$: no linear relationship at all (a shapeless cloud).
- $r = -1$: a perfect **downward** line (one goes up, the other goes **down**, exactly).

Here one axis is **bits per character** (lower = better compression) and the other is **benchmark score** (higher = smarter). Better compression means **fewer** bits, so a smarter model has a **lower** bpc and a **higher** score — the two move in opposite directions, hence the **minus** sign. The magnitude 0.95 (very close to 1) says the points hug that downward line astonishingly tightly. An r of -0.95 across dozens of independently-built models is a strikingly strong, clean empirical signal.

That is a genuinely surprising and weighty result. It says that learning to predict text efficiently — to compress it — **tracks** measured capability with near-linear fidelity, across models nobody coordinated. It is the best evidence we have that the slogan is more than a slogan.

KEY IDEA

Better compression of text correlates almost linearly with measured intelligence. This is the empirical backbone of the “compression = intelligence” program. It strongly supports one direction of the claim: **being intelligent implies compressing text well**. A capable model is, by that very capability, a good predictor and hence a good compressor.

But — and Chapter 61 already warned us — a correlation in one direction is not an identity in both. The honest 2025–2026 consensus is careful. The evidence supports **intelligence** → **good compression** (a smart model predicts text well). It does **not** establish the converse, **all intelligence = compression**:

- **Some capability need not show up as predictable token statistics.** Out-of-distribution generalisation, multi-step reasoning that diverges from the training distribution,

the use of tools and external memory — these may not register cleanly as lower bits-per-character on a held-out text corpus.

- **The correlation is measured on text.** It is about **language** modelling. Intelligence that is not primarily linguistic sits outside the measurement.
- **The brute economics cut against the slogan as a definition.** If being the world's best compressor requires a 140 GB model running at 1 MB/s, then “intelligence = compression” cannot be the **whole** story of intelligence, which is also about doing more with less — efficiency, transfer, the right answer the first time.

Myth. *that DeepMind's and Huang et al.'s results prove “intelligence is just compression.”*

Reality. They prove something more careful and, honestly, more interesting: **compression is a reliable measurable signature of capability.** A model that compresses text well is, with near-certainty, a capable model — that is the $r \approx -0.95$ result, and it is real. But “X is a strong signature of Y” is not “X is identical to Y.” Compression appears to be a **necessary** hallmark of capable language models, not a **complete definition** of intelligence. The identity **log-loss = code length** is exact and beautiful; the leap from there to **intelligence = compression** is a philosophical bet, not a theorem.

Where does that leave us? With one of the most satisfying convergences in all of computing. A field that began with Morse counting dots and dashes (Chapter 2), that Shannon turned into mathematics in 1948 (Chapter 18), that spent fifty years inventing ever-cleverer hand-built coders (Volume II), has arrived — via the very same identity it started with, $L = \sum -\log_2 p$ — at the doorstep of artificial intelligence. The best compressor and the best predictor turned out to be the same machine. That is not the end of the story; it is a hinge. But it is a hinge worth pausing on.

Chapter takeaways

- An autoregressive language model emits $p(x_t \mid \text{context})$; feed that to an arithmetic coder and you have a lossless compressor whose file size is exactly $\sum -\log_2 p$ — the model's cross-entropy. **Training a language model is building a compressor.**
- Log-loss in nats converts to bits-per-symbol by multiplying by $\frac{1}{\ln} 2 \approx 1.4427$. Always report **bits per original byte**, never per token, to compare against gzip/PNG/FLAC.
- DeepMind's **Language Modeling Is Compression** (2023) used a frozen Chinchilla 70B to reach **0.642 bpb on enwik9** and — astonishingly — to beat **PNG** on images (43.4% vs 58.5%) and **FLAC** on audio (16.4% vs 30.3%) with a **text-only** model. A good enough predictor is a universal compressor.
- Bellard's **nncp** learns the file online (nothing shipped, 0.85 bpb on enwik9); his **ts_zip** ships a small frozen RWKV-169M (1.1 bpb, 1 MB/s on a GPU). Both demand **bit-exact determinism** across hardware, or the file decodes to noise.

- Honest accounting charges **model bits + file bits**. A 140 GB model compressing a 1 GB file is only sensible amortised over a vast corpus — which is why tiny models are the practical sweet spot and why the Hutter Prize (won Oct 2024 by **fx2-cmix**, 110.8 MB) is **not** topped by giant LLMs.
- LLM compression is slow, fragile, and model-dependent: a research instrument and niche tool, not a replacement for `zstd`. Knowing **when not to use it** is part of the expertise.
- Huang et al. (COLM 2024) found bits-per-character correlates with benchmark “intelligence” at $r \approx -0.95$ across 31 models — strong evidence that **intelligence implies good compression**, but not that **intelligence is nothing but** compression.

7.8 Exercises

Exercise 62.1 ★

A language model reports an average validation loss of 0.95 nats per token on English text. (a) Convert this to bits per token. (b) If, on this corpus, one token corresponds on average to 4.1 bytes, what is the model’s compression rate in **bits per byte**? (c) Is that better or worse than `gzip`’s typical ≈ 3.1 bpb on text?

Exercise 62.2 ★

Explain in two or three sentences, to a friend who has never heard of compression, why a model trained only on English text was able to compress **photographs** better than PNG, a codec built specifically for images.

Exercise 62.3 ★★

The message `BANANA` is to be compressed with an order-1 model (predict the next byte from the **one** previous byte) that, in each context, starts with a count of 1 for each of the three symbols $\{A, B, N\}$ and adds 1 to a symbol’s count after seeing it. Work out, symbol by symbol, the probability the model assigns to each **actual** next symbol of `BANANA`, and sum the surprisals $-\log_2 p$ to get the total code length in bits. (Treat the first symbol as predicted from an “empty” context that is uniform over the three symbols.)

Exercise 62.4 ★★

In `tinyzip`, both `llm_compress` and `llm_decompress` consult the model’s distribution (via `cumulative / find`) **before** the current byte is known, and the per-symbol step calls `model.update(byte)` **after**. Suppose a careless programmer changed the **decoder only** so that it called `model.update` on the **previously** decoded byte an extra time before decoding the next one. The encoder is untouched. Describe precisely what goes wrong and at roughly which symbol the output first diverges.

Exercise 62.5 ★★

A team compresses a single 2 GB text archive using a frozen 50-billion-parameter model stored at 2 bytes per parameter, achieving 0.7 bits per byte on the data. (a) What is the size of the model-coded data alone? (b) What is the model’s size? (c) Under the honest two-part accounting $L(M) + L(D | M)$, what is the total, and how does it compare to just keeping the raw 2 GB file? (d) Over how large a **corpus** would the model need to be amortised before the model-coded approach beats raw storage?

Exercise 62.6 ★

Why does the Hutter Prize — which charges the size and runtime of the **decompressor itself** — fail to be won by a giant frozen LLM, even though such a model achieves a better **raw** bits-per-byte than the winning entry? Connect your answer to the two-part code $L(M) + L(D | M)$.

Exercise 62.7 ★★★

Extend `tinyzip`’s `Order2Model` into an `OrderKModel` that conditions on the previous k bytes (so $k = 2$ reproduces the current model). Discuss, without necessarily running it: as k grows, what happens to (a) compression on a long repetitive file, (b) the memory used by the `counts` dict, and (c) the **early-file** cost before contexts have been seen? What does this trade-off have to do with the difference between a tiny online model and a giant pretrained one?

Exercise 62.8 ★★★

The identity **log-loss** = **expected code length** is exact. The slogan **intelligence** = **compression** is contested. Write a tight argument (a) for why the identity is genuinely a theorem and not a metaphor, and (b) for **one** concrete kind of intelligent behaviour that the bits-per-character measure of Huang et al. might fail to capture. Reference the relevant earlier chapters.

7.9 Further reading

- Grégoire Delétang, Anian Ruoss, et al., **Language Modeling Is Compression**, ICLR 2024 — the headline paper: Chinchilla 70B beating PNG and FLAC. [arXiv:2309.10668](https://arxiv.org/abs/2309.10668)
- Chandra Shekhara Kaushik Valmeekam et al., **LLMZip: Lossless Text Compression Using Large Language Models**, 2023 — an independent, concurrent demonstration of the LLM-plus-arithmetic-coder recipe. [arXiv:2306.04050](https://arxiv.org/abs/2306.04050)
- Yuzhen Huang, Jinghan Zhang, Zifei Shan, Junxian He, **Compression Represents Intelligence Linearly**, COLM 2024 — the $r \approx -0.95$ correlation across 31 models. [arXiv:2404.09937](https://arxiv.org/abs/2404.09937)
- Fabrice Bellard, **NNCP: Lossless Data Compression with Neural Networks** and **ts_zip: Text Compression using Large Language Models** — the practical online and pretrained tools, with benchmark figures and determinism notes. bellard.org/nncp and bellard.org/ts_zip

- Matt Mahoney, **Large Text Compression Benchmark** — the enwik8/enwik9 leaderboard where nncp and the context-mixers are ranked. matmahoney.net/dc/text.html
- Marcus Hutter, **The Hutter Prize for Lossless Compression of Human Knowledge** — the prize that encodes the honest two-part accounting; fx2-cmix’s October 2024 record. prize.hutter1.net

COMING UP NEXT

We have just seen that the **running** of a giant model is the bottleneck — every byte costs a full forward pass, and the model itself is a multi-gigabyte millstone. That raises an irresistible question: can we compress **the model**? If a 140 GB network could be squeezed to a few gigabytes without losing its predictive power, the honest accounting of this chapter would shift dramatically, and neural compression — and neural inference everywhere — would get cheaper, faster, and far more portable.

Chapter 63 turns the compression lens **inward**, onto the network’s own weights. We will meet **quantization** — storing each weight in 8, 4, or even fewer bits — through **GPTQ** and **AWQ**, and then push to the information-theoretic edge with **BitNet b1.58**, where every weight is one of just three values $\{-1, 0, +1\}$, costing $\log_2 3 \approx 1.58$ bits apiece. The compressor, at last, compresses itself.

Model Compression I: Quantization

The goal is to find the most compact description that still lets you do the task.

— Sanjiv Kumar, Google Research

Imagine you spent six months training a language model that can answer medical questions, write poetry, and debug code — all in the same breath. It weighs 14 gigabytes. Your hospital wants to run it on a tablet at the bedside. The tablet has 4 GB of RAM. You have a problem.

This is the Model Compression Problem, and it is not a niche research curiosity. It is one of the most practically important problems in all of computing right now. Every time you use an AI assistant on a phone, a translation app in an airplane without Wi-Fi, or a voice assistant on a smart speaker, you are using a compressed model. The original version lived on a server with dozens of high-end GPUs. What you actually run is a carefully squeezed copy — smaller, faster, and only slightly dumber.

Over the next two chapters we explore three compression levers for neural networks: **quantization** (this chapter), and **pruning, distillation, and low-rank factorization** (Chapter 64). In Chapter 65 we return to the inference-time problem of compressing the network's **memory** during a long conversation (the KV-cache). Together, these three chapters bring us to the absolute frontier of the field.

This chapter is about quantization: the art and science of replacing the high-precision floating-point numbers that describe a neural network's weights with far fewer bits — and doing it without wrecking what the network knows.

WHERE WE ARE

In Chapter 39 we met **scalar quantization** for images: map a continuous value to the nearest step on a fixed grid and code the integer index. Chapter 42 showed how JPEG uses quantization to discard high-frequency DCT coefficients aggressively. Chapter 62 showed that a language model is itself a compressor — it assigns a probability to every possible next word. **This** chapter asks the reverse: how do we compress the model itself?

By the end of this chapter you will be able to:

- Understand what a neural network weight is and why it is stored in floating point
- Explain the difference between post-training quantization (PTQ) and quantization-aware training (QAT)
- Follow the GPTQ algorithm step by step and see why second-order information matters
- Understand AWQ's insight about salient channels and activation magnitudes
- Know why outlier activations make naive INT8 quantization fail
- Grasp the information-theoretic argument behind BitNet b1.58 and ternary weights
- Have a concrete sense of what FP8 is and why it is the sweet spot for today's hardware
- Be able to choose among quantization strategies for a given deployment scenario

8.1 What Is a Neural Network Weight, and Why Does It Matter?

A large language model is, at its mathematical core, an enormous collection of real numbers called **weights**. When you ask the model a question, the answer is computed by multiplying your input by these weights, adding them up, squishing the result through a non-linear function, and repeating that pattern dozens of times. The whole computation is called a **forward pass**.

A model with 7 billion parameters — which is considered relatively small in 2024 — has 7,000,000,000 individual weights. If each weight is stored as a 32-bit floating-point number (the IEEE 754 `float32` format), the model occupies $7 \times 10^9 \times 4 \text{ bytes} = 28 \text{ GB}$. That does not fit in consumer GPU memory. Even switching to 16-bit half-precision (`float16` or `bfloat16`), the standard for training and inference since about 2018, gives $7 \times 10^9 \times 2 \text{ bytes} = 14 \text{ GB}$. Still too large for a tablet.

Σ GO FURTHER · THE MATHS — Floating-Point Numbers

skip if familiar

A **floating-point number** stores a value as three parts: a **sign bit** (positive or negative), an **exponent** (the power of 2 that sets the scale), and a **mantissa** or **fraction** (the significant digits).

- `float32` (single precision): 1 sign + 8 exponent + 23 mantissa = 32 bits. Can represent values from about $\pm 3.4 \times 10^{38}$ with 7 decimal digits of precision.
- `float16` (half precision): 1 + 5 + 10 = 16 bits. Range $\pm 65,504$, 3 decimal digits.
- `bfloat16` (“brain float 16”): 1 + 8 + 7 = 16 bits. Same exponent range as `float32`, but less precise mantissa — easier for the hardware to handle without overflow.

For comparison, `int8` is simply a signed 8-bit integer: values -128 to 127 , with no fractional part at all.

The big intuition: a `float32` weight might be something like 0.032847192. Do you really need 7 significant digits of precision for a single weight in a 7-billion-weight network? Almost certainly not — most weights could be rounded to 0.033 without anyone noticing. Quantization is the systematic exploitation of this slack.

The key insight that makes model quantization possible is statistical: in a trained network, weights are **approximately Gaussian** (bell-curve shaped) and concentrated near zero. The network spent months learning the values that matter; a tiny rounding error in each of billions of weights will, on average, partially cancel out. With the right algorithm, you can introduce controlled rounding and the model keeps working almost as well.

8.2 The Zoo of Bit Widths

Before diving into algorithms, let us survey the landscape. The community has standardized on a shorthand: *WkAm* means “weights in k bits, activations in m bits.”

- **W16A16**: BFloat16 or float16 throughout. The standard starting point, 2 bytes/weight.
- **W8A8**: INT8 weights and INT8 activations. Hardware-accelerated on NVIDIA T4, A100, H100, and Apple Silicon. 4× smaller than float32.
- **W4A16**: 4-bit weights, 16-bit activations. The most common deployment sweet spot in 2024. Weights are decompressed on the fly before each matrix multiply. Very fast to load; not quite as fast for compute.
- **W4A8**: 4-bit weights, 8-bit activations. Faster compute but harder to get right.
- **W2A8** or **W3A16**: Aggressive, often with noticeable quality loss. An active research area.
- **W1.58**: The ternary frontier. BitNet b1.58. Only three values: $-1, 0, +1$.

KEY IDEA

Every bit you shave off a weight halves the storage. Going from float16 (16 bits) to INT4 (4 bits) is a 4× compression of the model. For a 7B-parameter model that is the difference between 14 GB and 3.5 GB — it fits on a consumer GPU with room to spare.

8.3 The Outlier Problem: Why Naive Quantization Fails

Here is the trap that caught early practitioners by surprise.

Suppose you take a trained 7B-parameter language model and naively round every weight to the nearest 8-bit integer. The weights look roughly Gaussian around zero, so you pick a scale factor (the range of values, divided by 256 steps) and clamp. For most layers this works well. But for a small fraction of layers — roughly 0.1% of the feature dimensions — the **activations** (the intermediate computed values during a forward pass, not the weights themselves) contain huge outliers. Values like 100 or 200, when most values are between -1 and 1 .

Why does this matter for weight quantization? Consider: a weight of 0.01 multiplied by a “normal” activation of 0.5 gives 0.005. Round the weight to 0 (because it is very small relative to the scale) and you lose 0.005. Fine. But a weight of 0.01 multiplied by an outlier activation of 150 gives 1.5. Round the same tiny weight to 0 and you lose 1.5 — a catastrophic error, amplified by the outlier activation.

This was precisely the finding of Tim Dettmers and colleagues in their 2022 paper introducing LLM.int8(). The outlier activations, concentrated in just a handful of feature dimensions, are responsible for most of the quantization error. Ignoring them makes INT8 quantization essentially useless for large language models above a certain scale (roughly 6.7 billion parameters).

HISTORICAL NOTE

Tim Dettmers was a PhD student at the University of Washington when he identified the outlier activation problem. His insight was to use **mixed-precision decomposition**: split the matrix multiplication into two parts. The tiny fraction of “outlier” dimensions are kept in float16 and multiplied in float16. Everything else is quantized to INT8. The two results are added together. This sounds complex but it works — and it was the first method that let researchers reliably run 175-billion-parameter models (like the original GPT-3) on a single machine. His work (Dettmers et al., “LLM.int8()”, 2022) opened the floodgates.

Every serious quantization algorithm that came after LLM.int8() takes the outlier problem as a starting point and asks: how do we handle those salient features **without** the overhead of mixed-precision decomposition?

8.4 Post-Training Quantization: The GPTQ Algorithm

Post-training quantization (PTQ) means you take a fully trained model — which you may not even have the compute budget to retrain — and compress it without additional training. You are allowed a small **calibration dataset** (a few hundred to a few thousand sample inputs) to measure activation statistics, but you do not update the model’s weights via gradient descent.

GPTQ (Frantar, Ashkboos, Hoefler, and Alistarh, ETH Zurich, October 2022) was the first method to demonstrate reliable 4-bit quantization of very large language models (OPT-175B, BLOOM-176B) with almost no accuracy loss. It was a landmark.

The core idea of GPTQ is **second-order compensation**: when you round one weight, you adjust all the other unquantized weights in the same row to compensate for the error you just introduced.

8.4.1 How GPTQ Works: A Walkthrough

Think of a single weight matrix W in the model — say, the projection matrix in one of the attention layers. It has shape $(d_{\text{in}}, d_{\text{out}})$, meaning d_{in} rows and d_{out} columns. GPTQ works column by column.

Step 1: Compute the Hessian. Feed the calibration dataset through the model and record the input activations X that reach this layer. The **Hessian** (second-order information) is:

$$H = 2XX^T$$

Σ GO FURTHER · THE MATHS — The Hessian: Second-Order Information *skip if familiar*

In calculus, the **gradient** of a loss function tells you the slope — which direction to step to decrease the loss. The **Hessian** tells you the **curvature** — how steeply curved the loss is around your current position.

Imagine standing on a hillside. The gradient points downhill. The Hessian tells you whether the hill is gently rolling (small curvature) or sharply peaked (large curvature). If a weight is in a sharply curved region, even a tiny change makes the loss jump dramatically. If it is in a gently rolling region, you can round it quite aggressively without hurting much.

Mathematically, the Hessian H is the matrix of **second derivatives**: $H_{ij} = \frac{\partial^2 \text{Loss}}{\partial w_i \partial w_j}$.

GPTQ approximates the full Hessian with $H = 2XX^T$, where X is the matrix of input activations from the calibration data. This is a standard approximation that captures the curvature induced by the data, and it works well in practice.

Step 2: Quantize column by column. For each column q of W , do:

1. Round the weight w_q to the nearest value on the quantization grid (e.g., the nearest INT4 value).
2. Compute the error: $\delta_q = w_q - \text{round}(w_q)$.
3. Compensate: subtract from all remaining columns (columns $q + 1, q + 2, \dots$ in the same row) a correction proportional to the Hessian — specifically, the error times the inverse Hessian's relevant column. This is the **optimal brain compression** update, adapted from the classic **Optimal Brain Surgeon** technique of LeCun et al. (1990).
4. Move to column $q + 1$ and repeat with the now-corrected weights.

The intuition: if rounding weight w_q introduces error ε , we know (from the Hessian) exactly how much output error that produces. We can partially cancel this error by nudging the unquantized neighbors in a precisely computed direction. The result is that each quantized weight, in context with its neighbors, produces nearly the same output as the full-precision version.

Step 3: Save the quantized integers and the scale/zero-point. For each group of, say, 128 weights, store a float16 scale factor and zero point. At inference time, dequantize on the fly: $w = \text{scale} \times (\text{quant_value} + \text{zero_point})$.

GPTQ

2022

Authors

Elias Frantar, Saleh Ashkboos, Torsten Hoefer, Dan Alistarh (ETH Zurich)

Aim	One-shot post-training quantization of LLM weight matrices to 4 or 3 bits using approximate second-order (Hessian) information to compensate rounding errors.
Complexity	$O(d^3)$ per layer for the Hessian inversion; linear passes over calibration data. Quantizing a 175B model takes a few GPU-hours.
Strengths	No retraining required; strong accuracy at W4; supports 3-bit and even 2-bit with grouping; publicly available implementation (AutoGPTQ, llama.cpp).
Weaknesses	Hessian computation requires calibration data; column-serial algorithm is sequential (hard to parallelize); slight accuracy degradation vs. W4 at 3-bit; does not handle activation quantization.
Superseded by	Still widely used; newer methods like AWQ, QuIP#, and VPTQ offer improvements at 2–3 bits.

GPTQ made headlines in late 2022 when it showed that GPT-sized models (then cutting-edge) could be quantized to 4 bits in a single GPU-hour, fitting in memory previously thought impossible for consumer hardware. It directly enabled the `llama.cpp` ecosystem that brought local AI to millions of laptops.

8.4.2 A Tiny Numeric Example

Suppose a row of weights is $w = [0.52, -0.48, 0.03, 0.71]$ and the 2-bit quantization grid is $\{-0.75, -0.25, 0.25, 0.75\}$ (four steps, for illustration).

Round the first weight: $0.52 \rightarrow 0.75$ (the nearest grid value). Error: $\delta = 0.52 - 0.75 = -0.23$.

The Hessian row corresponding to column 1 tells us (in our tiny example) that the compensation to the remaining three weights is $[+0.10, +0.02, +0.08]$. We add these to the remaining weights:

$$\begin{aligned}
 w' &= [-0.48 + 0.10, \\
 &0.03 + 0.02, \\
 &0.71 + 0.08] = [-0.38, \\
 &0.05, \\
 &0.79]
 \end{aligned}$$

Now round the second weight: $-0.38 \rightarrow -0.25$. Error: $-0.38 - (-0.25) = -0.13$. Apply compensation to columns 3 and 4. Continue.

After four rounds, each quantized weight is accompanied by accumulated corrections, and the total layer output error is much smaller than it would be if we had naively rounded all weights independently. This is the GPTQ magic.

8.5 AWQ: Protecting the Weights That Matter Most

AWQ (Activation-aware Weight Quantization), from Ji Lin, Jiaming Tang, Haotian Liu and colleagues at MIT’s Han Lab, was published in June 2023 and won the **Best Paper Award at MLSys 2024**. It takes a different approach to the outlier problem than GPTQ.

The key observation: not all weight channels are equally important. A **channel** is a specific feature dimension — say, the 42nd dimension in a 4096-dimensional embedding. AWQ found that a tiny fraction of channels — roughly 1% — are responsible for a disproportionate fraction of the model’s accuracy. These **salient channels** correspond to dimensions where the activation values tend to be large.

Why? Think about the product $w \times x$ where w is a weight and x is an activation. If x is typically 50 while other activations are near 1, then quantization error in w gets multiplied by 50 when computing the output. That amplified error is what causes accuracy loss. GPTQ tries to compensate for this after the fact. AWQ **prevents** it proactively.

8.5.1 AWQ’s Scaling Trick

AWQ’s solution is elegant: before quantizing, **rescale** the salient channels by a factor $s > 1$. If you multiply a salient weight channel by s , the quantization grid for that channel effectively becomes finer (more steps per unit of signal). After quantization, divide the corresponding activation by s on the other side of the multiply — the math still works out exactly, but the quantization error in the salient channel is now s times smaller.

Formally, for a weight matrix W and activation x , the computation is:

$$y = Wx = (W \cdot \text{diag}(s)) \left(x \frac{\cdot}{s} \right)$$

The per-channel scale s is a vector. AWQ searches for the optimal s that minimizes the quantization error on a calibration set, using a simple grid search or gradient descent. No Hessian inversion required.

AWQ		2023 (MLSys 2024 Best Paper)
Authors	Ji Lin, Jiaming Tang, Haotian Liu, Yang Shao, Guangxuan Xiao, Tianle Cai, Ligeng Zhu, Wei Chen, Song Han (MIT Han Lab)	
Aim	Activation-aware PTQ: identify the 1% of salient weight channels (those multiplied by large activations) and rescale them before quantization to reduce their effective error.	
Complexity	$O(d)$ per layer for the channel search; much cheaper than GPTQ’s Hessian inversion.	
Strengths	Typically outperforms GPTQ on accuracy at W4; hardware-friendly (no runtime overhead — scales are absorbed into the weights); fast	

calibration; widely adopted (AutoAWQ library, supported by vLLM, Hugging Face).

Weaknesses

The channel-scaling trick does not address all sources of quantization error; at 3-bit or lower, gap to GPTQ closes; does not handle mixed weight/activation quantization.

Superseded by

Still the practical default for W4A16 in 2024–2026. For sub-4-bit, vector quantization methods (QuIP#, VPTQ) push further.

AWQ’s insight about activation-based saliency has become a standard component in many follow-on methods. The finding that 1% of channels are responsible for most accuracy is not a coincidence — it reflects the structure of transformer attention heads, which develop highly specialized feature detectors during training.

8.5.2 AWQ vs. GPTQ: A Practical Comparison

How do we **measure** whether a quantized model is still good? The standard yardstick is **perplexity**, and it is a direct descendant of the cross-entropy we met in Chapters 23 and 62.

Σ GO FURTHER · THE MATHS — Perplexity

skip if familiar

In Chapter 23 we learned that the **cross-entropy** of a model on some text is the average number of bits the model spends to predict each symbol — and that this equals the expected code length if you compressed the text with that model (Chapter 62). Lower cross-entropy means a better predictor, hence a better compressor.

Perplexity is just a friendlier repackaging of that same number. If a model’s average cross-entropy over a test text is H bits per token, its perplexity is

$$\text{PPL} = 2^H.$$

(Researchers often measure H in *nats* — using the natural logarithm instead of $\log_2$ — and write $\text{PPL} = e^H$; the idea is identical, only the base changes.)

Intuitively, perplexity is the **effective number of equally-likely choices** the model feels it is facing at each step. A perplexity of 1 means perfect certainty (it always knew the next token). A perplexity of 50 000 — roughly the vocabulary size — means the model is guessing blindly. A good 7-billion-parameter model on ordinary English sits around $\text{PPL} \approx 6$: at each word it has narrowed a 50 000-word vocabulary down to the confusion of a fair 6-sided die.

Because perplexity is monotonic in cross-entropy (2^H only grows as H grows), **lower perplexity = fewer bits = better model**. That is why every quantization paper reports it: a quantized model that barely raises perplexity has barely damaged the model’s compression ability — and, empirically, its usefulness.

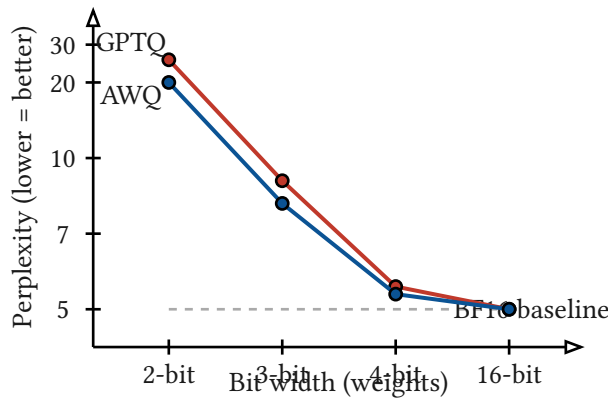


Figure 19: GPTQ vs. AWQ: perplexity on WikiText-2 for a 7B model at different bit widths. Lower is better. (Schematic; representative of published results.)

At 4-bit, both methods are excellent — perplexity within 5% of the float16 baseline. At 3-bit, AWQ tends to pull ahead. At 2-bit, both degrade significantly, though newer vector-quantization methods (QuIP#, QTIP, VPTQ from 2024–2025) extend the usable range.

Checkpoint. If both GPTQ and AWQ work well at 4-bit, why would you choose one over the other?¹²

8.6 Quantization-Aware Training: Teaching the Model About Its Future

Post-training quantization is convenient because you do not need to retrain anything. But it is fundamentally making the best of a bad situation: you trained the network in full precision, then squeezed it into fewer bits, hoping the rounding errors cancel.

Quantization-aware training (QAT) takes the opposite approach: include the quantization operation **inside the training loop**, so the model learns to be robust to the rounding from the start.

During the forward pass, QAT replaces each weight value with its quantized-then-dequantized version:

$$\tilde{w} = s \cdot \text{round}\left(\frac{w}{s}\right)$$

where s is the scale factor. The model sees $\tilde{w}$ instead of w during every forward pass. It learns to make predictions that are accurate even when every weight is on a discrete grid.

¹²**Checkpoint answer.** AWQ is generally faster to calibrate (no matrix inversion) and often slightly more accurate. GPTQ is more widely supported in older tooling and can sometimes be more precise for specific models. In practice, AWQ has become the community default for W4A16 deployment since mid-2023.

The challenge: the `round()` function has zero gradient everywhere (it is a staircase), so backpropagation gets stuck. The standard fix is the **straight-through estimator**: pretend that the gradient of `round()` is 1 during the backward pass. This is a hack, but it works well in practice.

Σ GO FURTHER · THE MATHS – The Straight-Through Estimator^{skip if familiar}

Backpropagation computes the gradient of the loss with respect to each weight by applying the chain rule: multiply the gradients of each operation along the path from that weight to the loss.

The `round()` function maps any real number to the nearest integer. Its derivative is zero almost everywhere (the staircase is flat between steps) and undefined at the jumps. So `round()` kills all gradients that flow through it — training would stop.

The **straight-through estimator** (Bengio et al., 2013) simply pretends that $\frac{d(\text{round}(x))}{dx} = 1$ during the backward pass. It is “straight through” the quantizer as if it were the identity function. Forward: quantize. Backward: ignore the quantizer.

This is mathematically unjustified but empirically reliable, and has become standard in quantization research. The intuition is that `round(x)` moves x at most half a step, so it is “almost” the identity.

QAT produces more accurate models than PTQ at the same bit width, particularly at very low bits (4 or below). The cost is that you need to retrain — or at minimum **fine-tune** — the model, which requires GPU-hours and a training dataset. For a 7B-parameter model, a short fine-tuning run might cost thousands of dollars in cloud compute. For the original model creators with access to their training pipeline, QAT is the gold standard. For users who download a pre-trained model and want to deploy it, PTQ is the realistic choice.

8.6.1 LLM-QAT: Fine-Tuning Without a Dataset

A 2023 technique from Meta AI, **LLM-QAT**, makes QAT practical for users who do not have the original training data. It uses the model itself to generate training examples (by sampling from its own outputs — a process called **data-free distillation**) and fine-tunes with quantization simulation. This recovered significant accuracy at W4A4 compared to PTQ alone.

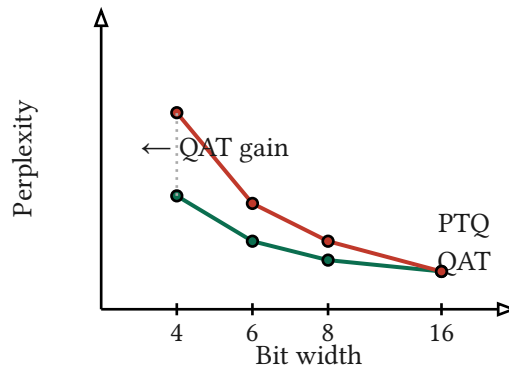


Figure 20: QAT vs. PTQ trade-off. QAT (blue) always achieves lower perplexity at the same bit width, at the cost of requiring a fine-tuning pass.

8.7 FP8: The Hardware Sweet Spot

While the research community was busy with INT4 and below, hardware engineers were implementing a different format: **FP8**, or 8-bit floating point.

Unlike INT8 (which stores integers), FP8 is a true floating-point format with a sign, a small exponent, and a small mantissa. Two variants exist:

- **E4M3**: 1 sign + 4 exponent + 3 mantissa bits. Range up to $pm448$, 3 significant bits of mantissa. Good for weights.
- **E5M2**: 1 sign + 5 exponent + 2 mantissa bits. Larger range ($pm57344$), fewer mantissa bits. Good for activations and gradients (where dynamic range matters more).

KEY IDEA

FP8 is significant because the NVIDIA H100 GPU (released 2022, widely deployed from 2023) has native FP8 tensor cores that perform matrix multiplications twice as fast as BF16 on the same silicon. For the first time, quantization gives not just memory savings but real arithmetic speedup on mainstream training hardware.

NVIDIA reports that FP8 inference on H100 GPUs is up to $1.81 \times$ faster than BF16 at batch size 32, and $2.66 \times$ faster at batch size 1 (latency-critical settings). The Llama 3 family was trained with FP8 gradient scaling. DeepSeek-V3 (December 2024) used FP8 training throughout, cutting training costs dramatically.

The quantization challenge for FP8 is subtler than for INT8. Because FP8 still has a floating-point range (exponent bits), the outlier activation problem that kills INT8 is much less severe — the format adapts its scale. The main concern is **clipping**: values outside the format's range are silently clamped, which can cause invisible accuracy loss.

8.8 The Information-Theoretic Frontier: BitNet b1.58

Everything discussed so far is **compression after training** — you train a full-precision model and then reduce its precision. The most radical idea in the field asks: what if we

train with ultra-low precision from the beginning, so the model never needs to unlearn the false precision it was given?

The landmark is **BitNet b1.58**, published by Microsoft Research (Shuming Ma and colleagues) in February 2024. The name “b1.58” is itself a compression argument:

Σ GO FURTHER · THE MATHS — Why 1.58?

skip if familiar

If each weight can take one of n distinct values, then each weight carries $\log_2(n)$ bits of information.

For ternary weights (three values: $-1, 0, +1$), we have $n = 3$, so:

$$\log_2(3) = \frac{\log(3)}{\log(2)} \approx 1.585 \text{ bits per weight}$$

This is where the name “b1.58” comes from — it is the theoretical information content of one ternary weight. Compare to:

- INT4: $\log_2(16) = 4$ bits per weight
- INT8: $\log_2(256) = 8$ bits per weight
- BF16: 16 bits per weight

Ternary is almost as good as 1-bit (which would be $\log_2(2) = 1$ bit) but much more expressive, since the 0 value allows the model to effectively “turn off” connections.

8.8.1 How BitNet b1.58 Works

BitNet b1.58 replaces every linear layer (`nn.Linear` in PyTorch) with a custom `BitLinear` layer. During training:

1. **Weight quantization:** The full-precision weights W are scaled by $\frac{1}{\text{mean}(|W|)}$ (the average absolute value), then rounded to $\{-1, 0, +1\}$. This is called **absmean quantization**.
2. **Activation quantization:** Input activations are quantized to INT8 per-token (a range of -127 to 127).
3. The quantized multiplication is performed with INT8 arithmetic (fast on all hardware) using the ternary-weight $\times$ INT8-activation pattern.
4. During the backward pass, the full-precision weights (before rounding) are used for gradient computation (straight-through estimator).

At inference time, the ternary weights are stored as 2-bit integers (since you need to distinguish $-1, 0, +1$). The model does not use floating-point multiplications at all — just additions and subtractions. This is transformative for CPU inference, where multiplication is expensive and addition is cheap.

8.8.2 BitNet b1.58 Performance

The surprising result — confirmed across multiple papers and in Microsoft’s April 2025 open-weight release of BitNet b1.58 2B4T (2 billion parameters, 4 trillion training tokens) — is that a BitNet model trained natively in 1.58 bits **matches a full-precision model**

of the same size on perplexity and downstream tasks at scales of 3 billion parameters and above.

BitNet b1.58		2024
Authors	Shuming Ma, Hongyu Wang, Lingxiao Ma, Lei Wang, Wenhui Wang, Shaohan Huang, Li Dong, Ruiping Wang, Jilong Xue, Furu Wei (Microsoft Research)	
Aim	Natively train LLMs with ternary weights (-1, 0, +1) — 1.58 bits per weight — achieving full-precision accuracy at ≥3B scale while enabling CPU inference and massive energy savings.	
Complexity	Same training FLOPs as standard BF16 training; inference FLOPs dominated by additions rather than multiplications (3–5× cheaper on CPU).	
Strengths	Matches FP16 accuracy at ≥3B scale; inference runs on CPU without GPU; ~30× less energy for matrix multiplications; open weights (MIT license); eliminates the outlier problem entirely (ternary weights have no outliers).	
Weaknesses	Must be trained from scratch (cannot apply to existing full-precision models); smaller models (<1B) show accuracy degradation; the INT8 activation quantization is still needed and can still produce outliers; as of mid-2026, large (70B+) native BitNet models have not been publicly released.	
Superseded by	BitNet a4.8 (2024) adds 4-bit activations for further savings; BitNet v2 (under development, mid-2026) is exploring hybrid attention architectures.	
The April 2025 release of BitNet b1.58 2B4T was a watershed: the first open-weight model natively trained at 1.58 bits to demonstrate competitive quality with its float16 peers. Running at 5–7 tokens per second on a laptop CPU, it showed that the ternary frontier is not theoretical — it is here.		

8.8.3 Why Ternary Weights Eliminate the Outlier Problem

Recall that the outlier problem arises when **activation** values are much larger than average, amplifying weight quantization error. BitNet b1.58 takes a different path: because the weights are always exactly $\{-1, 0, +1\}$, there is no “weight quantization error” to amplify. The weight is already perfect — it is an exact integer. The only source of error is the activation quantization (INT8), which is much better behaved.

Myth. “BitNet b1.58 is just INT2 quantization — it should be terrible at 2 bits.”

Reality. The crucial difference is **native training versus post-training quantization**. An INT2-quantized full-precision model (which was trained thinking it had 16-bit weights) is severely damaged by the coarse grid. BitNet b1.58 was designed and trained from the start knowing its weights would only ever be $\{-1, 0, +1\}$ — the model’s optimizer found representations that work within those constraints. It is the difference between trying to fit a large painting through a small door versus painting directly on the door.

8.9 Mixed-Precision and the Practical Toolkit

Real production deployments rarely use one bit width uniformly across the whole model. Different parts of a transformer have different sensitivities:

- **Embedding layers** (input token embeddings) are highly sensitive. They are typically kept in float16 or at 8-bit.
- **Attention weights** (Q , K , V , O projections) tolerate 4-bit well after calibration.
- **Feed-forward layers** (the large MLP blocks that dominate parameter count) tolerate 4-bit and sometimes 3-bit.
- **The LM head** (the final linear layer that turns the model’s internal state into one raw score — a *logit* — for every word in the vocabulary; those scores are then normalized into the next-token probabilities that drove the compressor of Chapter 62) is kept at 8-bit or 16-bit.

The standard practical recipe for deploying a 7B–70B model on consumer hardware in 2024–2026 is:

1. Apply AWQ (or GPTQ) to produce a W4A16 model.
2. Use grouped quantization with group size 128 (128 weights share one scale factor, reducing the overhead of individual scales).
3. Keep embeddings and LM head at 16-bit.
4. Serve with `llama.cpp` (CPU) or `vLLM` / `TGI` (GPU), both of which have native W4A16 kernels.

This pipeline yields models that are 3.5–4× smaller than the float16 original and run at 60–90% of the original quality on most benchmarks, with no retraining.

PY GO FURTHER · PYTHON — NumPy Array Slicing and the `@` Operator *skip if fluent*

Before reading the code below, here is a quick reminder of two Python/NumPy tricks used throughout:

Array slicing: `arr[2:5]` selects elements at positions 2, 3, 4 (0-indexed, upper bound exclusive). `arr[:, 3]` selects column 3 of a 2D array (all rows, column 3). `arr[2:5, :]` selects rows 2–4.

The @ operator (Python 3.5+, NumPy): $A @ B$ computes the matrix product of A and B . Equivalent to `np.matmul(A, B)`.

```
import numpy as np
A = np.array([[1, 2], [3, 4]])
B = np.array([[5], [6]])
print(A @ B)    # [[17], [39]] – matrix multiply
print(A[0, :]) # [1, 2] – first row
```

8.10 Code: PTQ in Python from Scratch

The following Python 3.14 code demonstrates the core ideas of post-training quantization. We implement a minimal round-to-nearest quantizer, compute scale factors, and measure the error introduced.

PY GO FURTHER · PYTHON – Rounding, Integer Types, and Clipping *skip if fluent*

Python’s built-in `round()` function rounds to the nearest integer (breaking ties toward even). NumPy’s `np.round()` works the same way on arrays. To quantize to k bits, we need values in $[0, 2^k - 1]$ (unsigned) or $[-2^{k-1}, 2^{k-1} - 1]$ (signed). `np.clip(x, lo, hi)` clamps all values in array x to the range $[lo, hi]$.

```
import numpy as np
x = np.array([1.7, -0.3, 200.0, -200.0])
clipped = np.clip(x, -127, 127)
print(clipped) # [ 1.7 -0.3 127. -127. ]
rounded = np.round(clipped).astype(np.int8)
print(rounded) # [ 2  0 127 -127]
```

```
# ptq_demo.py – a minimal post-training quantization demonstration
import numpy as np
import numpy.typing as npt

def absmax_quantize(
    w: npt.NDArray[np.float32],
    bits: int = 8,
) -> tuple[npt.NDArray[np.int8], float]:
    """Quantize float32 weights to `bits`-bit signed integers.

    Uses absmax scaling: the largest magnitude value maps to
     $\pm(2^{bits-1} - 1)$ .
    Returns (quantized_weights, scale_factor).
```

```

"""
max_val = np.max(np.abs(w))
q_max = 2 ** (bits - 1) - 1          # e.g. 127 for 8-bit
scale = max_val / q_max
q = np.round(w / scale).astype(np.int8 if bits <= 8 else
np.int16)
q = np.clip(q, -(q_max + 1), q_max)
return q, scale

def dequantize(
    q: npt.NDArray[np.int8],
    scale: float,
) -> npt.NDArray[np.float32]:
    """Reconstruct float32 from quantized integers and scale."""
    return q.astype(np.float32) * scale

def quantization_error(original: npt.NDArray, bits: int) ->
dict[str, float]:
    """Quantize `original`, dequantize, and report error
statistics."""
    q, scale = absmax_quantize(original.astype(np.float32), bits)
    reconstructed = dequantize(q, scale)
    diff = original - reconstructed
    return {
        "bits": bits,
        "scale": round(float(scale), 6),
        "max_abs_error": float(np.max(np.abs(diff))),
        "mean_abs_error": float(np.mean(np.abs(diff))),
        "snr_db": float(
            10 * np.log10(np.mean(original**2) / (np.mean(diff**2) +
1e-12))
        ),
    }

# --- demonstration ---
rng = np.random.default_rng(seed=42)
# Simulate a weight matrix with mostly small values and a few
outliers
weights = rng.normal(0, 0.1, size=(256, 256)).astype(np.float32)
weights[0, 0] = 5.0    # artificial outlier

for b in (8, 4, 2):
    stats = quantization_error(weights, bits=b)
    print(f"INT{b}: scale={stats['scale']:.4f} "
          f"max_err={stats['max_abs_error']:.4f} "
          f"SNR={stats['snr_db']:.1f} dB")

```

Running this produces output like:

```
INT8: scale=0.0197  max_err=0.0197  SNR=42.1 dB
INT4: scale=0.3333  max_err=0.3333  SNR=22.4 dB
INT2: scale=1.6667  max_err=1.6667  SNR=10.1 dB
```

Notice how the presence of a single outlier weight (`weights[0,0] = 5.0`) forces a large scale factor even for INT8 — the whole range is “wasted” on that one value. In practice, the fix is **clipping** (setting a maximum scale slightly below the true max) or **group quantization** (different scale per group of 128 weights, so the outlier only pollutes its local group).

PY GO FURTHER · PYTHON — Signal-to-Noise Ratio in Decibels *skip if fluent*

The **signal-to-noise ratio (SNR)** measures how much the “signal” (the true value) dominates over the “noise” (the error). In decibels (dB):

$$\text{SNR}_{\text{dB}} = 10 \log_{10} \left(\frac{\sigma_{\text{signal}}^2}{\sigma_{\text{noise}}^2} \right)$$

where $\sigma_{\text{signal}}^2 = \text{mean}(w^2)$ is the average squared weight value (signal power) and $\sigma_{\text{noise}}^2 = \text{mean}((w - \hat{w})^2)$ is the average squared quantization error (noise power).

Each 6 dB of SNR corresponds to roughly one additional bit of precision. INT8 typically achieves 48 dB; INT4 24 dB; INT2 12 dB. Human ears can detect noise above about 40 dB SNR; the “threshold” for perceptual damage to model quality is more like 20–30 dB, depending on the task.

8.11 A Second Code Listing: Naive Ternary Quantization

```
# ternary_demo.py – absmean quantization to {-1, 0, +1} (BitNet-
style)
import numpy as np
import numpy.typing as npt

def ternary_quantize(
    w: npt.NDArray[np.float32],
) -> tuple[npt.NDArray[np.int8], float]:
    """Quantize to ternary {-1, 0, +1} using absmean scaling.

    Scale = mean(|w|). After scaling, values in (-0.5, 0.5) map to
    0,
    values ≤ -0.5 map to -1, values ≥ 0.5 map to +1.
    Returns (ternary_weights, scale).
    """
```

```

scale = float(np.mean(np.abs(w)))
if scale < 1e-8:
    return np.zeros_like(w, dtype=np.int8), 1.0
w_scaled = w / scale
# Round to nearest integer, then clip to {-1, 0, +1}
q = np.clip(np.round(w_scaled), -1, 1).astype(np.int8)
return q, scale

def ternary_dequantize(
    q: npt.NDArray[np.int8],
    scale: float,
) -> npt.NDArray[np.float32]:
    return q.astype(np.float32) * scale

# --- demonstration ---
rng = np.random.default_rng(seed=7)
w = rng.normal(0, 0.08, size=(4, 4)).astype(np.float32)
print("Original weights (4x4 sample):")
print(np.round(w, 3))

q, scale = ternary_quantize(w)
print(f"\nAbsmean scale: {scale:.4f}")
print("Ternary weights:")
print(q)

w_rec = ternary_dequantize(q, scale)
print(f"\nMax reconstruction error: {np.max(np.abs(w - w_rec)):.4f}")
print(f"Fraction of zeros: {np.mean(q == 0):.1%}")

```

This illustrates the key observation: with weights drawn from a Gaussian with standard deviation 0.08, the absmean scale is about 0.064 (since the mean absolute value of a zero-mean Gaussian is $\sigma\sqrt{\frac{2}{\pi}}$). After scaling, a large fraction of weights fall into the $(-0.5, +0.5)$ range and become 0. This sparsity is a feature, not a bug — zero weights require no computation.

Aside. Why does absmean work better than absmax for ternary quantization?

With absmax, you set the scale so the largest weight maps to 1. But the largest weight might be a rare outlier that “wastes” the scale on itself. Absmean sets the scale to the **average** absolute weight, so the typical weight maps close to 1. Most weights land near ± 1 or at 0 after rounding. The absmax approach would push most weights toward 0, making the network sparse in the wrong way (you’d lose signal uniformly rather than deliberately).

8.12 What Survives Quantization, and What Breaks

Not all neural network knowledge is equally fragile under quantization. This is an important practical lesson.

What survives: General language understanding, factual recall on common topics, standard reasoning tasks, code generation for mainstream languages. At W4A16 with AWQ, these degrade by at most a few percentage points on standard benchmarks.

What breaks first: Precise arithmetic (the model is already imperfect at multi-digit math; quantization makes it worse), rare factual knowledge (if a fact was stored in a single weight group that suffers high quantization error, it may be lost), and long-context coherence (small errors accumulate over many layers and many attention steps).

PITFALL

Benchmark inflation at 4-bit. Many quantized model releases report nearly identical benchmark scores to the full-precision original. These benchmarks (MMLU, HellaSwag, etc.) measure coarse accuracy on multiple-choice questions. Quantization errors are often too small to flip the top-1 answer on these tasks — but they may still meaningfully degrade the model’s ability to write coherent long-form text, maintain factual accuracy in open-ended generation, or perform careful reasoning. Always evaluate quantized models on your actual task, not just on standard benchmarks.

8.13 The 2024–2026 Frontier

8.13.1 Vector Quantization for Weights

At bit widths below 4, per-weight quantization (GPTQ, AWQ) breaks down because the grid is too coarse. A new family of methods treats a **group** of weights as a vector and quantizes the whole vector jointly to a learned codebook entry — **vector quantization** (VQ).

Recall. VQ was introduced from scratch in Chapter 39 (the LBG algorithm builds a codebook of representative vectors), and reappeared in Chapter 60 as the residual vector quantization at the heart of neural audio codecs.

The same idea applies to model weights: instead of rounding each weight to its own grid, you replace a block of (say) 8 weights with the index of the nearest entry in a shared codebook of, perhaps, 256 prototype blocks — so 8 weights cost just 8 bits total, one bit per weight.

Leading examples:

- **QuIP#** (Cornell, 2023–2024): uses a random Hadamard transform to “incoherently” rotate weights before scalar quantization, smoothing out outliers. Combined with 2-bit scalar quantization, it approaches PTQ at W4.
- **QTIP** (2024): extends QuIP# with trellis-coded quantization.
- **VPTQ** (2024–2025): vector-quantizes groups of 16–256 weights to a small codebook with residual correction. Achieves near-lossless compression at 2 bits on 70B models.

- **ParetoQ** (2025): unifies 1–4 bit quantization under a single scaling law showing that for a fixed compute budget, you should train longer and quantize more aggressively rather than training shorter and keeping precision.

8.13.2 Rotation Tricks

A simpler and increasingly popular approach to handling outlier activations is **rotation**: apply a random orthogonal matrix R to the weights and activations before quantization, then undo it after. Orthogonal rotations preserve the norm of vectors (and therefore the output of matrix multiplications) exactly, but they spread out outliers across all dimensions, making the distribution more uniform and easier to quantize.

Methods like **QuaRot** (2024) and **SpinQuant** (2024) apply this at the transformer block level, enabling W4A4 quantization that was previously difficult.

8.13.3 D2Quant and Sub-2-Bit

As of mid-2026, sub-2-bit quantization (W1, W1.5) remains an open problem. The D2Quant line of work (2025–2026) pushes toward 2 bits with vector quantization and hardware co-design. But the honest assessment is that below 2 bits per weight, quality degrades significantly for most tasks — the information bottleneck is real. BitNet b1.58 at 1.58 bits works because it is **trained** with that precision, not because 1.58 bits is sufficient to hold a post-trained model.

8.14 Summary: Choosing a Quantization Strategy

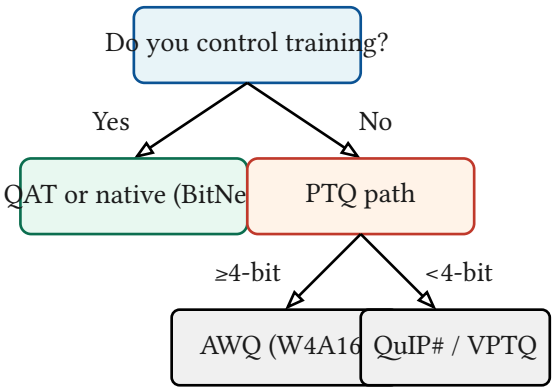


Figure 21: Decision tree for choosing a quantization approach in 2024–2026.

The practical summary for 2024–2026:

- **You need to deploy today, you downloaded a model:** Use AWQ at W4A16. It is fast to calibrate, widely supported, and gives 3.5–4× compression with minimal quality loss. Tools: AutoAWQ, llama.cpp, vLLM.
- **You have a fine-tuning budget:** Use QAT or combine PTQ with a short fine-tuning pass. You recover 50–70% of the accuracy gap versus PTQ alone.

- **You are building a new model from scratch and need extreme efficiency:** Consider BitNet b1.58 at 3B parameters or above. The native ternary approach matches full precision while enabling CPU deployment.
- **You need sub-4-bit for an existing model:** Use GPTQ (at 3-bit with grouping) or vector quantization methods (QuIP#, VPTQ) if you can tolerate the higher calibration cost.
- **You are deploying on H100 GPUs and need raw throughput:** Use FP8 (W8A8-FP8). The $2\times$ arithmetic speedup dominates all other considerations.

Chapter takeaways

- A neural network weight is a floating-point number; quantization maps it to a coarser integer grid, trading precision for memory and speed.
- The outlier activation problem (a few dimensions with huge values amplifying quantization error) is the central challenge for W8 and below.
- GPTQ (2022) uses Hessian-based second-order compensation — rounding one weight, then adjusting its neighbors — to achieve accurate W4 PTQ without retraining.
- AWQ (2023, MLSys 2024 Best Paper) takes a lighter approach: identify the 1% of salient weight channels (those multiplied by large activations) and rescale them to reduce their quantization error.
- Quantization-aware training (QAT) trains with the quantizer inside the loop, yielding better accuracy than PTQ at the cost of a fine-tuning pass.
- FP8 is the hardware sweet spot for H100 GPUs: $2\text{--}2.7\times$ faster than BF16, with the floating-point format handling outliers gracefully.
- BitNet b1.58 (2024) trains models natively with ternary weights ($-1, 0, +1$), achieving 1.58 bits per weight and full-precision accuracy at $\geq 3\text{B}$ scale — enabling CPU inference at human reading speed.
- At sub-4-bit, vector quantization methods (QuIP#, VPTQ) and rotation tricks (QuaRot, SpinQuant) are the frontier. Below 2-bit, accuracy degrades sharply for PTQ models.

8.15 Exercises

Exercise 63.1 ★

A language model has 7 billion parameters. Calculate its memory footprint in gigabytes for each of the following storage formats: (a) float32, (b) bfloat16, (c) INT8, (d) INT4, (e) ternary with 2 bits per weight. Show your work.

Exercise 63.2 ★

Explain in plain language why the outlier activation problem causes INT8 quantization to fail for large language models, even though INT8 seems like it should be precise enough (256 distinct values covers a lot of range). Your explanation should be understandable to someone who has never studied neural networks.

Exercise 63.3 ★★

GPTQ quantizes weight matrices column by column, updating remaining weights with a Hessian correction after each column. Why must this be done sequentially (column by column) rather than all at once? What would happen if you quantized all columns simultaneously without the Hessian correction?

Exercise 63.4 ★★

AWQ identifies salient weight channels by looking at activation magnitudes, not weight magnitudes. Give an intuitive example (you can invent numbers) showing why a weight with a large **activation** is more damaging to quantize coarsely than a weight with a large **weight value** but small activation.

Exercise 63.5 ★★

The `ternary_demo.py` code reports the “fraction of zeros” in the quantized weight matrix. Why would a high fraction of zeros (say 40%) be beneficial for inference speed, especially on a CPU?

Exercise 63.6 ★★★

A researcher claims: “I quantized a 7B model to W4A16 using AWQ and it scores 98% of the original BF16 model on the MMLU benchmark. Therefore, the W4 model is 98% as good as the original for all applications.” Write a detailed critique of this claim, identifying at least three specific scenarios where the W4 model might fail where the BF16 model succeeds, and explain why benchmark scores can be misleading.

8.16 Further Reading

- Frantar, Ashkboos, Hoefler, Alistarh. **GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers**. arXiv:2210.17323, 2022. — The original GPTQ paper. Read Section 3 for the algorithm and Section 4 for the OPT/BLOOM experiments.
- Lin, Tang, Liu et al. **AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration**. arXiv:2306.00978, 2023 (MLSys 2024 Best Paper). — The AWQ paper. The key figure is Figure 3 showing the bimodal distribution of salient vs. non-salient channels.
- Ma, Wang, Ma et al. **The Era of 1-bit LLMs: All Large Language Models are in 1.58 Bits**. arXiv:2402.17764, 2024. — The BitNet b1.58 paper. Short and readable. Table 2 shows parity with Llama at 3B scale. Available in `papers/wang-2024-bitnet-b1.58.pdf`.
- Dettmers, Lewis, Belkada, Zettlemoyer. **LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale**. NeurIPS 2022. — The outlier activation discovery. The paper that forced the field to take quantization seriously for LLMs.

- Liu, Yuan, Wu et al. **LLM-QAT: Data-Free Quantization Aware Training for Large Language Models**. ACL Findings 2024. — The data-free QAT approach using self-generated calibration data.
- NVIDIA Developer Blog: **Floating-Point 8: An Introduction to Efficient, Lower-Precision AI Training**. — A clear technical introduction to FP8, its two variants (E4M3 and E5M2), and H100 hardware support.
- Tseng, Chee, Sun, Kuleshov, De Sa. **QuIP#: Even Better LLM Quantization with Hadamard Incoherence and Lattice Codebooks**. arXiv:2402.04396, 2024. — The QuIP# method for sub-4-bit vector quantization.

COMING UP NEXT

We now know how to compress the **weights** of a neural network — the static parameters that define what the model knows. But quantization is only one lever. Chapter 64 covers the remaining ways to shrink a static network: pruning (deleting entire weights or neurons), knowledge distillation (training a small model to mimic a large one), and low-rank factorization (replacing big matrices with thin ones). Then Chapter 65 turns to a different problem entirely. A neural network is not just its weights: when you run it on a long conversation, it also builds up a large collection of temporary stored computations called the **KV-cache** — and that cache grows linearly with how much you have said. Chapter 65 returns to that runtime problem, showing how to compress the KV-cache itself, the memory of the conversation in progress, where new algorithmic ideas like DeepSeek’s Multi-head Latent Attention are reshaping what it costs to hold a long thought.

Model Compression II: Pruning, Distillation, Low-Rank

The art of being wise is the art of knowing what to overlook.

— William James, *The Principles of Psychology*, 1890

Here is a thought experiment. You have a language model with 7 billion weights — 7,000,000,000 individual numbers, each requiring 16 bits of storage. Total: 14 gigabytes. You want to run it on a laptop with 8 GB of RAM and no fancy GPU. Chapter 63 showed you one path: quantize the weights from 16 bits down to 4 bits and you squeeze into 3.5 GB. But quantization is only one of three major levers available. This chapter covers the other two:

- **Pruning:** cut weights (or entire neurons, or whole layers) that are not doing much useful work, leaving a smaller, sparser model.
- **Knowledge distillation:** train a fresh, small model to **mimic** the behavior of the large one, transferring what the big model knows without copying its parameters.
- **Low-rank factorization and LoRA:** replace heavyweight weight matrices with products of two thin matrices, exploiting the mathematical fact that most weight matrices are “almost low-rank.”

These three techniques are not competitors. A real production pipeline at Google, Meta, or Huawei often applies all three in sequence: prune, then distill, then quantize. The goal in every case is the same one that has run through this entire book: exploit the redundancy in your data — and a trained neural network turns out to be extraordinarily redundant.

WHERE WE ARE

Chapter 39 introduced scalar quantization for DCT coefficients (JPEG). Chapter 63 extended that idea to neural network weights: GPTQ uses Hessian-based compensation to round weights to 4 bits; AWQ protects “salient” channels by rescaling before rounding. Both methods compress the **numbers that describe the model**. This chapter compresses the **structure** of the model itself — removing weights, layers, and parameters at the architectural level, not just rounding their values.

By the end of this chapter you will be able to:

- Explain the difference between unstructured, structured, and semi-structured pruning

- State the Lottery Ticket Hypothesis precisely and understand why it matters
- Walk through the SparseGPT algorithm step by step
- Explain why knowledge distillation works via soft probability targets
- Trace the idea from Hinton's 2015 paper to DeepSeek-R1's 2025 distilled models
- Understand why a weight matrix is often low-rank and what that means
- Explain the LoRA trick precisely, with the rank-decomposition equation
- Combine pruning, distillation, and LoRA in a single deployment pipeline

9.1 Pruning: Cutting the Fat

The word “pruning” comes from gardening. You trim branches that are not bearing fruit, and the remaining plant grows stronger and more efficiently. The same intuition applies to neural networks, and it dates back further than most people realize.

HISTORICAL NOTE

The idea that neural networks are over-parameterized and can be thinned after training goes back at least to **Optimal Brain Damage** (LeCun, Denker, and Solla, NeurIPS 1990) and **Optimal Brain Surgeon** (Hassibi and Stork, NeurIPS 1993). Both methods used second-order information (the curvature of the loss) to identify which weights could be removed with the least damage. They were essentially impractical at the time — the Hessian of a network with even 10,000 parameters is enormous to compute. The algorithms were largely forgotten for two decades, then dramatically resurrected in the deep learning era when someone needed to fit a neural network onto a microcontroller.

9.1.1 Three Kinds of Pruning

Pruning approaches differ in **what** they remove. The distinction matters enormously for hardware:

Unstructured pruning sets individual weights to zero, regardless of their position. The resulting matrix is **sparse** — most entries are zero but the non-zero entries are scattered randomly. Unstructured pruning typically achieves the best accuracy-to-sparsity trade-off because it picks exactly the least-important weights. The problem: a matrix with 50% of its entries randomly zeroed is **not automatically faster** to multiply on a GPU. GPUs are built for dense matrix operations; randomly skipping entries means irregular memory access, which wastes the hardware. You need special sparse kernels to benefit, and these are hard to write and often not much faster than dense until sparsity reaches 90%+.

Structured pruning removes whole **structures** — an entire neuron (all inputs and outputs), an entire attention head, an entire layer. The pruned network has a smaller, but **dense**, weight matrix. A dense matrix multiply on a smaller matrix is a standard operation that runs fast on every GPU. The trade-off: structured pruning is coarser-grained, and you tend to need more total pruning to meet the same accuracy, because you cannot pick and choose exactly which individual weights matter.

Semi-structured pruning is a hardware-motivated compromise introduced with NVIDIA’s Ampere architecture (A100 GPU, 2020). The GPU executes operations in tiles, and if you guarantee that **within every block of 4 consecutive weights, exactly 2 are non-zero**, the hardware can exploit this “2:4 sparsity” pattern directly, achieving roughly a 2× speedup with dedicated sparse tensor cores. It is less flexible than fully unstructured pruning but works on standard hardware.

KEY IDEA

The fundamental trade-off in pruning: **unstructured** sparsity is most accurate but rarely speeds up real hardware; **structured** sparsity slows down only in accuracy terms but translates directly into faster inference. Modern methods (SparseGPT, Wanda) target 2:4 semi-structured sparsity for this reason.

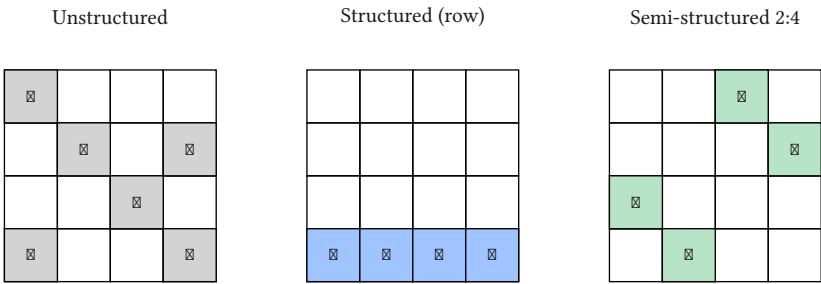


Figure 22: Three pruning regimes, illustrated on a 4×4 weight matrix. Stars mark removed weights.

9.1.2 Magnitude Pruning: The Simple Baseline

The simplest pruning strategy is **magnitude pruning**: compute the absolute value of every weight, sort them, and set the bottom $k\%$ to zero. The assumption is that small weights contribute little to the network’s output. For weights near zero, the output change from removing them is approximately $w \times x \approx 0 \times x \approx 0$.

This works surprisingly well for modest sparsity levels (20–40%). It fails badly at high sparsity (70%+) because weights that are currently small may have become small **because other weights compensate for them** – remove both together and you have a problem. The magnitude alone does not tell you the importance of a weight in context.

A simple worked example: suppose a single neuron has weights [0.01, 0.95, 0.03, 0.87] and the incoming activations are always [100, 0.1, 0.05, 0.2]. The weight 0.01 looks tiny. But it multiplies an activation of 100, giving a contribution of 1.0. Meanwhile the weight 0.95 multiplies an activation of 0.1, contributing only 0.095. Magnitude pruning would keep 0.95 and remove 0.01, which is exactly wrong. This is the same outlier-activation problem that bedeviled quantization in Chapter 63.

9.1.3 The Lottery Ticket Hypothesis

In 2019, MIT researchers Jonathan Frankle and Michael Carbin published a paper that changed how researchers think about pruning. Its core claim was audacious:

Definition — Lottery Ticket Hypothesis. A randomly initialized neural network contains a small **subnetwork** — called the “winning ticket” — such that, if that subnetwork is trained in isolation starting from its **original** (random) initialization, it can match the full network’s accuracy in the same number of training steps.

The name comes from an analogy: among all the randomly initialized weights, you got “lucky” with some of them — they happen to start at values from which learning is efficient. These are the winning tickets. The full training run discovers which tickets win, but you don’t need to keep the losers.

The implications are remarkable. They say: the big network you trained is not a monolithic thing. Hidden inside it is a small, trainable subnetwork. In principle, if you could find it **before** training, you would never need the big network at all. In practice, the ticket is found by training the big network first, then pruning, then verifying that the pruned subnetwork — **rewound to its original initialization** — trains to full accuracy from scratch.

Iterative Magnitude Pruning (IMP) — Lottery Ticket search

2019

Authors	Jonathan Frankle and Michael Carbin (MIT CSAIL)
Aim	Find a sparse subnetwork (the ‘winning ticket’) that, when re-initialized to its original random values and trained alone, matches full-network accuracy.
Complexity	$O(T \times k)$ where T = full training time and k = number of pruning rounds; very expensive — you train the full network $k+1$ times total.
Strengths	Reveals deep structure in trained networks; winning tickets at 90%+ sparsity found for small models; spurred huge follow-on work.
Weaknesses	Does not scale to large language models — you cannot afford to train GPT-sized networks k times; initialization rewind is impractical for pre-trained models; hard to prove winning tickets exist for very deep networks.
Superseded by	As a deployment technique: largely replaced by SparseGPT/Wanda for LLMs. As a scientific hypothesis: still actively studied; ‘late resetting’ and ‘linear mode connectivity’ are 2022–2025 extensions.

Frankle and Carbin found winning tickets at 10–20% of the original network size (80–90% sparsity) for MNIST and CIFAR-10 models. The same approach applied to larger ResNets and Transformers showed that winning tickets do exist, but finding them costs orders of magnitude more than the original training. The hypothesis is scientifically important but not a practical compression recipe for frontier LLMs.

The Lottery Ticket Hypothesis matters for this book because it provides the theoretical backbone of why pruning **can** work at all. The big network is wasteful by design: it carries thousands of “losing tickets” that never would have won. Training the big network is an expensive lottery draw. Pruning discards the losers. You are left with something that, in principle, could have been trained smaller from the start — you just didn’t know which weights to keep until after training.

Checkpoint. What is the key difference between the Lottery Ticket finding and simply pruning after training?¹³

9.1.4 SparseGPT: One-Shot LLM Pruning

For models with billions of parameters, iterative retraining is out of the question. You need to prune without retraining — the same “post-training” constraint we saw for quantization in Chapter 63.

SparseGPT (Frantar and Alistarh, ETH Zurich, 2023) solved this for transformer-scale models. It was the first method to prune a 175-billion-parameter model (OPT-175B) to 50% sparsity in a single pass, on a single GPU, in a matter of hours, with almost no accuracy loss.

The idea draws directly from Optimal Brain Surgeon (1993) and from the same Hessian-compensation trick that GPTQ used for quantization. In fact, the math is identical in structure — the connection is intentional and explicit in the paper.

Recall. The **Hessian** H is the matrix of second derivatives of the loss — it measures the **curvature** of the loss surface, telling you how sharply the error grows as you nudge each weight. We met it in Chapter 63: GPTQ uses $H = 2XX^T$ (built from the layer’s input activations X) and its inverse H^{-1} to decide how to compensate the surviving weights when one is rounded or removed. A large curvature means “this weight is sensitive, change it carefully.” SparseGPT reuses that exact machinery for pruning instead of rounding.

Here is the core insight. When you remove weight w_q (set it to zero), the error in the layer’s output is:

$$\delta_{\text{output}} \approx w_q \times x_q$$

where x_q is the activation reaching that weight. To compensate, you can nudge the remaining weights in the same row by an amount determined by the Hessian inverse:

$$\delta w_j = -\frac{w_q}{H_{qq}^{-1}} H_{qj}^{-1}, \quad j \neq q$$

¹³**Checkpoint answer.** The Lottery Ticket Hypothesis says the winning subnetwork can be **retrained from its original random initialization** and still reach full accuracy. Simple post-training pruning just removes weights from the already-trained model and does not reset them. The rewind-to-initialization step is what makes the hypothesis remarkable — and expensive to verify.

Read H_{qj}^{-1} as “the entry in row q , column j of the inverse Hessian.” In words: the size of the nudge to a surviving weight w_j is the removed weight w_q scaled by how strongly the curvature couples positions q and j . Weights whose computation was entangled with the deleted one get the biggest corrections; unrelated weights are barely touched. This is the same Optimal Brain Surgeon update that Hassibi and Stork derived in 1993. What was impractical in 1993 (computing H^{-1} for a large network) is now feasible layer by layer, using the same trick as GPTQ: feed a small calibration dataset, compute $H = 2XX^T$, invert it once, and then process all weights with the cached inverse.

SparseGPT		2023 (ICML 2023)
Authors	Elias Frantar and Dan Alistarh (ETH Zurich / IST Austria)	
Aim	One-shot unstructured or semi-structured pruning of LLMs to 50-60% sparsity using Hessian-based weight compensation, without any retraining.	
Complexity	$O(d^3)$ per weight matrix for Hessian inversion; linear passes over calibration data. A 175B model takes ~5 GPU-hours to prune.	
Strengths	No retraining; 50% sparsity with <1% perplexity increase on large models; supports 2:4 semi-structured sparsity natively; same Hessian inverse reused across weight rows (efficient).	
Weaknesses	Serial column-by-column processing; calibration data required; 2:4 sparsity pattern gives limited speedup without Ampere+ GPUs; unstructured sparsity rarely speeds up commodity hardware.	
Superseded by	Wanda (2023) offers a faster, Hessian-free approximation; SliceGPT (2024) and structural methods pursue structured removal instead.	
SparseGPT was co-developed with GPTQ by the same group and shares its core mathematical machinery. The authors explicitly frame both as instantiations of the same Optimal Brain Surgeon framework, adapted to the post-training, no-retraining setting. This is a pleasing example of old ideas (1993) becoming suddenly practical three decades later, once the hardware and the problem scale matched.		

9.1.5 Wanda: Pruning Without the Hessian

SparseGPT’s Hessian inversion is expensive. In 2024, **Wanda** (Mingjie Sun, Zhuang Liu, Anna Bair, and J. Zico Kolter, ICLR 2024) showed you can get almost the same result much more cheaply.

Wanda’s pruning score for weight w_{ij} is simply:

$$S_{ij} = |w_{ij}| \times \|X_j\|_2$$

That is: the magnitude of the weight, times the L_2 norm of the corresponding input activation X_j (averaged over a calibration batch). This is **not** a Hessian computation — it is a single forward pass to collect activations, followed by element-wise multiplication. No matrix inversion, no iterative solver.

Why does this work? The key insight from GPTQ’s outlier analysis (Chapter 63): a small weight multiplied by a large activation is dangerous to remove. Wanda’s score directly captures this interaction. It is the activation-aware saliency of AWQ, adapted for pruning instead of quantization.

The result: Wanda prunes 50% of weights with comparable accuracy to SparseGPT, in a fraction of the time, and supports both unstructured and 2:4 structured sparsity. For 2:4 sparsity on LLaMA-2-70B, Wanda matches SparseGPT’s accuracy within measurement noise.

PY GO FURTHER · PYTHON — NumPy Array Norms

skip if fluent

The **norm** of a vector is its “length” — a single number summarizing the magnitudes of all its entries. The most common is the L_2 norm (Euclidean length):

$$\|v\|_2 = \sqrt{v_1^2 + v_2^2 + \dots + v_n^2}$$

In NumPy you compute it as `np.linalg.norm(v)`. For a matrix of activations, `np.linalg.norm(X, axis=0)` gives the L_2 norm of each **column** — one number per input feature.

```
import numpy as np

# Suppose activations X has shape (batch=4, features=3)
X = np.array([[1.0, 100.0, 0.5],
              [0.9, 98.0, 0.3],
              [1.1, 102.0, 0.4],
              [0.8, 99.0, 0.6]])

col_norms = np.linalg.norm(X, axis=0)
print(col_norms)
# Output: [1.80, 199.5, 0.83] (the middle feature has huge norm
# - an outlier!)
```

In Wanda, those column norms are multiplied by the weight magnitudes to produce the pruning score. The middle column would make even a tiny weight score high, keeping it alive.

A tiny worked Wanda example: suppose one row of a weight matrix is $W = [0.01, 0.50, 0.03, 0.80]$ and the activation norms are $\|X\|_2 = [90, 0.2, 0.1, 0.4]$. Wanda scores:

$S = [0.01 \times 90,$
 $0.50 \times 0.2,$
 $0.03 \times 0.1,$
 $0.80 \times 0.4] = [0.90,$
 $0.10,$
 $0.003,$
 $0.32]$

To reach 50% sparsity in this row, prune the two lowest-scoring weights: positions 2 and 3 (the naive magnitude pruner would have kept position 2 because 0.50 looks large, while removing position 1 because 0.01 looks small — the exact opposite of Wanda’s correct answer).

PY GO FURTHER · PYTHON — Wanda pruning score — toy implementation *skip if fluent*

```

import numpy as np

def wanda_scores(W: np.ndarray, X: np.ndarray) -> np.ndarray:
    """
    W: weight matrix of shape (out_features, in_features)
    X: calibration activations of shape (batch, in_features)
    returns: score matrix same shape as W
    """
    # L2 norm of each input column across the calibration batch
    col_norms = np.linalg.norm(X, axis=0) # shape:
    (in_features,)
    # Each row of W is multiplied element-wise by the column
    norms
    return np.abs(W) * col_norms[np.newaxis, :]

def prune_50_percent(W: np.ndarray, scores: np.ndarray) ->
np.ndarray:
    """Set the bottom 50% of weights (by score) to zero."""
    W_pruned = W.copy()
    threshold = np.percentile(scores, 50)
    W_pruned[scores < threshold] = 0.0
    return W_pruned

# --- tiny self-test ---
W = np.array([[0.01, 0.50, 0.03, 0.80]])
X = np.array([[90.0, 0.2, 0.1, 0.4]] * 4) # batch of 4
identical rows

```

```

scores = wanda_scores(W, X)
print("Scores:", scores)           # [[0.9, 0.1, 0.003, 0.32]]
W_p = prune_50_percent(W, scores)
print("Pruned:", W_p)              # [[0.01, 0.0, 0.0, 0.80]] –
correct!

```

9.1.6 Structured Pruning: Removing Whole Neurons and Heads

Unstructured pruning, even with 2:4 patterns, requires special hardware support to translate into actual speedup. For general-purpose CPUs and older GPUs, **structured pruning** is more practical.

The most common structured targets in a transformer are:

- **Attention heads:** a transformer uses 32 or 64 parallel “attention heads” that each attend to different aspects of the input. Individual heads can be removed entirely after ranking them by importance (measured on a calibration set). Pruning 25–30% of heads typically costs less than 1% accuracy.
- **Feed-forward intermediate dimension:** the two feed-forward matrices in each transformer block have a large “intermediate” dimension (often 4× the model dimension). Reducing this dimension is a structured cut that immediately produces a smaller, faster matrix multiply.
- **Entire layers:** **SliceGPT** (2024, from Microsoft Research) removes entire weight columns and rows from matrices, effectively reducing the model’s hidden dimension. The result is a model with the same number of layers but uniformly narrower — easy to run on hardware optimized for smaller dense matrices.

Aside. LLM-Pruner (Ma et al., NeurIPS 2023) extended structured pruning to dependencies. In a transformer, removing a neuron from one layer forces you to remove corresponding neurons in other layers (since they feed the same residual stream). LLM-Pruner builds a dependency graph of the entire model, identifies groups of coupled weights, and estimates each group’s importance jointly. The result: 20% structured pruning of LLaMA with a single GPU-day of recovery fine-tuning.

9.2 Knowledge Distillation: Teach a Small Student

Pruning asks: which parts of the big model can we remove? Distillation asks a different question: instead of shrinking the big model, can we **teach** a fresh, smaller model to behave like the big one?

The key insight is subtle. When a model classifies an image of a cat, it does not just output “cat.” It outputs a probability distribution over all possible classes: “cat: 97%, lynx: 1.5%, fox: 0.8%, ...” Those near-zero probabilities on wrong classes are surprisingly informative. They reveal the model’s internal notion of **similarity**. The small probability of

“lynx” tells the student that cats are somewhat like lynxes. A hard label (“cat: 1, everything else: 0”) discards that information.

HISTORICAL NOTE

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean (2015) published “Distilling the Knowledge in a Neural Network” while at Google. The paper is remarkably short (9 pages) and remarkably clear. Hinton had been developing the idea since at least 2006 under the name “Dark Knowledge.” The soft probabilities he called “dark knowledge” — information hidden in the near-zero outputs of a trained model that hard labels cannot carry. His experiments showed that a small model trained on soft targets outperformed the same architecture trained on hard labels, sometimes by large margins. The paper now has over 25,000 citations and is a cornerstone of modern model deployment.

9.2.1 The Distillation Loss

Formally, suppose the teacher produces logits z^T (the raw scores before softmax) and the student produces logits z^S . The **temperature** T (not to be confused with the teacher model T) softens both distributions:

$$p_i^T = \frac{\exp\left(\frac{z_i^T}{\tau}\right)}{\sum_j \exp\left(\frac{z_j^T}{\tau}\right)}, \quad p_i^S = \frac{\exp\left(\frac{z_i^S}{\tau}\right)}{\sum_j \exp\left(\frac{z_j^S}{\tau}\right)}$$

Σ GO FURTHER · THE MATHS — Temperature in a Softmax

skip if familiar

Softmax converts a vector of raw scores into a probability distribution. Each entry z_i becomes:

$$\text{softmax}(z)_i = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

The **temperature** τ divides all scores before softmax. At $\tau = 1$, this is standard softmax. At $\tau > 1$, the resulting distribution is **flatter** (softer): small probabilities become a little larger and large probabilities become a little smaller. At $\tau \rightarrow 0$, the distribution collapses to a hard “1 for the top class, 0 for everything else.”

Example: scores $z = [4, 2, 1]$.

- $\tau = 1$: probs $\approx [0.84, 0.11, 0.04]$ — confidently “class 0.”
- $\tau = 4$: probs $\approx [0.49, 0.29, 0.22]$ — much softer, classes 1 and 2 get meaningful probability.

Distillation uses $\tau = 4$ –10 so the tiny probabilities on “wrong” classes become large enough to carry a gradient signal to the student.

Σ GO FURTHER · THE MATHS — Cross-Entropy: Measuring the Gap Between Distributions skip if familiar

Cross-entropy measures how surprised one probability distribution is by samples from another. If the true distribution is p and your model predicts q , the cross-entropy is:

$$\mathcal{L}_{\text{CE}(p,q)} = - \sum_i p_i \log_2 q_i$$

Intuition: $-\log_2 q_i$ is the codelength (in bits) you would assign to class i if you used distribution q as your codebook. Cross-entropy is the **average** codelength when samples actually come from p . Minimizing cross-entropy means making the model's predictions q as close as possible to the truth p .

When p is a one-hot hard label ($p_{\text{cat}} = 1$, all others 0), the sum collapses to a single term: $\mathcal{L}_{\text{CE}} = -\log_2 q_{\text{cat}}$. The loss is simply how many bits your model wastes on the wrong answer.

When p is a soft distribution (as in distillation), every non-zero class contributes. Getting the small probabilities right — the “lynx: 0.04” — lowers the loss just as much as getting the top class right.

The distillation loss is a mixture of two terms:

$$\mathcal{L}_{\text{distill}} = \alpha \cdot \mathcal{L}_{\text{CE}(p_{\tau}^S, p_{\tau}^T)} + (1 - \alpha) \cdot \mathcal{L}_{\text{CE}(p_1^S, y)}$$

where $\mathcal{L}_{\text{CE}}$ is cross-entropy, p_{τ}^S and p_{τ}^T are the student's and teacher's temperature-softened distributions, y is the hard one-hot ground-truth label, and α (typically 0.9) controls the trade-off. The first term teaches the student to **match the teacher's output distribution**. The second term keeps the student anchored to the true labels.

A critical detail: the teacher-matching cross-entropy must be scaled by τ^2 to keep gradients at the same magnitude as for the hard-label term. (This is because dividing by τ inside softmax makes the soft probabilities smaller by a factor of τ , which reduces the gradient by τ^2 .) Hinton 2015 is explicit about this and most implementations include the rescaling.

Knowledge Distillation (KD)

2015

Authors Geoffrey Hinton, Oriol Vinyals, Jeff Dean (Google Brain)

Aim Train a small 'student' model to match the soft probability outputs of a large 'teacher' model, transferring 'dark knowledge' that hard labels cannot carry.

Complexity	Student training cost, plus teacher inference cost for each training batch (teacher runs forward-only, so roughly $2\times$ student compute). No teacher backward pass required.
Strengths	Can close most of the accuracy gap between student and teacher; works without the teacher's training data (black-box distillation); student is architecturally free — any smaller model works; widely deployed (every major tech company uses some form of this).
Weaknesses	Requires access to either the teacher's logits or its outputs; very large teacher–student size gaps may not distill well (the 'capacity gap problem'); temperature tuning is finicky; teacher must be available at training time.
Superseded by	Still the dominant paradigm; extended to LLMs by MiniLLM (2024), DistiLLM (2024), DeepSeek-R1 distillation (2025), and many others.

The original paper demonstrated distillation on acoustic models for speech recognition (where it was originally developed by Hinton's group) and on MNIST. The acoustic model experiments showed that an ensemble of 10 teacher models could be distilled into a single model that outperformed any individual member of the ensemble — compression as a way to **exceed** the teacher through ensemble knowledge.

9.2.2 Why Soft Targets Work: An Intuition

Consider a model classifying handwritten digits (MNIST). When it sees a “7”, the hard label says: “7: 1, everything else: 0.” The teacher's soft output might say: “7: 0.92, 1: 0.04, 9: 0.03, 4: 0.01.” That 0.04 on “1” is not noise. It is the teacher's learned knowledge that 7s and 1s look similar — both have vertical strokes. The student trained on soft targets learns this similarity implicitly. The student trained on hard targets does not.

More formally: the soft targets provide a much richer training signal per example. A hard label has essentially 1 bit of useful information (which class). A soft probability vector over 1000 classes has up to $\log_2(1000) \approx 10$ bits. You need many fewer training examples to converge when each carries 10 bits of signal versus 1 bit.

KEY IDEA

Soft targets are a compressed representation of the teacher's world-model. The near-zero probabilities encode the teacher's similarity judgments — which concepts are close, which are far. Distillation is, literally, compression of knowledge.

9.2.3 Feature-Level Distillation

Hinton's original formulation matched only the final output layer. Later work realized you could match **intermediate layer activations** too — the features, attention patterns, or hidden states at each transformer layer.

FitNets (2015) trained a thin, deep student to mimic not just the teacher’s output but its intermediate representations. **TinyBERT** (Jiao et al., NeurIPS 2020) distilled BERT by matching attention matrices and hidden states at corresponding layers — a 4-layer student matching a 12-layer teacher’s representations at every other layer. The result: 7.5× smaller, 9.4× faster, and retaining 96.8% of BERT’s performance on the GLUE benchmark. TinyBERT showed that for transformers, matching intermediate attention patterns is far more efficient than matching only final outputs.

9.2.4 Distillation for Reasoning: DeepSeek-R1

The most striking demonstration of distillation’s power in 2024–2025 was **DeepSeek-R1** (released January 2025 by Chinese AI company DeepSeek). The story:

DeepSeek trained a large reasoning model (671 billion parameters, mixture-of-experts architecture — meaning only a fraction of the weights activate per token) using large-scale reinforcement learning with a specialized reward for mathematical and logical reasoning. This model (DeepSeek-R1) achieved performance competitive with OpenAI’s o1 model.

They then distilled it: they generated **800,000 examples** of the teacher’s chain-of-thought reasoning — not just final answers, but the full “think step by step” process — and used that data to fine-tune existing open models (Qwen and LLaMA) via standard supervised fine-tuning. No RL was needed for the students. The result: DeepSeek-R1-Distill-Qwen-7B (7 billion parameters, not 671 billion) matched or exceeded the performance of much larger models on most reasoning benchmarks, including GPT-4o and Claude 3.5 Sonnet, on mathematical reasoning tasks.

This is distillation as **reasoning transfer**: the student does not just copy output probabilities, it learns to **think like** the teacher by imitating the teacher’s internal reasoning traces.

PITFALL

Distillation is not magic. The student model has a limited **capacity** — a maximum amount of information it can store, determined by its number of parameters and architecture. If the teacher is much larger than the student, distillation will transfer only the parts of the teacher’s knowledge that fit. For mathematical reasoning, this turns out to be a surprisingly large fraction. For tasks requiring diverse world knowledge distributed across many rare facts, the capacity gap bites hard.

Checkpoint. A student model is trained with $\tau = 8$ and $\alpha = 0.9$. What do those numbers mean in practice?¹⁴

¹⁴**Checkpoint answer.** Temperature $\tau = 8$ means the teacher’s probability distribution is softened significantly before the student is trained to match it — near-zero probabilities on non-target classes are amplified, making the class-similarity information visible. $\alpha = 0.9$ means 90% of the loss comes from matching the teacher’s soft distribution and only 10% from matching the hard ground-truth labels — heavily teacher-guided training.

9.3 Low-Rank Factorization and LoRA

Here is a fact about matrices that seems unrelated to neural networks but turns out to be central: most large matrices in the real world are **approximately low-rank**. They can be closely approximated by the product of two much thinner matrices.

9.3.1 What “Rank” Means

Σ GO FURTHER · THE MATHS — Matrix Rank and SVD

skip if familiar

The **rank** of a matrix W (with shape $m \times n$) is the number of truly independent directions it encodes. A matrix of rank r can be **exactly** written as:

$$W = USV^T$$

where U has shape $m \times r$, S is an $r \times r$ diagonal matrix of **singular values** $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0$, and V^T has shape $r \times n$. This is the **Singular Value Decomposition (SVD)**.

The singular values measure how much each direction “matters.” If $\sigma_1 \gg \sigma_2 \gg \dots \gg \sigma_r$, the matrix is dominated by its first few directions and the rest are negligible.

The **low-rank approximation** keeps only the top $k < r$ singular values and sets the rest to zero:

$$W \approx U_k S_k V_k^T = AB$$

where $A = U_k S_k^{\frac{1}{2}}$ has shape $m \times k$ and $B = S_k^{\frac{1}{2}} V_k^T$ has shape $k \times n$.

Storage comparison: full matrix W costs $m \times n$ numbers. Low-rank approximation costs $m \times k + k \times n = k(m + n)$ numbers. For a 4096×4096 weight matrix and rank $k = 8$:

- Full: $4096 \times 4096 = 16,777,216$ numbers.
- Rank-8: $8 \times (4096 + 4096) = 65,536$ numbers. That is a **256× reduction**.

Why would a trained weight matrix be low-rank? The intuition: during training, the model solves a task (predict the next word, classify an image). The task has some effective dimensionality — the number of truly independent factors that matter for solving it. If the task has k important factors but the weight matrix has 4096 dimensions, the matrix only needs rank k to express all the relevant computation. The remaining $4096 - k$ dimensions are filled with near-zero singular values — mathematical noise.

Empirically, for large language models, the weight matrices in attention and feed-forward layers do have rapidly decaying singular values. Cutting off the bottom 80–90% of singular values often changes the layer output by less than 1%.

9.3.2 Classic Low-Rank Compression

The direct application: perform SVD on every weight matrix, keep only the top k singular values, and replace the original matrix with two smaller matrices A and B . This was

explored as early as 2013 for speech recognition models (Xue et al., ICASSP 2013) and generalized to many neural network architectures in the years that followed.

The challenge: choosing k per layer (different layers may have different effective ranks) and the accuracy loss from discarding singular values. A well-known result (Eckart–Young theorem) says the rank- k approximation by SVD is the **best possible** rank- k approximation in the *Frobenius norm* sense — the Frobenius norm being just the L_2 norm of a matrix flattened into one long vector, $\|W\|_F = \sqrt{\sum_{i,j} w_{ij}^2}$, so “small Frobenius distance” means “the two matrices differ by little, entry by entry.” But “best approximation of the weight matrix” is not the same as “best approximation of the model’s behavior” — you are minimizing the wrong objective.

9.3.3 LoRA: Low-Rank Adaptation

The most influential low-rank method of the last few years is not about compressing a **trained** model — it is about **fine-tuning** a pre-trained model efficiently. But the insight connects directly to compression.

LoRA (Low-Rank Adaptation) was published by Edward Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen at Microsoft in 2021 (published ICLR 2022). The premise: when you fine-tune a large pre-trained model on a new task, the **change** in the weights is approximately low-rank. You do not need to update all 7 billion parameters; you just need to update a small low-rank perturbation on top of the frozen pre-trained weights.

The mechanism. Take a weight matrix $W_0 \in \mathbb{R}^{m \times n}$ from the pre-trained model. Instead of updating W_0 directly (which requires $m \times n$ gradient computations and $m \times n$ stored parameters), freeze W_0 and introduce two small trainable matrices $A \in \mathbb{R}^{r \times n}$ and $B \in \mathbb{R}^{m \times r}$, where $r \ll \min(m, n)$. The adapted weight is:

$$W = W_0 + BA$$

BA is a rank- r matrix (at most). Only A and B are trained; W_0 is frozen. The forward pass uses $Wx = W_0x + BAx$. Since both W_0x and Ax then $B(Ax)$ can be computed separately and added:

- Compute W_0x (frozen, no gradient).
- Compute $h = Ax$ (small: $r \times n$ matrix times n -vector $\rightarrow r$ -vector).
- Compute Bh (small: $m \times r$ matrix times r -vector $\rightarrow m$ -vector).
- Output: $W_0x + Bh$.

The number of trainable parameters: $r \times n + m \times r = r(m + n)$. For GPT-3 ($m = n = 12288$, $r = 4$): $4 \times (12288 + 12288) = 98,304$ parameters per weight matrix, versus $12288^2 = 150,994,944$ for full fine-tuning. That is a $1536\times$ reduction in trainable parameters.

Authors	Edward Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen (Microsoft Research)
Aim	Efficient fine-tuning of large pre-trained models by learning only a low-rank perturbation ΔW to each weight matrix, keeping the original weights frozen.
Complexity	Training: $O(r)$ per weight matrix instead of $O(m \times n)$; inference: zero overhead if B and A are merged into W_0 before deployment.
Strengths	Dramatic reduction in fine-tuning parameters ($1000\times$ for $r=4$); no inference overhead after merging; multiple fine-tuned variants can coexist on the same frozen base; memory-efficient; enables fine-tuning on single GPUs.
Weaknesses	Does not reduce the size of the deployed base model; the rank r must be chosen (too low loses accuracy; too high negates the savings); may not capture fine-tuning changes of very high rank; limited to modifying existing weight matrices.
Superseded by	Extended by QLoRA (quantize base + LoRA, 2023), AdaLoRA (adaptive rank, 2023), DoRA (weight decomposition, 2024), GaLore (gradient low-rank projection, 2024).

LoRA was motivated by the observation that the weight update matrices during fine-tuning of GPT-3-scale models have very low *stable rank* — a soft, noise-tolerant count of how many singular values really matter, equal to the ratio of the squared Frobenius norm (sum of all squared singular values) to the squared largest singular value. A stable rank of, say, 3 means the update is effectively rank-3 even if technically full-rank. The empirical finding that $r = 4$ or $r = 8$ suffices for most fine-tuning tasks, while capturing nearly all the task-relevant adaptation, is the paper’s central experimental result. LoRA is now the standard method for fine-tuning almost every open language model.

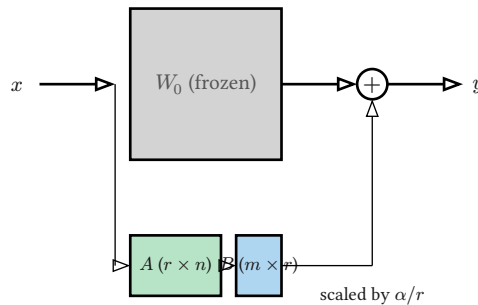


Figure 23: LoRA architecture. The frozen pretrained weight W_0 (grey) handles the main computation. Two small trainable matrices A and B add a low-rank correction BAx to the output. After fine-tuning, BA is merged into W_0 for zero inference overhead.

9.3.4 QLoRA: Quantize the Base, LoRA the Adapters

QLoRA (Dettmers, Pagnoni, Holtzman, Zettlemoyer, University of Washington, NeurIPS 2023) combined two ideas: quantize the frozen base model to 4-bit (NF4 format, a 4-bit data type optimized for normally distributed weights) and then apply LoRA adapters on top in 16-bit precision.

The result was startling: you could fine-tune a 65-billion-parameter model on a **single consumer GPU with 48 GB of VRAM**. Previously this required at least 780 GB (10 × A100 GPUs). QLoRA did it in 24 hours. The quality was comparable to full float16 fine-tuning.

QLoRA also introduced a key trick: **double quantization** — quantize the quantization constants themselves (the scale factors for each group of 64 weights) to save additional memory. And **paged optimizers**: when GPU memory fills up, offload optimizer states to CPU RAM and page them back in as needed, using the same memory-paging mechanism as a computer’s virtual memory system.

QLoRA made personal fine-tuning of frontier-class models a reality. It is now the standard first choice whenever you want to adapt a large open model to a specialized task on limited hardware.

9.3.5 GaLore: Gradient Low-Rank Projection

A 2024 development worth knowing: **GaLore** (Zhao et al., ICML 2024) extends the low-rank insight from **weight updates** to **gradients**. Recall from Chapter 56 that training adjusts each weight by following its *gradient* (the slope of the loss). Modern training does not use the raw gradient: the standard **Adam** optimizer keeps two extra running averages per weight — a smoothed gradient (its “momentum”) and a smoothed squared gradient (its “second moment”) — to take steadier steps. Storing the gradient plus this optimizer state costs 8–12 bytes per parameter — for a 7B model, that is 56–84 GB just for optimizer state. GaLore projects the gradient matrix into a low-rank subspace (using a periodically updated SVD basis) before accumulating into the optimizer state. This reduces optimizer memory by 4–8× with almost no accuracy loss.

GaLore is a genuine compression of the **training process**, not just the deployed model.

9.4 Combining the Three: The Production Pipeline

In practice, frontier deployment teams use all three methods in sequence:

1. **Start with the pre-trained model** (e.g., LLaMA-3 70B, float16, 140 GB).
2. **Distill** into a smaller architecture (e.g., 7B), using the large model to generate soft labels for fine-tuning data (or using chain-of-thought traces, as DeepSeek-R1 did).
3. **Prune** the distilled model: apply SparseGPT or Wanda to reach 50% sparsity in attention and feed-forward weights.
4. **Quantize** to W4A16 using AWQ, bringing each weight to 4 bits.

5. **Deploy** with LoRA slots available: ship the quantized model + hooks to load task-specific LoRA adapters at inference time, switching tasks without reloading the base model.

The total compression from step 1 to step 5: $140 \text{ GB} \rightarrow (7/70 \text{ from distillation}) \times (2/4 \text{ from quantization}) \times (\times 0.5 \text{ from pruning weight only sparsity}) \approx 140 \times 0.1 \times 0.5 \times 0.5 \approx 3.5 \text{ GB}$. Fits in a phone.

This is not science fiction — as of 2025, the MLC AI group ships exactly this kind of pipeline (WebLLM, MLC-LLM) for running 7B models in a browser using WebGPU, and on Android phones using Vulkan.

SCOREBOARD — Weight of a 7B language model through the compression pipeline

Technique	Bytes	Ratio	Notes
Stage	Method	Size (approx.)	Relative to original
Pre-trained float16	—	14 GB	1×
Post quantization INT4	AWQ / GPTQ (Ch. 63)	3.5 GB	4×
Post pruning 50%	Wanda / SparseGPT	3.5 GB (sparse)	4× (storage same, compute ½)
Distilled 7B from 70B	KD soft targets	14 GB float16	5–10× vs. 70B teacher
Distilled + INT4	KD + AWQ	3.5 GB	20–40× vs. 70B teacher

Chapter takeaways

- Pruning removes weights (unstructured) or entire structures (structured) from a trained network. Structured pruning gives real hardware speedup; unstructured pruning needs special sparse kernels.
- The Lottery Ticket Hypothesis says every large network hides a small trainable subnetwork. It is scientifically important but too expensive to apply to LLMs directly.
- SparseGPT and Wanda prune LLMs to 50% sparsity in one pass without retraining, using Hessian compensation and activation-weighted magnitude respectively.
- Knowledge distillation trains a small student on the large teacher's soft probability outputs. Soft targets carry 'dark knowledge' — similarity structure — that hard labels discard.
- Temperature $\tau > 1$ in softmax softens the teacher's distribution, amplifying near-zero probabilities so the student can learn from them.

- DeepSeek-R1 (2025) demonstrated chain-of-thought distillation: a 7B student matched 671B teacher performance on reasoning tasks by imitating the teacher's full thought traces.
- Most weight matrices in large models are approximately low-rank: their singular values decay rapidly, so they can be approximated by the product of two thin matrices.
- LoRA freezes pre-trained weights and learns a low-rank perturbation BA (rank $r \ll \min(m,n)$) for efficient fine-tuning, reducing trainable parameters by 100–1000×.
- QLoRA combines 4-bit quantization of the frozen base with 16-bit LoRA adapters, enabling fine-tuning of 65B models on a single consumer GPU.
- Production pipelines combine distillation, pruning, quantization, and LoRA in sequence to achieve 20–40× compression of frontier models for phone deployment.

9.5 Exercises

Exercise 64.1 ★

A language model outputs logits $z = [3.0, 1.0, 0.0, -1.0]$ for classes A, B, C, D. Compute the standard softmax ($\tau = 1$) and the temperature-softened softmax at $\tau = 3$. Verify that the sum of probabilities is 1 in each case. What happens to the probability of the least-likely class (D) as temperature increases?

Exercise 64.2 ★

A weight row is $W = [0.02, -0.60, 0.01, 0.45]$ and calibration activation norms are $\|X\|_2 = [80, 0.3, 50, 0.5]$. Compute Wanda scores for each weight. Which weights would be pruned to achieve 50% sparsity in this row?

Exercise 64.3 ★★

A weight matrix W has shape 1024×1024 . You apply a rank- r approximation $W \approx AB$ where $A \in \mathbb{R}^{1024 \times r}$ and $B \in \mathbb{R}^{r \times 1024}$.

(a) How many numbers does the full matrix store? How many do the two factor matrices store?

(b) For what value of r does the factorization use exactly half the storage of the original?

(c) At $r = 4$, what fraction of the original storage is used? Express as a percentage.

Exercise 64.4 ★★

Explain in your own words why knowledge distillation often produces a student that is **better** than a student trained directly on the hard labels, even though both see the same raw training data. Your explanation should mention the concept of “dark knowledge” and the information content of soft versus hard targets.

Exercise 64.5 ★★

The Lottery Ticket Hypothesis claims that within a large trained network there exists a sparse subnetwork that, when trained from the original random initialization, matches the full network’s accuracy. Why can this finding not be directly used as a practical compression recipe for large language models? What would need to be true for it to become practical?

Exercise 64.6 ★★★

You are deploying a 13B-parameter language model to run on a phone with 6 GB of RAM (available for the model). The base model is float16. Design a compression pipeline using at least three of the techniques from this chapter (quantization from Chapter 63 counts as one), and estimate the final model size. State your assumptions clearly and identify the biggest uncertainty in your estimate.

9.6 Further Reading

Frankle and Carbin, “The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks,” ICLR 2019 — the original ticket paper; clear, concise, and still well worth reading in full.

Frantar and Alistarh, “SparseGPT: Massive Language Models Can Be Accurately Pruned in One Shot,” ICML 2023 — the extension of Optimal Brain Surgeon to trillion-parameter scale; the mathematical connection to GPTQ is made explicit.

Sun et al., “A Simple and Effective Pruning Approach for Large Language Models (Wanda),” ICLR 2024 — shows that activation-weighted magnitude suffices; a clean and reproducible result.

Hinton, Vinyals, and Dean, “Distilling the Knowledge in a Neural Network,” NeurIPS 2015 workshop — the foundational distillation paper; 9 pages, extremely readable; the temperature scaling derivation is in the appendix.

Hu et al., “LoRA: Low-Rank Adaptation of Large Language Models,” ICLR 2022 — the LoRA paper; includes the stable-rank analysis that motivates why $r = 4$ suffices.

Dettmers et al., “QLoRA: Efficient Finetuning of Quantized LLMs,” NeurIPS 2023 — QLoRA; the double-quantization and paged-optimizer tricks are in Section 3.

DeepSeek-AI, “DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning,” January 2025 — describes the teacher model and the distillation procedure that produced the R1-Distill series.

Ma et al., “LLM-Pruner: On the Structural Pruning of Large Language Models,” NeurIPS 2023 — the dependency-graph approach to structured pruning; includes the recovery fine-tuning pipeline.

COMING UP NEXT

We have now exhausted the three classical levers for compressing the **model weights**: quantization (Chapter 63), pruning, distillation, and low-rank compression (this chapter). But there is a fourth place where memory explodes at inference time: the **KV-cache** — the stored key and value vectors for every token in the model’s context window. A model with a 128,000-token context window and 32 layers can accumulate gigabytes of KV-cache for a single conversation. Chapter 65 opens this new front: eviction strategies (dropping low-attention tokens), quantized caches, DeepSeek’s Multi-head Latent Attention architecture (which bakes cache compression into the model itself), and the 2025–2026 vector-quantization frontier for embedding spaces.

KV-Cache, Context, and Embedding Compression

The best way to serve a billion tokens is to never store them in the first place.

— Anonymous ML infrastructure engineer, 2025

Picture a librarian who reads extraordinarily fast. Every time you start a new conversation, this librarian reads your entire message history — every word, every sentence, going back to the beginning — just to answer your next question. That is, more or less, exactly what a large language model does. And the library it maintains grows one shelf longer with every reply.

That library has a name: the **key-value cache**, or KV-cache. By the summer of 2024, running a large language model on a sequence of 100,000 tokens required more GPU memory for the cache alone than the model’s own weights occupied. Teams at Google, Microsoft, Meta, and the Chinese AI lab DeepSeek all arrived at the same uncomfortable conclusion: the bottleneck to cheap, long-context AI is not the model — it is the **memory for the conversation so far**.

This chapter is about the compression tricks that attack that bottleneck. We will travel from the basic architecture of the KV-cache (why it exists, exactly what it stores, why it grows without bound), through three families of compression — **eviction**, **quantization**, and **low-rank architectural redesign** — to a fourth, complementary idea: compressing the **prompt itself** before it ever enters the model. Then we zoom out to the database side, where millions of vectors are stored for retrieval, and learn the elegant technique of **product quantization**, which shrinks a 768-number float vector into fewer bytes than a tweet.

By the end you will see a single principle running through all of it: every technique in this chapter is the same ancient move in a new costume — **predict and throw away what can be reconstructed, code only the residual**.

WHERE WE ARE

- In **Chapter 26** we built an arithmetic coder: given a probability for the next symbol, spend $-\log_2 p$ bits. The KV-cache problem is memory, not entropy — but the information-theoretic framing still applies.

- In **Chapter 39** we met **scalar quantization**: replace a real number with the nearest step on a fixed grid. Chapter 63 extended this to the weights of a neural network (GPTQ, AWQ). This chapter applies the same idea to a running cache of vectors.
- In **Chapter 62** we learned what an autoregressive language model actually does: it maintains a probability distribution over the next token, conditioned on every previous token. The mechanism that lets it “see” all those previous tokens at low cost is precisely the KV-cache we are about to meet.

By the end of this chapter you will be able to:

- Explain what the KV-cache is, why it exists, and why its memory cost grows with context length
- Describe the H2O eviction policy and the Heavy-Hitter Oracle insight behind it
- Explain SnapKV and how observation-window pooling selects which cache entries to keep
- Describe DeepSeek's Multi-head Latent Attention (MLA) and quantify how much memory it saves
- Understand KV-cache quantization and why rotation tricks tame outlier values
- Explain LLMingua's prompt compression and how it decides which tokens to drop
- Define product quantization (PQ), work through a complete numeric example, and explain OPQ's improvement
- Describe binary and Matryoshka embeddings and state when each is the right choice

10.1 The KV-Cache: What It Is and Why It Hurts

Before we can compress the KV-cache, we need to understand what it contains and why it exists. The explanation requires a brief tour of the **attention mechanism**, which is the mathematical engine inside every modern language model.

10.1.1 Attention in Plain English

A transformer language model processes text in layers. At each layer, every token looks at every other token to gather context — this is the **attention** operation. For each token at position i , the model computes three vectors from the token's current representation: a **query** vector q_i , a **key** vector k_i , and a **value** vector v_i .

The intuition: the query is what the token is looking for; the key is what it is advertising about itself; the value is what it will contribute if selected. To produce the attention output for token i , the model dots q_i with every other token's key k_j , turns those dot-products into probabilities with a softmax, then adds up all the value vectors v_j weighted by those probabilities.

Recall. From Chapter 64: the **softmax** turns a list of raw scores $z = (z_1, \dots, z_n)$ into probabilities that sum to 1 by exponentiating and normalising, $\text{softmax}(z)_i = e^{z_i} / \sum_j e^{z_j}$. The largest score gets the largest probability; negatives are allowed. In attention, the scores are the query–key dot products, so the most relevant tokens receive the most weight.

Σ GO FURTHER · THE MATHS — Dot product and what it measures *skip if familiar*

A **dot product** of two vectors $\mathbf{a} = (a_1, a_2, \dots, a_d)$ and $\mathbf{b} = (b_1, b_2, \dots, b_d)$ is:

$$\mathbf{a} \cdot \mathbf{b} = a_1 b_1 + a_2 b_2 + \dots + a_d b_d = \sum_{i=1}^d a_i b_i$$

It is a single number. When $\mathbf{a}$ and $\mathbf{b}$ point in the same direction, the dot product is large and positive. When they are perpendicular, it is zero. When they point opposite ways, it is negative.

Why attention uses it: if the query vector for the word “bank” (as in river bank) points in a similar direction to the key vector for “river,” their dot product is large, telling the model to weight “river” heavily when computing context. The dot product measures **relevance**.

Tiny example with $d = 2$: $\mathbf{a} = (3, 1)$, $\mathbf{b} = (2, 4)$. Dot product: $3 \times 2 + 1 \times 4 = 10$. With $\mathbf{c} = (-1, 3)$: dot product = $3 \times (-1) + 1 \times 3 = 0$ — these two are perpendicular (unrelated in the model’s space).

10.1.2 The Caching Opportunity — and the Memory Tax

Here is the critical observation: when the model generates token $t + 1$, it needs to attend over every previous token $1, 2, \dots, t$. But the keys and values for those earlier tokens were **already computed** when those tokens were processed. Recomputing them would waste time proportional to t^2 (each new token forces a full re-read of all history). So the model stores the keys and values for every past token in fast GPU memory — the KV-cache.

The price is steep. For a typical model with:

- 32 attention layers
- 32 heads per layer
- key/value dimension 128 per head
- BFloat16 precision (2 bytes per number)

Each token occupies $2 \times 32 \times 32 \times 128 \times 2 = 524,288$ bytes ≈ 512 KB — for the full 32-layer model. With a 128,000-token context (LLaMA-3 supported this in 2024), the KV-cache alone consumes $512 \text{ KB} \times 128,000 \approx 64 \text{ GB}$. The GPU has 80 GB total. The model weights — everything the model learned — take perhaps 14 GB. The cache is eating **four times more memory than the model itself**.

KEY IDEA

The KV-cache is a classic time–space trade-off: we spend memory to avoid recomputing keys and values. As contexts grow from thousands to millions of tokens, the memory bill becomes the dominant inference cost. Every technique in this chapter attacks that bill.

10.1.3 How Big Is “One Token” in the Cache?

Let us make the numbers concrete with a comparison. Different models make different architectural choices and pay different memory costs per token.

Model	Architecture	Bytes/token	Notes
LLaMA-3.1 405B	MHA, 128 heads	516 KB	Standard multi-head attention
Mistral 7B	GQA, 8 groups	32 KB	Grouped query: groups share K/V
Qwen-2.5 72B	GQA, 40 groups	327 KB	Large model, still expensive
DeepSeek-V3 671B	MLA latent	70 KB	Low-rank latent only; see section below

Table 1: KV-cache memory per token for representative 2024 models (all numbers approximate).

Grouped Query Attention (GQA) — where multiple query heads share one pair of key/value heads — already cuts memory considerably compared to full Multi-Head Attention (MHA). DeepSeek’s MLA shrinks it further still, by a fundamentally different mechanism.

10.2 Family 1: Eviction — Throwing Away What You Don’t Need

The simplest compression strategy is eviction: once the cache grows past a budget, drop some old entries. But which ones?

A naive policy — drop the oldest tokens — sounds reasonable, but it fails catastrophically. Older tokens often carry **structural** information: the opening sentence, the system prompt, the question being answered. Dropping them first is like throwing away the question before you write the answer.

10.2.1 The Heavy-Hitter Oracle (H2O)

The key observation behind H2O (Zhang et al., NeurIPS 2023) is empirical but robust: **across all heads and all layers, a small fraction of token positions consistently attract most of the total attention score**. These tokens are called **Heavy Hitters** (H2), and they are the tokens that are actually load-bearing for the model’s reasoning.

The H2O policy maintains a fixed-size cache budget B split between two pools:

- **Recent tokens** (the last $\frac{B}{2}$ or so): kept unconditionally, because attention to the very recent past is always important.
- **Heavy hitters** (the remaining budget): kept based on cumulative attention scores.

At each new decoding step, the model computes attention from the new query to all cached keys. H2O **accumulates** those attention scores over time. When the cache exceeds the

budget, it evicts the non-recent token with the lowest cumulative score — the one that has received the least total attention weight from every subsequent token. The intuition: if no future token has found this past token relevant, it is unlikely to become relevant now.

H2O --- Heavy-Hitter Oracle KV-Cache Eviction

2023

Authors	Zhenyu Zhang, Ying Sheng, Tianyi Zhou, et al. (UT Austin / UC Berkeley / CMU)
Aim	Reduce KV-cache memory during autoregressive decoding by evicting low-attention tokens while retaining heavy hitters and recent tokens
Complexity	$\mathcal{O}(B)$ extra memory per layer; $\mathcal{O}(n)$ work per decoding step to update the attention score accumulator
Strengths	Simple post-hoc optimization; no retraining needed; 29x throughput improvement reported on OPT-30B at 20% heavy-hitter budget; works across OPT, LLaMA, GPT-NeoX
Weaknesses	Eviction is irreversible --- a dropped token cannot be recalled later; accuracy degrades on tasks that require looking far back beyond the recent window; optimal budget split is task-dependent
Superseded by	SnapKV (2024), which uses observation-window pooling; StreamingLLM (2023), which keeps only 'sink' tokens plus recent; various learned eviction policies in 2024--2025

H2O formalizes the eviction problem as maximizing a **submodular** function — a class of set functions where adding an element to a smaller set is at least as valuable as adding it to a larger one. This gives a theoretical guarantee: the greedy score-based eviction policy achieves a constant-factor approximation of the best possible offline eviction policy. In practice, using only the last-layer accumulated attention scores works nearly as well as using future-token scores, making the method fully causal (no peeking ahead).

10.2.2 SnapKV: What the Model Already Knows It Needs

H2O accumulates scores during decoding. SnapKV (Li et al., arXiv:2404.14469, NeurIPS 2024) asks a smarter question: **before generating**, can the model tell which prompt tokens it will need most?

The answer is yes. The intuition is that the last few tokens of the prompt — the actual question being asked — already “look ahead” to the parts of the context they will need. SnapKV calls this the **observation window**: typically the last 16–64 tokens.

The algorithm:

1. Process the full prompt normally, computing all keys and values.

2. For each attention head, compute the attention weights from the observation window tokens to every earlier prompt token. Average (pool) those weights using a small kernel (size 5) to prevent single-token spikes from dominating.
3. Keep the top- k prompt positions per head (where k is the memory budget).
4. Generate with this compressed cache, adding only new generated tokens.

SnapKV achieves a 3.6x generation speedup and 8.2x memory reduction at 16K-token context with almost no accuracy loss on standard benchmarks. The key insight — that the model “knows what it needs before generation” — is a beautiful example of using the model’s own attention machinery as a compressor.

Checkpoint. H2O and SnapKV both keep a “recent” window unconditionally. Why?

¹⁵

10.3 Family 2: Architectural Compression — DeepSeek’s Multi-Head Latent Attention

Eviction works post-hoc: we build the full cache and then throw parts away. DeepSeek’s approach is different: **redesign the architecture so the cache never gets large in the first place**. The result, Multi-head Latent Attention (MLA), was introduced in DeepSeek-V2 (May 2024) and carried forward into DeepSeek-V3 (December 2024).

10.3.1 The Low-Rank Bottleneck Trick

Standard multi-head attention caches, for each token, separate key and value vectors for each of H heads. If each head has dimension d_h and there are H heads, the cache per token per layer is $2Hd_h$ floating-point numbers.

MLA’s key insight: **keys and values across all heads are highly correlated**. The information they carry lives in a much lower-dimensional space. Instead of caching $2Hd_h$ numbers, cache a single **latent vector** c of dimension d_c (where $d_c \ll 2Hd_h$), trained to contain everything needed to reconstruct the keys and values.

Formally:

$$c = W_c x$$

where x is the token’s current hidden state and W_c is a learned compression matrix of shape $d_c \times d_{\text{model}}$. At inference, the keys and values for all heads are recovered:

$$K = W_K c, \quad V = W_V c$$

where W_K and W_V are learned expansion matrices. The critical point: **only c is stored in the cache**. The expansion matrices are part of the model weights, already loaded into GPU memory. So the per-token cache cost drops from $2Hd_h$ numbers to d_c numbers.

¹⁵**Checkpoint answer.** Very recent tokens carry the local coherence of the sentence being generated. The model almost always attends strongly to them for grammar, reference resolution, and continuation. Evicting them first would break the most immediate context.

Σ GO FURTHER · THE MATHS — Low-rank matrix factorization *skip if familiar*

A matrix M of shape $m \times n$ has **rank** r if it can be written as a product of two smaller matrices:

$$M = AB, \quad A \in \mathbb{R}^{m \times r}, B \in \mathbb{R}^{r \times n}$$

When $r \ll \min(m, n)$, this is a **low-rank factorization**. The number of stored values shrinks from mn to $r(m + n)$.

Intuition: if the rows of M are not all independent but instead lie near a low-dimensional “plane” in $\mathbb{R}^n$, then r basis vectors can describe them all. The rank is the dimension of that plane.

Example: $M = \begin{pmatrix} 2 & 4 & 6 \\ 3 & 6 & 9 \\ 1 & 2 & 3 \end{pmatrix}$ has rank 1, because every row is a multiple of $(1, 2, 3)$. We can write $M = ab^T$ with $a = (2, 3, 1)^T$ and $b = (1, 2, 3)^T$ — storing 6 numbers instead of 9.

MLA’s bet: the full K and V matrices across all heads, for a given token, lie near a low-rank plane. The model is trained end-to-end to learn the right low-dimensional representation.

10.3.2 How Much Does MLA Save?

In DeepSeek-V2 with its MLA configuration:

- Standard MHA per token per layer: $2 \times 128 \text{ heads} \times 128 \text{ dim/head} = 32,768$ numbers.
- MLA latent per token per layer: 512 numbers (plus a small decoupled position vector of 64 numbers).

That is roughly a 60x reduction per layer. The actual reported figure — 70 KB/token for the full DeepSeek-V3 model versus 327–516 KB/token for comparable MHA models — reflects approximately a 93% reduction in KV-cache memory, while the model **matches or exceeds** the quality of full attention on standard benchmarks.

HISTORICAL NOTE

MLA was the architectural surprise of 2024. Prior attempts to compress KV-caches had all been post-hoc (train normally, apply a patch). DeepSeek’s team, based in Hangzhou, baked the compression into the training objective itself. DeepSeek-V3, released December 2024 as a fully open-weights model (671B parameters, mixture-of-experts), demonstrated that the quality cost of this compression was essentially zero. This was widely recognized as one of the most practically important architectural innovations in the transformer era, and several other frontier labs (including Moonshot AI’s Kimi k1.5, 2025) adopted MLA or close variants.

Authors	DeepSeek-AI (arXiv:2405.04434 for V2; arXiv:2412.19437 for V3)
Aim	Eliminate most of the KV-cache by storing a low-rank joint latent vector instead of per-head keys and values; trained end-to-end so the latent captures everything needed for high-quality attention
Complexity	$\mathcal{O}(d_c)$ cache per token instead of $\mathcal{O}(H d_h)$; matrix multiplications at each attention step to expand the latent to keys/values, but these matrices are already in GPU memory as model weights
Strengths	~93% KV-cache memory reduction; quality on par with full MHA; enables very long contexts (128K+ tokens) without special eviction logic; no post-hoc degradation since it is trained in
Weaknesses	Requires training from scratch (converting an existing MHA model needs full retraining); adds matrix multiplications at inference; interaction with rotary positional embeddings requires careful design
Superseded by	Ongoing; MLA variants continue to appear in frontier models through 2025--2026

MLA also introduced a subtle positional-encoding trick: a small separate “decoupled” key vector carries position information while the main latent stays position-free. This allows the expansion $\mathbf{W}_K \mathbf{c}$ to be algebraically “absorbed” into the query projection at inference time, eliminating one of the two matrix multiplications from the hot path — a compiler-style optimization baked into the mathematical structure of the mechanism.

10.4 Family 3: KV-Cache Quantization — Fewer Bits per Number

Even without eviction or architectural change, we can shrink the cache by storing each key and value in fewer bits. Chapter 63 covered this idea thoroughly for model weights (GPTQ, AWQ, 4-bit quantization). The KV-cache presents the same challenge — and the same enemy: **outliers**.

10.4.1 Why KV-Cache Quantization Is Hard

When we quantize a vector of 128 numbers to 4 bits each, we are saying: “map the entire range of values to 16 grid steps.” If most values cluster near zero but one value is 10x larger, that outlier forces the grid to spread wide, leaving the clustered values with terrible precision. (Recall the dead-zone and clipping tricks from Chapter 39: a single outlier wrecks a uniform quantizer’s average error.) This is the same problem that makes naive INT8 quantization fail for activations (Chapter 63), and it is **worse** for the KV-cache because the key vectors encode position information through **Rotary Position**

Embeddings (RoPE) — a scheme that injects a token’s position by rotating its key vector by an angle proportional to that position, which produces particularly structured, spiky coordinate distributions that are awkward to quantize.

10.4.2 PolarQuant: Rotation as a Pre-Conditioner

PolarQuant (published at AISTATS 2026) attacks the outlier problem geometrically. The insight: a random orthogonal rotation matrix $\mathbf{R}$ preserves all the information in a vector (it just rotates it in space), but it **redistributes** the variance uniformly across all coordinates. After rotation, each coordinate independently follows a distribution close to a Gaussian, and a Gaussian quantizes beautifully with a standard scalar quantizer.

The algorithm:

1. Sample a random orthogonal matrix $\mathbf{R}$ once (shared across all tokens and all heads).
2. Before caching key/value vector $\mathbf{k}$, compute $\mathbf{k}' = \mathbf{R}\mathbf{k}$.
3. Quantize $\mathbf{k}'$ to 4 bits per coordinate with a per-head scalar quantizer.
4. At attention time, dequantize: the query is also rotated, so $\mathbf{q}'^T \mathbf{k}' = (\mathbf{R}\mathbf{q})^T (\mathbf{R}\mathbf{k}) = \mathbf{q}^T \mathbf{k}$ — the rotation is mathematically invisible.

The rotation does not change the attention output. Only the **quantization grid** changes, from one that is distorted by outliers to one that is well-matched to a spread-out Gaussian.

10.4.3 TurboQuant: Online Vector Quantization at 3 Bits

TurboQuant (Google Research, ICLR 2026, arXiv:2504.19874) pushes further. Instead of scalar quantization post-rotation, it applies **online vector quantization**: each rotated key/value vector is quantized jointly rather than coordinate-by-coordinate, exploiting correlations between coordinates within a single vector.

TurboQuant achieves 3-bit compression with, according to its authors, effectively zero accuracy loss on standard LLM benchmarks. A companion system, KVTC (NVIDIA, 2025), uses PCA-based decorrelation instead of random rotation and reaches approximately 20x compression — storing the KV-cache in roughly 1.6 bits per coordinate — with less than one percentage point accuracy penalty across models from 1.5B to 70B parameters.

Recall. PCA (Principal Component Analysis), met in Chapter 38 as the statistical cousin of the Karhunen–Loève transform, finds the orthogonal rotation that aligns the axes with the directions of greatest variance in the data. After this rotation the coordinates are decorrelated and most of the energy concentrates in the first few — exactly the property that makes quantization cheap. Where a *random* rotation spreads variance evenly by luck, PCA computes the *best* rotation from a sample of real cache vectors.

PITFALL

“Zero accuracy loss” in quantization papers almost always means “no statistically significant degradation on the benchmarks we tested.” Long-context tasks — multi-hop reasoning chains, retrieval over very long documents — are much more sensitive to small numerical errors than short-answer benchmarks. Aggressive KV-cache quantization

should always be validated on the actual deployment task, not just on the standard short-context suite.

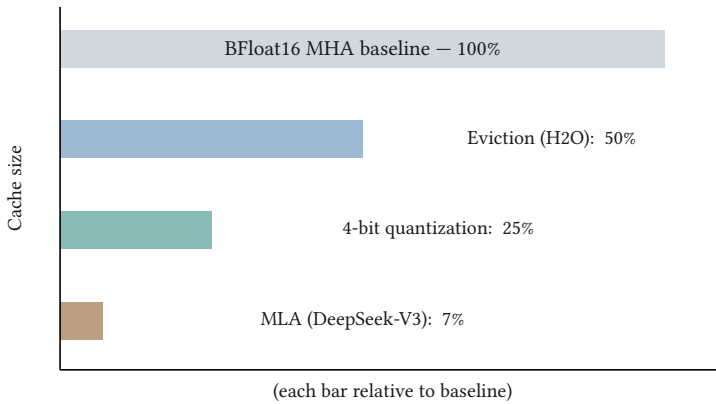


Figure 24: KV-cache compression families and approximate memory relative to BFloat16 MHA baseline.

10.5 Family 4: Prompt and Context Compression

All three families above shrink the **stored** cache. A fourth strategy attacks the problem at source: compress the **input text** before it ever enters the model, so fewer tokens are processed and fewer cache entries are ever created.

10.5.1 LLMingua: Drop the Unimportant Tokens

LLMingua (Microsoft Research, EMNLP 2023) uses a small, fast language model — much smaller than the target LLM — to evaluate **how surprising** each token in the prompt is, given its context. Unsurprising tokens (tokens with low **perplexity**, i.e., high model probability) are candidates for removal: if the small model already expected this token, the big model probably does too, and removing it loses little information.

Recall. From Chapter 18: **perplexity** of a sequence under model M is 2^H where H is the cross-entropy. Low perplexity means the model was unsurprised — the sequence was predictable.

The LLMingua pipeline:

1. A **budget controller** decides the overall compression ratio (e.g., 4x) and how to allocate it across the prompt’s structural parts (instructions, context, examples, question).
2. A small model (e.g., LLaMA-7B) scores each token in the prompt with its conditional perplexity.
3. Tokens with probability above a threshold — the predictable ones — are dropped.
4. A post-processing step reconnects the remaining tokens into pseudo-sentences that the target LLM will still parse correctly.

LLMingua achieves up to 20x compression with minimal performance loss on benchmarks involving long-context retrieval and question answering. LLMingua-2 (2024) improved on the original by training a dedicated BERT-scale classifier on GPT-4-distilled

data, framing compression as token classification rather than perplexity thresholding — this allows it to use bidirectional context and better preserve rare but critical tokens.

LLMLingua / LLMLingua-2

2023 / 2024

Authors	Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, Lili Qiu (Microsoft Research)
Aim	Drop low-information tokens from a long prompt to reduce the context fed to a large, expensive target LLM --- fewer tokens means faster decoding, smaller KV-cache, lower API cost
Complexity	One forward pass through a small proxy model to score all tokens; $O(n)$ time where n is prompt length
Strengths	Up to 20x compression achievable; no modification to target LLM; integrated into LlamaIndex; LLMLingua-2 achieves better recall of critical tokens via bidirectional context
Weaknesses	Target LLM must tolerate grammatically fragmented input; can drop rare high-perplexity tokens that are actually load-bearing facts; requires a proxy model running at serving time
Superseded by	AutoCompressors (2023, soft token approach); selective context summarization methods; KV-cache reuse systems like gisting

The key insight connecting LLMLingua to the rest of this book: dropping a token because a small model finds it predictable is exactly the compression move we have seen everywhere — **a good prediction makes a cheap code**. Here the “code” is zero bits (omission), and the “decoder” is the target LLM, which fills in the predictable word implicitly from context. This is lossy compression of the prompt, with the target model acting as a soft decoder.

KEY IDEA

LLMLingua exploits cross-entropy as a measure of information content: a token that the proxy model assigns high probability to carries little new information beyond what came before. Dropping it is the same bet we make in LZ77 when we replace a repeated string with a back-reference — we bet the decoder can reconstruct it from context.

10.6 Product Quantization: Compressing Vectors for Retrieval

So far we have been inside the language model. Now we zoom out to a different but related problem: storing and searching enormous collections of **embedding vectors** efficiently.

10.6.1 The Embedding Problem

Modern retrieval systems — the “R” in RAG (Retrieval-Augmented Generation) — work like this: convert every document chunk into a **dense vector embedding** produced by a neural model. Store millions of these vectors. At query time, convert the query into a vector of the same kind, then find the nearest stored vectors (approximate nearest-neighbour search, ANN).

A typical embedding might have $d = 768$ or $d = 1536$ floating-point dimensions. In `float32`, one vector costs $768 \times 4 = 3,072$ bytes. A million vectors: 3 GB. A billion vectors: 3 TB. That is too large for fast in-memory search.

Product Quantization (PQ) — introduced by Jégou, Douze, and Schmid in their landmark 2011 IEEE TPAMI paper “Product Quantization for Nearest Neighbor Search” — is the solution that almost every large-scale vector database uses today.

10.6.2 The PQ Idea: Divide and Conquer

PQ splits each d -dimensional vector into M non-overlapping sub-vectors of dimension $\frac{d}{M}$ each, then separately quantizes each sub-vector.

Each sub-vector is quantized by finding the nearest of $K = 256$ **centroids** (cluster centers) learned from training data by running k-means on that subspace. The result is a single byte (one of 256 options) per sub-vector. So a 768-dimensional vector with $M = 96$ sub-vectors of dimension 8 each becomes 96 bytes — a **32x compression** from `float32`.

Σ GO FURTHER · THE MATHS — K-means clustering

skip if familiar

K-means is an algorithm for grouping n data points into K clusters. Starting with K random centroids (representative points), it alternates between:

1. Assign each data point to its nearest centroid.
2. Move each centroid to the mean of its assigned points.

Repeat until assignments stop changing. The result: K cluster centers that best “summarize” the data.

Why 256 centroids? Because $2^8 = 256$, so each centroid index fits in exactly one byte. With 8-bit codes, each sub-vector’s centroid assignment is one byte, and all 256 centroids for one sub-space fit in a tiny lookup table. The training cost is $O(ndK)$ for n vectors.

10.6.3 A Worked Numeric Example

Suppose we have 4-dimensional vectors and split into $M = 2$ sub-vectors of dimension 2. (In practice $M = 64$ or $M = 96$; we use 2 for clarity.)

Training: run k-means on sub-space 1 (dimensions 1–2) and sub-space 2 (dimensions 3–4) separately.

- Codebook 1: centroid A = (0.1, 0.2), centroid B = (0.9, 0.8), centroid C = (0.5, 0.5) (256 in practice; 3 here).

- Codebook 2: centroid $X = (0.0, 1.0)$, centroid $Y = (1.0, 0.0)$, centroid $Z = (0.5, 0.5)$.

Encoding vector $v = (0.08, 0.19, 0.95, 0.12)$:

- Sub-vector 1: $(0.08, 0.19)$. Distances: to $A = \sqrt{(0.08 - 0.1)^2 + (0.19 - 0.2)^2} \approx 0.022$; to $B \approx 1.14$; to $C \approx 0.46$. Nearest: **A**. Code: byte 0.
- Sub-vector 2: $(0.95, 0.12)$. Distances: to $X \approx 1.29$; to $Y = \sqrt{0.0025 + 0.0144} \approx 0.13$; to $Z \approx 0.66$. Nearest: **Y**. Code: byte 1.

Stored representation: 2 bytes instead of 4 floats (16 bytes). **8x compression** for this toy example.

Fast distance computation: to compare query q against all stored vectors, pre-compute **distance tables**: for each sub-space m , the distance from the corresponding sub-vector of q to each of the 256 centroids. Then the approximate distance to any stored vector is the sum of M table lookups — one memory access per sub-space. For a billion vectors and $M = 96$, this is 96 billion lookups, but each lookup is a single array read — far faster than computing 768 floating-point distances.

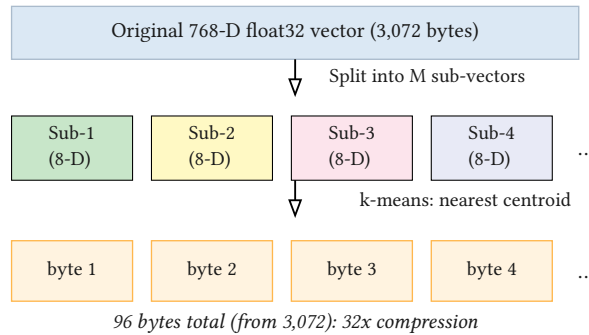


Figure 25: Product quantization: split the vector into M sub-vectors, find the nearest centroid in each sub-space, store only the centroid indices.

10.6.4 OPQ: Optimized Product Quantization

Standard PQ splits the vector at fixed dimension boundaries, ignoring correlations between sub-spaces. Optimized Product Quantization (OPQ, Ge, He, Ke & Sun, IEEE TPAMI 2013–14) adds a learned rotation before splitting: find the rotation matrix $\mathbf{R}$ such that the quantization error after PQ on $\mathbf{R}v$ is minimized. Equivalently, rotate the embedding space so variance is evenly distributed across sub-spaces before the fixed splits.

The rotation is learned iteratively — alternating between optimizing the PQ codebooks for a fixed $\mathbf{R}$ and optimizing $\mathbf{R}$ for fixed codebooks — on a sample of training vectors. At query time, the query is also rotated by $\mathbf{R}$ before the lookup-table distance computation.

OPQ typically gives 10–20% better recall than vanilla PQ at the same compression ratio, at the cost of one matrix–vector product per query. It is available in FAISS as `IndexIVFOPQ`.

Product Quantization (PQ) / Optimized PQ (OPQ)^{2011 (PQ) / 2013--14 (OPQ)}

Authors	Hervé Jégou, Matthijs Douze, Cordelia Schmid (INRIA, France); OPQ: Tiezheng Ge, Kaiming He, Qifa Ke, Jian Sun (Microsoft Research)
Aim	Compress high-dimensional float vectors to a few bytes, while enabling fast approximate distance computation in the compressed domain via lookup tables
Complexity	Training: $\mathcal{O}(n \cdot d \cdot K \cdot M)$ for n vectors; Encoding: $\mathcal{O}(d \cdot K)$ per vector; Query: $\mathcal{O}(n \cdot M)$ per query via table lookups, much faster than $\mathcal{O}(n \cdot d)$ exact search
Strengths	32x or greater memory compression; fast lookup-table distance computation; FAISS (Meta AI, 2017) makes billion-scale PQ search accessible; widely deployed in production vector databases
Weaknesses	Lossy --- approximate distances, so some true nearest neighbours are missed (recall typically 80--95%); sensitive to sub-space dimension choice; OPQ training cost scales with corpus size
Superseded by	ScaNN (Google, 2020) for top-recall scenarios; HNSW (graph-based index) for high-recall; neural quantizers like Qinc2 (arXiv:2501.03078, 2025) for better recall at same compression; PQ remains the default in FAISS for high-compression regimes

PQ was originally proposed for image retrieval — finding similar images in billion-image databases, at a time when deep embeddings did not yet exist. It transferred seamlessly to the neural-embedding era. FAISS, the open-source library Meta AI built around PQ (released 2017), now underpins vector search in most major LLM applications. Kaiming He, a co-author of OPQ, later co-invented ResNet (2015) and is now one of the most-cited researchers in computer vision.

10.7 Binary and Matryoshka Embeddings

Beyond PQ, two other compression strategies for embeddings have become practically important in 2024–2026.

10.7.1 Binary Quantization: 1 Bit per Dimension

Binary quantization is the most aggressive form of scalar quantization: map each floating-point coordinate to a single bit (positive $\rightarrow$ 1, negative $\rightarrow$ 0). A 768-dimensional `float32` vector (3,072 bytes) becomes a 768-bit binary vector — 96 bytes, a **32x compression**.

Approximate distances in the binary domain are computed with the **Hamming distance**: the number of bit positions where two binary vectors differ. Modern CPUs can

compare 64 bits in a single instruction using the hardware `POPCNT` instruction, making binary search extremely fast.

The accuracy cost is surprisingly small for modern embedding models. Research from Hugging Face and the MTEB (Massive Text Embedding Benchmark) leaderboard in 2024 shows that binary quantization of state-of-the-art sentence embeddings retains 85–95% of the retrieval recall of the full-precision vectors. FAISS, ScaNN, and virtually every production vector database support binary search as a first-pass filter (retrieve the top 10x candidates, then re-rank with full precision).

PY GO FURTHER · PYTHON — Simulating binary quantization and Hamming distance *skip if fluent*

Here is a tiny simulation showing how to binary-quantize a float embedding and compute Hamming distance.

```
def binary_quantize(vec: list[float]) -> bytes:
    """Convert a float vector to a packed binary vector (1 bit
    per dim)."""
    n_bytes = (len(vec) + 7) // 8
    result = bytearray(n_bytes)
    for i, x in enumerate(vec):
        if x >= 0.0:
            result[i // 8] |= (1 << (i % 8))
    return bytes(result)

def hamming_distance(a: bytes, b: bytes) -> int:
    """Count differing bits between two packed binary
    vectors."""
    return sum(bin(x ^ y).count("1") for x, y in zip(a, b))

# Demo with 8-dimensional toy vectors
v1 = [0.3, -0.1, 0.8, -0.5, 0.2, 0.9, -0.4, 0.6]
v2 = [0.2, -0.3, 0.7, -0.1, 0.1, 0.8, -0.6, 0.5]

b1 = binary_quantize(v1)  # 1 byte encodes all 8 dimensions
b2 = binary_quantize(v2)

print(f"v1 signs: {[1 if x>=0 else 0 for x in v1]}")
print(f"v2 signs: {[1 if x>=0 else 0 for x in v2]}")
print(f"Hamming distance: {hamming_distance(b1, b2)}")
# → 0 (signs agree on all 8 dims in this example)

# A vector where the signs differ:
v3 = [-0.2, 0.1, 0.8, -0.5, 0.2, -0.9, -0.4, 0.6]
b3 = binary_quantize(v3)
```

```
print(f"Hamming(v1, v3): {hamming_distance(b1, b3)}")
# → 2 (dims 0 and 5 flipped)
```

In production, numpy does this as `(vec >= 0).view(np.uint8)` packed with `numpy.packbits`, and FAISS handles the bitpacking and POPCNT distance computation internally. Binary search can be 100x faster than float dot-product at billion scale.

10.7.2 Matryoshka Representation Learning

Binary quantization reduces precision. What if instead we reduce **dimensionality**? Matryoshka Representation Learning (MRL, Kusupati et al., NeurIPS 2022) trains embeddings so that the **first k dimensions already form a meaningful representation** for any prefix length $k \in \{8, 16, 32, 64, 128, 256, 512, 1024\}$.

The training trick: optimize the standard retrieval loss simultaneously at **multiple** output sizes, front-loading the most important information into the earliest dimensions. The name “Matryoshka” refers to Russian nesting dolls — each shorter prefix is a valid, useful embedding nested inside the larger one.

Aside. OpenAI’s `text-embedding-3-small` and `text-embedding-3-large` models, released in early 2024, explicitly support MRL-style truncation via an API `dimensions` parameter — the first production embedding models to ship with built-in multi-resolution support. Truncating from 3072 to 256 dimensions costs roughly 1% recall on standard benchmarks.

10.7.3 The Combination: MRL + Binary

The real leverage comes from combining both techniques:

1. Take a 1024-dimensional embedding.
2. Truncate to 128 dimensions (MRL). Memory: $128 \times 4 = 512$ bytes.
3. Binary-quantize the 128 dimensions. Memory: $\frac{128}{8} = 16$ bytes.

Total compression: $1024 \times \frac{4}{16} = 256 \times$ relative to the original `float32` 1024-D vector, while retaining roughly 90% of retrieval recall on standard benchmarks (per Vespa’s published evaluation, 2024). This combination is the default configuration for large-scale retrieval in cost-sensitive deployments.

SCOREBOARD — KV-cache and embedding compression: representative savings (illustrative figures)

Technique	Bytes	Ratio	Notes
Full BFloat16 MHA KV-cache	512 KB/tok	1.0x	Baseline: 32 layers, 32 heads, 128 dim/head
H2O eviction (20% budget)	102 KB/tok	5.0x	Evicts 80%; some loss on long-range tasks

Technique	Bytes	Ratio	Notes
4-bit KV quantization	128 KB/tok	4.0x	PolarQuant / TurboQuant rotation + INT4
MLA (DeepSeek-V3 style)	37 KB/tok	13.8x	Low-rank latent; 93% reduction vs. MHA
Float32 768-D embedding	3,072 bytes	1.0x	Baseline embedding for retrieval
PQ (M=96 sub-spaces, K=256)	96 bytes	32x	80–95% ANN recall; FAISS default
Binary quantization 768-D	96 bytes	32x	Fast Hamming; 85–95% recall
MRL 128-D + binary	16 bytes	192x	90% recall; Vespa / FAISS deployment

10.8 Putting It All Together: The Inference Memory Stack

An LLM inference system in 2025–2026 stacks these techniques in layers:

Architecture level: choose MLA or GQA at training time. This is the highest-leverage intervention — a 93% reduction before any further tricks.

Quantization level: store the KV-cache in 4 bits with rotation-based quantization (PolarQuant, TurboQuant). Independent of the architecture choice; roughly a 4x further reduction.

Eviction level: for very long contexts, combine with SnapKV or H2O-style policies. The three levels compose multiplicatively.

Prompt level: for RAG and multi-turn chat, use LLMingua-style prompt compression to reduce the number of tokens that ever enter the model.

Retrieval level: store document embeddings in PQ or binary-quantized form in FAISS or a vector database; the ANN index fits in RAM rather than requiring expensive GPU memory.

The result: a system that can serve 128K-token contexts on hardware that could barely handle 4K tokens two years earlier, with quality that is, in most practical tasks, indistinguishable from the uncompressed version.

Myth. “Compressing the KV-cache is just quantization — the same as compressing model weights.”

Reality. KV-cache compression and weight quantization share the same underlying arithmetic, but the target and the constraints differ. Weights are fixed after training; the cache grows dynamically with each new token and must be compressed

in real time, with zero budget for expensive calibration passes. Weight quantization can afford one-shot calibration (GPTQ uses Hessian inverses computed over a calibration set). KV-cache quantization must be **online** — each new key/value vector is quantized the instant it is computed. This is why rotation-based methods (PolarQuant, TurboQuant) matter: the rotation can be pre-applied as a cheap matrix multiply, making the subsequent per-token scalar quantization trivially fast.

PY GO FURTHER · PYTHON — A minimal KV-cache size estimator*skip if fluent*

This function computes KV-cache memory (in bytes) for a standard MHA model and for an MLA model, so you can see the difference concretely.

```
def kv_cache_bytes_mha(
    n_layers: int,
    n_heads: int,
    head_dim: int,
    n_tokens: int,
    bytes_per_float: int = 2,  # bfloat16
) -> int:
    """Estimate KV-cache bytes for standard multi-head
    attention."""
    # Factor of 2: one K matrix and one V matrix per layer
    return 2 * n_layers * n_heads * head_dim * n_tokens *
    bytes_per_float

def kv_cache_bytes_mla(
    n_layers: int,
    latent_dim: int,  # d_c, the compressed latent size
    decoupled_dim: int,  # small positional key kept separately
    n_tokens: int,
    bytes_per_float: int = 2,
) -> int:
    """Estimate KV-cache bytes for MLA (DeepSeek-V2/V3
    style)."""
    # Only the latent vector + small decoupled positional key
    return n_layers * (latent_dim + decoupled_dim) * n_tokens *
    bytes_per_float

# Approximate DeepSeek-V3 config
mha = kv_cache_bytes_mha(
    n_layers=61, n_heads=128, head_dim=128, n_tokens=10_000
)
mla = kv_cache_bytes_mla(
    n_layers=61, latent_dim=512, decoupled_dim=64,
```

```

n_tokens=10_000
)

print(f"MHA KV-cache at 10K tokens: {mha / 1e9:.2f} GB")
print(f"MLA KV-cache at 10K tokens: {mla / 1e9:.2f} GB")
print(f"Reduction: {mha / mla:.1f}x")
# MHA KV-cache at 10K tokens: 40.31 GB
# MLA KV-cache at 10K tokens: 0.70 GB
# Reduction: 57.6x

```

The numbers are somewhat larger than DeepSeek's reported 14x because DeepSeek uses GQA as the baseline for their comparison, not full MHA. But the principle is clear: only `latent_dim + decoupled_dim` numbers are cached per token per layer, instead of `2 * n_heads * head_dim`.

Checkpoint. In product quantization with $M = 64$ sub-spaces and $K = 256$ centroids per sub-space, how many bytes does one encoded vector occupy? How many entries are in the distance lookup table for a single query?¹⁶

Chapter takeaways

- The KV-cache stores pre-computed keys and values for every past token, enabling fast attention at the cost of memory that grows linearly with context length --- at 512 KB per token for a large MHA model, this dominates inference memory
- H2O eviction retains 'heavy hitter' tokens (highest cumulative attention scores) and recent tokens, evicting everything else once a budget is hit; 29x throughput reported on OPT-30B at 20% budget
- SnapKV uses the last few prompt tokens as an observation window to identify important cache positions before generation begins, enabling 3.6x speedup with near-zero quality loss
- DeepSeek's MLA trains a joint low-rank compression of all heads' keys and values into a single latent vector, achieving approximately 93% KV-cache reduction as a first-class architectural choice
- KV-cache quantization is complicated by outlier values; random rotation pre-conditioning (PolarQuant, TurboQuant) redistributes variance uniformly, enabling accurate 3--4-bit scalar quantization
- LLMingua uses a small proxy model's perplexity scores to drop low-information tokens from prompts, compressing context length by up to 20x before tokens ever enter the model

¹⁶**Checkpoint answer.** Exactly 64 bytes: one byte per sub-space (since $K = 256 = 2^8$ centroids, the index fits in 8 bits). The lookup table has $M \times K = 64 \times 256 = 16,384$ entries — computed once per query from the query sub-vectors and then read once per stored vector.

- Product quantization splits an embedding into M sub-vectors, codes each sub-vector as a byte-sized centroid index, and enables fast approximate nearest-neighbour search via M table lookups per query; 32x compression typical
- Binary quantization (1 bit per dimension) and Matryoshka truncation each give approximately 32x compression; combined they reach ~192x while retaining about 90% retrieval recall --- the standard for cost-sensitive deployment in 2025--2026

10.9 Exercises

Exercise 65.1 ★

A model has 40 attention layers, 40 heads per layer, and 128 dimensions per head. Weights are stored in BFloat16 (2 bytes each). Calculate the KV-cache size for a 32,000-token context. Express your answer in gigabytes.

Exercise 65.2 ★

Explain in your own words why H2O keeps **both** heavy hitters and recent tokens, rather than keeping only heavy hitters.

Exercise 65.3 ★★

A 512-dimensional embedding is compressed with PQ using $M = 64$ sub-spaces (dimension 8 each) and $K = 256$ centroids per sub-space. (a) How many bytes does the compressed vector occupy? (b) How many floating-point numbers are stored in all 64 codebooks combined? (c) What is the compression ratio versus the original float32 vector?

Exercise 65.4 ★★

MLA's low-rank latent has dimension $d_c = 512$ while the full key-value cache for the same model (128 heads, head dimension 128) has $2 \times 128 \times 128 = 32,768$ floats per layer per token. (a) What is the compression ratio? (b) MLA requires a matrix multiplication of shape $512 \times 32,768$ to expand the latent to keys/values at each attention step. Why does this matrix not add to the **cache** memory?

Exercise 65.5 ★★

LLMLingua uses a small proxy model to identify low-perplexity tokens for removal. One token has perplexity 1.2 under the proxy model; another has perplexity 85. (a) Which is the better candidate for removal? (b) Explain why, connecting to the information-theoretic definition of perplexity from Chapter 18. (c) Give one type of token that might have low perplexity but still be critical to keep.

Exercise 65.6 ★★★

Design a combined KV-cache compression scheme for a 32-layer, 32-head, 128-dim/

head transformer serving a 64,000-token context. Target: fit the cache in 4 GB of GPU memory. (a) Calculate the uncompressed BFloat16 cache size. (b) Choose which combination of MLA, quantization, and/or eviction reaches the target, showing your arithmetic. (c) Identify one quality risk of your chosen scheme and how you would validate it.

10.10 Further Reading

- Zhang et al. (2023). **H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models**. *NeurIPS 2023*. The original H2O paper; includes the submodular formalization and ablations across OPT, LLaMA, and GPT-NeoX.
- Li et al. (2024). **SnapKV: LLM Knows What You are Looking for Before Generation**. *NeurIPS 2024*. Observation-window pooling for smarter prefix compression; 3.6x speedup with near-zero quality cost.
- DeepSeek-AI (2024). **DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model**. *arXiv:2405.04434*. MLA is introduced and analyzed here; the appendix derives the absorb-form optimization.
- DeepSeek-AI (2024). **DeepSeek-V3 Technical Report**. *arXiv:2412.19437*. Scaling MLA to 671B parameters with full open weights; reports the 70 KB/token figure.
- Google Research (2026). **TurboQuant: Online Vector Quantization for KV-Cache Compression**. *ICLR 2026*. 3-bit KV-cache via rotation and online VQ; companion to PolarQuant.
- PolarQuant authors (2025). **Quantizing KV Caches with Polar Transformation**. *AISTATS 2026*. Rotation-based outlier taming for key-cache quantization.
- Jiang et al. (2023). **LLMLingua: Innovating LLM Efficiency with Prompt Compression**. *EMNLP 2023*. First system to systematically drop low-perplexity prompt tokens; up to 20x compression.
- Liang et al. (2024). **A Survey on Efficient Inference for Large Language Models**. *arXiv:2407.18003*. — in `papers/liang-2024-kvcache-survey.pdf`. Comprehensive survey covering eviction, quantization, speculative decoding, and architectural approaches.
- Jégou, Douze & Schmid (2011). **Product Quantization for Nearest Neighbor Search**. *IEEE TPAMI*. The paper that established PQ as the standard technique for large-scale ANN search.
- Ge, He, Ke & Sun (2013–14). **Optimized Product Quantization**. *IEEE TPAMI*. The rotation-optimization extension of PQ.
- HuggingFace (2024). **Introduction to Matryoshka Embedding Models**. Matryoshka training and truncation, with practical guidance.

COMING UP NEXT

We have now seen every major compression technique for the **serving** side of modern AI: how to shrink what the model stores during inference (KV-cache) and how to shrink what retrieval systems store (embeddings). The next chapter steps out of the AI world entirely and into the universe of **scientific data** — the outputs of climate simulations, cosmological N-body codes, and fluid dynamics solvers. These petabyte-scale floating-point arrays present a completely different compression challenge: the user does not care about exact reconstruction, but they care deeply about **error bounds** — and they need random access into the compressed data. Chapter 66 introduces zfp and SZ, the two dominant error-bounded lossy compressors for HPC, and explains why a DCT-like orthogonal transform is the right tool for decorrelating simulation data.

Solutions to Exercises

Solution 56.1. Weighted sum for $(1, 3, 4)$: $2(1) + (-1)(3) + 0.5(4) + (-1) = 2 - 3 + 2 - 1 = 0$. ReLU of 0 is 0. For $(0, 0, 0)$: the weighted sum is $0 + 0 + 0 + (-1) = -1$; ReLU clamps negatives to 0, so the output is 0 again. (Both happen to be zero — a reminder that ReLU is “off” for any input whose weighted sum lands at or below zero.)

Solution 56.2. Two linear layers compute $W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2)$, which is itself one linear map $W'x + b'$ — no gain. The squash is **nonlinear**, so it cannot be absorbed into the surrounding matrices; it lets each layer bend the input in a way the next layer can build on, which is what gives depth its power.

Solution 56.3. Update rule: $w \leftarrow w - \eta(\partial L)/(\partial w) = 3 - 0.1 \times 4 = 3 - 0.4 = 2.6$. The weight **decreased**. The gradient is positive, meaning L rises as w rises; to **reduce** L we must move w the **opposite** way — downhill — which is exactly what subtracting $\eta \times$ gradient does. The learning rate η controls how big the step is.

Solution 56.4. As $\lambda \rightarrow \infty$ the distortion term dominates, so the optimizer cares almost only about reconstruction quality and will spend bits freely: **large files, faithful (low-distortion) reconstructions** — the high-rate, low-distortion end of the curve. As $\lambda \rightarrow 0$ the rate term dominates and the optimizer minimizes file size at the expense of quality: **tiny files, blurry/high-distortion reconstructions** — the low-rate end. Sweeping λ from large to small traces the codec’s operating points along the rate–distortion frontier of Chapter 21, each λ picking one point.

Solution 56.5. Local slopes: $(da)/(dx) = 2$; $(db)/(da) = 2a$; $(dL)/(db) = 1$. Multiply along the chain: $(dL)/(dx) = (dL)/(db) \cdot (db)/(da) \cdot (da)/(dx) = 1 \cdot 2a \cdot 2 = 4a = 4(2x) = 8x$. At $x = 3$: $8 \times 3 = 24$. (Backpropagation is just this multiplication of local slopes, done backward and reused across all weights.)

Solution 56.6. Rounding $\hat{y} = \lfloor y \rfloor$ is a staircase: flat between half-integers (slope 0 everywhere it is defined) with undefined jumps. A zero gradient tells backpropagation “changing y doesn’t change the output,” so no learning signal flows through the quantizer — training stalls. The fix replaces hard rounding, **during training only**, with adding uniform noise u drawn evenly from $(-1/2, +1/2)$: $\hat{y} \approx y + u$. This is smooth (gradients flow) and its spread of $\pm 1/2$ matches the actual error that real rounding introduces (a value can be off by up to half a step), so the network trains against a faithful, differentiable stand-in for quantization. At test time you switch back to true rounding.

Solution 56.7. Change the data generator to `data = np.hstack([a + 0.05*rng.normal(size=(2000,1)) for _ in range(3)])` and call `train_autoencoder(data, latent=1, eta=0.2)` — a **smaller** learning rate, because

with three inputs the original $\eta = 0.5$ takes steps so large the loss overflows to infinity (a vivid demonstration of the Pitfall above: too big a step and gradient descent diverges instead of descending). **Prediction:** since all three pixels share the same underlying value a , the smart code is again their average, so the learned encoder row should come out roughly proportional to $(1, 1, 1)$ (each pixel contributing equally). **Compression ratio:** three input numbers are represented by one latent number, a 3 : 1 ratio. **Why MSE stays small:** the three pixels are nearly identical, so one number (their shared value) reconstructs all three almost perfectly; the only error is the tiny per-pixel noise the bottleneck cannot recover. This is the same principle as transform coding (Chapter 38): concentrate the energy into few coefficients and drop the rest.

Solution 57.1. During training the network learns by gradient descent, which needs a non-zero slope to send a learning signal back through every step. Rounding has zero slope almost everywhere, so it blocks the signal and the encoder cannot learn. Adding uniform $(-\frac{1}{2}, \frac{1}{2})$ noise reproduces the same statistical jitter quantization would cause but has a clean slope of 1, so gradients flow. At test time the codec rounds honestly to integers and entropy-codes them — there is no learning happening, so no gradient is needed, and exact integers are required for the entropy coder to be reversible. Using random noise at test time would make the output non-deterministic and the values non-integer, so encoder and decoder could not agree and the file could not be decoded correctly.

Solution 57.2. At $\sigma = 16$: $\log_2 16 + 2.05 = 4 + 2.05 = 6.05$ bits. At $\sigma = 4$: $\log_2 4 + 2.05 = 2 + 2.05 = 4.05$ bits. The main saving is $6.05 - 4.05 = 2.0$ bits per value. Subtract the 0.3 side bits: net saving $2.0 - 0.3 = 1.7$ bits per value. (Note the saving is just $\log_2(\frac{16}{4}) = 2$ bits — halving the scale twice saves two bits, before paying for the side info.)

Solution 57.3. By Chapter 18, $-\log_2 p(s)$ is the surprisal: the bits an ideal coder spends on symbol s under model p . It is small exactly when $p(s)$ is large — when the model *expected* that symbol, i.e. predicted it well. Averaging over the latent gives the cross-entropy $H(q, p)$. By the theorem in this chapter, $H(q, p) = H(q) + D_{\text{KL}}(q \parallel p)$, so minimizing the rate (with the fixed true entropy $H(q)$ as a floor) minimizes the KL-divergence $D_{\text{KL}}(q \parallel p)$ between the model and the true distribution. Its smallest possible value is 0, attained exactly when $p = q$ — a perfect predictor, the cheapest possible code. This is Chapter 23's thesis made literal: better prediction *is* fewer bits.

Solution 57.4. The spatial autoregressive model conditions each latent *position* on its already-decoded spatial neighbours (above and to the left). Because position n needs position $n - 1$, decoding is strictly serial — hundreds of thousands of sequential steps for one image. The channel-wise model instead splits the latent's *channels* into a few groups and conditions each group on the previously-decoded groups, not on spatial neighbours. Within a group, every spatial position is conditionally independent, so they all decode *in parallel*; only the small number of groups (a handful) must run in sequence. You trade a little compression for a colossal speed-up: a few parallel passes instead of a per-element serial crawl.

Solution 57.5. The hyperprior encodes a compressed *description of the main latent's local statistics* — for each region (or element) the parameters of its Gaussian, the scale σ

and often the mean μ — not any image content. It is small because there are far fewer statistics to describe than there are latent values to code (one scale can govern a whole neighbourhood), and the statistics are themselves smooth and predictable. Analogy: the main latent is the dense detail drawn all over a map; the hyperprior is the tiny *legend* in the corner that tells you how to read the symbols — short, but it makes the whole map cheaper to interpret.

Solution 57.6.

```
import math
def gauss_pmf(value, mu, sigma):
    sigma = max(sigma, 1e-6); z = (value - mu) / sigma
    h = math.exp(-0.5*z*z) / (sigma*math.sqrt(2*math.pi))
    return min(max(h, 1e-12), 1.0)

def total_loss(orig, recon, mus, sigmas, lam):
    n = len(orig)
    D = sum((o - r)**2 for o, r in zip(orig, recon)) / n      # MSE
    R = sum(-math.log2(gauss_pmf(r, mu, sg))
            for r, mu, sg in zip(recon, mus, sigmas))         # bits
    return R + lam * D
```

As `lam` grows large, the distortion term dominates: the optimizer prefers reconstructions that minimize MSE no matter the bit cost — high quality, large files (the high end of the rate–distortion curve). As `lam` shrinks toward zero, bits dominate: it prefers cheap latents even if the picture suffers — small files, lower quality. Sweeping `lam` traces out the rate–distortion curve, and in a real learned codec each `lam` trains a *different* network living at a different point on that curve.

Solution 57.7. The refutation rests on $H(q, p) \geq H(q)$, with equality iff $p = q$ (proved via $D_{\text{KL}} \geq 0$). The reported rate is the cross-entropy $H(q, p)$ of the true latent distribution q under the model p — and the *real* entropy coder also uses p , so it spends exactly those bits (up to coder overhead). The model cannot make $H(q, p)$ small while the real cost stays large, because they are the same quantity. Nor can the model fake a rate below $H(q)$: cross-entropy is bounded below by the true entropy, so the only way to drive the reported (hence real) rate down is to make p genuinely closer to q — to predict the latent better. “Grading its own homework” would be a problem only if minimizing the reported rate did *not* minimize the real bits; the theorem proves it does. The one honest caveat is the train/test gap: q at test time may differ slightly from the training distribution, so the model is honest but not omniscient.

Solution 58.1. At low bitrates, many pixels cannot be stored precisely — the decoder has uncertainty about each pixel’s true value. MSE is minimized by outputting the **expected value** (average) of all plausible pixel values, because averaging reduces the average squared error. When the true pixel could be anywhere from light to dark grey, outputting middle grey is mathematically optimal for MSE. Applied across millions of pixels, this

produces smooth, blurry averages instead of sharp textures — not a bug, but the correct solution to the wrong problem.

Solution 58.2. Consider a portrait compressed at 0.05 bpp. **Scenario A:** The MSE-optimal decoder outputs the pixel-average face — correct mean color, low MSE, but indistinct blur for every feature. **Scenario B:** A GAN decoder invents sharp-looking eyes, crisp hair, realistic pores — high MSE because each invented pixel differs from the original, but humans rate it as “photographic.” The GAN output has higher distortion (more MSE) but better perceptual quality (looks more like a real face). Blau–Michaeli predicts exactly this: at this bitrate, you cannot have both.

Solution 58.3. With $\lambda_{\text{dist}} = 1.0$ (large), the distortion term dominates. The decoder would be trained primarily to minimize pixel MSE, producing blurry but pixel-accurate reconstructions — high PSNR, poor perceptual quality (similar to a standard MSE codec). The GAN term with $\lambda_{\text{GAN}} = 0.075$ would be too weak to force realistic texture generation. The perception–distortion tradeoff implies that pushing toward low distortion pulls you away from high perceptual quality: by weighting distortion heavily, we slide along the tradeoff curve toward the “low D, low perception” end, not the “high perception” end.

Solution 58.4. (a) $0.003 \times 512 \times 512 = 786.432$ bits, round to 786 bits. (b) $\frac{786}{8} \approx 98$ bytes. (c) With $K = 1024$ codebook entries, each index takes $\log_2 1024 = 10$ bits. A 16×16 grid has 256 latent positions, so total bits = $256 \times 10 = 2560$ bits. That is larger than 786 bits — indicating that either the grid is much smaller, the codebook is much smaller, or (more likely) entropy coding of the indices reduces the cost well below 10 bits each if the index distribution is skewed. Realistic PerCo systems use additional entropy coding that achieves average code lengths far below $\log_2 K$ for likely indices.

Solution 58.5. LPIPS (Learned Perceptual Image Patch Similarity) compares a specific reconstruction to its specific original by measuring distance in VGG/AlexNet feature space — it is a **reference-based**, per-image metric. It answers: “Does this reconstruction look like this original?” FID (Fréchet Inception Distance) compares the **distribution** of reconstructed images to the **distribution** of real images across an entire test set — it is a **distribution-level** metric. It answers: “Do all the reconstructions, taken together, look like real photographs?” For (a), LPIPS is appropriate. For (b), FID is appropriate. A generative codec can have poor LPIPS (each reconstruction differs from its original) but good FID (the set of reconstructions looks natural) — this is normal and expected.

Solution 58.6. Pros of perceptual compression in radiology: Storage cost reduction could be enormous (hospitals store millions of images); bandwidth savings for telemedicine; faster transfers. **Cons and dangers:** (1) Hallucinated detail is the core risk — a diffusion decoder could remove a real nodule or invent one, directly affecting diagnosis. (2) Medical imaging is regulated: DICOM standards and FDA guidelines require that archived images faithfully represent the original acquisition. (3) Legal liability — if a missed diagnosis is later linked to a perceptual codec altering an image, the liability would be severe. (4) Diffusion decoders are non-deterministic in stochastic sampling modes; bit-exact reproducibility (required for medicolegal purposes) requires additional engineering. **Conclusion:** Perceptual codecs are contraindicated for archival radiology

images. Lossless or tightly controlled lossy codecs (e.g., JPEG 2000 at 10:1 with bounded error, as permitted in some DICOM profiles) are appropriate. Perceptual codecs might be acceptable for **preview thumbnails** in a radiology workflow system, never for the primary archived image.

Solution 58.7.

```
import math

def rdp_curve(
    lambdas: list[float],
    source_variance: float = 1.0,
) -> list[tuple[float, float, float]]:
    """
    Compute toy rate-distortion-perception tradeoff curve for a
    Gaussian source.

    Each lambda controls the perception weight in the joint loss:
        L = rate + lam_dist * distortion + lambda * perception_loss

    Large lambda → minimize perception loss → generative, high
    distortion.
    Small lambda → minimize distortion → MSE codec, low perceptual
    quality.

    For a Gaussian source with variance  $\sigma^2$ , the rate-distortion
    function gives:
         $D = \sigma^2 * 2^{(-2R)}$  so  $R = -0.5 * \log_2(D / \sigma^2)$ 

    We model perception loss as how far the distortion is from "zero
    distortion"
    (perfect perception requires the reconstruction distribution to
    match source).
    For a Gaussian, perception_loss  $\approx D$  (simplified model).
    """
    results = []
    for lam in lambdas:
        # Trade off: higher lambda → accept more distortion for
        # lower perception loss.
        # Simplified model: equilibrium distortion =
        source_variance / (1 + 1/lam)
        # (As lam → ∞, distortion → source_variance; as lam → 0,
        # distortion → 0)
        distortion = source_variance / (1.0 + 1.0 / (lam + 1e-9))

        # Rate from rate-distortion function:  $R = 0.5 * \log_2(\sigma^2 / D)$ 
        ratio = source_variance / max(distortion, 1e-9)
```

```

bits = max(0.0, 0.5 * math.log2(ratio))

# Perception loss: how non-Gaussian is the reconstruction?
# Simplified: proportional to distortion (lower D = more
perceptual)
perception_loss = 1.0 / (1.0 + distortion) # 0 (worst) to 1
(perfect)

results.append((distortion, 1.0 - perception_loss, bits))

return results

# Example:
curve = rdp_curve(lambdas=[0.01, 0.1, 1.0, 10.0, 100.0],
source_variance=1.0)
print(" lambda | distortion | perception_loss | bits")
for lam, (d, p, r) in zip([0.01, 0.1, 1.0, 10.0, 100.0], curve):
    print(f" {lam:6.2f} | {d:.4f} | {p:.4f} | {r:.4f}")

```

Solution 59.1. (a) A block motion vector assigns a single displacement $(\Delta x, \Delta y)$ to the whole 16×16 block — every pixel in the block is assumed to have moved by the same amount. A dense optical flow field assigns a separate displacement $(\Delta x, \Delta y)$ to **every** pixel individually, capturing smoothly varying motion (e.g., a spinning object where each pixel moves differently), partial occlusions, and fine-grained motion boundaries that block vectors miss. (b) A dense flow field has $H \times W$ motion vectors rather than $\lceil \frac{H}{16} \rceil \times \lceil \frac{W}{16} \rceil$ block vectors — roughly $256 \times$ more data. This must be compressed by the motion codec. DVC handles this with a learned image codec applied to the 2D flow field; the flow field is smooth enough that the codec compresses it well, but the raw size is larger.

Solution 59.2. In DCVC, the encoder still transmits the full latent $\hat{y}_t$ of the current frame — it does not subtract anything. What changes is how those latents are **entropy coded**: the entropy model is conditioned on temporal context φ_t (feature maps from previously decoded frames), so it assigns very short codes to latent coefficients that the context predicts well, and longer codes to coefficients that are surprising. The “information about what changed” is implicitly carried by the latent coefficients that the entropy model **could not** predict — those are the ones that cost significant bits. The decoder reconstructs $\hat{x}_t$ from $\hat{y}_t$ and φ_t using the learned decoder network; the network has learned to combine current-frame latents with temporal context to produce a good reconstruction.

Solution 59.3. (a) In DVC residual coding: Y is the residual $r = x_t - \tilde{x}_t$ and X is the warp prediction $\tilde{x}_t$. The context X is used in pixel space before encoding — literally subtracted. The encoder then compresses r , which has no further access to X .

(b) In DCVC: Y is the full current-frame latent $\hat{y}_t$ and X is the temporal context φ_t (features from past decoded frames). The entropy model explicitly computes $p(\hat{y}_t \mid \varphi_t)$ and uses it for coding.

(c) Residual coding uses X in pixel space; if motion estimation is imperfect, the residual still contains large values. Conditional coding uses X in **latent** space, where the network has learned a much richer representation. The entropy model can condition on patterns (texture statistics, object-level features, scene semantics) that have no simple pixel-domain description but are highly predictive of what latent coefficients will look like — squeezing more from the same temporal context.

Solution 59.4.

```
def nerv_params(H: int, W: int, n_frames: int, hidden_dim: int) ->
int:
    # Embedding: n_frames vectors of size hidden_dim
    embed = n_frames * hidden_dim
    # Two 3x3 conv blocks (depth-preserving, hidden_dim ->
hidden_dim)
    conv_block = 2 * (hidden_dim * hidden_dim * 3 * 3)
    # Final 1x1 conv: hidden_dim -> 3
    final = hidden_dim * 3 * 1 * 1
    return embed + conv_block + final

# 720p, 100 frames, hidden_dim=128
params = nerv_params(720, 1280, 100, 128)
print(f"Parameters: {params:,}")
# embed = 100 * 128 = 12,800
# conv_block = 2 * 128*128*9 = 294,912
# final = 128*3 = 384
# total = 308,096

raw_bytes = 100 * 1280 * 720 * 3 # 3 bytes per pixel
print(f"Raw pixels: {raw_bytes:,} bytes")
# = 276,480,000 bytes ≈ 263 MB
```

Parameter count: 308,096. At 4 bytes each (float32) that is 1,232,384 bytes $\approx$ 1.2 MB. Raw pixel data: 276,480,000 bytes $\approx$ 263 MB. The network represents the video at roughly 1.2 MB/263 MB $\approx$ 0.45% of its raw size — before we have even applied weight quantization or entropy coding. Of course at this extreme compression the reconstruction quality will be poor; a real NeRV network uses many more upsampling stages and larger channels.

Solution 59.5. In a panning shot, the camera moves slowly left, so the content visible at the right edge of frame t is the content that was just off-screen to the right in frame $t - 1$, and everything else shifts one position to the left. In the latent space (a compressed grid of feature vectors), this manifests as a spatial **shift** of the entire feature map by a small amount. The transformer would need to learn: “when I see a sequence of latent grids where each one is the previous one shifted slightly in the same direction, the next one will be shifted by the same amount again.” This is like predicting the next step in a geometric sequence. The attention mechanism (see the transformer box earlier in this chapter) allows the transformer to look at multiple past frames simultaneously and infer

the shift direction and magnitude, then predict the corresponding shifted version of the previous latent as the most likely next latent. No explicit “optical flow” is needed — the transformer learns to do shift prediction as a special case of pattern continuation.

Solution 59.6. (a) The compute cost is worth it in scenarios where encoding happens **once** and the video is served **many times**: video-on-demand libraries (Netflix, YouTube), where a film is encoded once and watched by millions. A 21% bitrate saving on a film streamed 100 million times saves enormous CDN bandwidth cost. The GPU cost of encoding is amortized over all playback events. Also worth it: high-value, ultra-low-latency scenarios where quality per bit matters more than encoding hardware cost — 8K sports broadcasting, medical video archival.

(b) Not worth it for: live streaming (sports, gaming, video calls) where there is no time to encode offline and the GPU must be doing other work. Consumer cameras and smartphones where dedicated H.264/HEVC silicon encodes at zero marginal energy cost. Any context where the decoder is a classical hardware chip that cannot run neural inference — which is still most end-user devices in 2026.

(c) Changes that could shift the trade-off: (1) NPU silicon designed for DCVC-RT inference shipped in devices — once the decoder has dedicated hardware, energy and speed become competitive. (2) Standardization: if DCVC-RT (or a successor) becomes an ISO/IEC standard, browser and OS vendors implement it, and CDN pipelines support it. (3) Continued GPU/NPU efficiency improvements: at current improvement rates, the compute gap between neural and classical decoding narrows by roughly $2\times$ every 2–3 years.

Solution 60.1. (a) $\frac{24000}{320} = 75$ frames per second. (b) Each frame produces 6 tokens, each 10 bits ($\log_2 1024 = 10$): $75 \times 6 \times 10 = 4500$ bits/s = 4.5 kbps. (c) $4500 \times 0.75 = 3375$ bits/s ≈ 3.4 kbps.

Solution 60.2. Codebook collapse is when some codebook entries are selected for every (or almost every) encoder output, while other entries are never used. It happens because the training objective does not directly penalise unused entries — gradient descent just reinforces whatever is already working. DAC prevents it by periodic **random restart**: any entry that has not been chosen for a fixed number of steps is replaced by a randomly selected encoder output from the current minibatch, forcing it back into use.

Solution 60.3. Level 1: Distances from $(3.0, 1.5)$: $e_0: (2.0)^2 + (1.0)^2 = 5.0$; $e_1: (0.5)^2 + (0.5)^2 = 0.5 \leftarrow$ nearest; $e_2: (1.0)^2 + (0.5)^2 = 1.25$. So $k_1 = 1$, $\hat{z}_1 = (3.5, 2.0)$. Residual $r_1 = (3.0 - 3.5, 1.5 - 2.0) = (-0.5, -0.5)$.

Level 2: Distances from $(-0.5, -0.5)$: $e_0: (1.0)^2 + (1.0)^2 = 2.0$; $e_1: (0.0)^2 + (0.0)^2 = 0.0 \leftarrow$ exact match; $e_2: (1.3)^2 + (0.3)^2 = 1.78$. So $k_2 = 1$, $\hat{z}_2 = (-0.5, -0.5)$.

$(k_1, k_2) = (1, 1)$. $\hat{z} = (3.5 - 0.5, 2.0 - 0.5) = (3.0, 1.5)$. Error = 0 (exact reconstruction in this small example).

Solution 60.4. (a) The semantic model generates token stream 0 (or tokens from a self-supervised speech model), which encodes linguistic/content information. The acoustic

model generates streams 1 through $N_q - 1$, conditioned on stream 0, encoding voice quality and fine acoustic detail.

(b) Stream 0 alone is a much shorter sequence (or uses tokens that are more compressible because they have linguistic structure). The semantic model can be a smaller, faster autoregressive model. The acoustic model can generate all streams in parallel conditioned on stream 0, avoiding the slow sequential generation for those streams. A single model generating all streams autoregressively would face a much longer sequence and harder joint distribution.

(c) VALL-E’s acoustic model is conditioned on: the text transcript (encoded as text tokens), the first RVQ stream generated by the autoregressive semantic model, and a three-second reference audio clip of the target speaker (also as EnCodec tokens).

Solution 60.5. (a) The function $k = \operatorname{argmin}_k \|z - e_k\|_2$ outputs an integer index. Changing z slightly does not change k until z crosses the boundary between two Voronoi cells, at which point k jumps discontinuously. A function that is constant except at isolated jump points has zero gradient almost everywhere — there is no smooth slope for gradient descent to follow.

(b) Forward pass: the quantized vector $\hat{z} = e_{k^*}$ is used normally in the computation. Backward pass: the gradient $\partial_{\hat{z}} \mathcal{L}$ is passed directly to z as if the quantization were the identity — “pass straight through.” The codebook entries themselves are not updated by backpropagation; they are updated by EMA.

(c) The commitment loss adds a term $\beta \|\operatorname{sg}(e_{k^*}) - z\|_2^2$ to the training loss, where “sg” means “stop gradient.” This pushes z (the encoder output) toward the chosen codebook entry e_{k^*} , preventing the encoder from producing outputs that drift far from any codebook entry, which would destabilise training.

Solution 60.6.

```
import math

def bitrate_from_rvq(n_q: int, K: int, frame_rate: float) -> float:
    """Raw bits per second for an RVQ codec (before entropy
    coding)."""
    bits_per_frame = n_q * math.log2(K)
    return frame_rate * bits_per_frame

def min_codebook_bits(target_bps: float,
                      frame_rate: float,
                      n_q: int) -> int:
    """Minimum codebook size K (power of 2) to reach target_bps."""
    bits_per_frame_needed = target_bps / frame_rate
    bits_per_code = bits_per_frame_needed / n_q
    K_exact = 2 ** bits_per_code
    # Round up to next power of 2
```

```

K_min = 2 ** math.ceil(math.log2(K_exact))
return K_min

# SoundStream: 24000 / 320 = 75 fps, n_q=8, K=1024
ss_rate = bitrate_from_rvq(8, 1024, 75.0)
print(f"SoundStream raw bitrate: {ss_rate:.0f} bps =
{ss_rate/1000:.1f} kbps")
# → 6000 bps = 6.0 kbps

# EnCodec with 25% entropy saving: effective rate ~6 kbps from 8
kbps raw
# (8 kbps raw × 0.75 = 6 kbps after entropy coding)
ec_raw = bitrate_from_rvq(8, 1024, 75.0)
ec_effective = ec_raw * 0.75
print(f"EnCodec effective bitrate: {ec_effective:.0f} bps =
{ec_effective/1000:.1f} kbps")
# → 4500 bps = 4.5 kbps (illustrative; actual EnCodec configuration
varies)

# Minimum K to hit 12 kbps with n_q=4, 75 fps
K = min_codebook_bits(12000.0, 75.0, 4)
print(f"Minimum K for 12 kbps, n_q=4: {K}") # → K = 16 (4 bits
each)

```

Solution 61.1. The memorizer’s “model” is the answer key — a description as long as the data it explains, so $L(M) \approx L(D)$ and the total description $L(M) + L(D|M)$ is no shorter than storing the raw data: zero compression. The rule-learner’s model is one short rule, $L(M) \ll L(D)$, and once you know the rule the data’s residual surprise $L(D|M)$ is small too, so the total is far shorter: real compression. On a fresh exam, the rule-learner wins, because (by the Occam/PAC-Bayes bound) a short hypothesis that fits the practice data has a small generalization penalty $\propto \sqrt{L(M)/n}$, whereas the memorizer’s enormous $L(M)$ gives a vacuous guarantee — it has no reason to work on questions it has not memorized. Compressing and generalizing are the same success here.

Solution 61.2. The friend confuses *surprise* with *understanding*. True random noise is maximally surprising bit-by-bit, yes — but that is exactly why it is *incompressible*: it has no shorter description, $K(x) \approx |x|$, no program prints it but a verbatim copy. “Understanding,” in this chapter, means finding a *short description*, i.e. compressing. There is nothing to understand in noise — no rule, no pattern — so an intelligent system compresses it the *worst* (cannot beat storing it raw), not the best. A system that claimed to compress noise would either be lying or have detected hidden structure, meaning it was not truly random after all. Maximum surprise and maximum understanding are opposites here, not the same thing.

Solution 61.3. Null M_0 : costs 0 bits to state; codes each of 200 bits at 1 bit, so $L(D|M_0) = 200$ bits. Total = $0 + 200 = 200$ bits.

Biased M_1 ($p = 0.75$ for a 1): costs 8 bits to state. The residual is $-150 \log_2(0.75) - 50 \log_2(0.25) = 150(0.415) + 50(2) = 62.25 + 100 = 162.25$ bits. Total = $8 + 162.25 = 170.25$ bits.

MDL prefers M_1 ($170.25 < 200$). The bias is worth declaring: stating the model costs 8 bits but saves about 38 bits of residual, a net win of ≈ 30 bits — and that net saving is the compression achieved by “understanding” the coin is biased.

Solution 61.4. The penalty is $\sqrt{(L(h)(0.693) + 3.0)/20000}$.

(a) $L(h) = 50$: numerator = $50(0.693) + 3.0 = 34.65 + 3.0 = 37.65$; penalty = $\sqrt{37.65/20000} = \sqrt{0.00188} \approx 0.043$. So $e(h) \leq 0 + 0.043 = 4.3\%$ — a strong guarantee.

(b) $L(h) = 5000$: numerator = $5000(0.693) + 3.0 = 3465 + 3.0 = 3468$; penalty = $\sqrt{3468/20000} = \sqrt{0.173} \approx 0.416$. So $e(h) \leq 41.6\%$ — nearly useless.

Both hypotheses fit the training data perfectly, yet the short one is *certified* to err at most 4.3% on new data while the long one’s guarantee is vacuous. The bound makes “prefer the shorter description” a quantitative, provable preference: compression buys generalization.

Solution 61.5. Solomonoff’s bound says the total prediction error ever accumulated, when the true source is a computable distribution μ , is at most a constant times $K(\mu)$ — the Kolmogorov complexity of the true source, i.e. the length of the shortest program that is the source’s rulebook. A “simple” universe has a short rulebook (small $K(\mu)$): the learner pays a small one-time bit-debt and then predicts almost perfectly forever. A “complicated” universe has a long rulebook (large $K(\mu)$): more total error before convergence. An incompressible universe ($K(\mu) \approx$ its length) cannot be learned at all. The single governing quantity, $K(\mu)$, is *exactly the compressed size of the world’s laws* — so “easy to learn” and “easy to compress” are the same number, which is the chapter’s thesis at the ideal limit.

Solution 61.6. (a) **Learning:** Solomonoff’s universal mixture M (often written ξ) — the Bayesian average over all computable environments weighted by $2^{-\text{length}}$ — models the world and updates as observations arrive. (b) **Acting:** the “max over actions” of expected total future reward — ordinary sequential decision theory, picking whichever action M predicts will yield the most reward. (c) Without a reward signal there is nothing to maximize, so the decision layer has no job; all that remains is M predicting the next observation — which is precisely Solomonoff induction. AIXI is therefore “Solomonoff induction plus a goal,” and the compressor (M) is the part doing the intelligence in both.

Solution 61.7. The honest, MDL-fair size is $L(M) + L(D|M)$: the *model’s own* description length plus the coded residual. For a 70B model at, say, fp16, $L(M) \approx 140$ GB, so $L(M) + L(D|M) \approx 140 \text{ GB} + 100 \text{ MB} \gg 1 \text{ GB}$ — far *larger* than the raw file. By the two-part accounting the model has not compressed this file; it has moved bits into its weights (an “expander,” not a compressor, for a single file). It genuinely compresses only when amortized over a corpus far larger than the model: if the same frozen model codes, say, 10 TB of text, the 140 GB is a one-time cost spread thin and the *per-corpus* total beats classical codecs. This is the MDL “cannot be fooled” theorem applied honestly: a model

can only win in total bits if its structure plus residual beats raw storage — a memorizer (or an under-amortized giant) pays for itself in $L(M)$ and cannot profit. The thesis’s own bookkeeping both powers and caps the LLM-compression claims.

Solution 61.8. (a) **Steel-man:** By the cross-entropy identity (Chapter 23), modelling a source is coding it; by MDL and the Occam/PAC-Bayes bound, the shortest description that fits provably generalizes best; by Solomonoff’s bound, the optimal predictor’s lifetime error is the compressed size $K(\mu)$ of the true world; by AIXI’s Pareto-optimality theorem, the optimal *agent* is that predictor plus reward-seeking. So the best compressor is the best inductor is (with a goal) the best agent — intelligence and compression coincide at the ideal limit. The empirical $r \approx -0.95$ between LLMs’ bits-per-character and benchmark scores (Huang et al., 2024) is strong real-world corroboration.

(b) **Objections:** (1) **Direction.** The -0.95 correlation shows *intelligence implies compression* (a capable model necessarily models/compresses language well) but not the converse that *all intelligence reduces* to compression; correlation is not identity. (2) **Scope and accounting.** AIXI assumes unbounded computation, a clean agent/environment split, and a given reward — drop any and optimality breaks; bounded rationality, the origin of goals, alignment/wireheading, and reasoning that does not reduce to next-symbol statistics sit outside the frame; and the two-part code shows the headline LLM “wins” only after amortizing a model that dwarfs the data. The fair verdict: compression is a *necessary signature* of capable prediction and learning, proven; it is an *open conjecture* as a complete account of agency and intelligence.

Solution 62.1. (a) $0.95 \times 1.4427 \approx 1.37$ bits/token. (b) Each token covers 4.1 bytes, so bits per byte $= \frac{1.37}{4.1} \approx 0.334$ bpb. (c) Far better than gzip’s ≈ 3.1 bpb — roughly a $9 \times$ improvement in this idealised accounting. (Caveat: this ignores the model’s own size and the tokenizer’s contribution; it is the $L(D \mid M)$ term only.)

Solution 62.2. Photographs, stored as a stream of numbers, are full of predictable patterns — pixels next to each other are usually similar colours. The text model had learned, from language, a very broad sense of “what real-world data tends to look like,” and that general talent for guessing the next number worked surprisingly well on image data too. PNG only knows a few fixed image tricks; the model’s general prediction beat them. (Key point: compression is just good guessing, and a good-enough guesser is general.)

Solution 62.3. Three symbols, so a fresh context starts $(A\{:\}1, B\{:\}1, N\{:\}1)$, total 3. Process B A N A N A :

- B from empty context (uniform $\frac{1}{3}$): $p = \frac{1}{3}$, surprisal $\log_2 3 \approx 1.585$.
- A from context B (fresh, uniform $\frac{1}{3}$): $p = \frac{1}{3}$, ≈ 1.585 . Now B’s row has $A\{:\}2$.
- N from context A (fresh, $\frac{1}{3}$): $p = \frac{1}{3}$, ≈ 1.585 . A’s row now $N\{:\}2$.
- A from context N (fresh, $\frac{1}{3}$): $p = \frac{1}{3}$, ≈ 1.585 . N’s row now $A\{:\}2$.
- N from context A (now $A\{:\}1, B\{:\}1, N\{:\}2$, total 4): $p(N) = \frac{2}{4} = \frac{1}{2}$, surprisal 1.0. A’s row now $N\{:\}3$.
- A from context N (now $A\{:\}2, B\{:\}1, N\{:\}1$, total 4): $p(A) = \frac{2}{4} = \frac{1}{2}$, surprisal 1.0.

Total $\approx 1.585 \times 4 + 1.0 + 1.0 = 6.34 + 2.0 = 8.34$ bits. The model started clueless and paid full price early, then began to learn the $A \rightarrow N$ and $N \rightarrow A$ alternation and paid less — the online-learning curve of nncp in miniature.

Solution 62.4. The decoder’s model would have advanced one update further than the encoder’s, so its context (the sliding two-byte window) and its counts would be one step ahead. For the **very first** symbol there is no prior byte, so it might survive symbol 1, but from the **second** symbol onward the decoder’s `cumulative / find` answer from a **different** distribution than the encoder used to narrow the interval. Since arithmetic decoding depends on matching distributions exactly, the decoder recovers a wrong byte at that point and every subsequent byte cascades into garbage — the round-trip invariant “consult the **same** distribution, then apply the **same** update,” identical on both sides, is broken. This is a controlled illustration of the determinism trap.

Solution 62.5. (a) 0.7 bpb on $2 \text{ GB} = 2 \times 10^9$ bytes gives $(\frac{0.7}{8}) \times 2 \times 10^9 = 0.175 \times 2 \times 10^9 = 0.35 \times 10^9$ bytes $\approx 0.35 \text{ GB}$ of coded data. (b) 50×10^9 params $\times 2$ bytes $= 100 \times 10^9$ bytes $= 100 \text{ GB}$. (c) Total $\approx 100.35 \text{ GB}$ versus 2 GB raw — the two-part code is **fifty times larger** than just keeping the file. For a single file, catastrophic. (d) The model adds a fixed 100 GB ; each GB of data costs $\approx 0.175 \text{ GB}$ coded versus 1 GB raw, saving 0.825 GB per GB . To repay 100 GB of model you need $\frac{100}{0.825} \approx 121 \text{ GB}$ of data. Beyond roughly **120 GB of corpus**, the amortised model-coded approach finally beats raw storage. This is exactly why LLM compression only makes sense at corpus scale.

Solution 62.6. The Hutter Prize scores essentially the **honest** two-part code: it counts the bits of the data **plus** the size of the decompressor (which, for an LLM coder, includes the model), and it enforces strict memory and time limits. A giant LLM has a superb $L(D \mid M)$ (low raw bpb) but a gigantic $L(M)$ (the model) and a punishing runtime, so its **total** score is poor and it likely violates the resource limits outright. The October 2024 winner, fx2-cmix (110.8 MB on enwik9), is a self-contained context-mixing compressor whose decompressor is small and fast — it optimises the **whole** $L(M) + L(D \mid M)$, which is what the prize actually measures, rather than just $L(D \mid M)$.

Solution 62.7. Implementation sketch: store the context as a `tuple` of the last k bytes (or, faster, an integer rolling hash); `cumulative / find / update` are otherwise unchanged — only the size of the context key grows. (a) Larger k captures longer-range structure, so on long repetitive or structured text compression **improves** — up to a point. (b) But the number of **possible** contexts is 256^k , so the `counts` dict can blow up; memory grows fast and most contexts are seen rarely or never. (c) With big k , almost every context is **new** early in the file, so the model is uniform (maximally surprised) for a long warm-up, paying full price — the “cold start” worsens. This is the **online model’s** core tension: rich context needs data to populate it. A **giant pretrained** model sidesteps the cold start entirely — it arrives already knowing a vast distribution learned from terabytes — at the cost of being a huge fixed dependency that must be shipped and amortised. The trade between “learn it cheaply on the fly but start clueless” and “know everything up front but be enormous” is exactly nncp-vs-Chinchilla, in miniature.

Solution 62.8. (a) The identity is a theorem because of Chapter 23’s result: for a model assigning probability $p(x_t \mid \text{context})$, an arithmetic coder (Chapter 26, achieving $\approx -\log_2 p$ per symbol to within a bounded constant for the whole message) produces a file of $\sum -\log_2 p$ bits, and the **expectation** of $-\log p$ over the data-generating distribution **is** the cross-entropy that training minimises (Chapter 18). The two quantities are equal by the definition of expectation and the coder’s near-optimality — not by analogy. (b) Many valid answers; one good one: **multi-step tool-augmented reasoning** — e.g., a model that solves a problem by calling a calculator or searching external memory. Its competence lives partly **outside** its own next-token statistics on a static held-out corpus, so its bits-per-character on that corpus need not reflect it; the model could be a mediocre raw predictor yet highly capable in deployment. This is the gap between **intelligence** $\rightarrow$ **compression** (well supported, $r \approx -0.95$) and **intelligence** = **compression** (not established), the careful line Chapter 61 drew.

Solution 63.1. Each parameter in: (a) float32 = 4 bytes, so $7 \times 10^9 \times \frac{4}{10^9} = 28$ GB. (b) bfloat16 = 2 bytes, so 14 GB. (c) INT8 = 1 byte, so 7 GB. (d) INT4 = 0.5 bytes, so 3.5 GB. (e) 2 bits per weight = 0.25 bytes, so 1.75 GB. Note: real models also carry overhead for activations, KV cache, and metadata, but these calculations capture the weight-only footprint.

Solution 63.2. Imagine you are packing a suitcase. You have 100 T-shirts and one giant inflatable castle. The suitcase has 256 compartments. If you size each compartment to fit the castle, the T-shirts rattle around in compartments far too big for them — you can barely distinguish a size-S from a size-M shirt because they both look “small” next to the castle. That is what happens with outlier activations. A few feature dimensions have values like 150, while most are near 0.1. The quantizer must cover the whole range $[-150, +150]$ in 256 steps, making each step about 1.2 units wide. A typical weight contributing to the small-valued dimensions gets rounded to the nearest 1.2-unit mark — an error of up to 0.6 for a weight that should only matter in the hundredths place. The error is giant relative to the signal.

Solution 63.3. The sequential structure is essential because each correction depends on the errors introduced in previous steps. If you quantize all columns simultaneously, you have no information about the other columns’ rounding errors and cannot compensate for them. The compensation of column q assumes that columns 1, ..., $q - 1$ have already been quantized (their errors already fixed) and that columns $q + 1$, ... are still in full precision (and can absorb the correction). If you tried to quantize all at once without correction, you get the naive round-to-nearest result — every weight independently rounded, errors not compensated — which is equivalent to standard PTQ without any second-order information. GPTQ’s contribution is exactly this sequential compensation loop.

Solution 63.4. Suppose weight $A = 10.0$ (large weight) multiplied by activation $x_A = 0.001$ (tiny activation). The contribution to output is $10.0 \times 0.001 = 0.010$. If we quantize A to 9.0 (rounding error = 1.0), the output error is $1.0 \times 0.001 = 0.001$ — tiny. Now weight $B = 0.01$ (small weight) multiplied by activation $x_B = 200$ (outlier activation). Contribution = $0.01 \times 200 = 2.0$. If we quantize B to 0 (rounding error = 0.01), the

output error is $0.01 \times 200 = 2.0$ — enormous. AWQ’s insight is to look at **output error** = **weight_error** × **activation**, not just weight magnitude or activation magnitude alone.

Solution 63.5. Zero weights do not require any computation. In a standard matrix-vector multiply, each output element is a sum of products $y_j = \sum_i w_{ij}x_i$. If $w_{ij} = 0$, the term $0 \times x_i = 0$ contributes nothing and can be skipped. On a CPU, multiplications are much more expensive than additions, and loading and multiplying by zero is still work. A sparse ternary weight matrix can be implemented as: for each non-zero weight (which is either +1 or −1), just add or subtract the corresponding activation. No multiplication at all. With 40% zeros, you skip 40% of the additions, and you replace all remaining multiplications with simple sign flips — a substantial speedup, especially on CPUs without dedicated SIMD float multiply instructions.

Solution 63.6. The claim conflates benchmark performance with general capability. Three specific failure scenarios: (1) **Long-form factual writing**: MMLU tests multiple-choice recall. A small quantization error that nudges a probability from 0.92 to 0.89 does not change the top-1 answer but might cause the model to subtly misremember rare facts when generating paragraphs. The error is invisible on MMLU, but matters in deployment. (2) **Multi-step arithmetic**: Models already struggle with precise arithmetic. W4 quantization reduces the effective precision of the model’s “working memory” across layers. A problem requiring 5 sequential arithmetic steps accumulates small errors at each step; the W4 model may fail on step 4 while the BF16 model succeeds. MMLU rarely tests this deeply. (3) **Calibration and uncertainty**: If the model must express calibrated probabilities (useful for safety-critical applications), the small score shifts from quantization may systematically bias the model’s confidence. Finally, MMLU is a coarse benchmark — it is designed to distinguish random guessing (25%) from human expert (90%+). The 2 percentage points of “headroom” above a chance-inflated base rate are much larger than the 2% gap the researcher measured. Real capability differences can hide in that range.

Solution 64.1. Standard softmax ($\tau = 1$): Compute $e^{\{3.0\}} = 20.09$, $e^{\{1.0\}} = 2.72$, $e^{\{0.0\}} = 1.00$, $e^{\{-1.0\}} = 0.37$. Sum = 24.18. Probabilities: A = 0.831, B = 0.112, C = 0.041, D = 0.015.

Temperature $\tau = 3$: Divide logits by 3 first: $[1.0, 0.333, 0.0, -0.333]$. $e^{\{1.0\}} = 2.718$, $e^{\{0.333\}} = 1.395$, $e^0 = 1.000$, $e^{\{-0.333\}} = 0.717$. Sum = 5.830. Probabilities: A = 0.466, B = 0.239, C = 0.172, D = 0.123.

As temperature increases: D’s probability rises from 1.5% to 12.3% — a large relative increase. This amplification of near-zero probabilities is exactly what makes soft targets useful for distillation.

Solution 64.2. Wanda scores: $|W| \times \|X\|_2$:

- Position 0: $0.02 \times 80 = 1.60$
- Position 1: $0.60 \times 0.3 = 0.18$
- Position 2: $0.01 \times 50 = 0.50$
- Position 3: $0.45 \times 0.5 = 0.225$

Sorted: 1.60, 0.50, 0.225, 0.18. For 50% sparsity, prune the two lowest-scoring: positions 1 (score 0.18) and 3 (score 0.225). Note: naive magnitude pruning would prune positions 0 and 2 (magnitudes 0.02 and 0.01), keeping the weights that multiply tiny activations while discarding those multiplied by activations of magnitude 80 and 50 — the exactly wrong choice.

Solution 64.3. (a) Full matrix: $1024 \times 1024 = 1,048,576$ numbers. Factor matrices: $1024 \times r + r \times 1024 = 2048r$ numbers.

(b) Set $2048r = 1,048,576$, $\frac{576}{2} = 524,288$. So $r = 524, \frac{288}{2048} = 256$. At rank 256, you use exactly half the storage.

(c) At $r = 4$: $2048 \times 4 = 8,192$ numbers. Fraction: $8, \frac{192}{1,048,576} \approx 0.0078 = 0.78\%$. Less than 1% of the original storage!

Solution 64.4. Hard labels convey only which class is correct: “cat: 1, everything else: 0.” They carry roughly 1 bit of information per example (which class among N). Soft targets from a trained teacher convey the teacher’s entire probability distribution over all classes: “cat: 0.92, lynx: 0.04, fox: 0.02, ...” These near-zero probabilities on wrong classes are “dark knowledge” — the teacher’s learned judgment that cats somewhat resemble lynxes (both have vertical ears, pointed face) but not chairs. This similarity structure is invisible in the hard label. Training on soft targets, the student learns not just “this is a cat” but “cats are like lynxes but unlike chairs” — a richer world model packed into each training example. The effective information per example rises from 1 bit (hard label) toward $\log_2 N$ bits (full soft distribution over N classes), which is why fewer examples suffice to train a capable student.

Solution 64.5. The Lottery Ticket procedure requires: (1) train the full network to convergence, (2) prune the trained weights by magnitude, (3) reset remaining weights to their original random initialization, (4) train the pruned subnetwork again. For a model like GPT-3 (175B parameters), steps 1 and 4 each cost millions of GPU-hours and millions of dollars. Step 3 requires having saved the original random initialization — impractical for pre-trained models released without their checkpoints. The method scales poorly: winning tickets found for CIFAR-10 (a tiny dataset) do not transfer to ImageNet without modification; it is not clear whether large transformer architectures have winning tickets at the same sparsity ratios as small networks. For it to become practical, we would need either: (a) a way to find the winning ticket without training the full network (a lottery-ticket “oracle”), or (b) mathematical guarantees that translate the hypothesis into a polynomial-time algorithm for identifying the right subnetwork.

Solution 64.6. One valid pipeline:

1. **Knowledge distillation** from a 70B teacher: train a 13B student on soft targets from the teacher’s logits on a 2-billion-token dataset. This is the most expensive step but preserves quality. No size change yet: $13B \times 2 \text{ bytes} = 26 \text{ GB}$.
2. **4-bit quantization** (AWQ or GPTQ): compress weights from float16 to INT4. Storage: $13B \times 0.5 \text{ bytes} = 6.5 \text{ GB}$. But we need to store quantization constants (one float16 scale per 128 weights): adds 100 MB. Still too large.

3. **Structured pruning** (remove 20% of attention heads + reduce FFN intermediate dimension by 20%): removes roughly 20% of parameters. New size: $6.5 \times 0.8 \approx 5.2$ GB.
4. After merging and sparse representation, target ≈ 5 GB, within the 6 GB budget.

Biggest uncertainty: the accuracy of the distilled 13B model relative to the 70B teacher. Distillation closes much of the teacher–student gap for instruction-following tasks, but may lose 5–15% relative accuracy on specialized domains. The pruning step adds additional 1–3% accuracy loss. Final model may run at 85–92% of the original teacher’s quality — acceptable for most phone use-cases.

Solution 65.1. Per token per layer: $2 \times 40 \times 128 \times 2 = 20,480$ bytes (the factor of 2 accounts for both K and V). Over 40 layers and 32,000 tokens: $40 \times 32,000 \times 20,480 = 26,214,400,000$ bytes ≈ 26.2 GB.

Solution 65.2. Heavy hitters capture long-range dependencies — the semantically important tokens established early in the context (the question, key facts, instructions). Recent tokens capture local coherence: the model almost always attends strongly to the last few generated tokens to stay grammatically and semantically connected to what comes next. Using only heavy hitters would cause the model to lose track of the immediate “thread” of the current sentence. Using only recent tokens would cause it to forget the key facts from earlier in the conversation. The combination handles both scales of dependency.

Solution 65.3. (a) 64 bytes — one byte per sub-space index ($2^8 = 256$ options fits in 8 bits). (b) $64 \times 256 \times 8 = 131,072$ floats across all codebooks — but these are shared across all stored vectors, not per-vector overhead. (c) Original: $512 \times 4 = 2,048$ bytes. Compressed: 64 bytes. Ratio: $2, \frac{048}{64} = 32 \times$.

Solution 65.4. (a) $32, \frac{768}{512} = 64 \times$ per layer. (b) The expansion matrix is a fixed model parameter — it is loaded once when the model starts and stays in GPU memory throughout. The cache is the **per-token, growing** buffer of new information. Model weights are static; cache entries are dynamic. Only the latent c (512 floats, 1 KB in BFloat16) is new for each token; the expansion matrix is the same whether we process 1 token or 100,000 tokens.

Solution 65.5. (a) The token with perplexity 1.2 is the better candidate for removal. (b) Perplexity equals 2^H where H is the cross-entropy. Low perplexity means low surprisal: the model expected this token with high probability, so it carries little new information. Dropping it is like removing a predictable symbol — the “code length” (information content) is nearly zero. The high-perplexity token was unexpected and carries more information, so it should be kept. (c) A proper noun or specific number that happens to appear in a grammatically predictable slot (“the answer is 42”) has low perplexity (the grammar predicts a number will follow “the answer is”) but is semantically essential. Domain-specific technical terms in predictable syntactic positions can also be low-perplexity but factually critical.

Solution 65.6. (a) Uncompressed BFloat16: $2 \times 32 \times 32 \times 128 \times 64,000 \times 2 = 16.78$ GB. (b) Option A — 4-bit quantization alone: BFloat16 is 2 bytes/float; INT4 is 0.5 bytes/float; ratio $= \frac{16.78}{4} = 4.19$ GB — just fits. Option B — H2O at 20% budget alone: $\frac{16.78}{5} = 3.36$ GB — fits with margin. Safer: combine both ($\frac{16.78}{2} = 8.39$ GB), leaving ample headroom for

model weights and activations. (c) Quality risk: 4-bit quantization may cause precision loss on long-context multi-hop reasoning (e.g., tracking entities across 64K tokens). Validate by comparing exact-match accuracy on a long-document QA benchmark (e.g., SCROLLS QuALITY or QASPER with full 64K context) against the BFloat16 baseline, and flag any degradation exceeding 2 absolute points as unacceptable.